# FDDT

## Fraud Document Detection Tool

Complete project documentation

# Contents


**Project**

FDDT — Fraud Document Detection Tool

**Introduction**

FDDT documentation

**User manual**

FDDT User Manual

FDDT Company User Manual

FDDT Platform Administrator Manual

**Architecture**

Architecture overview

End-to-end data flow

Multi-tenancy

Processing queues, fairness, rate limits and database connections

**Pipeline**

01 — Upload & intake

1a. Bulk upload (a zip of cases)

02 — Extraction & normalization

03 — Field validation

04 — Cross-document consistency

05 — Issuer verification

06 — Metadata forensics

6a. Font consistency

06b — Spelling (English)

07 — Error Level Analysis & copy-move detection

07a — Deleted / replaced content (ghost text)

08 — Visual inconsistency review & AI-generation assessment

09 — Signature & stamp verification

10 — Duplicate & near-duplicate detection

11 — Risk scoring engine

12 — Short check summaries

**Database**

Database schema

Entity-relationship diagram

**API**

API reference

API endpoint reference

**Frontend**

Frontend screens

Frontend components

**Reports**

Per-case forensic report (PDF)

**Deployment**

Local setup
Environment variables
Azure migration notes
Production deployment (Azure)
Azure production deployment — step-by-step runbook
Azure demo deployment

# FDDT — Fraud Document Detection Tool

(Formerly the "Document Authenticator Tool" / DocAuth. The product name is a display-level rename only — the repo folder, `docauth` database/containers and code identifiers are unchanged. Its single source is `APP_NAME` / `APP_FULL_NAME` in `backend/app/core/config.py` and `frontend/src/lib/appInfo.ts`.)

AI-powered document authentication, fraud detection, and case-management platform. Full documentation — architecture, every pipeline stage, database schema, API reference and deployment notes — is in `docs/`.

**What it does:** users submit a case of supporting PDF documents (vendor invoices, school documents, quotations, travel claims, payment evidence) — each upload is validated first (PDF only, ≤ 10 MB, not corrupted, not password-protected). A Celery pipeline OCRs each document (Azure Document Intelligence), then classifies it and extracts its fields (Azure OpenAI, behind the `LLMService` abstraction — see `app/services/llm_service.py`). It then runs field validation, issuer verification, cross-document consistency checks, and PDF forensics (metadata, Error Level Analysis, copy-move, ghost text on converted scans, font consistency, spelling (English), duplicate detection, visual-inconsistency review, signature/stamp detection). A transparent, weighted rules engine turns the results into an explainable risk score, and the case goes to a Reviewer L1 who approves, rejects or escalates it; an escalated case moves to the Reviewer L2 tier, which resolves it. Every step is written to an append-only audit log, and reviewers can export a per-case forensic report as PDF.

**Stack:** FastAPI + PostgreSQL (Row-Level Security, PgBouncer) + Celery/Redis (three queues, fair-share, global Azure rate limits) backend; React + TypeScript (Vite) frontend with login, case queue, case detail (live processing status, overlays, risk reasons, audit timeline), new-case/upload, company settings and platform-admin screens. Multi-tenant: several client companies share one deployment, each seeing only its own data.

## User Manuals

- 🏢 **Company User Manual** — Step-by-step operational guide for client organizations covering all 3 company roles (**User/Submitter**, **Reviewer L1/Analyst**, **Reviewer L2/Supervisor**), case submission, bulk uploads, review workflows, automated checks, and company settings.
- ⚙️ **Platform Administrator Manual** — Comprehensive guide for platform operators covering tenant provisioning, company storage/file limits, user account management across all 4 roles, password resets, global rule templates, queue monitoring, usage tracking, and audit governance.

## Prerequisites

- Docker Desktop (for Postgres, Redis and PgBouncer)
- Python 3.11+ (developed/tested against 3.14)
- Node.js 20+ (developed/tested against 24)

## Setup

```
# 1. Start Postgres + Redis + PgBouncer
docker compose up -d

# 2. Backend: create venv, install deps, configure env
cd backend
python -m venv .venv
./.venv/Scripts/activate        # Windows
# source .venv/bin/activate     # macOS/Linux
pip install -r requirements.txt
cp .env.example .env            # edit if you changed any docker-compose ports/creds

# 3. Apply migrations
alembic upgrade head

# 4. Seed the platform admin (admin@example.com / ChangeMe123!)
python seed.py

# 5. Run the API
uvicorn app.main:app --reload
```

API docs: http://localhost:8000/docs

```
# 6. Celery workers — one per processing queue, each in its own terminal
#    (scale each independently; see docs/processing-queues.md)
cd backend
./.venv/Scripts/activate   # or the equivalent for your shell
celery -A app.tasks.celery_app worker -Q extraction_queue -n extraction@%h --pool=threads -c 4 --
loglevel=info
celery -A app.tasks.celery_app worker -Q vision_queue     -n vision@%h     --pool=threads -c 4 --
loglevel=info
celery -A app.tasks.celery_app worker -Q forensics_queue,celery -n forensics@%h --pool=threads -c 4 --
loglevel=info
# optional: the scheduler + the worker for its jobs (queue metrics every minute,
# stuck-document recovery every 5 minutes, nightly usage reconciliation)
celery -A app.tasks.celery_app beat --loglevel=info
celery -A app.tasks.celery_app worker -Q housekeeping_queue -n housekeeping@%h --pool=solo --loglevel=info
```

On Linux/macOS use `./start-worker.sh extraction|vision|forensics` and `./start-beat.sh` (beat and the housekeeping worker in one process — `--beat` does not work on Windows; pool sizes from `*_WORKER_CONCURRENCY`; forensics uses prefork, one process per core). On Windows use `--pool=threads` (prefork doesn't work there). Or run them in Docker: `docker compose --profile workers up -d --scale forensics-worker=3`. Without the workers, uploaded documents sit at `processing_status: "pending"` forever — the API only enqueues the jobs.

**PgBouncer (recommended):** `docker compose up -d` also starts PgBouncer on :6432 (transaction pooling). Set `DATABASE_POOLER_HOST=localhost` in `backend/.env` to route the app through it (each process then keeps no pool of its own). See `docs/processing-queues.md` for the connection arithmetic, fair-share scheduling and the calibrated Azure rate limits ( `python -m scripts.calibrate_llm_tokens` re-measures real token usage).

**Multi-tenancy:** `admin@example.com` is a *platform admin* (no company). Create a company under **Platform › Companies** and its users under **Platform › Users**, then sign in as one of them to submit cases. The running app connects to Postgres as the RLS-enforced `fddt_app` role created by the migration ( `DATABASE_APP_*` in `.env` ). See `docs/multi-tenancy.md`.

```
# 7. In a third terminal: frontend
cd frontend
npm install
npm run dev
```

Open http://localhost:5173 and sign in with `admin@example.com` / `ChangeMe123!`. See `frontend/README.md` for details, and `docs/deployment/production-deployment.md` for the production plan on Azure (architecture, service tiers, costs, runbook). The demo is live on Azure: https://docauth-web.icysand-160befae.eastus.azurecontainerapps.io/

Login: admin@example.com / tTHt0pRc1ISDF6!9

## Verifying the document processing pipeline

1. Confirm `AZURE_DOCUMENT_INTELLIGENCE_ENDPOINT` / `_KEY` and `AZURE_OPENAI_KEY` / `_ENDPOINT` / `_DEPLOYMENT_NAME` are set in `backend/.env` (see `.env.example`) and the Celery worker (step 6 above) is running.
2. In the UI (as a company user): submit a new case, upload a PDF, then open the case detail page. Each document shows a status badge (Pending → Processing… → Complete/Failed) that updates on its own every few seconds while still in flight (the page polls `GET /cases/{id}` — no manual refresh needed). Once complete, the document type, its core fields (issuer/date/amount/reference #), and every other field the model found are shown inline, each flagged `uncertain` if the model wasn't confident.
3. To inspect the raw result directly (e.g. to review extraction quality against a source document, including Arabic text) instead of via the UI:

   ```
   # Case detail via API — includes document_type/processing_status/
   # extracted_fields per document:
   curl http://localhost:8000/cases/<case_id> -H "Authorization: Bearer <token>"

   # Or straight from Postgres, including the full OCR text
   # (documents.ocr_text) that the LLM actually saw — useful for
   # checking OCR accuracy on its own, independent of the LLM step:
   docker exec docauth-postgres psql -U docauth -d docauth -c \
     "SELECT document_type, processing_status, extracted_fields, ocr_text FROM documents WHERE id =
   '<document_id>';"
   ```

4. A failed document shows its `processing_error` inline in the UI (and in `documents.processing_error` / the `audit_log` table) instead of silently staying incomplete.
5. `backend/tests/test_document_processing.py` covers the task's success/ failure/unknown-id logic against mocked OCR/LLM services (no live Azure calls, no Celery worker needed) — run it with the rest of the suite via `pytest`. It does not (and cannot) verify real-world OCR/LLM accuracy — that has to be checked against actual sample documents, per step 3 above.

## Case report export

Reviewers can click **Export report** on Case Detail to generate a standalone PDF ( `POST /cases/{id}/reports` ; history at `GET /cases/{id}/reports` ). Sections, in order: 1 Case Details, 2 Executive Summary, 3 Explainable Findings, 4 All Checks, 5 Exceptions, 6 Limitations and Assumptions, 7 Audit Trail, 8 Appendix (fields, SHA-256 hashes, technical parameters), 9 PDF Highlighted Regions. Every highlight has a region ID (R1, R2, …) that Section 5 links to. Generation is synchronous ( `app/services/case_report_*.py` ). Reports are stored in Blob Storage under `companies/{company}/cases/{case}/reports/` with a row in `case_reports` ; every export adds a new report, none are overwritten. The findings drawn onto the report's page renders are separate from the live Case Detail overlays, which

never save anything, and the original files are only ever read. Run `alembic upgrade head` to create the `case_reports` table.

Highlight colours (live overlays and the report use the same convention): solid red = ELA, solid orange = copy-move, dashed blue = model judgment (visual review, signature — "approximate"), solid purple = rule-based field exception (a failed field-validation sub-check, or a cross-document mismatch, shown on **both** documents), solid fuchsia = font mismatch, solid teal = deleted / shortened text (ghost text). Every check and finding is shown short first (one line with the values), the full explanation behind it.

### Field locations

Field exceptions can only be highlighted if each extracted field's position is known. At extraction time the OCR word boxes are now kept long enough to match each extracted value back onto the page, and the result is stored on the field as a normalized `bounding_box` ( `app/services/field_locator_service.py` ). This is text matching, so positions are approximate; a field that can't be matched simply has no highlight. Documents extracted before this existed can be backfilled (re-runs only the Azure Layout call — no LLM, no value changes):

```
cd backend
python backfill_field_locations.py CASE-XXXXXXXX      # or no args for every document
```

## Tests

```
cd backend
pytest
```

Tests run against an in-memory SQLite DB, independent of the Postgres container, so `pytest` works without `docker compose up` . The Row-Level Security, PgBouncer and rate-limiter suites need the real services — see `docs/deployment/local-setup.md` .

## Project layout

```
backend/    FastAPI app, Celery tasks, Alembic migrations, tests
frontend/   React + TypeScript SPA (Vite)
docs/       Architecture, pipeline, database, API and deployment docs
sample-documents/   Sample PDFs for testing
docker-compose.yml  Local PostgreSQL + Redis + PgBouncer (+ optional Linux workers)
pgbouncer/          PgBouncer config for local development
```

See `docs/README.md` for the full documentation index.

# FDDT documentation

**FDDT — Fraud Document Detection Tool** authenticates third-party documents (vendor invoices, school documents, quotations, travel claims and the payment evidence behind them) *before* an approval or payment is made. A user submits a **case** of PDF documents; the system extracts their data, runs a battery of automated checks, turns the results into an **explainable** risk score, and hands the case to a human reviewer (Reviewer L1, with escalation routing a case to Reviewer L2). Every step is written to an append-only audit log. Several client companies share one deployment, each isolated from the others by the application and by PostgreSQL Row-Level Security.

These docs describe the system **as implemented**, checked against the code. The capabilities included in the current build, and those outside its scope, are collected in the table in architecture/overview.md.

## Start here

| If you want to… | Read |
|---|---|
| Learn to use the app as a Company User (Submitter, Reviewer L1, Reviewer L2) | COMPANY_USER_MANUAL.md |
| Manage the multi-tenant system as a Platform Administrator | PLATFORM_ADMIN_MANUAL.md |
| Understand the whole system in ten minutes | architecture/overview.md |
| Understand companies, roles and tenant isolation (RLS) | multi-tenancy.md |
| Tune processing queues, worker pools and Azure rate limits | processing-queues.md |
| Follow one case from upload to report | architecture/data-flow.md |
| Run it on your machine | deployment/local-setup.md |
| Know what to configure and where to get keys | deployment/environment-variables.md |
| Understand what each check detects and its limits | the pipeline docs below |
| Call or change the API | api/README.md → Swagger at `http://localhost:8000/docs` |
| Change the database | database/schema.md, database/erd.md |
| Work on the UI | frontend/screens.md, frontend/components.md |
| **Deploy to production on Azure, step by step** (network/IP/port plan, every command, scaling, alerts, release) | deployment/azure-production-runbook.md |
| Understand the production design on Azure (architecture, service tiers, costs) | deployment/production-deployment.md |
| See the remaining pre-production gaps | deployment/azure-migration-notes.md |
| See how the Azure demo is deployed | deployment/azure-demo-deployment.md |
| Read what changed in the multi-tenancy / queues work, and why | MULTI_TENANCY_CHANGE_REPORT.md |

## Contents

### User Manuals


- COMPANY_USER_MANUAL.md — Comprehensive guide for client organizations covering all 3 company roles (User/Submitter, Reviewer L1, Reviewer L2), case submission, bulk intake, review workflows, automated forensic

checks, overlays, and company settings. Includes high-resolution interface screenshots.
- PLATFORM_ADMIN_MANUAL.md — Complete operational manual for Platform Administrators covering tenant provisioning, company limits, user administration across all 4 roles, password resets, rule templates, queue monitoring, usage analytics, and immutable audit history. Includes high-resolution interface screenshots.
- user-manual.md — Consolidated role-by-role overview guide.

### Architecture

- overview.md — system diagram, tech stack, the key design decisions and their rationale, the scope-of-the-current-build table, security posture.
- data-flow.md — sequence diagram and step-by-step walkthrough, Celery task names and queues.
- multi-tenancy.md — companies, roles, two-layer tenant isolation (application + RLS), platform-admin support access, blob layout, billing & usage.
- processing-queues.md — the three queues, fair-share scheduling, idempotent writes, calibrated Azure rate limits, database connections and PgBouncer, load tests.

### Pipeline — what each stage does

Each page covers what it detects, the algorithm and libraries, the real input/output shape, known limitations and the risk rules it feeds.

1. Upload & intake · 1a. Bulk upload (a zip of cases)
2. Extraction & normalization
3. Field validation
4. Cross-document consistency
5. Issuer verification
6. Metadata forensics · 6a. Font consistency · 6b. Spelling (English)
7. ELA & copy-move · 7a. Deleted / replaced content (ghost text)
8. Visual review & AI-generation
9. Signature verification
10. Duplicate detection
11. Risk scoring engine
12. Short check summaries (what the case page and report show first)

### Reference

- API + `openapi.json` (generated from the code)
- Database schema and ERD (generated from the live database by `tools/gen_db_docs.py`)
- Frontend screens and components
- Per-case forensic report

### Deployment

- Local setup · Environment variables
- Azure production runbook — the step-by-step production deployment: address plan, ports and NSGs, static IPs, DNS, Key Vault, YAML for every container app, autoscaling, alerts, release/rollback, troubleshooting and go-live checklist
- Production deployment — Azure Container Apps architecture, S/M/L service tiers with monthly costs (UAE North list prices), deployment runbook, operations and go-live checklist
- Azure migration notes — what is Azure-ready and the remaining gaps
- Azure demo deployment — the low-cost demo topology and operating notes

### Change records

- MULTI_TENANCY_CHANGE_REPORT.md — every change, decision and problem in the multi-tenancy, processing-queue and follow-up work

## Ground rules the design follows

These principles explain many choices you will meet in the code:

- **No model is trained or hosted.** Intelligence is a managed cloud API or a classical library.
- **A transparent rules engine, not a model, produces the risk score**, with versioned, admin-tunable rules.
- **One generic, template-free extraction pipeline** for every document type.
- **Original files are immutable** and hash-verified; everything else derives from them.
- **The audit log is append-only.**
- **Weak signals are labelled weak** — signature comparison, AI-generation detection and ELA are advisory, never verdicts, and never described as "verified".
- **The UI is English-only**; only extracted Arabic values need right-to-left handling.

## Keeping these docs true

- The **OpenAPI snapshot** and the **database docs** are generated — regenerate them rather than editing: `api/openapi.json` (command in api/README.md); `database/*.md` with `cd backend && .venv/Scripts/python ../docs/tools/gen_db_docs.py` against the migrated local Postgres — re-run after every migration.
- The **pipeline docs** quote real constants, thresholds and result shapes. When you change an algorithm or a default rule, update its page and the rule tables (the default rules live in `backend/app/services/risk_rule_seed.py` and the platform's `risk_rule_templates`).
- When you add or change a capability listed in the overview's scope table, update that table.
- The **PDFs in `docs-pdf/`** are built from these Markdown files (and `api/openapi.json`) — never edit them by hand. After changing any doc, rebuild: `cd docs/tools && npm install && npm run build` (see tools/README.md).

# FDDT User Manual

**Fraud Document Detection Tool** — a guide for submitters, reviewers and platform administrators.

This manual describes the application **as implemented** (see `docs/README.md` for the technical documentation this is derived from). It covers what each of the four roles can do, how a case moves from upload to decision, how to read the automated checks and the risk score, and — deliberately — what the tool does **not** do, so nobody relies on it for more than it promises.

> **One deployment, several companies.** The application serves several client companies at once. Every user belongs to exactly one company and only ever sees that company's cases, documents, settings and audit log. The platform operator's own staff (platform admins) sit outside every company.
>
> *Screenshots were captured before multi-company support was added; some labels (for example "Admin") are older than the text, but the screens work as described here.*

## Who this is for

| Role | Can do |
|---|---|
| **User** (submitter) | Submit a case with supporting documents, track its status, see the reviewer's decision. Never sees the risk score, tier or triggered reasons — only a neutral "In review" flag — so the checks that catch fraud are never exposed to the person submitting the documents. |
| **Reviewer L1** | Everything a user can do, plus: the full review queue and dashboard, every case's automated findings and risk score, approve / reject / escalate, audit history, and exporting the per-case forensic report. Once a case is escalated to L2, a Reviewer L1 can still open and read it but can no longer approve, reject or re-escalate it. |
| **Reviewer L2** | Everything a Reviewer L1 can do, plus: approve or reject cases that have been escalated to L2, and manage **your company's** Issuer Registry and Risk Rules (Settings). |
| **Platform Admin** | Belongs to no company. Creates companies and every user account, edits the default rule set new companies start from, sees billing & usage per company and the processing-queue monitor. Can **read** any company's cases, audit log and settings for support — every such view is recorded in a platform-only log — but can never approve, reject, escalate, upload, create cases or export reports. |

Every rule above is enforced by the backend on every request, not just hidden in the interface — reaching a screen you're not permitted to use still returns "access required" or a 404, never data. A case or document of another company simply does not exist for you (404).

## Signing in

Go to the application URL and sign in with the email and password a platform admin created for you (there is no self-service sign-up, and no password-reset link — if you're locked out, ask the platform operator). The top bar shows your company and role. Sessions are a plain bearer token (8 hours) with no server-side logout. If your company is suspended, sign-in stops working for everyone in it.

### Changing your password

Every user can change their own password: click the **key icon** in the top bar (on a small screen, open the menu and choose **Change password**), enter your current password, then the new one twice (8–128 characters, different from the current one). The change is recorded in the audit log (never the password). It needs your current password — if you

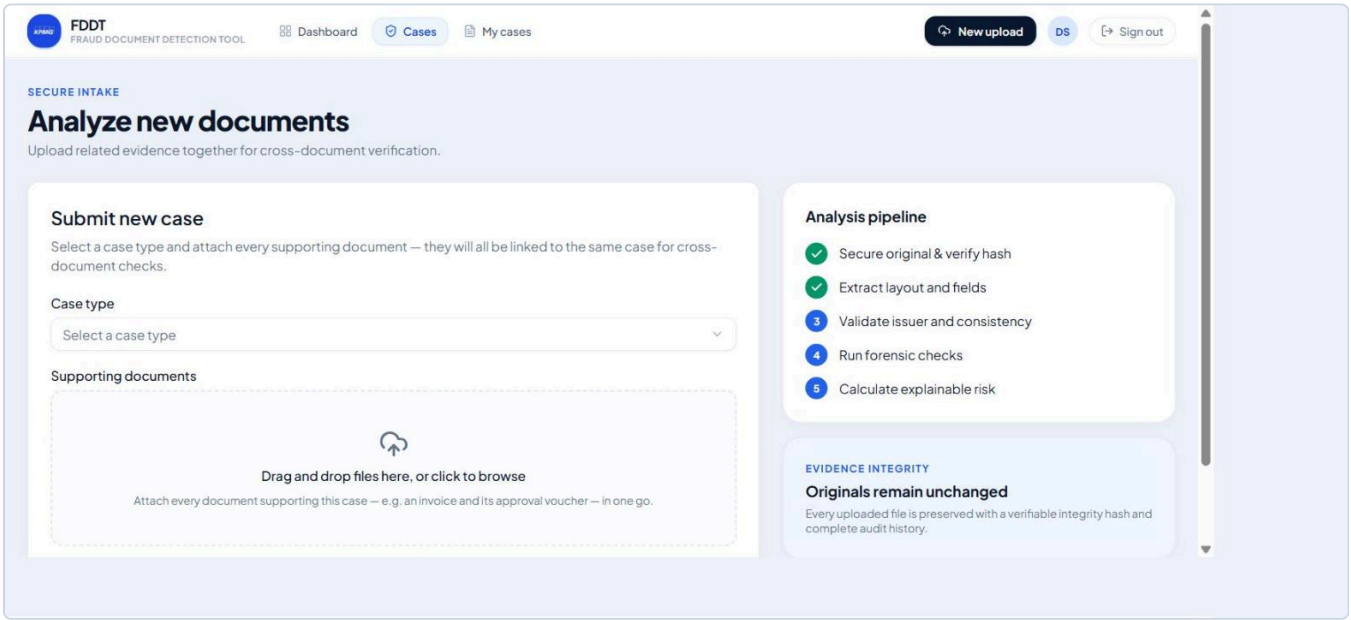
have forgotten it, a platform admin resets it for you (Settings → Users → Reset password). Sessions already signed in stay valid until they expire.

> The sign-in screen also shows a "Trust this device" checkbox and a "Sign in with SAML / Okta SSO" button. Neither does anything: the checkbox isn't wired to the login form (authentication doesn't have a device-trust concept to back it), and the SSO button is disabled. Treat the screen as plain email/password sign-in only — the badges and extra controls are decorative UI that hasn't been wired up or removed yet.

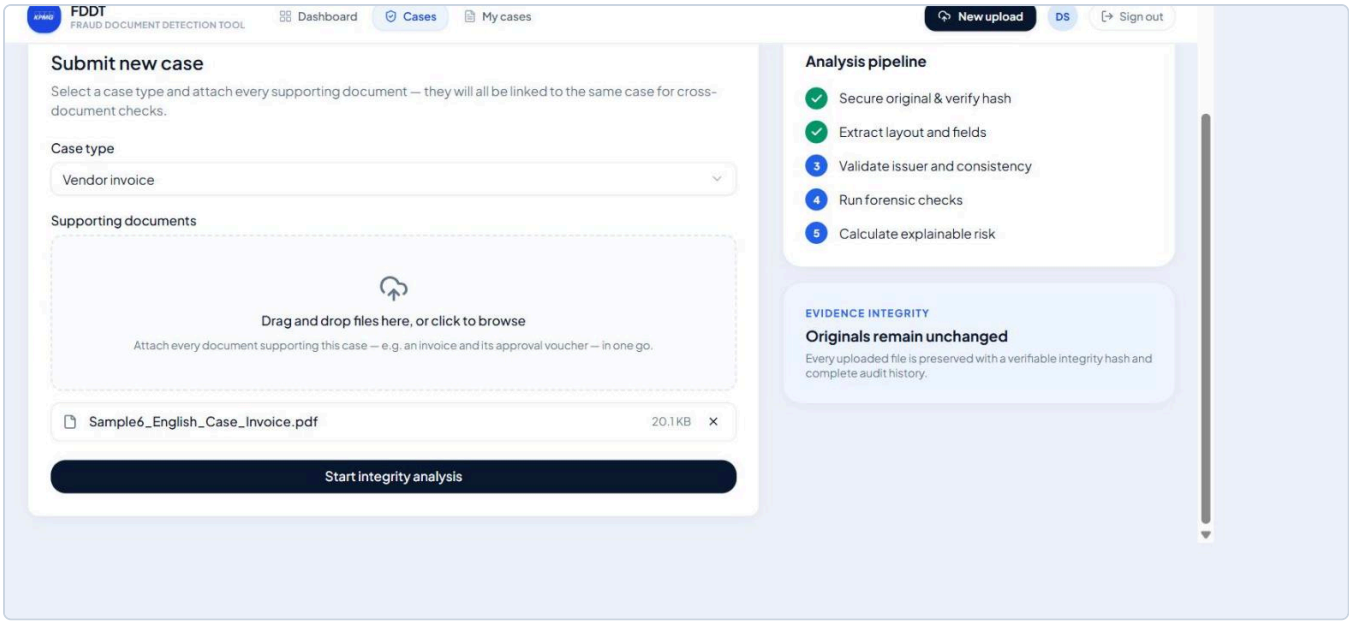
## Submitting a case (User)

1. Open **New Case**.
2. Choose the case type: *School / educational document*, *Vendor invoice*, *Commercial invoice*, *Procurement documentation*, *Quotation*, *Travel / accommodation reimbursement*, or *Other*.
3. Attach the supporting document(s) — the original invoice, receipt, quotation, evidence of payment, etc. **Only PDF files are accepted, each within your company's size limit** (10 MB unless your company has a different limit; the upload screen shows the limit that applies to you). If you have a photo or scan as an image, save or scan it to PDF first. A file is refused immediately, with the reason shown next to it, if it is:
   - not a PDF (for example a JPEG, PNG or Word file, even if renamed to `.pdf`);
   - empty, or larger than your company's limit (the message shows the file's size and the limit);
   - **password-protected** — remove the password and upload it again (the system never asks for document passwords);
   - corrupted or cut off (for example a partly downloaded file).
4. Upload. Each file uploads independently with its own progress bar; if one fails you can fix it and retry without losing the others, remove the rejected file by clicking its `[X]` icon to proceed with the remaining documents, or click **Reset form** to clear the intake form and start fresh.
5. **Optional — set a reference signature:** for an uploaded document, you can draw a box around a signature or stamp so the system compares it against the one on other documents. Skip this and no signature comparison runs for that document.
6. The case now moves into the automated pipeline, and you're taken to its detail page.

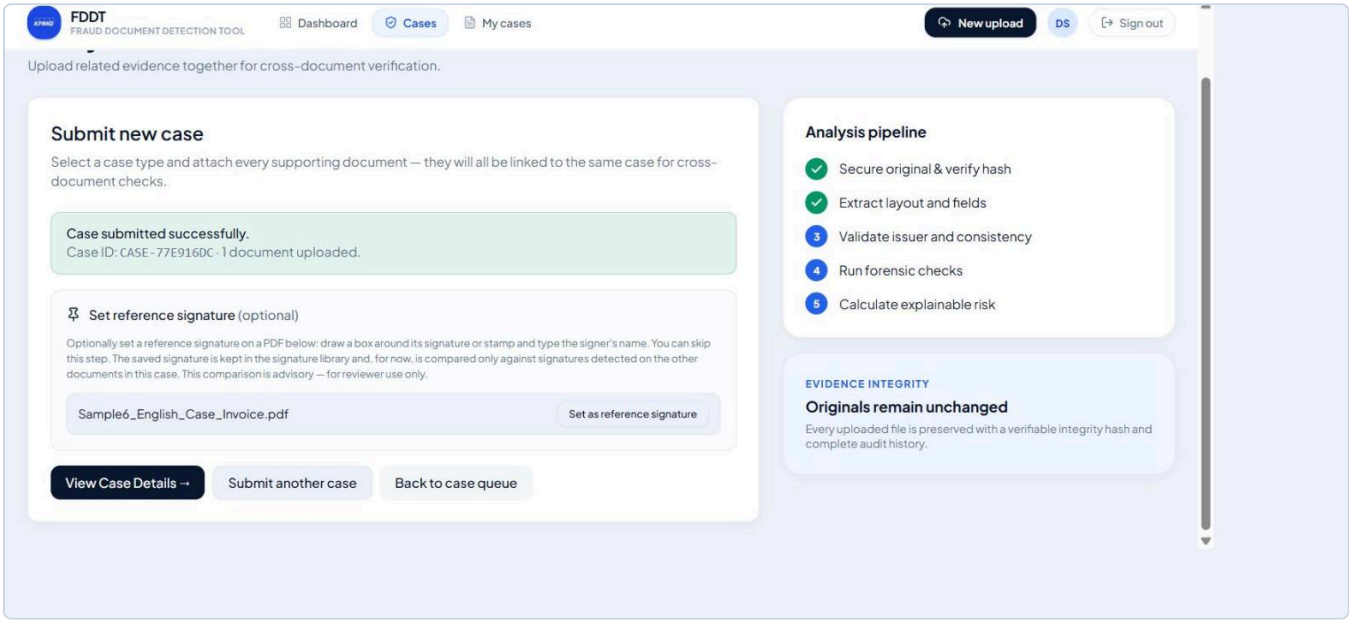
Attach every document of a case in this one submission — there is no "add a document" button on an existing case.




*The New Case form*




*A case type chosen and a document attached, ready to submit*




*Confirmation after submission, with the optional reference-signature step*

## Submitting many cases at once (bulk upload)

From **New case**, choose **Bulk upload (zip of cases)**.

1. Make one zip file with **one folder per case**, each holding that case's PDFs directly (no subfolders). The folder name becomes the case's reference label; it doesn't have to be unique.
2. Pick the case type. It applies to every case in the zip.
3. Drop the zip (within your company's zip and file limits, shown on the screen; 300 MB and 10 MB per PDF by default) and select **Upload and create cases**.

You land on the bulk upload summary. Every case folder is listed straight away, and problems found without opening the files (a subfolder inside a case folder, an empty folder, an empty or oversized file) are shown immediately. The cases are then created one by one, and each shows its own progress: **Queued** → **Processing** → **Done**. You don't have to wait for the whole zip; open any finished case while the rest are still running. A bad file (corrupted, password-

protected, not a PDF) is rejected on its own with the reason and the rest of its case goes ahead. A case is only *Not created* if none of its files could be accepted.

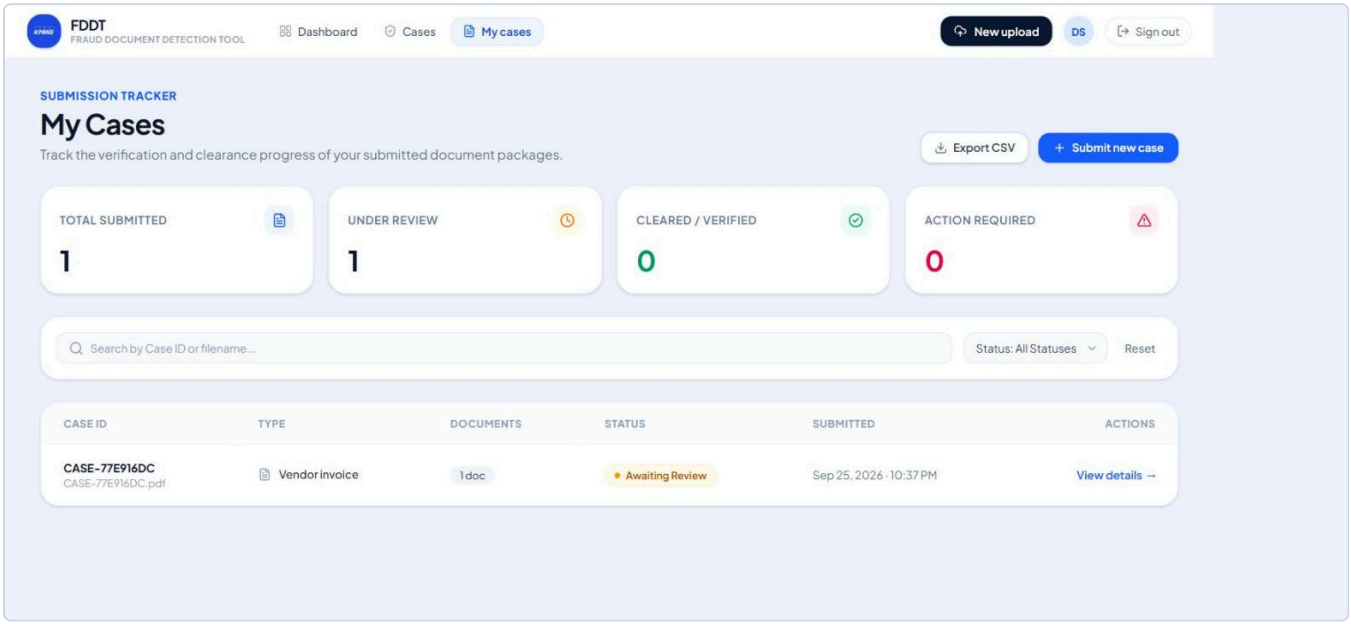
## Tracking your cases (User)

**My Cases** lists everything you've submitted, filterable by *Under review*, *Cleared* or *Action required*, with search. You'll see the outcome (approved / rejected, with the reviewer's reason when rejected) but not the risk detail behind it — that's intentional (see below).

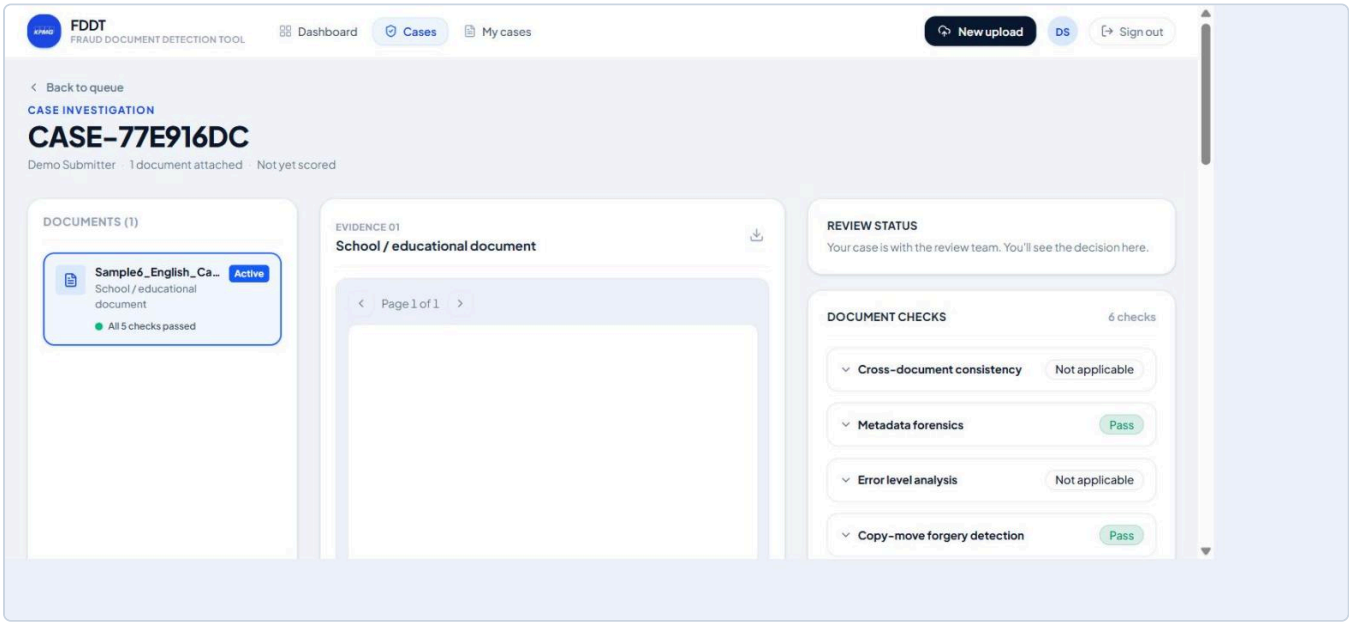
Opening one of your cases shows its documents, their extracted fields, and an activity timeline, but:

- No risk score, tier or list of triggered findings.
- No reviewer decision controls beyond a read-only history of what happened.

This is deliberate: if a submitter could see exactly which signals fired, a bad actor could learn what to avoid next time.




*My Cases: a submitter's own cases and their status*




*A submitter's view of their own case — no risk score or tier, by design*

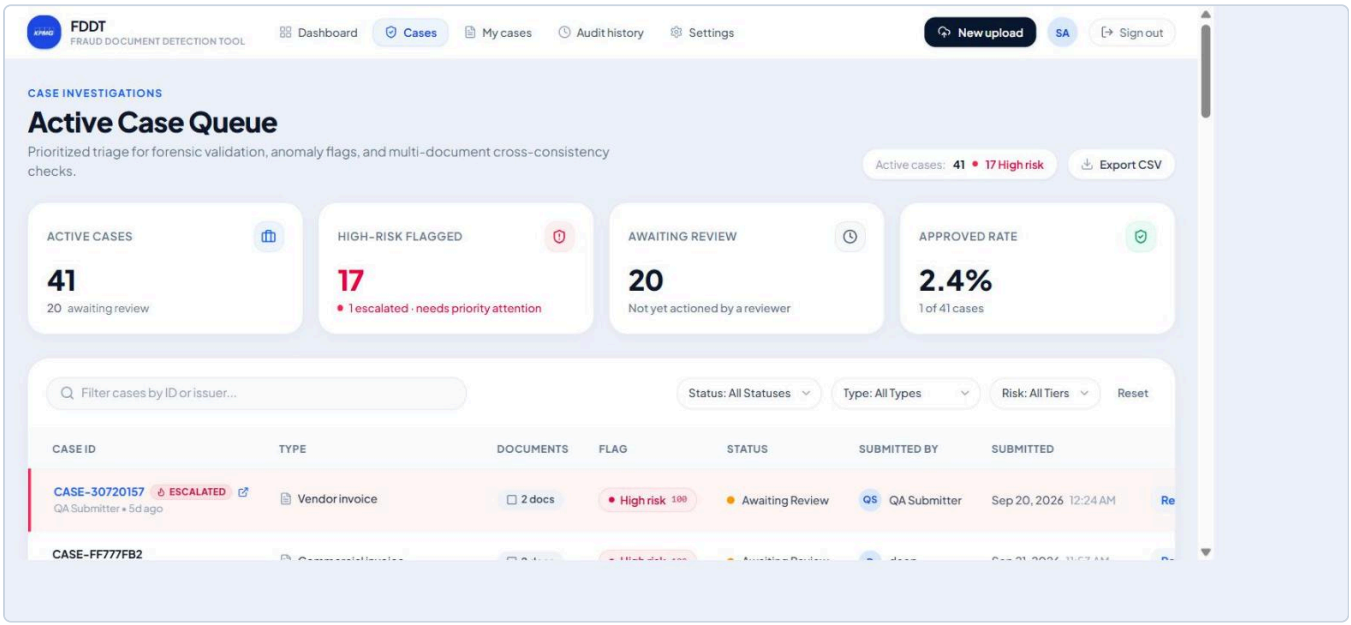
## The review queue & dashboard (Reviewers)

**Review Queue** (also the home page, `/cases`) is the master case list: counts of open / awaiting review / cleared / escalated cases, and a table of cases with their type, status, risk flag, escalation badge, document count and submitter. Search and the status / type / flag filters run instantly in your browser.

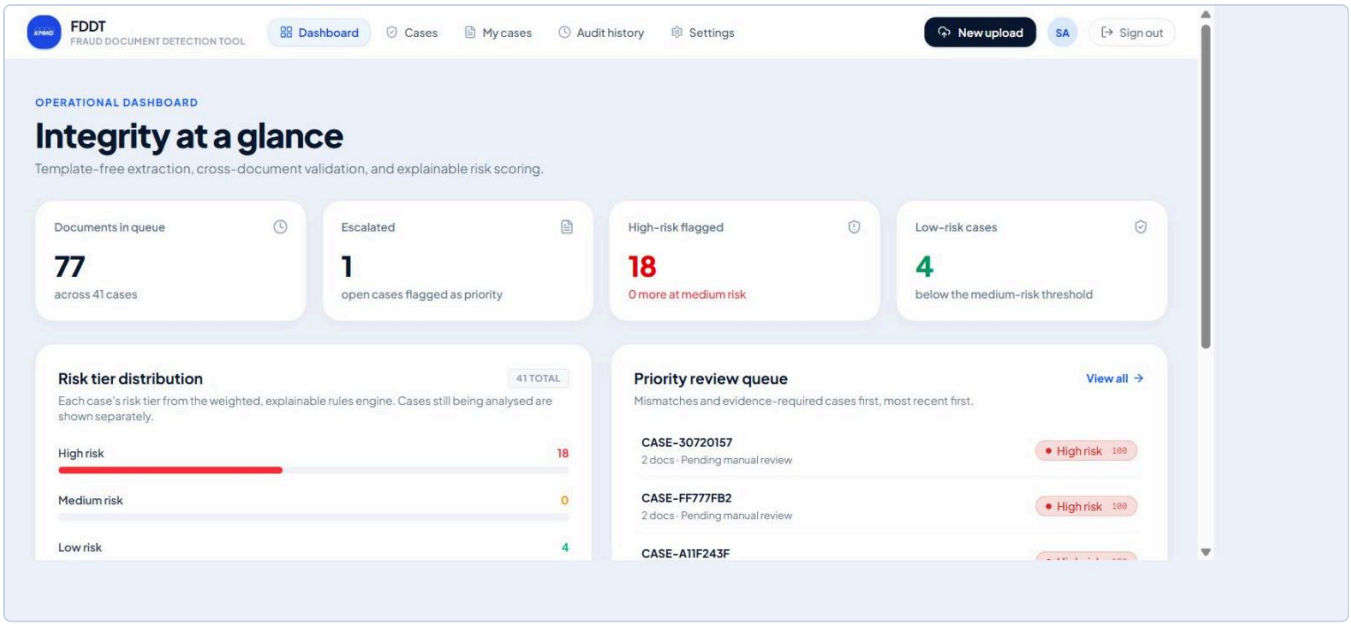
- **Reviewer L1** sees two tabs: **My queue** (the cases you can act on — the default) and **Escalated to L2 · view only** (cases handed to L2; you can open them to follow what happened, but not act on them).
- **Reviewer L2** sees **All cases** (the default — open L2 cases always sort to the top) and **Escalated queue (L2)** (only escalated cases).
- **Platform admins** pick one company at a time (company picker at the top) and see that company's list read-only.

> **"Escalated" is a reviewer tier, not a status.** An escalated case can be `submitted`, `pending_manual_review`, or already decided — escalation means the case now belongs to the Reviewer L2 tier; it doesn't change what state the case is in.

**Dashboard** is an at-a-glance operational summary computed entirely from the same case list: counts by risk flag, a risk-distribution panel, your top five open cases that you can act on (escalated first, then highest score, then newest), and a summary strip (total cases, documents analyzed, escalated, cleared). It is **not** an organisation-wide compliance/MIS report — there's no fraud-rate trend, average-resolution-time metric, or export here; that class of reporting is out of scope for this build (see *Known limitations*).




*The Review Queue: counts, filters, and the case table (admin view)*



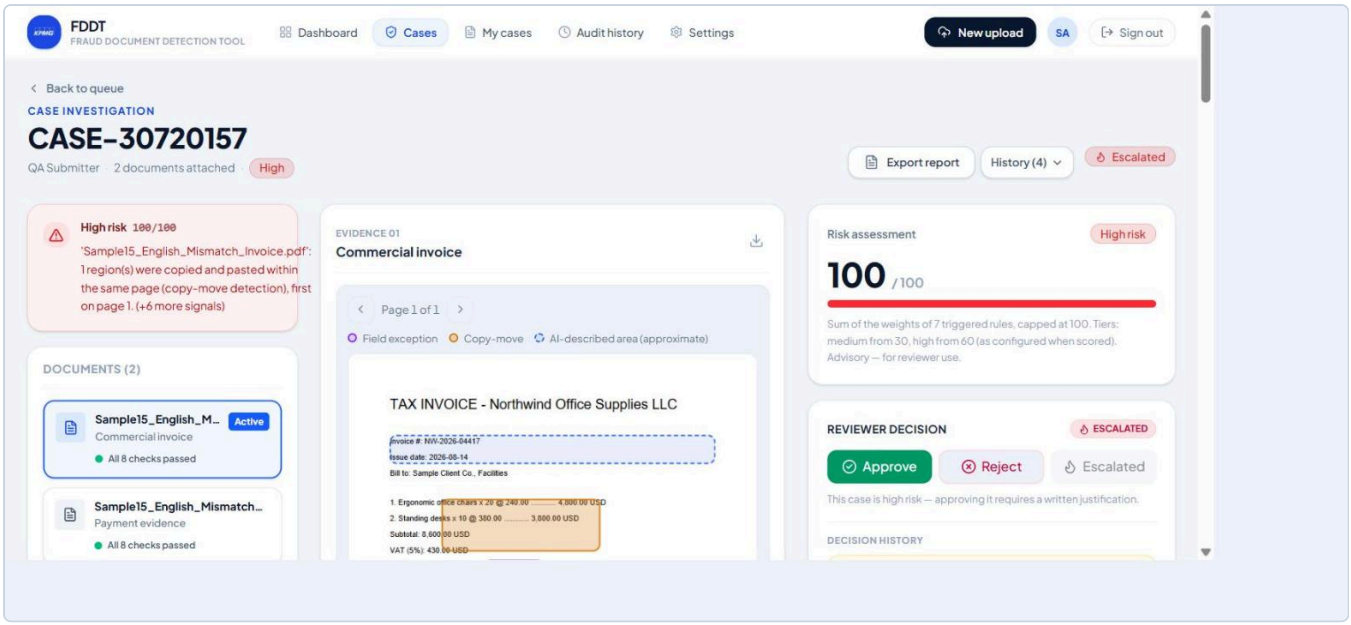

*The Dashboard: KPI cards, risk-tier distribution, and the review queue*

## Working a case: the case detail page

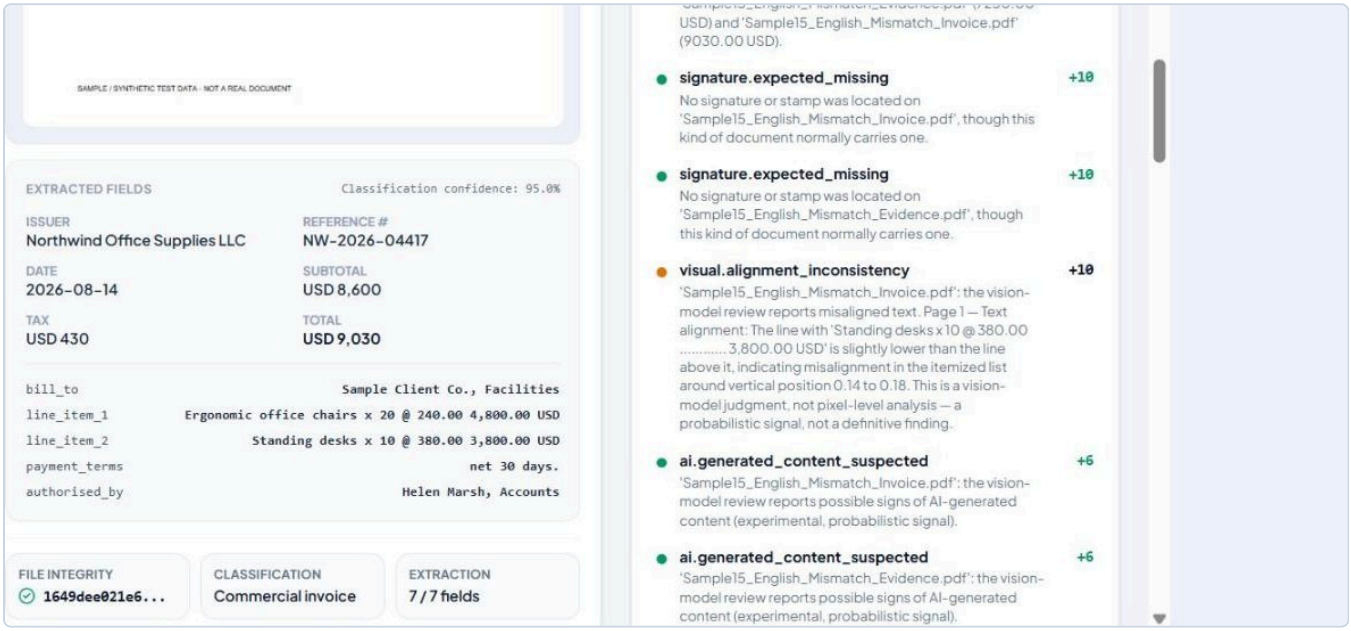
Opening a case ( `/cases/:caseId` ) shows everything about it in three panels plus a timeline:

- **Left — Documents:** every file in the case, its type and processing status. Click one to inspect it.
- **Middle — Document viewer & checks:** the selected PDF with live overlays drawn on top of it showing exactly where a check found something; the document's extracted core fields (issuer, reference number, date, amounts — issuer is the only one marked **"(uncertain)"** when the model wasn't confident); and one expandable card per automated check, plus a cross-document card and the signature comparison card if applicable.
- **Right — Risk & decision:** the score, tier, and every triggered reason in plain language; the approve/reject/escalate controls; and (reviewers only) the button to export a forensic report. A platform admin sees an amber **"Support view — read-only"** banner instead of any controls.
- **Bottom — Activity timeline:** the case's full audit trail, newest first — every automated check result and every human action, in order.

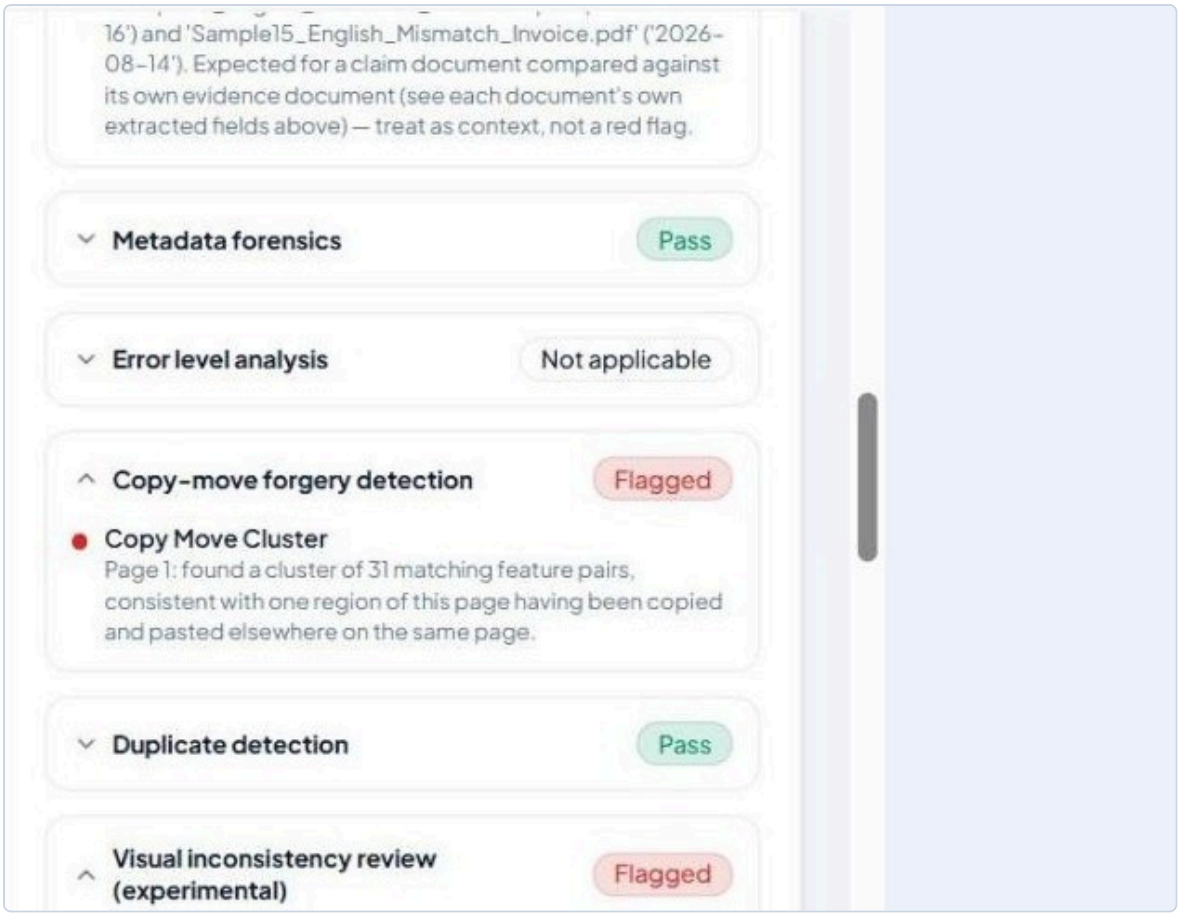
The page **polls automatically every 5 seconds** while the pipeline is still running, so you don't need to refresh — findings and the risk score simply appear as each check finishes.




*A high-risk case: documents, the PDF viewer with live overlays, the score, and the decision panel*




*Extracted fields and the full list of triggered findings in plain language*




*The per-check cards — each shows its result and, when flagged, the specific finding*

## Understanding the automated checks

Every uploaded document goes through the same generic pipeline regardless of its type — there's no separate logic per document type, so a school certificate and a vendor invoice get exactly the same scrutiny. In plain language, here's what each check looks for and how far to trust it:

| Check | What it looks for | How much to trust it |
|---|---|---|
| **Extraction & classification** | Reads the document (OCR, including printed Arabic), classifies its type, and pulls out issuer, date, amounts, reference number and any other identifiable fields — without a per-type template. | Low-confidence fields are marked; the case still goes to manual review either way. |
| **Field validation** | Rule-based checks on the extracted data: is the date in the future, do the line items add up (each column of a multi-column table on its own; installments and total rows not counted twice), do the tax and the printed rate agree, are the dates in order (dated → printed → PDF made), does a TAX INVOICE show a TRN, is a "Scanned with CamScanner" line on a file with no scan, several invoices in one file (each checked on its own; two for the same month flagged), does the stamp name the issuer. | Deterministic and reliable — it's arithmetic and date logic, not a model judgment. |
| **Issuer verification** | Fuzzy-matches the extracted issuer name against the known-issuer registry (vendors, schools, tax IDs the organisation has already vetted). | Matches on **name only** — there's no letterhead/logo comparison, and a legitimate new issuer will simply not be in the registry yet. |

| Check | What it looks for | How much to trust it |
|---|---|---|
| **Cross-document consistency** | Once a case has 2+ documents, compares shared fields (amount, date, vendor) pairwise across them and flags mismatches. | Deterministic comparison of extracted values; only as good as the extraction behind it. |
| **Metadata forensics** *(PDF only)* | Inspects the PDF's own internal structure: a modify date long after the create date, editing-software fingerprints (e.g., Photoshop) in the tool history, stripped metadata, multiple incremental-save markers. | A strong, mostly-deterministic signal, but presence of an edit marker doesn't by itself prove fraud — legitimate re-saves happen. |
| **Font consistency** | Text set in a different font from the text around it, a number with mixed character sizes, or text in a second embedded copy of a font the page already uses (text typed in during a later edit). On scans, an estimate from OCR. | Exact on a PDF's text layer; an estimate on scans, scored only when several words agree. |
| **Spelling (English)** | Misspelled ordinary English words in the printed text ("Certifcate", "Recieved") — genuine issuers print from proofread templates, so a typo can mean the text was retyped. Names, addresses, codes, British spellings and words OCR misread are left out; an AI model confirms each word before it is shown. Arabic words are not checked. | **A weak signal** — small issuers make typos too, so it adds only a little to the score (once per document). Each misspelled word is boxed on the page with its correction; the check also lists the unknown words it decided were fine (names, terms), so you can see what it set aside. |
| **Deleted / replaced content (ghost text)** | On a scan converted to editable text: faint traces of the original text left in the scanned background with no text on top (deleted) or running past the text (shortened). Shows an enhanced image of the trace and, for a deleted block, a vision model's guess at what it was. | Strong when it fires; the guess is only a hint — the trace is usually too faint to read. Not applicable to plain scans or born-digital files. |
| **Error Level Analysis (ELA)** | Looks for recompression differences that can indicate a pasted-in or edited region of an image. | **A signal, not a verdict.** Weak against a printed-and-rescanned tamper, and can false-positive on a legitimately recompressed scan — weighted lightly in scoring for this reason. |
| **Copy-move detection** | Looks for a region duplicated within the same image (e.g., a copied stamp or number). | Classical computer-vision matching — solid at finding literal copy-paste, not a semantic read of what changed. |
| **Duplicate / near-duplicate detection** | Perceptual-hashes every uploaded page and compares it against everything your company ever uploaded, including other cases; a match is then compared on invoice number, date, amount and student. | Reliable for catching the same file resubmitted. A page that only shares the layout (same template, other values) is shown for review, not scored. It never compares against another company's documents. |

| Check | What it looks for | How much to trust it |
|---|---|---|
| **Visual review & AI-generated-content assessment** | An AI vision pass describing visual inconsistencies as a secondary check, plus one experimental question about whether the image looks AI-generated. | **Explicitly experimental** — real-world accuracy on scanned business documents is unproven. Treated as one weak signal among many, never a standalone verdict. |
| **Signature / stamp detection & comparison** | Locates a signature/stamp region on the page (and flags a stamp that is typed text and drawn lines in the file rather than an image of a real stamp), and — only if a reference signature was set — gives an advisory visual comparison ("visually consistent with reference on file") against it. | **Presence/placement confirmation for a human, not an identity match.** This is a known weak point; the wording deliberately never says "verified". |

You will not see a single automated "fraud / not fraud" verdict anywhere in the product — every finding above feeds the risk score as one input among several, and a human always makes the final call.

## Understanding the risk score

Every case is scored by a **transparent, weighted rules engine** — not a black-box model. Every point in the score traces back to a named rule and a plain-English reason a reviewer can read.

- Each rule that fires adds its configured weight; the total is capped at 100.
- The score maps to a tier: **Low** (0–29 by default), **Medium** (30–59), **High** (60–100) — your company's Reviewer L2s can retune these thresholds under Settings.
- Next to the score you'll see every **triggered reason** in plain language (e.g. "*Issuer name 'X' did not match any known vendor in the registry*"), not just a number.
- **Every scored case goes to the reviewer queue.** Nothing auto-approves, regardless of score — a low score means "no rules fired," not "cleared."
- A case can re-score itself while you're looking at it (for example, when a signature comparison finishes after you opened the case) — a small delay before the numbers settle is expected.
- **When a rule's weight is later retuned, past cases keep the score they were originally given** — the rule versions and thresholds active at scoring time are frozen with that case forever, so tuning the engine never silently rewrites history.

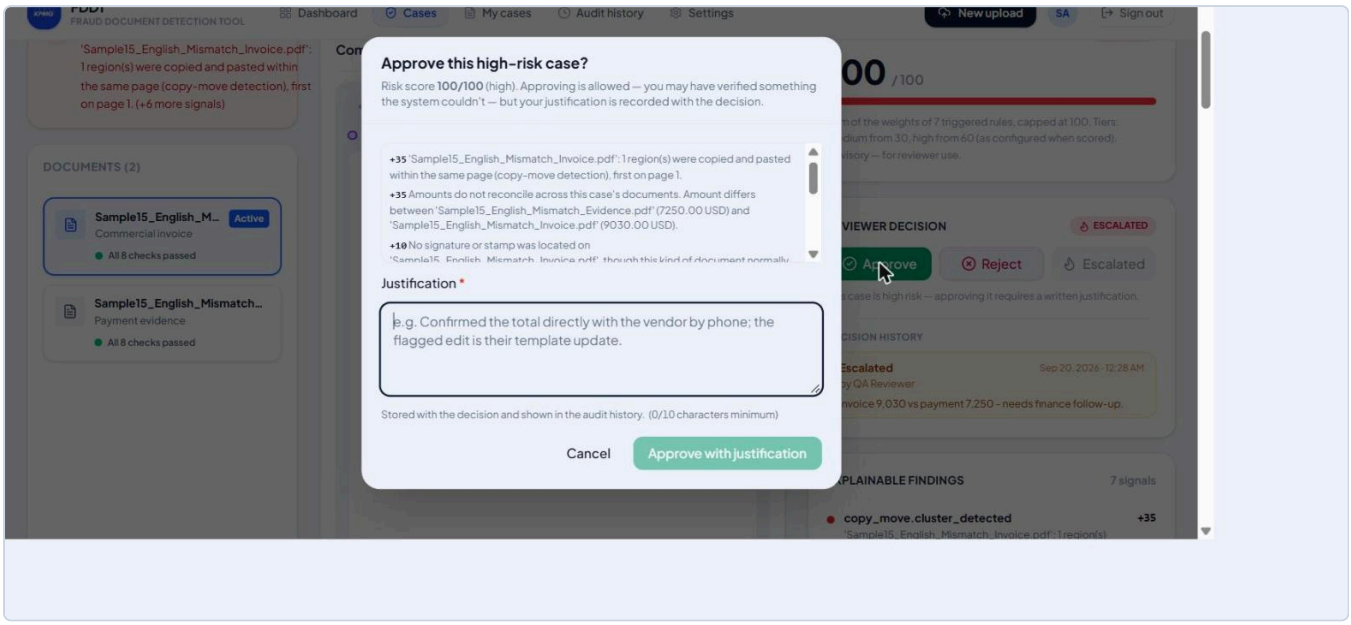
## Making a decision: approve / reject / escalate

From the case detail page, a reviewer can do the following. On a case escalated to L2, only a Reviewer L2 can; a Reviewer L1 sees a read-only panel explaining that the case is with L2, and the backend refuses the actions too. Platform admins can never act on a case.

- **Approve.** Disabled until every automated check has finished and the case has a risk assessment (a list of what's still pending is shown). Approving a **low-risk** case is a single click. Approving anything **medium or high** opens a confirmation showing the score and every triggered reason, and requires a **written justification of at least 10 characters** — you may have verified something the system couldn't, but that reasoning is recorded with the decision and shown in the audit history.
- **Reject.** Always requires a reason. It's stored, shown in the audit trail, and visible to the submitter.

- **Escalate.** Requires a reason. This hands the case to the **Reviewer L2** tier — it does **not** change the case's status. Escalation is one-way: there is no "send back to L1". A Reviewer L2 resolves the case with the ordinary Approve / Reject. From then on, Reviewer L1s see the case read-only.

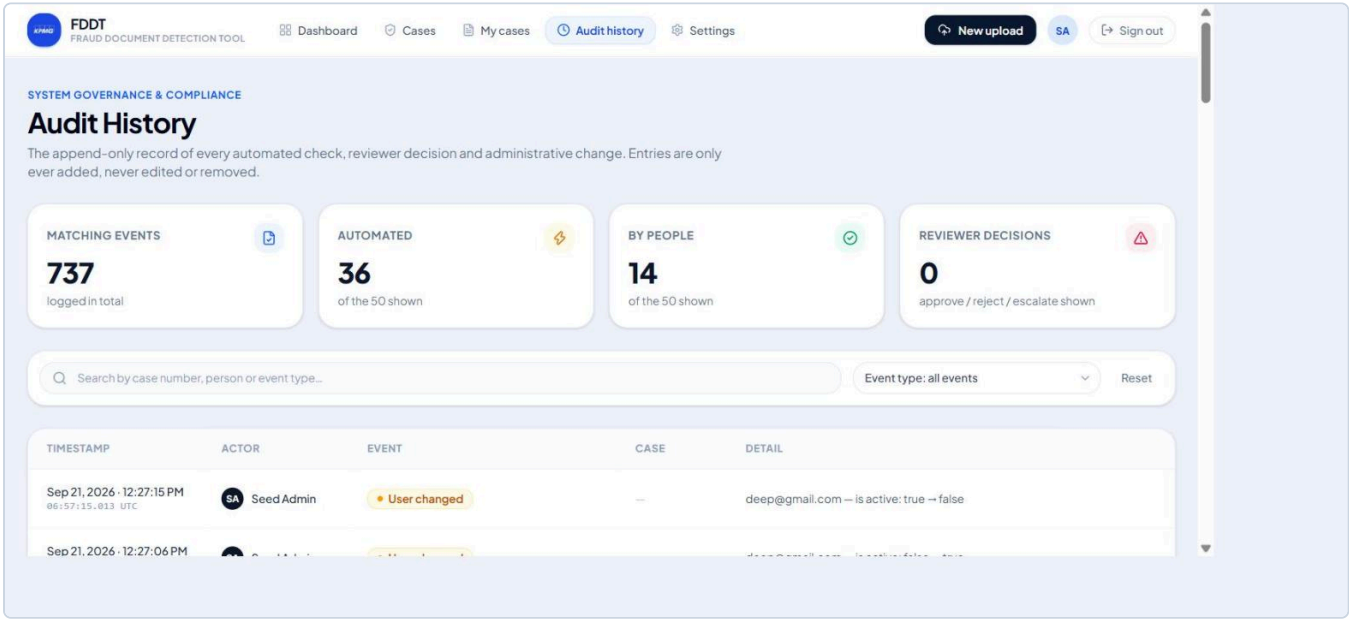
Once a case is approved or rejected, the decision panel becomes read-only.




*Approving a high-risk case requires a written justification — the triggered reasons are shown alongside*

## Audit history

**Audit History** ( `/audit-history` , reviewers only) is a searchable, filterable read of your company's **append-only** audit log — every automated check result and every human action, across every case of the company, written in plain sentences. Nothing in this log is ever edited or deleted; a case's own timeline (on its detail page) is the same log filtered to that case. Platform admins can read a company's log (and a separate platform-level log); their own support views are recorded only in the platform log.




*The system-wide, append-only audit log with filters and search*

## Exporting the forensic report

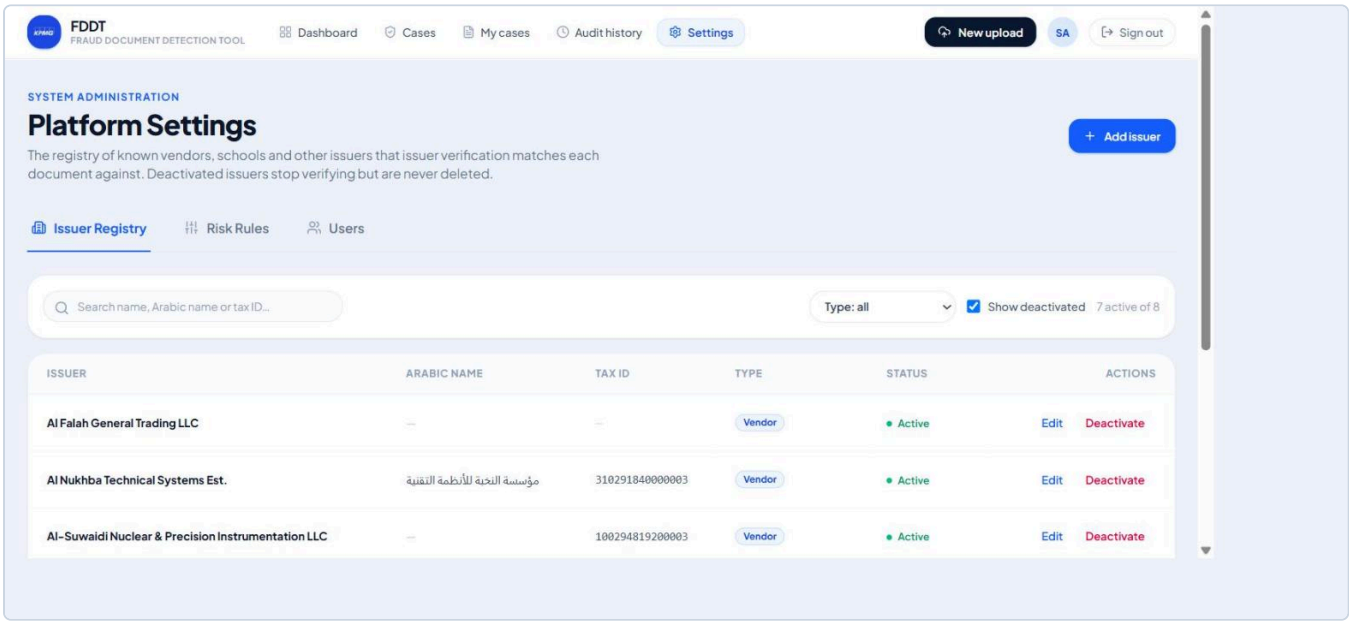
From a case's detail page, a reviewer can generate a standalone PDF report — **Export report** — built to be understood by someone who has never used the application. It contains:

1. Case details (number, type, status, submitter, documents, decision).
2. Executive summary (score, tier, decision, pointers to the sections below).
3. Every triggered risk reason, with severity and the exact region on the page it refers to.
4. The complete checklist — every check on every document, plus the cross-document and signature results.
5. Exceptions — everything that did **not** pass, spelled out concretely.
6. Limitations & assumptions — what the report is (a system-assisted analysis to support human review, not a certified forensic opinion), which checks are deterministic versus model judgment, and which checks actually ran.
7. The full audit trail (oldest first; later events beyond the first 150 are counted, not dropped).
8. An appendix: extracted fields with confidence, each original file's SHA-256 with live tamper-evidence verification, and technical parameters of each check.
9. One annotated page per flagged document page, with every finding highlighted and a legend.

Generating a report takes a few seconds and never re-runs a check or changes the score — it's a frozen, point-in-time snapshot. Every generation is kept; earlier reports for the same case remain downloadable, and two reports for the same case can legitimately show different numbers if the case was re-scored in between.

## Administration: issuer registry (Reviewer L2)

**Settings** → **Issuer Registry** maintains **your company's** list of known vendors, schools and other issuers (name, Arabic name, tax ID, type, active/inactive) that the issuer-verification check matches against. A new company starts with an empty registry — until it is filled in, almost every document will be flagged "issuer not in registry". Deactivating an issuer keeps its history — rows are never deleted, only marked inactive.



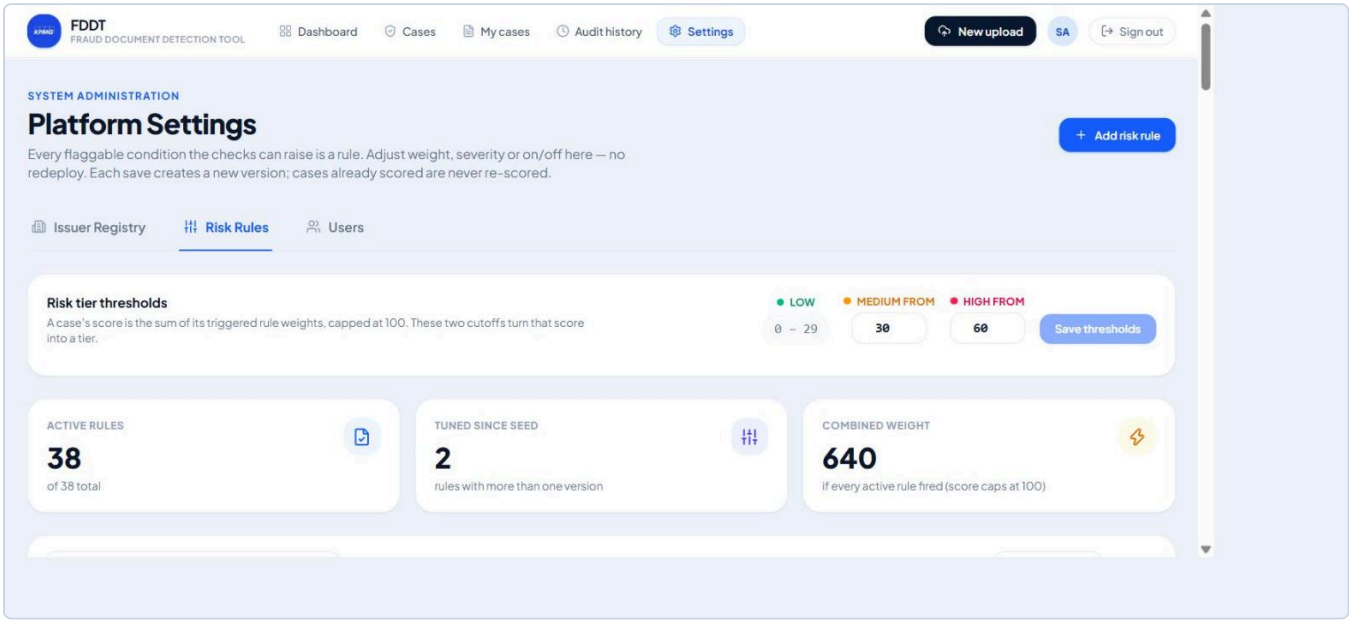

*The issuer registry, including right-to-left Arabic names*

## Administration: risk rules & thresholds (Reviewer L2)

**Settings** → **Risk Rules** lets you view and tune **your company's** scoring engine without a redeploy (every company starts from the platform's default rule set and then owns its copy):

- Adjust a rule's **weight**, **severity** or **active** flag. This **inserts a new version** — the old one is kept, and every case already scored keeps the version that was active when it was scored.
- View each rule's change history.
- Adjust the **Low / Medium / High thresholds** (default 30 / 60).
- Add a new rule from the catalog of match types the engine understands — you pick a condition and its parameters from a guided list; nobody hand-writes rule logic, and a combination the engine can't evaluate is rejected outright.

Changes apply to your company's cases scored **from that point on**. They are never applied retroactively, and never affect another company.




*Risk-tier thresholds and the active rule count — tunable without a redeploy*

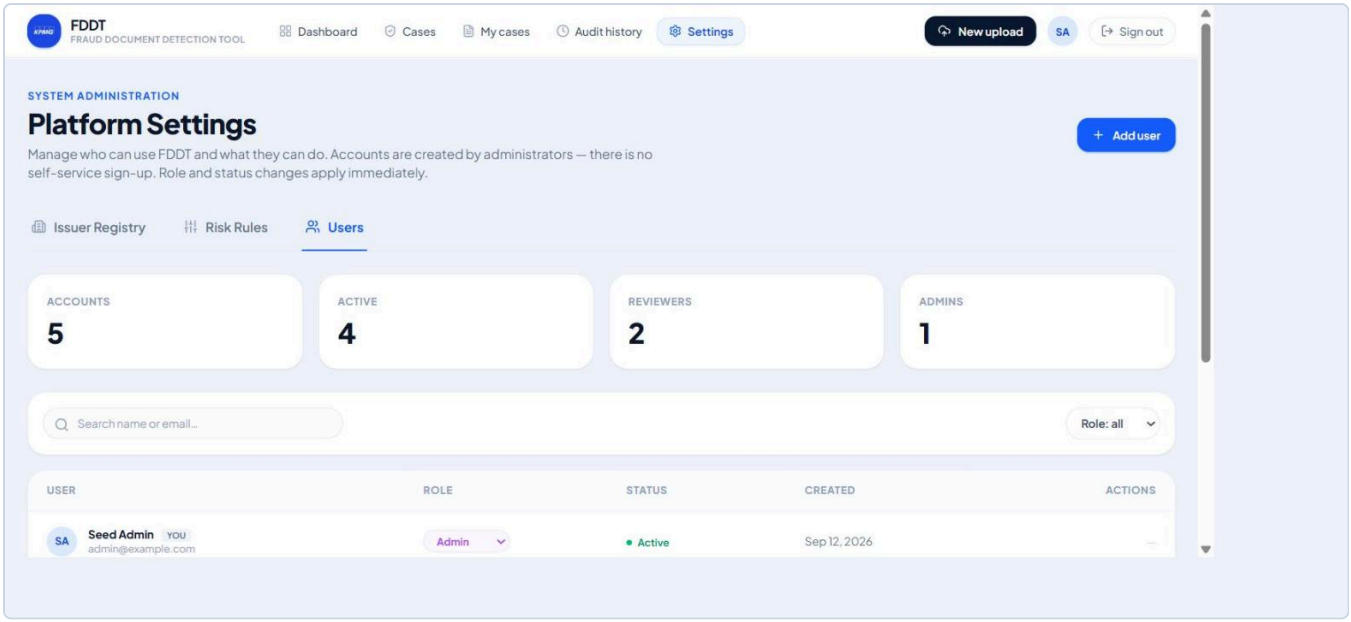
## Platform administration (Platform Admin)

The **Platform** area is visible only to platform admins:

- **Companies** — set each company's **upload limits** with **Edit** (max file size and max zip size in MB, 10 MB / 300 MB by default; takes effect on the company's next upload, and every change is recorded in the platform audit log with the old and new values); create a company (it receives the default rule set and thresholds; its issuer registry starts empty), rename it, or suspend / reactivate it. Suspending blocks sign-in for all its users.
- **Users** — create and manage every account; there's no self-service sign-up and companies cannot manage their own users. Each account gets a company and a role (User, Reviewer L1, Reviewer L2), or is a Platform Admin with no company. Moving a user to another company signs them out. A platform admin cannot change their own role or deactivate their own account.
- **Rule templates** — the default rules every **new** company starts from. Editing them never changes an existing company's rules.
- **Billing & usage** — per company: cases created, documents uploaded, files stored and storage used, for this month, last month, the last 30 days, this year, all time or a custom range; **Reconcile now** recounts from the source data.
- **Processing queues** — live view of the three processing queues (waiting, running, oldest waiting task, waits, rate-limiter activity, outstanding work per company).

For support, a platform admin can also open any company's case list, cases, audit log and settings (choosing the company at the top of the page). These views are **read-only** and every one is recorded in the platform-only audit log;

the company's users don't see those records. A platform admin may change a company's issuer registry or risk rules on its behalf — such a change is recorded in that company's own audit log.




*Account management — roles and status, no self-service sign-up*

## Case statuses & flags — quick reference

| What you see | Meaning |
|---|---|
| **Analyzing** | The automated pipeline is still running ( `submitted` / `under_automated_review` ). |
| **Awaiting Review** | The pipeline finished; the case is in the reviewer queue ( `pending_manual_review` ). |
| **Approved** | A reviewer approved it. |
| **Rejected** | A reviewer rejected it, with a recorded reason. |
| **Escalated · L2** (badge, not a status) | A reviewer handed it to the Reviewer L2 tier; it keeps whatever status it already had. |
| **Low / Medium / High** (flag, reviewers only) | The risk tier from the scoring engine. A submitter sees only a neutral "In review" instead. |

## Known limitations — what this tool does not do

Being direct about this is part of using the tool responsibly:

- **No signature is ever "verified."** Signature/stamp checks confirm presence and placement, and give an advisory visual comparison at best — never a reliable identity match.
- **AI-generated-content detection is experimental.** It's one weak signal inside the visual review, not a proven detector, and its real-world accuracy on scanned business documents hasn't been validated.
- **ELA is a signal, not a verdict** — weak against print-and-rescan tampering, and can flag a legitimately recompressed scan.
- **Only PDFs (within the company's size limit, not password-protected) can be uploaded**, because every forensic check works on the PDF itself. There is no page-count limit and no virus scan.

- **A failed check contributes nothing to the score, silently.** If a check errors out, the case still scores — the failure is visible in the audit log and the check's own card, but no rule fires because of it.
- **The absence of a flagged finding does not guarantee a document is genuine.**
- Not built in this version: notifications/email, SLA tracking and deadlines, organisation-wide MIS/compliance reporting, letterhead/logo matching, cross-case signature-library matching, integration with an external ticketing platform (ServiceNow, Jira, etc.), password reset, and company-managed user accounts.
- **The interface is English-only.** Documents may be in English or printed/typed Arabic; **handwritten Arabic is not reliably read.** Only extracted Arabic field values render right-to-left — the application itself is not translated.

## Glossary

| Term | Meaning |
|---|---|
| **Company** | One client organisation (tenant). Its users, cases, settings and audit log are invisible to every other company. |
| **Case** | One submission, grouping one or more related documents (e.g., an invoice plus its payment evidence) under one `case_number` (`CASE-XXXXXXXX`). |
| **Document** | One uploaded PDF within a case. |
| **Tier** | Low / Medium / High — the risk-score band that drives the case flag and whether approval needs a justification. |
| **Triggered reason** | One rule that fired on a case, shown as a plain-language sentence with its severity and weight. |
| **Tier (L1 / L2) / Escalated** | Which reviewer tier owns a case. Every case starts at L1; escalating moves it to L2 for good — not a status. |
| **Pipeline** | The set of automated Celery jobs (OCR, classification, extraction, validation, forensics, scoring) that run on every uploaded document. |
| **Audit log** | The append-only record of every automated result and human action, never edited or deleted. |

# FDDT Company User Manual

## Fraud Document Detection Tool — Guide for Submitting & Reviewing Organizations

Welcome to the **Fraud Document Detection Tool (FDDT)**. This manual provides a complete, step-by-step operational guide for client organizations using FDDT to authenticate supporting documents, detect digital tampering and alterations, and streamline the review process for invoices, reimbursements, quotations, and official records.

---

## Table of Contents


1. System Overview & Architecture
2. Company User Roles & Permissions
3. Signing In & Navigation
4. User (Submitter) Guide
    - Submitting a Single Case
    - Submitting Cases in Bulk (Zip Upload)
    - Tracking Submissions (My Cases)
    - Understanding the Submitter Case View
5. Reviewer L1 (Analyst) Guide
    - Operational Reviewer Dashboard
    - Managing the Review Queue
    - Full Case Analysis & Risk Breakdown
    - Forensic & Automated Document Checks
    - Interactive PDF Visual Overlays
    - Review Decisions (Approve, Reject, Escalate)
    - Exporting Case Forensic Reports
6. Reviewer L2 (Senior Reviewer / Supervisor) Guide
    - Managing Escalated Cases
    - Company Issuer Registry
    - Company Risk Rules & Thresholds
7. Getting Help & Administrative Requests


---

## 1. System Overview & Architecture

The **Fraud Document Detection Tool (FDDT)** is an enterprise document verification platform. When third-party supporting documents (such as vendor invoices, school fee receipts, procurement quotations, or travel receipts) are submitted, the automated verification engine performs:

1. **Intake Validation**: Checks file format (PDF), verifies integrity, ensures the file is not password-protected, and validates file size limits.
2. **Text Extraction & Normalization**: Automatically reads all printed text (including multi-lingual content such as Arabic and English) and structures key fields (dates, amounts, invoice numbers, vendor names).

3. **Forensic Analysis**: Scans for copy-move cloning, digital alteration artifacts (Error Level Analysis), text deleted or rewritten in converted scans (ghost text), retyped text in a different font or a second copy of the font, misspelled English words (a sign of retyped text; a weak signal), suspicious PDF metadata histories (including documents built as web pages), and visual inconsistencies.
4. **Consistency & Verification**: Validates calculations, compares cross-document values within the case, checks the vendor against your company's known **Issuer Registry**, and performs signature/stamp detection.
5. **Explainable Risk Scoring**: Aggregates findings into a transparent risk score (0–100) mapped to risk tiers (**Low**, **Medium**, **High**) with clear explanations.

> **Tenant Data Privacy**: Your organization operates in an isolated workspace. Users within your company only ever see cases, documents, and settings belonging to your company.

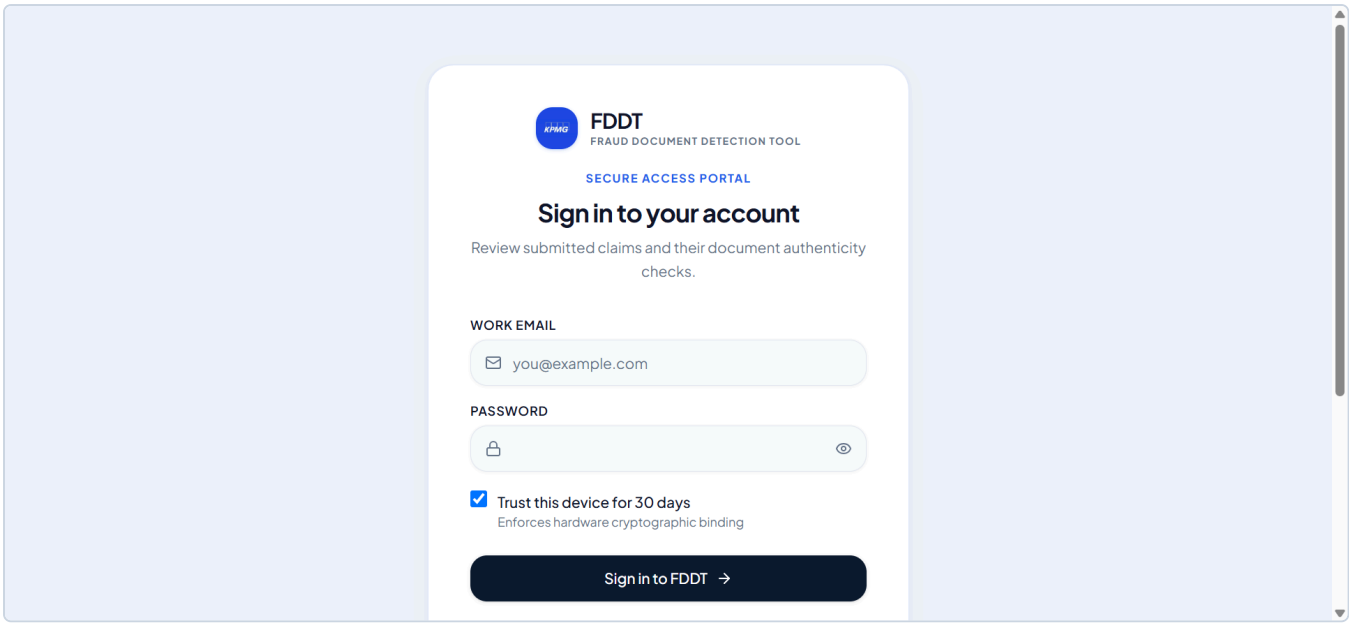
## 2. Company User Roles & Permissions

Within your company, three distinct user roles exist to enforce separation of duties:

| Role | Primary Responsibility | Available Screens & Capabilities |
|---|---|---|
| **User** *(Submitter)* | Document Intake & Tracking | • **New Case**: Submit individual cases with PDF files and optional signature references.<br>• **Bulk Upload**: Upload a zip file containing multiple case folders.<br>• **My Cases**: Track status (*Under review*, *Cleared*, *Action required*) of own submissions.<br>• **Case Detail (Submitter View)**: View uploaded documents and extracted fields without risk scoring details. |
| **Reviewer L1** *(Analyst)* | Initial Verification & Risk Triage | • *All User capabilities*, plus:<br>• **Dashboard**: High-level case volume and risk distribution metrics.<br>• **Review Queue**: Filter cases by risk tier, document category, and status.<br>• **Full Case Detail**: View risk scores (0–100), triggered fraud reasons, and automated forensic checks.<br>• **Visual Overlay Viewer**: Inspect bounding box anomalies directly on document pages.<br>• **Decision Actions**: Approve, Reject, or Escalate cases.<br>• **Export Report**: Download full forensic audit reports. |
| **Reviewer L2** *(Senior Reviewer)* | Final Adjudication & Company Governance | • *All Reviewer L1 capabilities*, plus:<br>• **Escalated Cases Resolution**: Final approve/reject authority on complex cases escalated by L1 analysts.<br>• **Issuer Registry**: Add, edit, verify, or deactivate trusted vendors and issuers for your company.<br>• **Company Risk Rules**: Customize fraud rule weights and adjust Low/Medium/High risk tier score thresholds. |

## 3. Signing In & Navigation

Access the platform URL provided by your organization using your business email address and password.




*FDDT Sign In Screen*

1. Enter your corporate email address and assigned password.
2. Click **Sign in**.
3. Upon successful sign-in, the top navigation bar displays your organization name, user role badge, and relevant navigation links based on your permissions.

> *Note: If you have forgotten your password or require a new user account, please email your administrator for this.*

**Changing your password.** Once signed in, click the **key icon** in the top navigation bar (or **Change password** in the mobile menu), enter your current password and the new one twice (at least 8 characters). Your current password is required — a forgotten password can only be reset by your administrator.

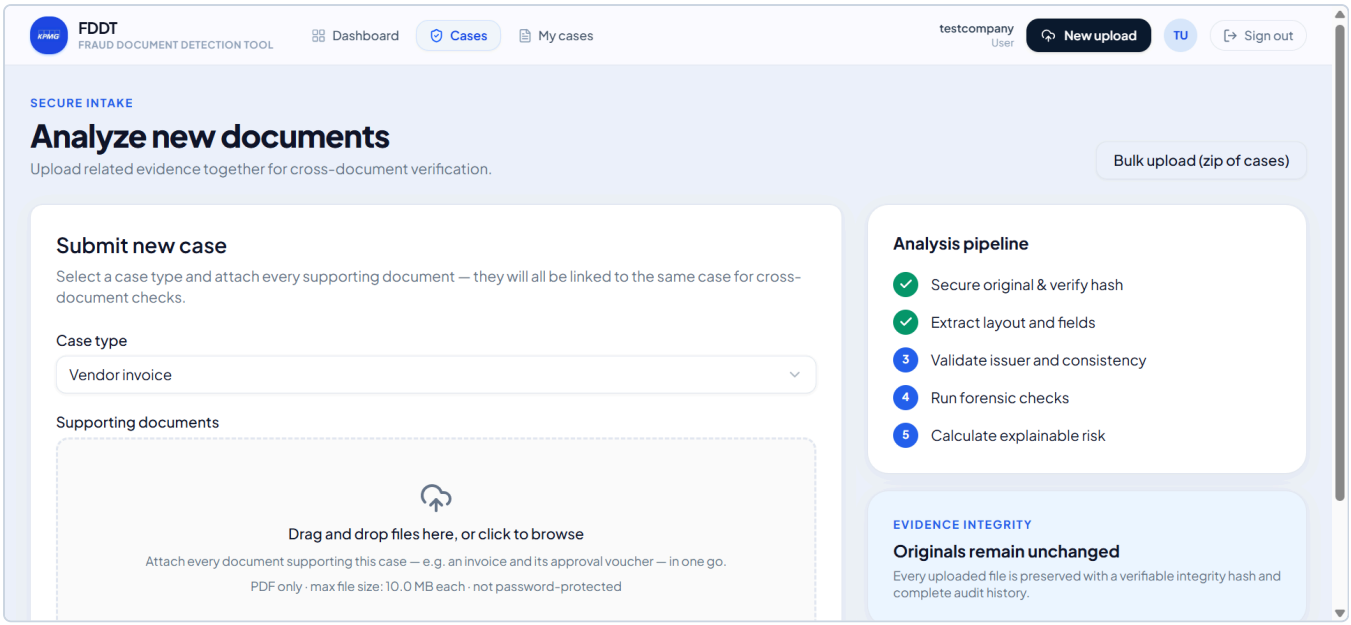
---

## 4. User (Submitter) Guide

### Submitting a Single Case

To submit documents for verification:

1. Click **New Case** (or **New upload**) in the top navigation bar.
2. Select the **Case Type** that matches your submission:
   - *School / educational document*
   - *Vendor invoice*
   - *Commercial invoice*
   - *Procurement documentation*
   - *Quotation*
   - *Travel / accommodation reimbursement*
   - *Other*



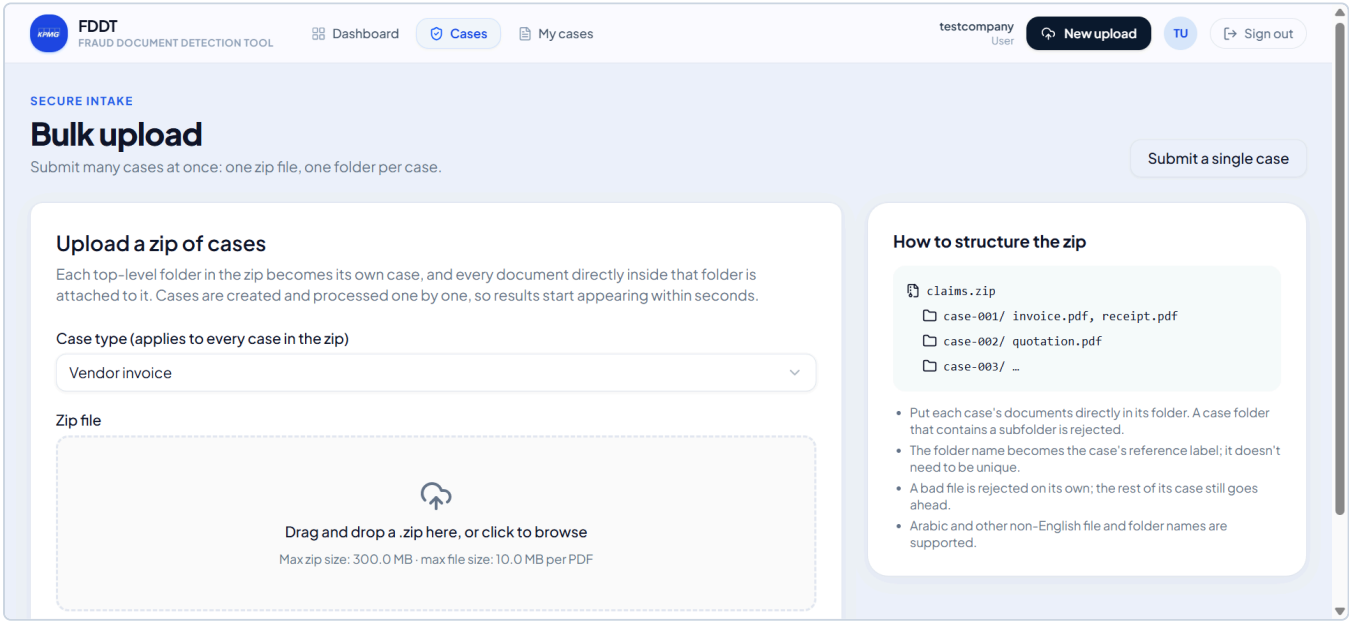

*New Case Intake Form*

3. **Attach PDF Files**:
   - Drag and drop your PDF documents into the upload zone or click **Browse files**.
   - **File Requirements**: Files must be valid PDF documents within your company's maximum file size limit (standard limit is 10 MB per document). Password-protected PDFs are rejected immediately for security.
4. **Optional Reference Signature**:
   - If your document contains an authorized signature or stamp that should be compared against other documents in the case, you can define a reference signature box.
5. Click **Submit Case**. The documents will begin automated processing immediately.

---

## Submitting Cases in Bulk (Zip Upload)

When managing high volumes of invoices or expense claims, use the **Bulk Upload** feature:




*Bulk Upload Interface*

1. Navigate to **New Case** and select **Bulk upload (zip of cases)**.

2. Prepare a standard `.zip` file on your computer structured with **one folder per case**:

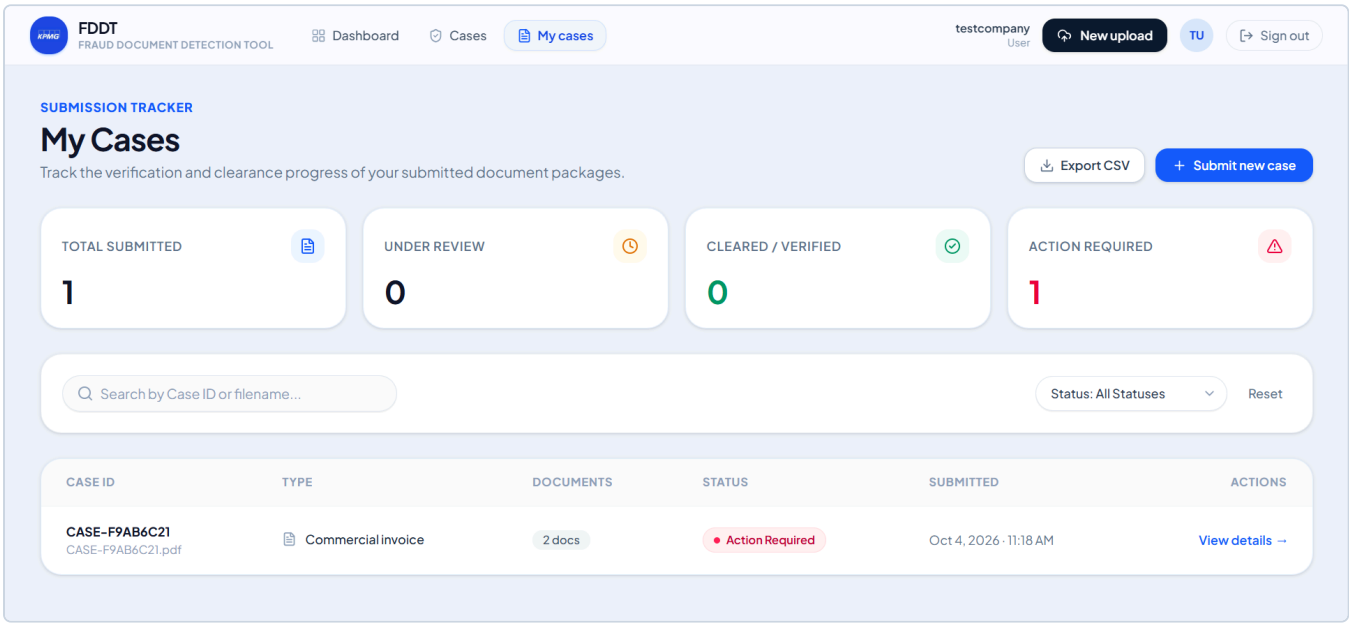
```
batch_october.zip
├── CASE_INV_1001/
│   ├── invoice_1001.pdf
│   └── purchase_order.pdf
├── CASE_INV_1002/
│   └── invoice_1002.pdf
└── CASE_INV_1003/
    ├── tuition_receipt.pdf
    └── student_id.pdf
```

3. Select the default **Case Type** for the batch.
4. Drop the `.zip` archive into the intake area.
5. The system automatically extracts each folder, verifies the PDF files, and creates individual cases with automated progress indicators (**Queued** → **Processing** → **Complete**).

---

## Tracking Submissions (My Cases)

Submitters can track all their submitted cases under **My Cases**:



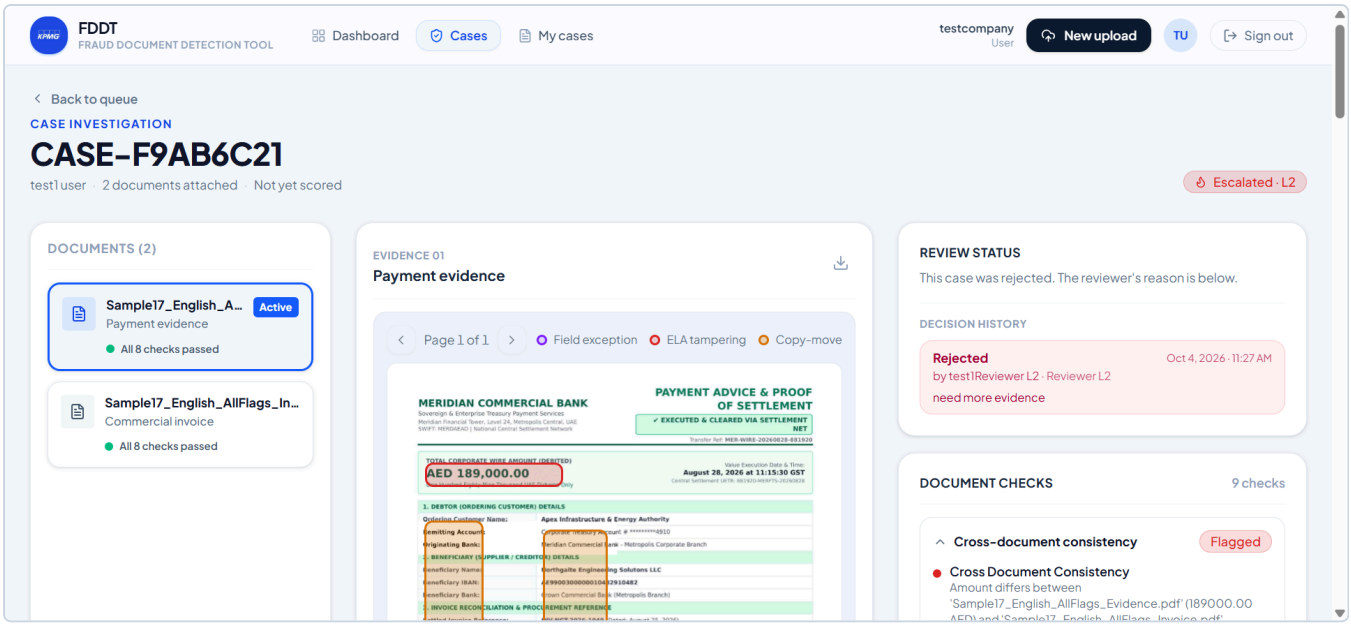

*My Cases Dashboard*

- **Search**: Search by case reference number or document name.
- **Filters**:
  - **All**: All cases submitted by your user account.
  - **Under review**: Cases currently being analyzed by the automated pipeline or queued for reviewer triage.
  - **Cleared**: Cases approved by the reviewing team.
  - **Action required**: Cases rejected with feedback notes from the reviewer.

---

## Understanding the Submitter Case View

When a submitter opens one of their cases, a clean, submitter-tailored view is presented:




*Submitter Case Detail View*

- **Document Overview**: Lists all uploaded files, page counts, file sizes, and processing statuses.
- **Extracted Information**: Displays values recognized by the OCR engine (such as invoice number, total amount, currency, and date).
- **Activity Timeline**: Displays a chronological event log showing when the case was submitted, when files were processed, and final decisions.

> **Security & Fraud Prevention Design**: Submitter accounts do **not** display numerical risk scores, triggered fraud check rules, or forensic detection algorithms. This prevents malicious actors from probing or attempting to bypass verification thresholds.

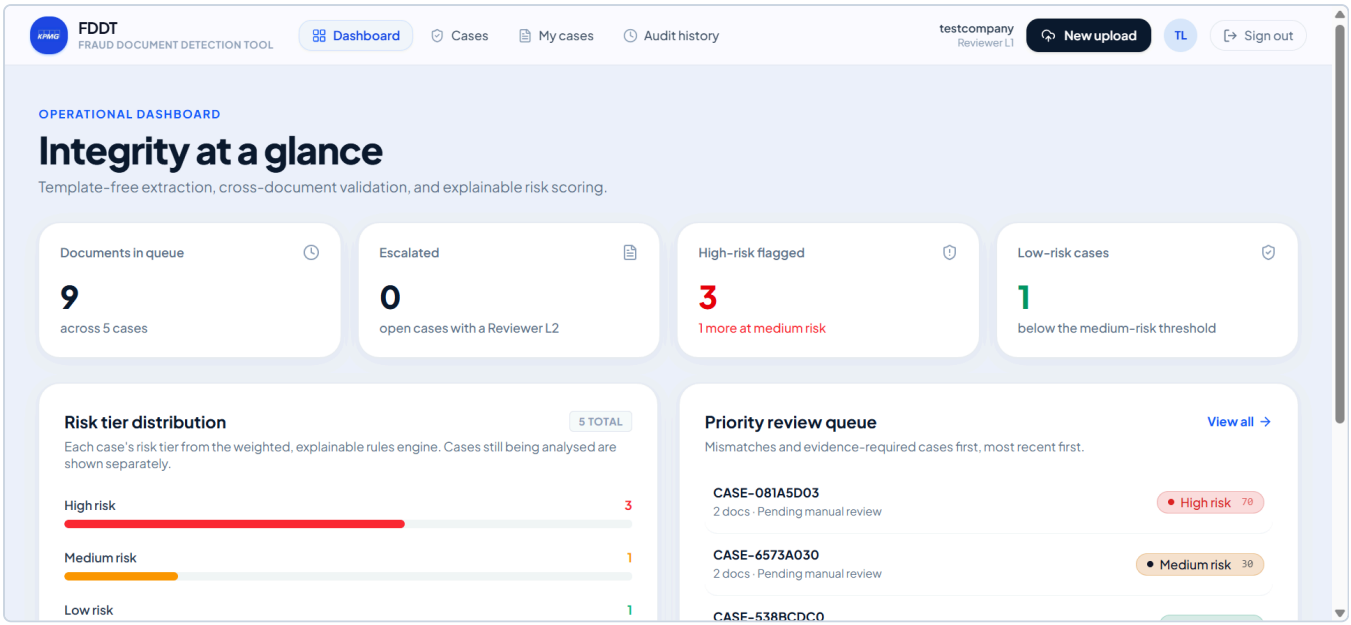
---

## 5. Reviewer L1 (Analyst) Guide

Reviewer L1 analysts assess automated findings, inspect flagged anomalies, and make informed operational decisions.

### Operational Reviewer Dashboard

The **Dashboard** provides real-time visibility into your company's verification queue:



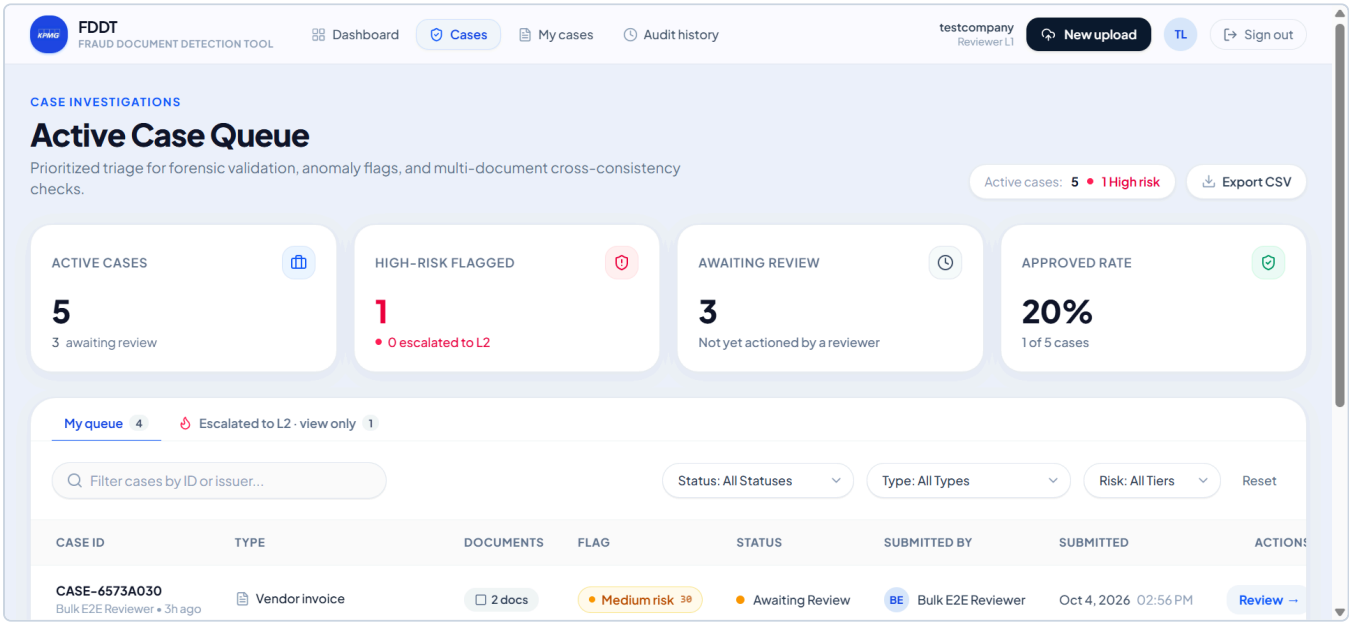

*Reviewer Operational Dashboard*

- **Key Metrics**: Total active cases, cases requiring immediate review, cleared cases, and flagged cases.
- **Risk Distribution**: High, Medium, and Low risk breakdowns across recent submissions.
- **Review Workload**: Cases sorted by submission date and SLA urgency.

---

## Managing the Review Queue

Click **Cases** in the navigation bar to access the full **Review Queue**:




*Review Queue*

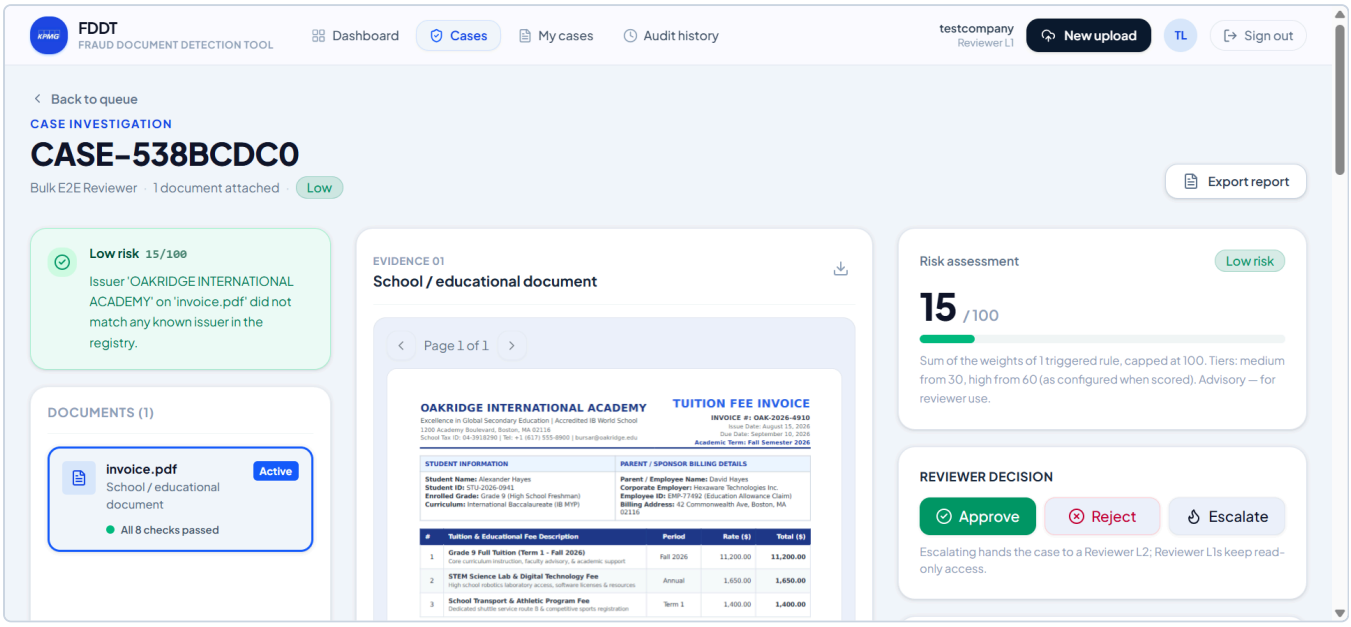
- **Risk Tier Badges**:
  - **HIGH RISK** (Score 70–100): Critical anomalies detected (e.g., severe metadata tampering, copy-move duplication, unrecognized issuer).
  - **MEDIUM RISK** (Score 30–69): Discrepancies or warnings (e.g., date mismatches, mathematical errors, missing expected fields).
  - **LOW RISK** (Score 0–29): Clear document with verified issuer and consistent fields.

- **Filters & Search**: Filter by Risk Tier, Case Type, Review Status, or keyword.

---

## Full Case Analysis & Risk Breakdown

Opening any case from the queue reveals the complete **Case Detail** analysis:



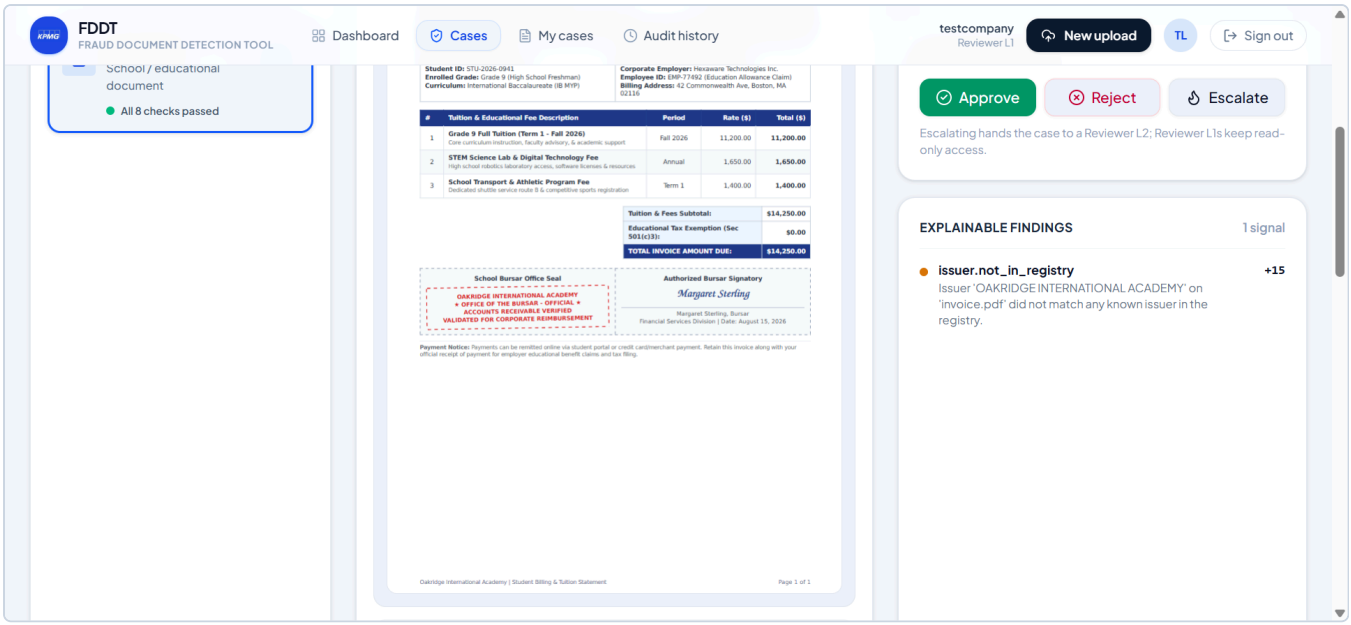

*Reviewer Case Overview*

1. **Risk Score Header**:
   - Prominently displays the composite Risk Score (0–100) and assigned Tier.
   - Shows case metadata: submitter email, case category, submission timestamp.
2. **Triggered Risk Reasons**:
   - Clear, explainable list of factors that contributed points to the score.
   - Example: *"Issuer not found in verified registry (+25 pts)"*, *"PDF modified in image editor after creation (+20 pts)"*.

---

## Forensic & Automated Document Checks

Scroll down on the Case Detail page to inspect the comprehensive **Checks & Forensics Panel**:




*Automated Checks & Forensics Panel*

Every uploaded document undergoes an exhaustive battery of independent tests:

| Check Name | What It Inspects |
|---|---|
| **Issuer Verification** | Cross-references extracted vendor name, tax ID, and address against your company's trusted Issuer Registry. |
| **Field Validation** | Verifies arithmetic totals (subtotal + tax = total), valid date formats, future dates, and required fields. |
| **Cross-Document Check** | Validates consistency between multiple documents in the same case (e.g., matching invoice amount with purchase order). |
| **Metadata Forensics** | Inspects PDF creation software, modification history, suspicious editing tools, and timestamp inconsistencies. |
| **Copy-Move Forgery** | Detects cloned regions (content copied and pasted from one part of the document to another). |
| **Error Level Analysis (ELA)** | Analyzes compression resave artifacts to identify spliced text or altered numbers. |
| **Duplicate Detection** | Computes cryptographic and perceptual hashes to detect duplicate submissions across historical cases. |
| **Signature / Stamp Detection** | Detects signature regions and compares them against reference samples for consistency. |

## Interactive PDF Visual Overlays

To visually inspect anomalies:

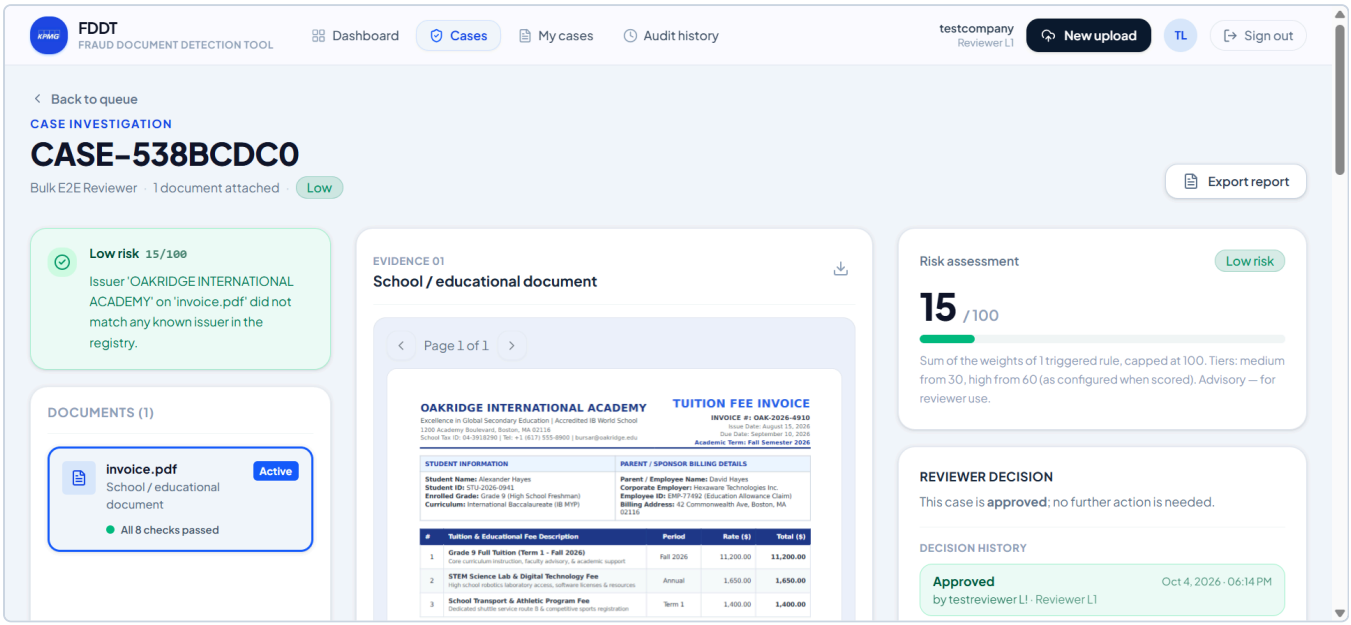
- Open the document preview within the Case Detail view.
- Color-coded bounding boxes highlight exact regions of concern:
  - **Red Box**: Tampering or Error Level Analysis anomaly.
  - **Orange Box**: Copy-move duplication source and target.
  - **Purple Box**: Arithmetic or field validation mismatch.
  - **Fuchsia Box**: Text set in a different font (or a second copy of the font) — likely typed in later.
  - **Teal Box**: Deleted or shortened text — the faint trace of erased text in a converted scan.

- **Blue Box**: Detected signature or official stamp.

---

## Review Decisions (Approve, Reject, Escalate)

In the top-right header of the Case Detail screen, three decision actions are available:




*Reviewer Decision Modal*

1. **Approve**:
   - Use when all checks pass or minor flags have been verified as legitimate.
   - Enter optional approval notes and confirm. The case status updates to **Approved**.
2. **Reject**:
   - Use when fraudulent alteration or non-compliance is identified.
   - Select a mandatory **Rejection Reason** category and provide notes detailing the decision.
3. **Escalate**:
   - Use when a case is complex, high-value, or requires supervisor adjudication.
   - Adds an escalation note and routes the case directly to the **Reviewer L2** queue.

---

## Exporting Case Forensic Reports

Click **Export report** at any time on the Case Detail page to generate an official, courtroom-ready **PDF Forensic Report**:

- Includes complete executive summary, breakdown of all automated checks, risk scoring explanation, audit trail, SHA-256 cryptographic document hashes, and high-resolution visual evidence pages with highlighted anomaly boxes.

---

# 6. Reviewer L2 (Senior Reviewer / Supervisor) Guide

Reviewer L2 users have all analyst privileges plus senior decision authority and configuration control over their company's risk parameters.

## Managing Escalated Cases

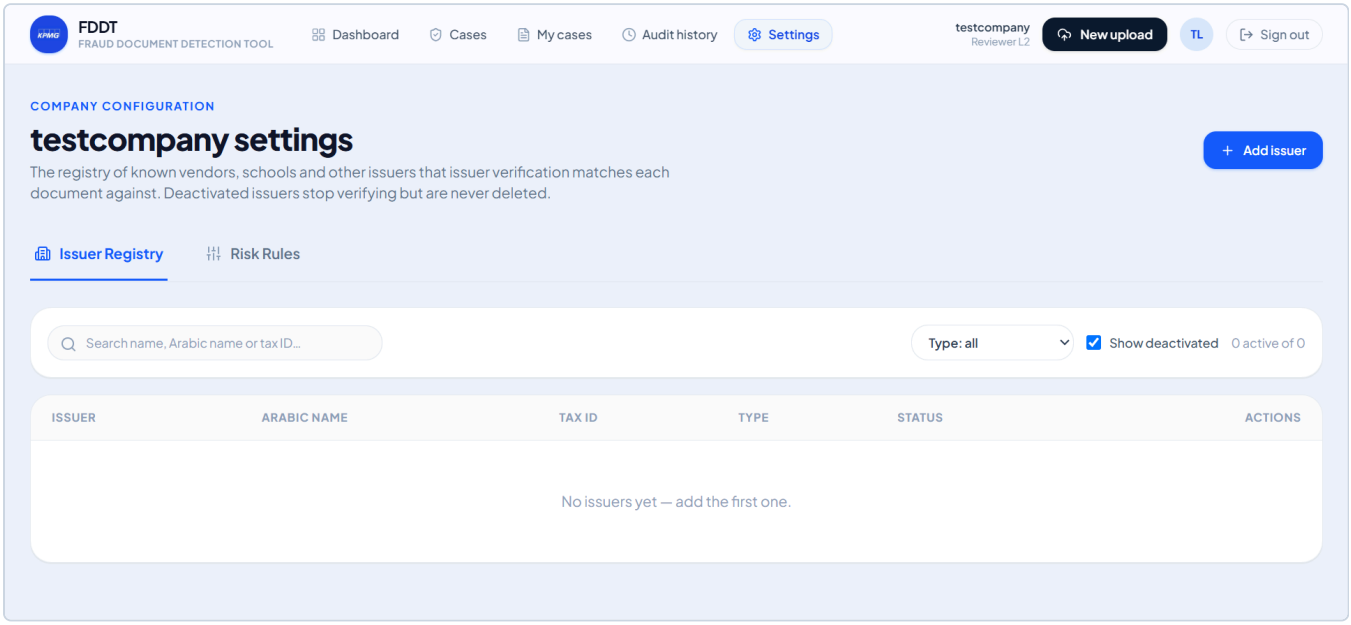
Cases escalated by L1 analysts appear in the review queue with an **Escalated** status badge.

- Reviewer L2 users conduct deep reviews of these cases.
- Only Reviewer L2 users have the authority to grant final **Approval** or execute final **Rejection** on escalated files.

---

## Company Issuer Registry

Under **Settings › Issuer Registry**, Reviewer L2 supervisors manage the company's verified vendors, schools, and partners:



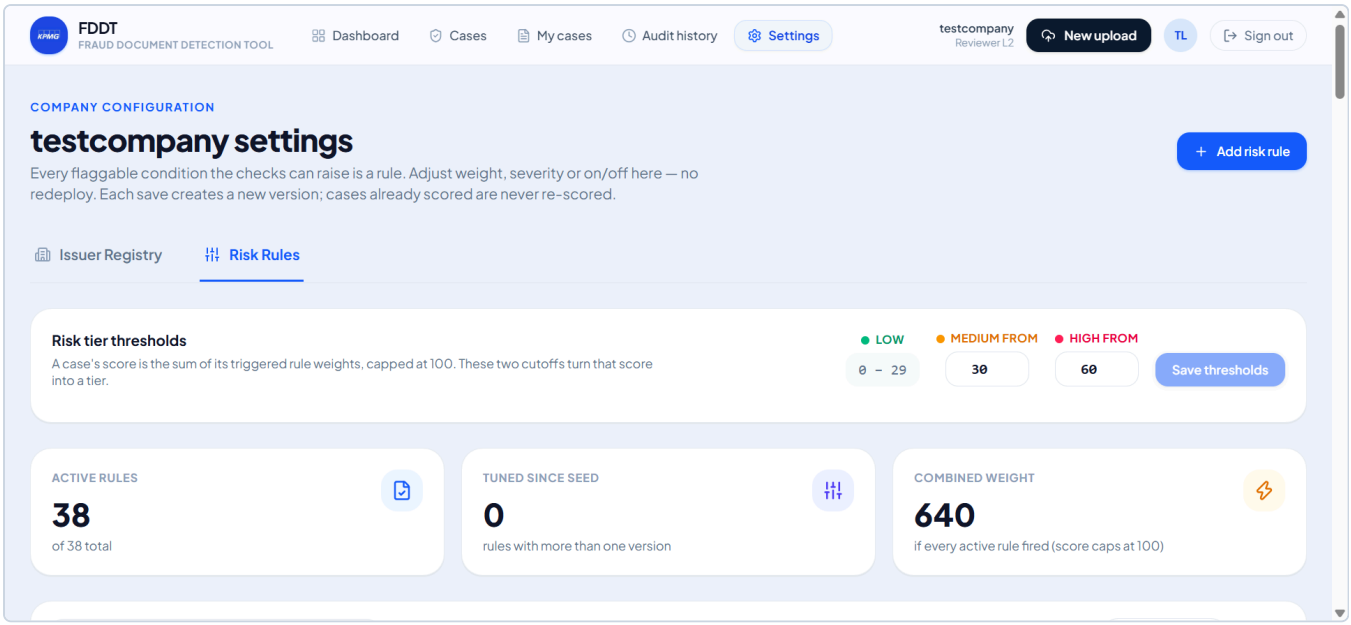

*Issuer Registry Management*

1. **Viewing Registry**: Lists all active and inactive registered issuers.
2. **Adding an Issuer**:
   - Click **Add Issuer**.
   - Enter Organization Name, Tax ID / Business Registration Number, official address, and known email domains.
   - Save the record. Incoming documents matching these details will pass Issuer Verification automatically.
3. **Deactivating / Reactivating**:
   - Deactivate obsolete vendors or reactivate previously deactivated entities with a single click.

---

## Company Risk Rules & Thresholds

Under **Settings › Risk Rules**, Reviewer L2 supervisors customize how risk scores are calculated for their organization:




*Company Risk Rules Management*

1. **Adjusting Risk Rules**:
   - Toggle individual fraud detection checks on or off based on your business requirements.
   - Adjust point weights for specific findings (e.g., increasing penalty points for metadata anomalies).
2. **Configuring Risk Thresholds**:
   - Customize score boundaries for **Low**, **Medium**, and **High** risk classifications to match your company's risk tolerance.

---

## 7. Getting Help & Administrative Requests

To maintain company security and data integrity, certain platform-level configurations are managed centrally.

> **Need assistance?**
>
> - To create new user accounts or adjust user roles
> - To reset an account password
> - To request an increase in maximum document upload size limits
> - To update your company profile or contact details
>
> **Please contact or email your administrator at `admin@example.com` (or your company's designated IT administrator) for this.**

# FDDT Platform Administrator Manual

## Fraud Document Detection Tool — Comprehensive System Administration & Governance Guide

This manual provides an exhaustive operational reference for **Platform Administrators** operating the **Fraud Document Detection Tool (FDDT)**. Platform Administrators manage multi-tenant companies, user accounts, system-wide risk rule templates, processing worker queues, resource usage, and audit compliance across the entire platform.

---

## Table of Contents


1. Platform Architecture & Security Posture
2. Platform Administrator Role & Scope
3. Company & Tenant Management
   - Viewing Companies & Statuses
   - Creating a New Client Company
   - Configuring Company Upload & Storage Limits
   - Suspending & Reactivating Companies
4. User Account & Identity Management
   - Comprehensive Role Breakdown (All 4 User Types)
   - User Directory & Filtering
   - Creating New User Accounts
   - Modifying Roles & Company Assignments
   - Account Deactivation & Lockout
   - Platform Admin Password Reset System
5. Global Risk Rule Templates
6. Worker Queues & Processing Engine Monitor
   - The Three Specialized Queues
   - Fair-Share Scheduling Algorithm
   - Queue Health & Alert Thresholds
7. System Usage & Resource Analytics
8. System Audit History & Governance
   - Immutable Audit Trail
   - Audited Platform Support Access
   - Event Types & Audited Operations


---

## 1. Platform Architecture & Security Posture

The **Fraud Document Detection Tool (FDDT)** is engineered as a multi-tenant enterprise system. Several client organizations operate concurrently on a single deployment while maintaining complete isolation:

- **Two-Layer Tenant Isolation**:
  1. *Application-Layer Filtering*: Every database query executed in tenant context automatically filters on `company_id`.

2. *PostgreSQL Row-Level Security (RLS)*: Database policies strictly enforce that connections under the `fddt_app` role cannot view or modify data outside the tenant session's `app.current_company_id`.

- **Platform Separation**: Platform administrators belong to **no company** (`company_id IS NULL`). They operate from an administrative plane (`fddt_platform`), granting global administrative privileges while ensuring tenant separation remains impenetrable.

---

## 2. Platform Administrator Role & Scope

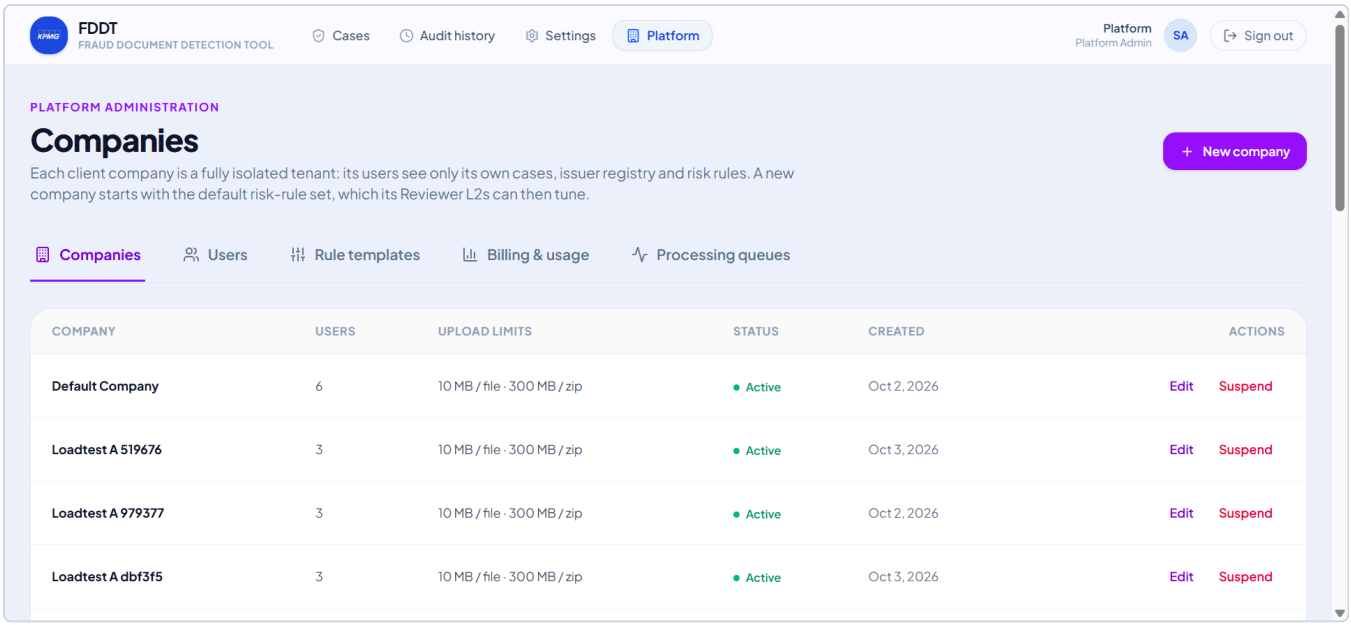
Platform Administrators hold system-wide authority over the platform infrastructure and user access:

| Capability | Platform Admin Authority | Tenant Users Authority |
|---|---|---|
| **Tenant Provisioning** | **Full**: Create, configure, suspend, and reactivate client companies. | None. |
| **User Administration** | **Full**: Create, update, deactivate, and reset passwords for any account across all roles. | None. |
| **Risk Rule Templates** | **Full**: Maintain master baseline rule templates applied to new companies. | Can only tune company-specific rules (Reviewer L2). |
| **Queue & Worker Monitoring** | **Full**: Real-time visibility into extraction, vision, and forensics Celery queues. | None. |
| **Resource & Billing Analytics** | **Full**: Cross-tenant tracking of pages processed, AI tokens, and storage quotas. | None. |
| **Support Access to Cases** | **Audited Read-Only**: Can inspect cases across companies for technical support; all views are logged. | Confined strictly to company's own cases. |
| **Case Adjudication** | **Deliberately Restricted**: Cannot approve, reject, escalate, or upload cases. | Performed exclusively by company reviewers. |

---

## 3. Company & Tenant Management

Navigate to **Platform › Companies** (`/platform/companies`) to administer tenant organizations.




*Platform Companies Management*

## Viewing Companies & Statuses

The Companies dashboard displays all onboarded client organizations:

- **Company Name & Identifier**: Unique business name and system UUID.
- **Status**:
  - `Active` (Green): Normal operations enabled.
  - `Suspended` (Red): All company users are immediately blocked from logging in or uploading.
- **Created Date**: Timestamp of tenant onboarding.
- **Active Limits**: Configured maximum file size and zip upload constraints.

## Creating a New Client Company

1. Click **+ Add Company** in the top-right corner.
2. In the modal, provide:
   - **Company Name**: Official legal or operational entity name.
   - **Max File Size (MB)**: Maximum size per uploaded PDF (defaults to 10 MB).
   - **Max Zip Size (MB)**: Maximum batch zip upload size (defaults to 300 MB).
3. Click **Create Company**.
   - The platform creates the tenant record, initializes company storage containers, and automatically seeds the default baseline Risk Rules from the master template.

## Configuring Company Upload & Storage Limits

To customize file intake thresholds for high-volume or enterprise tenants:

1. Click the **Edit** action button next to the target company.
2. Adjust `max_file_size_mb` or `max_zip_size_mb`.
3. Save changes. The updated limits take effect immediately without requiring worker or server restarts.

## Suspending & Reactivating Companies

If a client contract expires or a security breach is suspected:

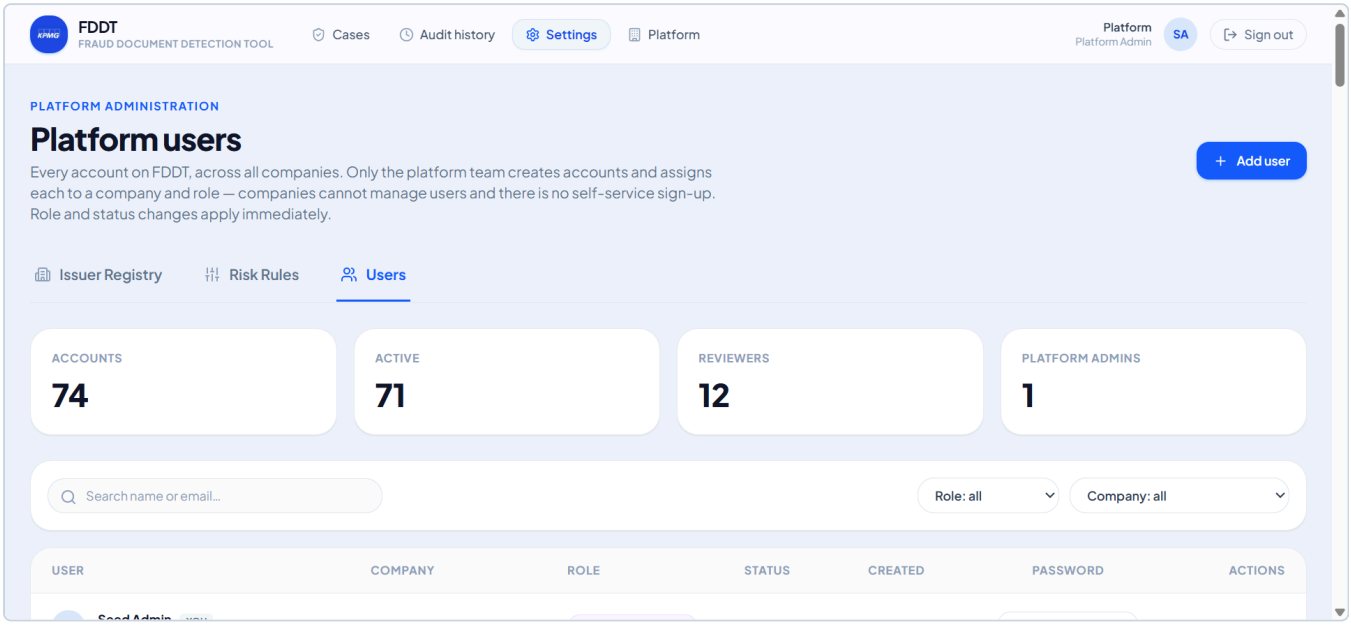
- Click **Suspend**.

- The change is instantaneous: active JWT sessions for that company become invalid on their next API request, and all user logins are blocked.
- Click **Reactivate** at any time to restore full platform access.

---

## 4. User Account & Identity Management

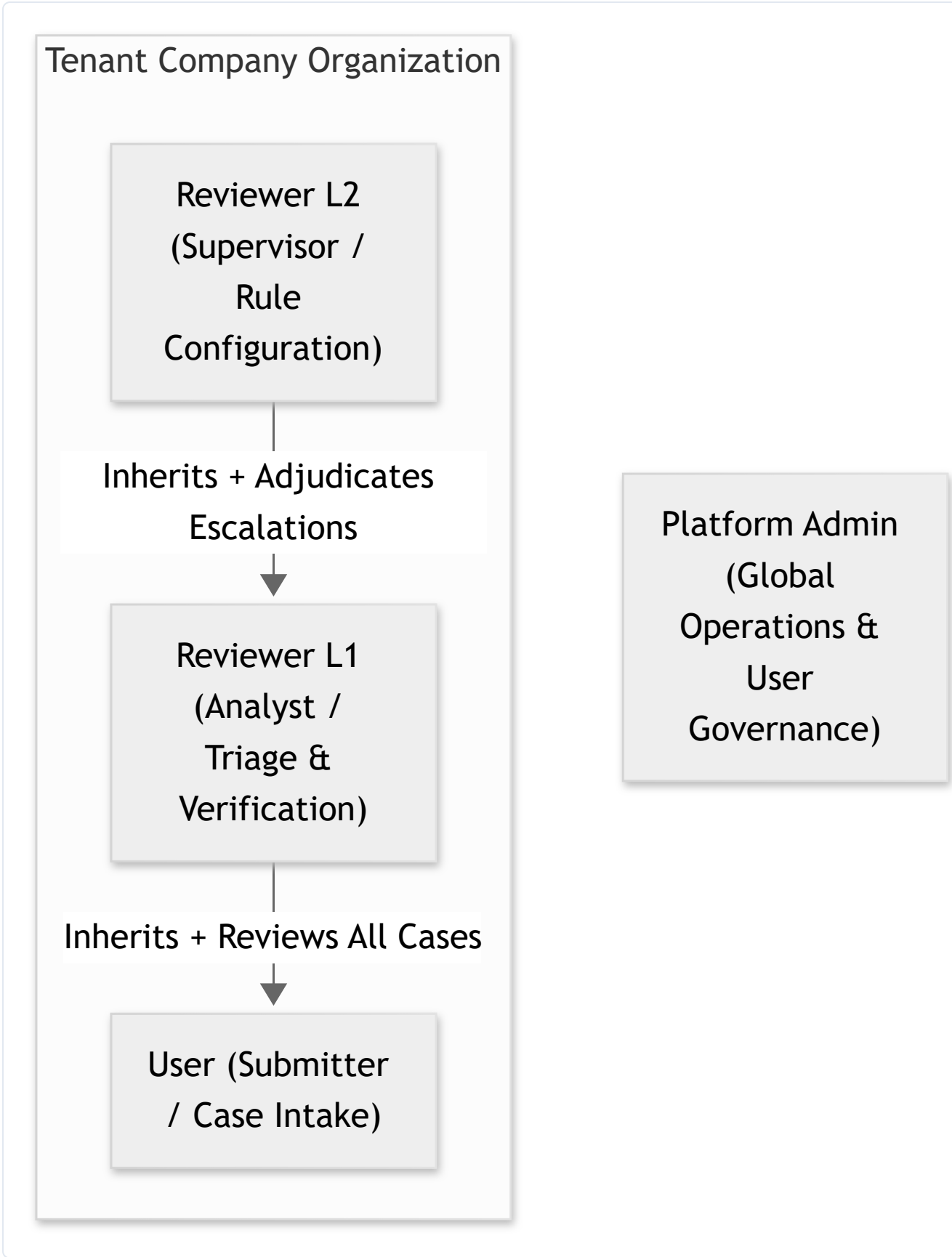
Navigate to **Platform › Users** ( `/settings/users` ) to view and manage user credentials and permissions across all organizations.




*Platform User Management*

### Comprehensive Role Breakdown (All 4 User Types)

FDDT defines four distinct roles across the platform:




1. `user` **(Submitter)**:
   - **Belongs to**: A specific company ( `company_id NOT NULL` ).
   - **Permissions**: Submits new cases (single or bulk zip), tracks personal submissions under *My Cases*, and inspects submitted documents.
   - **Security Restriction**: Never sees risk scores (0–100), risk tiers, or triggered fraud detection rules.

2. `reviewer_l1` **(Analyst)**:
   - **Belongs to**: A specific company ( `company_id NOT NULL` ).
   - **Permissions**: Full access to the company review queue, operational dashboard, risk scoring breakdown, automated check details, visual overlays, and decision actions (Approve, Reject, Escalate to L2). Can export forensic PDF audit reports.
3. `reviewer_l2` **(Senior Reviewer / Company Supervisor)**:
   - **Belongs to**: A specific company ( `company_id NOT NULL` ).
   - **Permissions**: All Reviewer L1 privileges, plus final decision authority on cases escalated to L2. Manages the company's **Issuer Registry** (trusted vendors) and **Risk Rules** (tuning weights and tier threshold boundaries).
4. `platform_admin` **(System Administrator)**:
   - **Belongs to**: Outside all companies ( `company_id IS NULL` ).
   - **Permissions**: System-wide governance, company provisioning, user management, password resets, queue monitoring, usage tracking, and system audit history.
   - **Deliberate Boundary**: Cannot act as a reviewer (cannot approve, reject, or upload cases) to maintain strict separation of duties.

---

## User Directory & Filtering

The user table lists all accounts across the platform:

- **Email**: User login address.
- **Company**: Associated client company (or *Platform* for administrators).
- **Role**: Current assigned role badge.
- **Status**: Active (Green) or Inactive (Gray).
- **Password**: Dedicated reset action column.

---

## Creating New User Accounts

1. Click **+ Add User** on the Users management screen.
2. Complete the form:
   - **Email**: Valid corporate email.
   - **Password**: Temporary initial password (minimum 8 characters).
   - **Role**: Select `user` , `reviewer_l1` , `reviewer_l2` , or `platform_admin` .
   - **Company**: Select the client company (required for company roles; omitted for Platform Admins).
3. Click **Create User**. The account is created immediately and is ready for login.

---

## Modifying Roles & Company Assignments

- You can change an existing user's role (e.g., promoting a `reviewer_l1` to `reviewer_l2` ) directly via the dropdown in the user table.
- Role and company changes take effect **immediately** on the user's next API call (tokens verify the database record in real time).

---

## Account Deactivation & Lockout

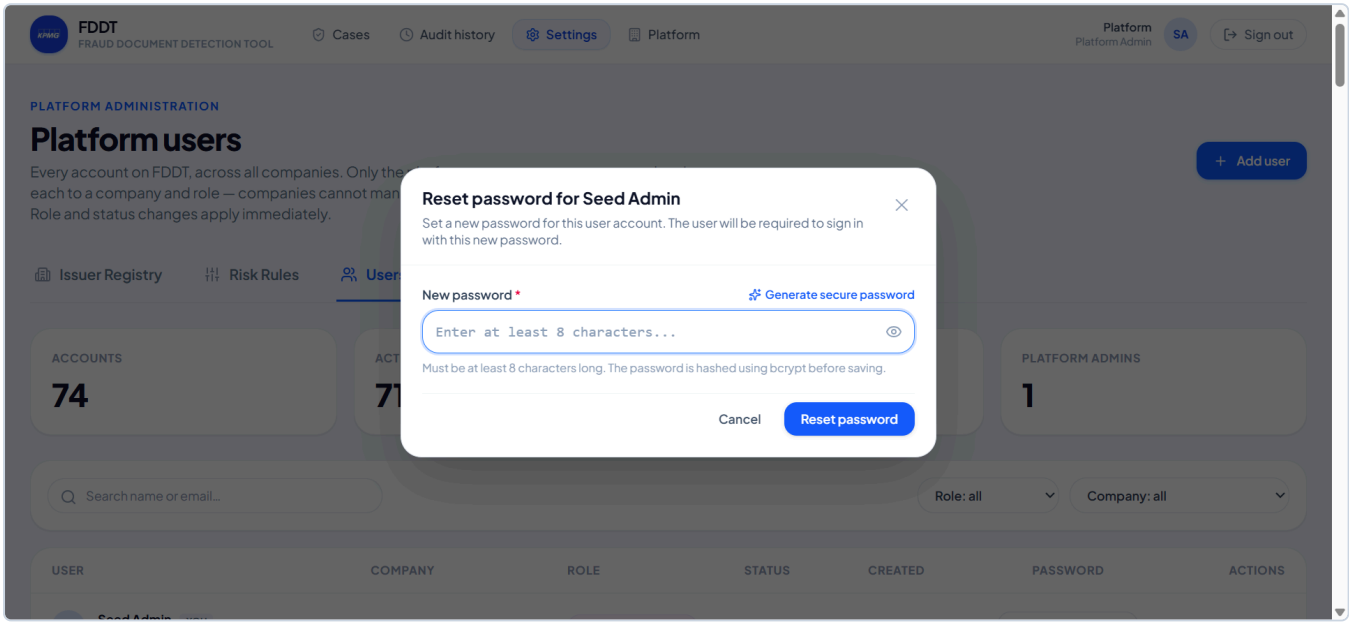
- Toggle the **Active** switch on any user row to deactivate the account.

- Deactivation instantly revokes access: existing bearer tokens are rejected with `401 Unauthorized`, and future sign-in attempts fail until reactivated.

---

### Platform Admin Password Reset System

Only Platform Administrators have the authority to reset account passwords. Users who still know their password change it themselves (key icon in the top bar → current password + new password, `POST /auth/me/password`, audited as `user_password_changed`); a reset is only needed when a password is forgotten.




*Platform Admin Password Reset Modal*

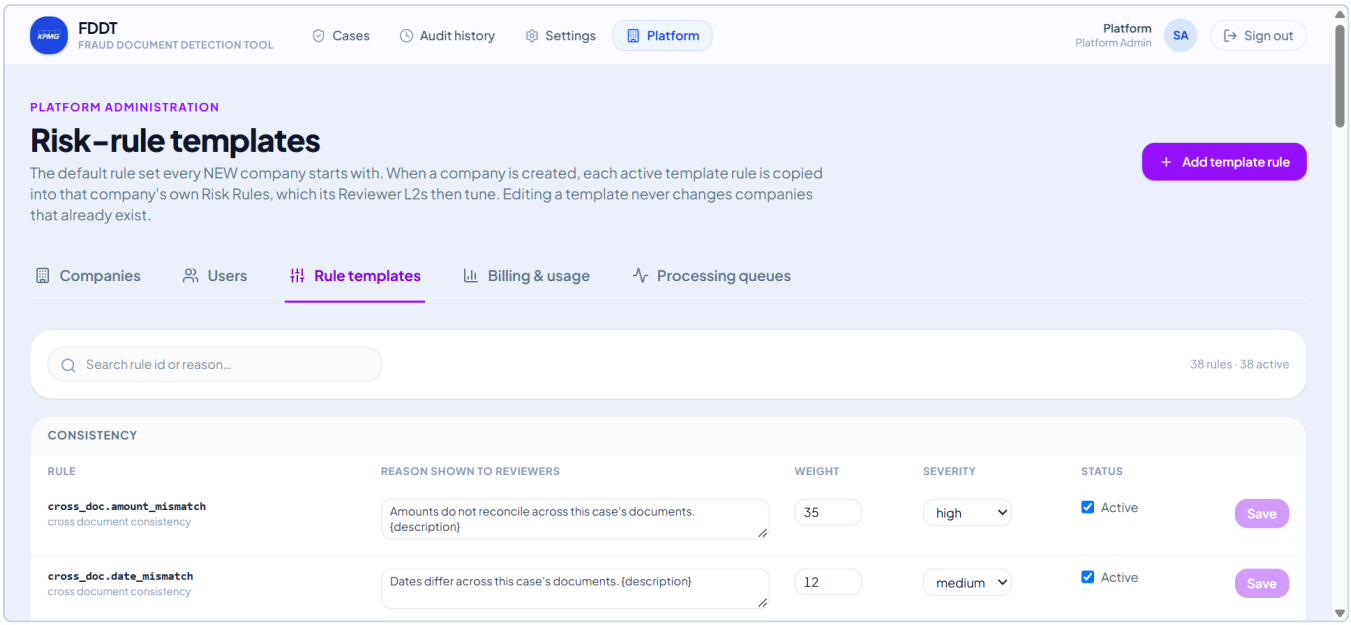
#### Step-by-Step Password Reset Workflow:

1. Locate the target user in the Users table.
2. In the **Password** column, click the `[ 🔑 Reset password ]` pill button.
3. The **Reset Password Modal** opens, displaying the target user's email and role.
4. **Choose a Password Method**:
   - *Manual Entry*: Type a new secure password (minimum 8 characters, maximum 128 characters). Click the **Eye Icon** to show or hide the password.
   - *One-Click Generator*: Click **Generate secure password**. The system uses a cryptographically secure random value generator to create a high-entropy password (e.g., `k9$Qm2#pL8!vR4wZ`).
5. **Copy to Clipboard**: Click **Copy** to copy the generated credentials to your clipboard with immediate visual confirmation ("Copied!").
6. Click **Reset Password**.
7. **Security & Audit Actions**:
   - The password is encrypted with bcrypt (`hash_password`) before being saved to the database.
   - The action is recorded in the permanent audit trail under the `user_password_reset` event.
   - Provide the new credentials to the user through your organization's approved secure communication channel.

---

## 5. Global Risk Rule Templates

Navigate to **Platform › Rule Templates** (`/platform/rule-templates`) to manage baseline risk scoring parameters.



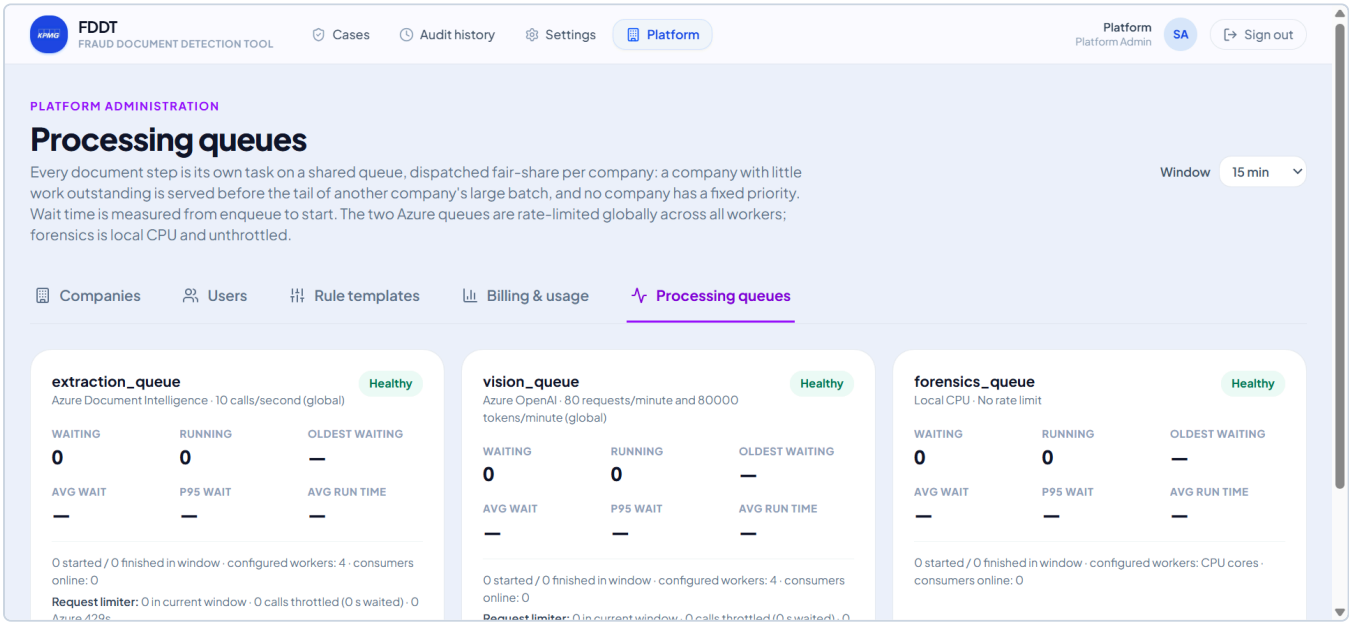

*Global Risk Rule Templates*

- **Purpose**: Defines the master library of fraud checks, default severities, and baseline weights.
- When a new company tenant is provisioned, its company-specific risk rule set is automatically cloned from this template.
- Adjusting global rule templates standardizes detection policies across all future client onboardings.

---

## 6. Worker Queues & Processing Engine Monitor

Navigate to **Platform › Queues** ( `/platform/queues` ) to monitor the health and throughput of background processing workers.




*Platform Worker Queues Monitor*

### The Three Specialized Queues

Document processing is divided into three asynchronous Celery queues:

1. `extraction_queue` (Azure Document Intelligence):

- Handles optical character recognition (OCR), layout parsing, and tabular text extraction.
- Rate-limited and metered to prevent external service throttling.

2. `vision_queue` (Azure OpenAI Vision):
   - Executes multi-modal visual analysis, signature detection, stamp localization, and visual inconsistency checks.
   - Calibrated against strict token reservations.
3. `forensics_queue` (Local CPU Worker Pool):
   - Computes local algorithmic checks: Error Level Analysis (ELA), copy-move forgery detection, PDF metadata forensics, and cryptographic hash comparisons.
   - Operates fully on local CPU cores without external API dependencies.

---

### Fair-Share Scheduling Algorithm

FDDT implements a **Fair-Share Dispatching Mechanism** ( `app/tasks/fairshare.py` ):

- Rather than standard FIFO (First-In, First-Out) queuing which could allow a single high-volume client submitting thousands of documents to starve smaller clients, jobs are dispatched based on each company's outstanding task count.
- Companies with lower active backlog receive priority dispatching, ensuring balanced throughput across all tenants.

---

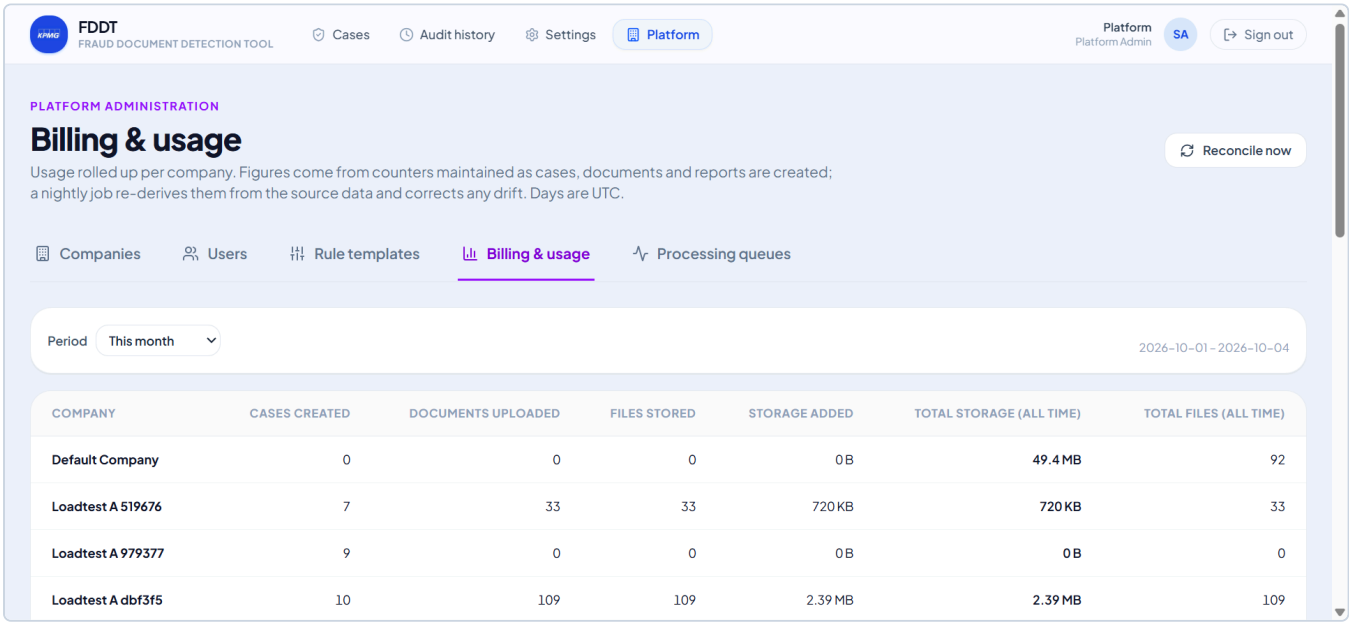
### Queue Health & Alert Thresholds

- **Active & Reserved Tasks**: Displays current worker workload.
- **Oldest Waiting Task Alert**: If the oldest task in any queue exceeds `QUEUE_ALERT_OLDEST_WAITING_SECONDS` (default: 300 seconds), a visual warning badge and system log alert are triggered, signaling the need to scale worker replicas.

---

## 7. System Usage & Resource Analytics

Navigate to **Platform › Usage** ( `/platform/usage` ) to review resource consumption and tenant billing metrics.




*Platform Usage & Analytics*

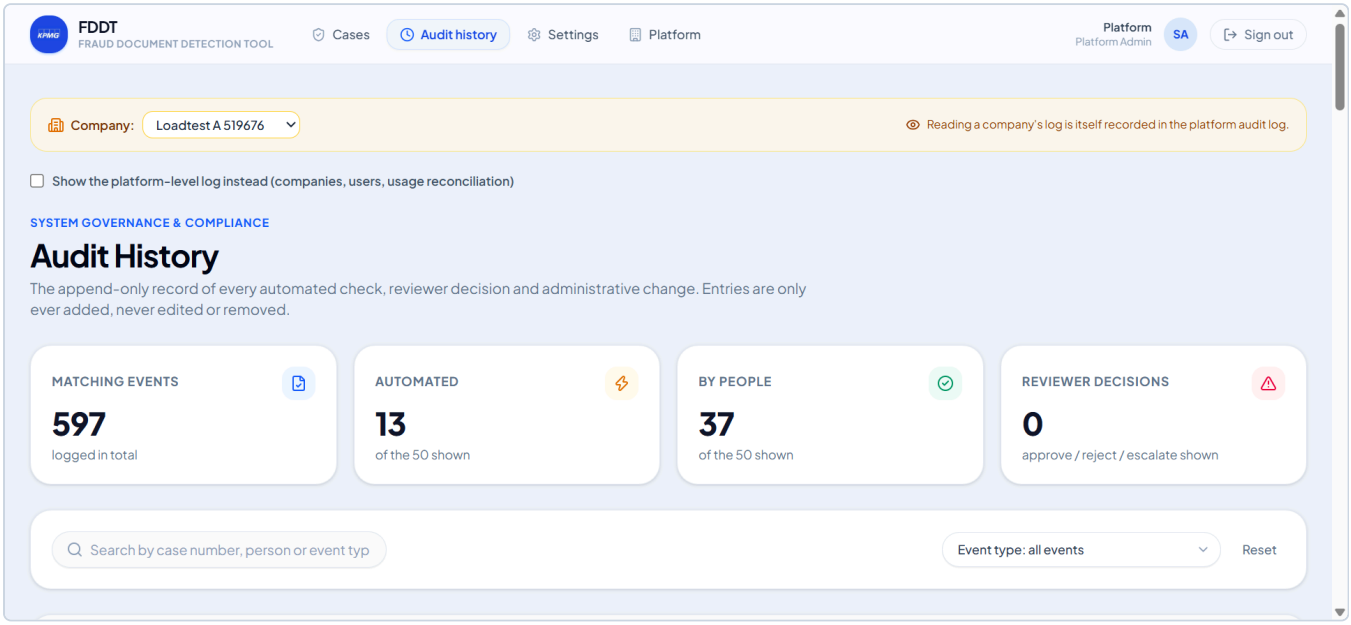
- **Total Documents Processed**: Cumulative PDF intake across all tenants.

- **OCR Pages Extracted**: Exact page count submitted to Document Intelligence.
- **AI Vision Token Consumption**: Aggregated prompt and completion tokens consumed per tenant.
- **Storage Utilization**: Storage footprint per company in Azure Blob Storage.
- **Tenant Billing Reconciliation**: Exportable monthly usage metrics for customer invoicing.

---

## 8. System Audit History & Governance

Navigate to **Audit History** ( `/audit-history` ) to inspect the centralized governance log.




*Platform System Audit History*

### Immutable Audit Trail

FDDT maintains an **append-only** audit log in the database. Audit rows cannot be modified or deleted by any user or administrator.

### Audited Platform Support Access

When a Platform Administrator inspects a tenant company's cases or documents for technical support, the system automatically records a `platform_support_view` event. This guarantees complete transparency and regulatory compliance for enterprise clients.

### Event Types & Audited Operations

| Event Type | Category | Description |
|---|---|---|
| `user_password_reset` | Security & Admin | Platform admin executed a credential reset for a user account. |
| `user_password_changed` | Security & Admin | A user changed their own password (with their current password). |
| `user_created` / `user_updated` | User Admin | New user account created or role/status modified. |
| `company_created` / `company_updated` | Tenant Admin | New company provisioned or limits adjusted. |
| `case_created` / `case_submitted` | Case Lifecycle | Submitter uploaded a new case and supporting documents. |

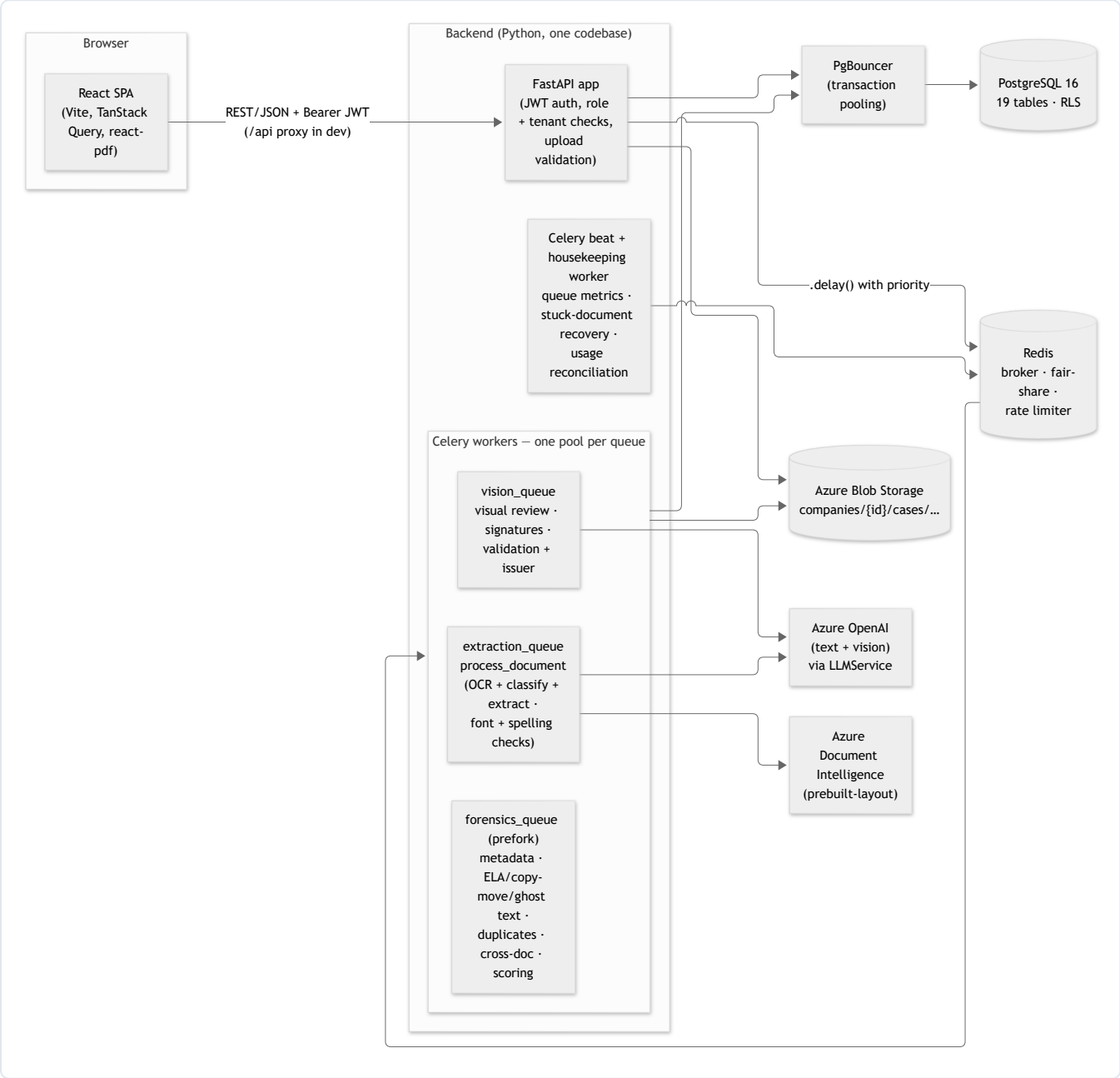
| Event Type | Category | Description |
|---|---|---|
| `case_approved` / `case_rejected` | Review Decision | Reviewer completed adjudication with notes and reasoning. |
| `case_escalated` | Review Decision | Reviewer L1 escalated a case to the L2 supervisor tier. |
| `document_processing_completed` | Pipeline | OCR extraction, classification, and field parsing succeeded. |
| `font_consistency_completed` / `font_consistency_failed` | Pipeline | Font consistency check finished (or failed — the extraction still completes). |
| `spelling_check_completed` / `spelling_check_failed` | Pipeline | Spelling check (English) finished (or failed — the extraction still completes). |
| `tampering_checks_completed` | Pipeline | ELA, copy-move detection and the ghost-content check completed. |
| `risk_assessment_completed` | Pipeline | Composite risk score calculated and tier assigned. |

# Architecture overview

FDDT (Fraud Document Detection Tool) authenticates third-party documents — vendor invoices, school documents, quotations, travel claims and the payment evidence that supports them — *before* a payment or approval is made. A user submits a **case** containing one or more PDF documents. The system extracts their data, runs a battery of automated checks, turns the results into an explainable risk score, and hands the case to a human reviewer (L1, escalating to L2 when needed) who approves, rejects or escalates it. Every step is written to an append-only audit log.

Several client **companies** share one deployment; each company's users see only that company's data, enforced twice — in the application and by PostgreSQL Row-Level Security (multi-tenancy.md).

## System diagram




Everything intelligent is a managed cloud API or a classical library. **No model is trained or hosted** (a design rule of the platform).

## Tech stack

| Layer | Technology | Used for |
|---|---|---|
| API | FastAPI 0.121, Pydantic 2, Uvicorn | REST endpoints, validation, OpenAPI |
| Auth | PyJWT (HS256), bcrypt | Email/password login, 8 h bearer tokens carrying `role`, `company_id`, `is_platform_admin` |
| Database | PostgreSQL 16, SQLAlchemy 2, Alembic, psycopg2 | Persistence, migrations (25), Row-Level Security per company |
| Connection pooling | PgBouncer (transaction mode) | Multiplexes API + worker connections onto ≤ 60 server connections |
| Queue | Celery 5.4 + Redis 7 | Three queues (extraction / vision / forensics), fair-share priorities, beat jobs |
| Rate limiting | Redis sliding windows ( `services/rate_limiter.py` , `services/login_throttle.py` ) | Global caps on Azure calls across every worker; sign-in throttling per email address and per IP |
| File store | Azure Blob Storage ( `azure-storage-blob` ) | Immutable originals, signature crops, report PDFs (company-prefixed paths) |
| OCR | Azure Document Intelligence `prebuilt-layout` | Text, tables, key-values and word geometry |
| LLM | Azure OpenAI (chat completions, strict JSON-schema output) | Classification, field extraction, issuer semantic match, spelling judgment, visual review, signature detect/compare |
| PDF forensics | pikepdf, PyMuPDF | Upload validation, metadata, xref/incremental-update analysis, page rendering, report PDF |
| Image forensics | OpenCV (headless), Pillow, NumPy | ELA, copy-move (BRISK), ghost text (traces of erased text in converted scans), signature crop/pixel comparison |
| Duplicates | imagehash (pHash) | Near-duplicate page detection (within a company) |
| Spelling | pyspellchecker (bundled word-frequency lists) | Dictionary pass of the English spelling check, before the LLM judgment |
| Fuzzy match | rapidfuzz | Issuer registry lookup, signer-name comparison |
| Workflow | `transitions` | Case state machine |
| Frontend | React 19, TypeScript 6, Vite 8, React Router 7 | SPA, route-level code splitting |
| UI kit | Tailwind CSS 4, Radix primitives, shadcn-style `ui/` components, lucide-react | Styling and base components |
| Data | TanStack Query 5 | Server state, polling |
| Forms | React Hook Form + Zod | Form state and validation |
| PDF view | react-pdf (pdf.js) | Live viewer with overlays, signature-region drawing |

## Key architectural decisions

### 1. Modular monolith

One FastAPI app and the Celery workers share one codebase ( `backend/app/` ), one database and one settings object. Modules are separated by responsibility — `api/` (HTTP), `services/` (logic, mostly pure functions), `tasks/` (Celery

wrappers that own DB sessions and audit events), `models/`, `schemas/`, `db/` (sessions and tenancy) — not by deployable unit. The same image runs as API, as each worker pool and as beat, differing only in the start command. The convention that makes this work: **services contain the algorithm and are testable without a database or network; tasks wrap them with I/O.** For example `services/forensics/ela.py` `run_ela_check(pages)` is pure; `tasks/tampering_checks_task.py` downloads the file, renders it, calls the pure function and stores the result.

## 2. Azure OpenAI behind an `LLMService` abstraction

Classification, field extraction, issuer semantic matching, visual review and signature analysis all use Azure OpenAI. `services/llm_service.py` defines the `LLMService` interface and one implementation, `AzureOpenAILLMService`. Callers depend only on the interface, so switching to another LLM provider means adding one implementation class, not rewriting call sites. The same pattern guards `StorageService` (Azure Blob) and `OCRService` (Document Intelligence). All LLM calls request **strict JSON-schema output** (`response_format: json_schema`, `strict: true`), so every response is validated by a Pydantic model (`DocumentAnalysis`, `PageVisualAnalysis`, `SignatureComparisonResult`, …). Vision calls require a vision-capable deployment (`gpt-4o`, `gpt-4.1`, `gpt-4.1-mini`, …) and raise `LLMConfigurationError` otherwise.

## 3. Three queues, fair-share, globally rate-limited

Uploads return immediately. The upload endpoint validates the file, commits the document row and enqueues **one task per document per step** — six of them. Each task goes to the queue of the service it depends on: `extraction_queue` (Document Intelligence), `vision_queue` (Azure OpenAI) or `forensics_queue` (local CPU, prefork). Each queue has its own worker pool, so they scale independently. Dispatch is **fair-share per company**: a task's priority comes from how much work its company already has outstanding, so one company's 100-document batch can't starve another company's single upload. Azure calls pass through a **Redis-backed global rate limiter** (requests and tokens per minute, calibrated against real usage) shared by every worker. Tasks commit before every slow step (no transaction is held across a download, an Azure call or CPU work), and check results are written **idempotently** (one row per document + check), so a redelivered task overwrites rather than duplicates. The browser polls `GET /cases/{id}` (every 5 s while the pipeline is incomplete). Details: processing-queues.md.

## 3b. Event-driven case workflow

State changes go through `services/workflow_service.py`, which drives a `transitions` state machine and **emits** `WorkflowEvent`s (`case_status_changed`, `case_approved`, `case_rejected`, `case_escalated`) to subscribers instead of calling side effects directly. The only subscriber today writes the audit log. A future external case-management integration would be one more `subscribe()` call plus the already-present nullable `cases.external_ref_id` column — no change to the state machine.

## 4. Immutable, validated originals

Only PDFs are accepted, and every upload is validated **before** it is stored: within the company's own size limit (per-company, set by a platform admin; 10 MB by default), not empty, a PDF by its header bytes, not corrupted, not password-protected (pipeline/01-upload-intake.md). A valid file is written to Blob Storage exactly once, at `companies/{company_id}/cases/{case_id}/documents/{document_id}_{sha256}{ext}`, with `overwrite=False`; `StorageService` exposes no update or delete. Its SHA-256 is stored on `documents.file_hash` so integrity can be re-verified later — the PDF report does exactly that and prints the result. Nothing downstream mutates the original: forensics render pages **in memory**; the signature crop and the report are *new* blobs (`…/signatures/…`, `…/reports/…`). The private container means the API never hands out the raw blob URL; it mints short-lived SAS URLs (15 minutes by default, 30 for OCR), and refuses to sign a path under another company's prefix.

## 5. One normalized bounding-box convention

Every located thing — ELA/copy-move regions, visual-review findings, extracted-field positions, signature/stamp regions, signature references — is stored as `{"page": 1-based int, "x", "y", "width", "height"}` where the four

numbers are **fractions (0–1) of the page's width/height, origin top-left**. It is resolution-independent, so the same box works on a 200-DPI server render, a browser canvas, and the report. The frontend draws boxes live over the real PDF ( `PdfOverlayViewer` ) and never saves an annotated image; only the exported report burns findings into *its own* rendered pages.

### 6. A transparent rules engine, not a model

Risk scoring is a weighted rules engine ( `services/risk_scoring_service.py` ). Each rule is a row in the company's `risk_rules` describing *what to match* in already-stored check results; there is no per-rule Python. Rules are **immutable and versioned** — editing inserts `version + 1` — and every scoring run ( `case_risk_assessments` ) freezes the rendered reason text, the exact rule versions and the thresholds it used, so retuning never changes a historical score. New companies start from the platform rule template ( `risk_rule_templates` ). See pipeline/11-risk-scoring-engine.md.

### 7. Template-free extraction

One generic pipeline handles every document type: Azure Layout OCR → a single Azure OpenAI call that both classifies the document and extracts core fields plus arbitrary additional fields as `{name, value}` pairs. No per-type extractors exist, so new document types need no engineering.

### 8. Two-layer tenant isolation

Every tenant-owned table carries `company_id` . The application binds each request's (and each task's) database session to one company and filters every ORM query automatically; independently, PostgreSQL Row-Level Security refuses any row of another company even to SQL with no filter at all. No database role has BYPASSRLS; the platform path sees all rows through an explicit `platform_all` policy. A cross-company id is a 404. See multi-tenancy.md.

### 9. Explainability over accuracy claims

Weak signals are labelled as such everywhere in code and UI: ELA is a "signal, not a verdict"; signature/stamp detection is presence-and-placement only; signature comparison uses advisory wording ("visually consistent with reference on file", never "verified"); the visual review's AI question is "experimental". Weights reflect that.

## Scope of the current build

What each capability area does today, and what is intentionally outside the current scope.

| # | Area | Implementation | Where |
|---|---|---|---|
| 1 | Classification & extraction | Azure OpenAI via `LLMService` , one combined call per document | `services/llm_service.py` |
| 2 | Signature/stamp detection | Azure OpenAI vision model locates signature and stamp regions (presence/placement only) | `services/signature_detection_service.py` |
| 3 | AI-generated-content assessment | One question inside the visual-review vision call; experimental, one signal among several. There is no separate content-detection service | `services/visual_inconsistency_service.py` |
| 4 | Metadata forensics | pikepdf + PyMuPDF read the PDF's own object model | `services/forensics/metadata_forensics.py` |

| # | Area | Implementation | Where |
|---|---|---|---|
| 5 | Accepted file types | **PDF only**, within the company's per-file limit (10 MB by default), validated at upload (type by header bytes, corruption, password). Every forensic check is PDF-based | `services/upload_validation.py` |
| 5a | Bulk upload | One zip (within the company's zip limit, 300 MB by default) with one folder per case. Ingested in the background case by case, each document validated like a single upload and queued as soon as its case exists. Partial success, live per-case summary | `services/bulk_upload_service.py` |
| 5b | Upload limits | Per company ( `companies.max_file_size_mb` / `max_zip_size_mb` ), changed only by a platform admin, audited old → new, effective on the next upload; one source for every upload path | `services/upload_limits.py` |
| 6 | Error Level Analysis | Runs only on pages containing an embedded raster image; born-digital text pages return `not_applicable` | `services/forensics/ela.py` |
| 7 | Case workflow | Statuses in use: `submitted` , `pending_manual_review` , `approved` , `rejected` . Every scored case goes to the reviewer queue; nothing auto-approves. Escalation moves the case to the L2 reviewer tier ( `assigned_tier` ), not a status | `services/workflow_service.py` |
| 8 | Low-confidence fields | The per-field `uncertain` flag is stored and displayed to the reviewer; it does not change routing, because every case reaches manual review | `services/llm_service.py` |
| 9 | Multi-tenancy | Companies, `company_id` everywhere, application filter + PostgreSQL RLS, platform admin with audited read-only support access | `db/tenancy.py` , `api/tenant_access.py` |
| 10 | Billing & usage | Per-company counters (cases, documents, files, storage) with nightly reconciliation; platform-admin dashboard | `services/usage_service.py` |
| 11 | Processing queues | Three queues, fair-share per company, global Azure rate limiter, queue monitor, JSON queue metrics + `queue_alert` log line | `tasks/celery_app.py` , `tasks/fairshare.py` |
| 12 | Notifications | Not part of the current scope (no notifications table, no email sending) | — |
| 13 | SLA tracking | Not part of the current scope (no deadline column, no overdue checks) | — |
| 14 | Organisation-wide MIS reporting | Not part of the current scope. The *per-case* forensic report is fully built | `services/case_report_service.py` |
| 15 | Frontend libraries | Tables are hand-built per page; no charting library is used | `frontend/package.json` |
| 16 | Signature verification | Detection + reviewer-created references + in-case vision comparison + a classical pixel-identical-reuse check. Cross-case library comparison is not built (the `is_library` flag only stores data) | `services/signature_*` |
| 17 | Evidence export | Per-case PDF report with file hashes and annotated renders; no bundle of the original files | `services/case_report_service.py` |
| 18 | Arabic values | Extracted Arabic field values render with the default text direction on the case detail page; Arabic issuer names render right-to-left in the Issuer Registry settings page | `frontend/src/pages/` |

| # | Area | Implementation | Where |
|---|---|---|---|
| 19 | Document labels | `school_document, vendor_invoice, commercial_invoice, procurement_documentation, quotation, travel_invoice, payment_evidence, other`. Note the **case** type is `travel_reimbursement` while the **document** label is `travel_invoice` | `services/classification_service.py`, `models/case.py` |
| 20 | Cloud deployment | Not built — local development only. The production plan and costs are in deployment/production-deployment.md | — |

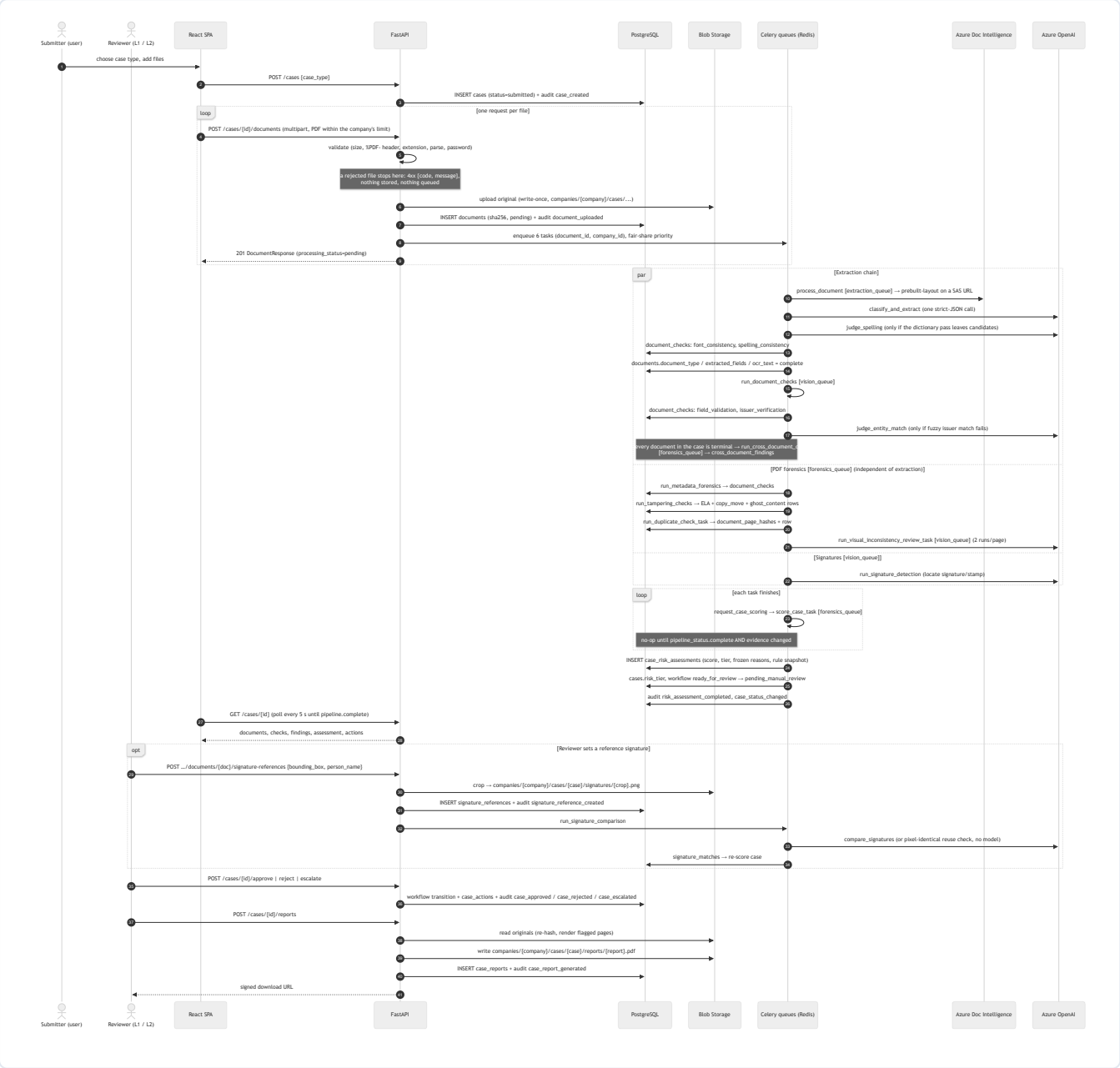
## Security posture

- **Two mandatory checks on every endpoint:** a role check (`require_company_role` / `require_reader` with a minimum rank — `user` < `reviewer_l1` < `reviewer_l2` — or `require_platform_admin`) and a tenant check (the request's session is bound to the caller's company by `get_tenant_db`). The user is re-read from the database on every request; a token whose company or platform flag no longer matches, or whose company is suspended, is refused. Case actions additionally check the case's tier (`app/api/case_access.py` `can_act_on_case`).
- **Tenant isolation is enforced twice** — application-layer filtering and PostgreSQL Row-Level Security — and a cross-company id is a **404**. Platform admins belong to no company, are read-only on company data, and every one of their accesses writes a platform-only audit row that no company user can see.
- **Submitters are deliberately blinded:** `GET /cases` and `GET /cases/{id}` return a neutral "In review" flag and no risk tier/assessment to a `user`, and the case timeline (`GET /cases/{id}/audit-log`) omits the scoring event and strips result details (results, finding and match counts, score, tier, rules fired) from the automated check events — so a bad actor cannot learn the score. A case owned by someone else is a **404, not 403**. *Note:* the per-document check cards on the case page (e.g. "Issuer verification: Flagged") are still shown to the submitter — see the change report R6 for this open decision.
- **Case-content routes are guarded** (`api/case_access.py`). Document upload and `GET …/documents/{id}/file-url` require that the caller is the case's **owner or a reviewer of its company** — anyone else gets a **404** (`load_visible_case`), and a rejected upload writes nothing to storage. The three `signatures` endpoints follow the same rule: the **uploader sets the reference signature during upload**, so the case owner may use them, as may any reviewer; a stranger gets a 404. Covered by `tests/test_case_authorization.py`. A submitter can still read their own case's signature comparison results (`GET …/signature-matches`), unlike the risk assessment, which is withheld.
- **Uploads are validated before storage** (PDF only, within the company's size limit, not corrupted, not password-protected), so a malformed file never reaches Blob Storage, a worker or an Azure API.
- Tokens are stored in `localStorage` and there is no server-side logout, refresh or password-reset endpoint. `JWT_SECRET_KEY` defaults to `changeme-in-.env` — production must set it (see deployment/production-deployment.md).

# End-to-end data flow

This walks one case from upload to exported report. Names in `monospace` are real Celery task names, functions, audit `event_type`s or table names; every one can be found by grepping the code.

## Sequence




Every request runs bound to the caller's company and every task to the document's company, so each arrow that touches the database is confined to one company by the application filter **and** by PostgreSQL Row-Level Security (multi-tenancy.md). Azure calls from any worker go through the global rate limiter (processing-queues.md).

## Step by step

### 1. Case creation

`POST /cases` creates a `cases` row with `status=submitted`, `assigned_tier=l1`, and a case number `CASE-` + the first 8 hex characters of the case UUID (unique without a counter table), stamped with the submitter's `company_id`. Audit event `case_created`; the company's `cases_created` usage counter is incremented in the same transaction.

### 2. Upload (one file per request)

The frontend (`NewCasePage`) calls the upload endpoint once per file with an XHR so it can show progress (and refuses non-PDF, empty and over-limit files before sending; the limit is the company's own, from `GET /auth/me/upload-limits`). The endpoint first **validates** the file (PDF only by its header bytes, within the company's per-file limit, not corrupted, not password-protected) and rejects a bad one with a specific `{code, message}` before anything is stored or queued. A valid file is hashed (SHA-256), stored in Blob Storage, its `documents` row inserted, `document_uploaded` written and the company's usage counters bumped; then **six tasks** are enqueued (each carrying the company, each with a fair-share priority) and the endpoint returns. See 01-upload-intake.

### 3. Extraction chain (`process_document` → `run_document_checks`)

`process_document` marks the document `processing`, gets a 30-minute SAS URL, runs Azure Layout OCR, then one Azure OpenAI call that classifies the document and extracts fields. It stores `document_type`, `ocr_text` and `extracted_fields` (each field with a normalized value, confidence and — best effort — a bounding box) and marks the document `complete` (or `failed` with `processing_error`). On success it enqueues `run_document_checks`, which writes two `document_checks` rows: `field_validation` and `issuer_verification`. See 02, 03, 05.

Two checks run **inside** `process_document`, before the document is marked complete, because they need its OCR words: font consistency (06a) and spelling (06b — English words only, confirmed by one LLM call when the dictionary pass leaves candidates). Both are best effort: each writes its row inside a savepoint, so a failure is stored as a failed check and never fails the extraction.

### 4. Cross-document check (case level)

After each `run_document_checks` — and also after a failed extraction — the task checks whether **every** document in the case has reached `complete` or `failed`; if so it enqueues `run_cross_document_checks`, which compares amount/date/issuer pairwise across completed documents and writes `cross_document_findings` (deleting and recomputing any previous rows for the case). Needs at least two completed documents. See 04.

### 5. Forensics (in parallel)

Independently of extraction, four tasks read only the file: `run_metadata_forensics`, `run_tampering_checks` (renders once, runs ELA and copy-move, plus the ghost-content check on scans converted to editable text) and `run_duplicate_check_task` on the CPU-bound `forensics_queue`, and `run_visual_inconsistency_review_task` on `vision_queue`; plus `run_signature_detection` (`vision_queue`). Uploads are PDF-only, so every document gets all of them (documents stored before that rule may be images; the forensic tasks skip those silently). Duplicate page matches are judged on the extracted fields once both documents have them (resubmission vs. same template — `refine_duplicate_check`, run by document checks after extraction). See 06, 07, 07a, 08, 09, 10.

### 6. Scoring

Every task ends by calling `request_case_scoring(case_id)`. `score_case` does nothing until `pipeline_status(case).complete` — extraction terminal for every document, `field_validation` and `issuer_verification` for every completed document, and the five forensic checks for every PDF, plus a cross-document check newer than the newest upload. It then evaluates every current rule, and — unless the evidence

fingerprint equals the latest assessment's — inserts a new `case_risk_assessments` row, sets `cases.risk_tier`, and on the first assessment moves the case to `pending_manual_review` (`case_status_changed`). See 11.

### 7. Reviewer decision

The case detail page polls until the assessment exists, then unlocks the decision panel. `POST /cases/{id}/approve` requires a finished pipeline and — above low risk — a justification of at least 10 characters; `reject` requires a reason; `escalate` (reason required) moves `assigned_tier` to `l2`, after which only a `reviewer_l2` of the company may act on it (an L1 keeps read access). Platform admins can never act on a case. Each writes a `case_actions` row **and** an `audit_log` row via the workflow event subscriber. Decided cases (approved/rejected/closed) reject further decisions with 409.

### 8. Report export

`POST /cases/{id}/reports` synchronously builds the nine-section PDF, stores it under `companies/{company}/cases/{case}/reports/`, and inserts a `case_reports` row (with the PDF's SHA-256); the company's `files_stored` / `storage_bytes` counters go up. See reports/forensic-report.md.

## Task names

The names in the diagram are the Python functions. The worker registers each under a Celery name (`@celery_app.task(name=...)`); three differ from the function name, which matters when reading worker logs or calling `celery_app.send_task`. The queue comes from `TASK_QUEUES` in `tasks/celery_app.py` (anything unlisted goes to `forensics_queue`, which always has workers):

| Python function | Registered Celery name | Queue | Module |
|---|---|---|---|
| `process_document` | `process_document` | `extraction_queue` | `tasks/document_processing.py` |
| `run_document_checks` | `run_document_checks` | `vision_queue` | `tasks/document_checks.py` |
| `run_cross_document_checks` | `run_cross_document_checks` | `forensics_queue` | `tasks/document_checks.py` |
| `run_metadata_forensics` | `run_metadata_forensics` | `forensics_queue` | `tasks/metadata_forensics_task.py` |
| `run_tampering_checks` | `run_tampering_checks` | `forensics_queue` | `tasks/tampering_checks_task.py` |
| `run_duplicate_check_task` | **`run_duplicate_check`** | `forensics_queue` | `tasks/duplicate_check_task.py` |
| `run_visual_inconsistency_review_task` | **`run_visual_inconsistency_review`** | `vision_queue` | `tasks/visual_inconsistency_task.py` |
| `run_signature_detection` | `run_signature_detection` | `vision_queue` | `tasks/signature_detection_task.py` |
| `run_signature_comparison` | `run_signature_comparison` | `vision_queue` | `tasks/signature_comparison_task.py` |
| `score_case_task` | **`score_case`** | `forensics_queue` | `tasks/risk_scoring_task.py` |

| Python function | Registered Celery name | Queue | Module |
|---|---|---|---|
| `reconcile_usage_stats` | `reconcile_usage_stats` (beat, nightly) | `forensics_queue` | `tasks/usage_tasks.py` |
| `log_queue_metrics` | `log_queue_metrics` (beat, every minute) | `forensics_queue` | `tasks/usage_tasks.py` |

## Timing and idempotency notes

- **Everything is at-least-once** ( `task_acks_late` : a task whose worker crashed is redelivered). Each task is safe to repeat: check results are upserted (one `document_checks` row per (document, check), one page hash per (document, page), one signature match per (reference, document, scope), all backed by unique constraints); `run_cross_document_checks` deletes then recomputes; `score_case` skips when the evidence fingerprint is unchanged (one assessment per fingerprint is a unique constraint).
- `signature_stamp_detection` **is deliberately not in the "required checks" list**, because it postdates older cases (which would otherwise never score). Its completion still re-triggers scoring.
- **Signature comparison is not part of the pipeline.** It runs only after a reviewer creates a reference. It re-scores the case when it finishes, so a case can be re-assessed after the reviewer has already seen a score.
- **The audit trail is the source of truth for "what happened when".** Distinct event types seen in a working install: `case_created` , `document_uploaded` , `document_processing_completed/failed` , `document_checks_completed` , `cross_document_check_completed` , `metadata_forensics_completed` , `tampering_checks_completed` , `duplicate_check_completed` , `visual_inconsistency_review_completed` , `signature_stamp_detection_completed` , `signature_reference_created` , `signature_comparison_completed` , `risk_assessment_completed` , `case_status_changed` , `case_approved` , `case_rejected` , `case_escalated` , `case_report_generated` , the settings events `issuer_created/updated/deactivated/reactivated` , `risk_rule_created/updated` , `risk_thresholds_updated` , `user_created/updated` (written into the **user's company's** log, actor shown to the company as "Platform team"; a platform admin account's own creation is platform-level), and the platform-level events ( `company_id` NULL, visible only to platform admins) `company_created/updated` , `risk_rule_template_created/updated` , `usage_stats_reconciled` and `platform_admin_access` . A failed forensic task writes `<check>_failed` (e.g. `tampering_checks_failed` ). A rejected upload writes no event. The three decision events carry `actor_role` (the actor's role at the time, e.g. `reviewer_l1` ); `case_escalated` also carries `from_tier` / `to_tier` ( `l1` → `l2` ) and the required `reason` .

# Multi-tenancy

FDDT serves several client companies from one deployment. Each company's users see **only** that company's data; the platform operator's own team (platform admins) sits outside every company.

## Roles

| Role | Company | Can |
|---|---|---|
| `user` | exactly one | submit cases/documents, track own cases |
| `reviewer_l1` | exactly one | review the company's queue; approve / reject / escalate (L1 tier) |
| `reviewer_l2` | exactly one | everything L1 can, plus escalated (L2) cases, plus the company's own **Settings › Issuer Registry** and **Settings › Risk Rules** |
| `platform_admin` | **none** | create companies; create every user and assign company + role; edit the **platform risk-rule template** new companies start from; billing & usage dashboard; processing-queue monitor; **read-only, audited** support access to any company's cases, audit log and settings (and may edit a company's issuer registry/risk rules on its behalf, also audited) |

- Company roles are ranked ( `has_rank` / `hasRank` ); `platform_admin` is deliberately *not* on that ladder, so it never inherits "reviewer" powers — it cannot approve, reject, escalate, upload, create cases or generate reports.
- There is no company-level user management: **Settings › Users** is platform-admin only.
- The former single-tenant `admin` role was migrated to `platform_admin` .

## Data model

- `companies (id, name, is_active, created_at, updated_at)` . `is_active = false` suspends a company: its users can't sign in and existing tokens stop working on the next request.
- `company_id` (FK, indexed) on every tenant-owned table: `users` , `cases` , `documents` , `document_checks` , `document_page_hashes` , `cross_document_findings` , `case_actions` , `case_risk_assessments` , `risk_scores` , `risk_rules` , `risk_settings` , `issuer_registry` , `signature_references` , `signature_matches` , `case_reports` , `audit_log` .
  - NOT NULL everywhere except `users` (NULL = platform admin; a CHECK constraint enforces `role = 'platform_admin' ⇔ company_id IS NULL` ) and `audit_log` (NULL = platform-level event such as *company created*, *template edited*, *usage reconciled*, *platform-admin access*). *User created / updated* for a company user is written into **that company's** log; the company sees the actor as "Platform team" (the platform admin's own user row is invisible to a company session).
  - Extracted fields are a column on `documents` and duplicate-detection hashes live in `document_page_hashes` — both covered by those tables' `company_id` .
- `risk_rules` , `risk_settings` (tier thresholds) and `issuer_registry` are **per company**. Rule uniqueness is `(company_id, rule_id, version)` . The issuer registry starts empty.

- `risk_rule_templates` (platform-level, **no** `company_id`) holds the default rule set. Platform admins edit it under **Platform › Rule templates**. When a company is created, every **active** template row is copied into that company's own `risk_rules` (version 1, `company_id` stamped) plus default thresholds (`seed_risk_rules(db, company_id)`). From then on the copy is the company's own: editing the template never changes an existing company — only companies created afterwards (the same "no retroactive change" principle as rule versioning). The migration seeded the template from the Default Company's current rules (38 rules — its 42 rows were 38 rules plus 4 older versions). Only `fddt_platform` has access to the table.
- `company_usage_stats (company_id, usage_date, cases_created, documents_uploaded, files_stored, storage_bytes)` — see Billing & usage.

## Tenant isolation — two independent layers

### 1. Application layer (`app/db/tenancy.py`, `app/api/auth.py`, `app/api/tenant_access.py`)

- The JWT carries `role`, `company_id` (null for platform admins) and `is_platform_admin`. Every request re-reads the user from the database; if the token's company or platform flag no longer matches (user moved to another company, role changed across the platform boundary) the token is refused (401), and a suspended company's users are refused too.
- Every protected endpoint applies **two mandatory checks**: the role check (`require_company_role`, `require_reader`, `require_platform_admin`) **and** the tenant check — the request's data session is bound to the caller's company by the `get_tenant_db` dependency, so a route cannot forget it.
- Every session is in one of three states:
  - **unbound** — refuses to query or write any tenant-owned table (raises `TenantContextError`);
  - **company** — every ORM SELECT/UPDATE/DELETE touching a tenant model gets `company_id = <bound company>` added automatically (`with_loader_criteria`, including relationship loads and `session.get()`); every INSERT is stamped with that company and a row of another company is refused at flush; objects of other companies are evicted from the identity map on binding;
  - **platform** — the explicit RLS-bypassing path (below), no automatic filter.
- On top of that backstop, service functions that read tenant data take `company_id` explicitly and filter by it themselves (`load_current_rules`, `get_risk_settings`, `match_issuer`, `run_duplicate_check`, `score_case`, `pipeline_status`, `latest_assessment`, `get_case_flags`, …).
- An id that belongs to another company is a **404** — never a silent empty result and never a 403 that would confirm the id exists.
- Celery tasks are enqueued as `task.delay(document_id, company_id)` and open a session bound to that company (`app/tasks/tenant.py`), so workers are confined exactly like requests.
- Duplicate detection compares a document only against **its own company's** history (matching across companies would leak another tenant's filenames and case numbers).

### 2. Database layer — PostgreSQL Row-Level Security

The migration `a7c1e9d3f5b2_multi_tenancy` creates two login roles and the policies:

| Role | Attributes | Used for |
|---|---|---|
| `fddt_app` (`DATABASE_APP_USER`) | NOSUPERUSER **NOBYPASSRLS** | every company-scoped request and pipeline task |
| `fddt_platform` (`DATABASE_PLATFORM_USER`) | NOSUPERUSER **NOBYPASSRLS** — full access through an explicit `platform_all` policy on every table | sign-in/token identity lookup, platform-admin screens, usage reconciliation, resolving a legacy task's company |

| Role | Attributes | Used for |
|---|---|---|
| owner ( `DATABASE_URL` ) | owns the tables; needs CREATEROLE, **not** superuser | Alembic migrations and `seed.py` only |

**No role has BYPASSRLS, and no superuser is needed anywhere.** Platform access is policy-based: `CREATE POLICY platform_all ON <table> FOR ALL TO fddt_platform USING (true) WITH CHECK (true)` on each RLS table — creating a policy only needs table ownership, so this works unchanged on managed PostgreSQL (Azure Flexible Server). Verified by running the **entire** migration chain as a NOSUPERUSER/CREATEROLE owner on a fresh database and re-running the RLS suite there. (PostgreSQL 16 requires superuser even to *mention* `SUPERUSER` / `BYPASSRLS` in `ALTER ROLE`, so those attributes are set on `CREATE ROLE` only and the migration asserts afterwards that the roles really are unprivileged.)

- RLS is enabled on every tenant-owned table (plus `companies` and `company_usage_stats`) with a `tenant_isolation` policy for `fddt_app`: `USING (company_id = fddt_current_company_id()) WITH CHECK (same)`, where `fddt_current_company_id()` reads the transaction-local setting `app.current_company_id`.
- The app sets that setting at the start of **every** transaction of a company-bound session ( `after_begin` hook, `set_config(..., is_local => true)` ), so a pooled connection never carries one request's company into the next. No setting ⇒ no rows.
- A bug that forgets the company filter therefore still cannot read, update, delete or insert another company's rows — the database refuses. `tests/test_rls_postgres.py` proves this with raw SQL that has **no** company filter at all.
- Grants are least-privilege and also make the append-only/immutable tables read+insert-only **in the database** for both app roles: `audit_log`, `risk_rules` (versioned), `case_risk_assessments`, `case_reports`. `users` is SELECT-only for `fddt_app` (user management is platform-only).
- The platform engine is reachable only through a session explicitly bound to platform mode ( `get_system_db`, `system_session()` ); the default request session is unbound and then company-bound.

**Adding a tenant-owned table later:** give it `TenantScopedMixin`, and in its migration `ENABLE ROW LEVEL SECURITY`, create the `tenant_isolation` policy ( `enable_tenant_rls()` in the multi-tenancy migration shows how) and grant the two app roles explicitly. There are deliberately no default privileges, so a new table is unreachable by the app until that is done.

## Platform-admin support access (audited)

A platform admin can open any company's case, case list, audit log or settings. The route resolves which company the resource belongs to through the platform session, binds the request to **that** company (so the rest of the request is confined by both layers), and writes a `platform_admin_access` audit row, e.g.:

> Platform admin ops@example.com viewed Company B / Case CASE-1A2B3C4D

**These rows are platform-only.** They are written through the platform session with `company_id = NULL` (the target company is in `event_data.company_id`), so:

- RLS hides them from every company session — a company query cannot return them even without a filter;
- every company-facing view (Audit History, its event-type filter, a case's timeline, the case report) also excludes the event type explicitly;
- company users have no indication that platform access happened.

Platform admins see every row — in the platform-level log, and in a company's log view (which shows the company's own events plus the platform-access rows about it). **Every access is recorded individually**: `PLATFORM_ACCESS_AUDIT_DEDUP_SECONDS` defaults to `0` (a value > 0 would collapse identical reads).

Rows written before this change were moved to `company_id = NULL` by migration `c9e3a1b5d7f4` (no row removed, no content changed).

Platform admins have no cross-company case list: the case queue takes one company at a time.

## Blob storage layout

```
companies/{company_id}/cases/{case_id}/documents/{document_id}_{sha256}{ext}
companies/{company_id}/cases/{case_id}/reports/{report_id}.pdf
companies/{company_id}/cases/{case_id}/signatures/{crop_id}.png
```

`ensure_company_blob()` refuses to sign a URL under another company's prefix. Files written before the migration keep their original paths ( `{case_id}/…` , `reports/…` , `signatures/…` ) — originals are immutable and are not moved; their database rows are company-scoped like everything else.

## Billing & usage

**Platform › Billing & usage** shows one row per company (never a mixed list of cases): cases created, documents uploaded, files stored (documents + generated reports), storage added in the period, and each company's current all-time storage and file count, for this month / last month / last 30 days / this year / all time / a custom UTC date range.

- Counters live in `company_usage_stats` , one row per company per UTC day, bumped with an atomic `INSERT … ON CONFLICT DO UPDATE SET n = n + delta` **in the same transaction** that creates the case, document or report ( `app/services/usage_service.py` ). The dashboard sums a few small rows — never a COUNT/SUM over the case tables.
- Storage is the exact byte size recorded when each file was written ( `documents.file_size_bytes` , `case_reports.file_size_bytes` ). Signature crops are small derived images and are not counted.
- `reconcile_usage_stats` (Celery beat, nightly at `USAGE_RECONCILIATION_HOUR_UTC` , default 02:00 UTC) recomputes every (company, day) from the source tables and corrects any drift, writing a platform-level `usage_stats_reconciled` audit row when it changed something. **Reconcile now** on the dashboard runs the same job on demand.
- Nothing deletes cases, documents or reports today, so counters only increase. If deletion is added it must decrement them (the nightly job would otherwise correct it the next night).

## Migrating an existing single-tenant database

`alembic upgrade head` (as the owner role):

1. creates `companies` / `company_usage_stats` and one company named `DEFAULT_COMPANY_NAME` ("Default Company");
2. adds `company_id` everywhere and assigns **every** existing row to that company;
3. turns existing `admin` accounts into `platform_admin` (no company); everyone else keeps their role in the default company;
4. backfills the usage counters from the existing data;
5. creates `fddt_app` / `fddt_platform` with the configured passwords, grants, and the RLS policies (tenant isolation for the app role, `platform_all` for the platform role).

Then `c9e3a1b5d7f4` (production hardening): removes BYPASSRLS from an existing `fddt_platform` (only if it has it), adds the `platform_all` policies, moves platform-access audit rows to platform-only, creates and seeds

`risk_rule_templates`, and enforces one row per check / page hash / assessment / signature match (collapsing duplicates earlier retries left: on the dev database 6 check rows, 1 page hash, 1 assessment).

The migrations need a table-owner role with **CREATEROLE**; no superuser.

`alembic downgrade` reverses it (keeping only the oldest company's rules/thresholds, since a single tenant can't hold several companies' copies).

## Tests

| Spec check | Where |
|---|---|
| Platform admin creates companies A and B and their users | `tests/test_multi_tenancy.py::tenants` fixture |
| A's reviewer_l2 sees A's cases, issuers, rules; 404 for every B resource by id (case detail/audit/file URL/signatures/reports/actions/issuer PATCH/rule history, `company_id=B` params) | `test_reviewer_l2_sees_own_company_and_gets_404_for_the_other`, `test_a_rule_history_of_another_company_is_a_404` |
| A's reviewer_l2 cannot reach Settings › Users | `test_reviewer_l2_cannot_reach_settings_users` |
| Platform admin opens B's case; a platform-only audit row is recorded (visible to platform admins, never to A or B) | `test_platform_admin_reads_any_case_and_each_access_is_audited` |
| Billing per company matches uploads; manual cross-check; drift corrected | `test_billing_dashboard_separates_companies_and_matches_uploads` |
| RLS alone blocks cross-tenant rows with no company filter in the SQL | `tests/test_rls_postgres.py` (needs `FDDT_RLS_TEST_DATABASE_URL`) |
| No BYPASSRLS anywhere; platform access via policies; platform-only audit rows invisible to the app role | `tests/test_rls_postgres.py` (`test_no_role_has_bypassrls`, `test_every_rls_table_has_the_platform_policy`, …) |
| Company users never see platform-access rows (Audit History, filter, timeline) | `tests/test_multi_tenancy.py::test_company_users_never_see_platform_admin_access` |
| Every idempotent write (page hashes, check rows, usage counters) works under the least-privilege app role | `tests/test_rls_postgres.py::test_every_upsert_the_pipeline_uses_works_under_the_app_role` |
| New company gets the template; template edits don't change existing companies | `tests/test_rule_templates.py` |
| Tenant context never leaks through PgBouncer transaction pooling | `tests/test_pgbouncer_rls.py` (needs PgBouncer) |
| Pre-migration data intact and attributed to the default company | verified against the real dev database (see the change summary) |

# Processing queues, fairness, rate limits and database connections

## Design

- **One task per document per step.** An upload enqueues six independent tasks for that document (extraction, metadata forensics, ELA + copy-move + ghost text, duplicate hashing, visual review, signature detection), each carrying the document's company. No task ever processes a whole case or batch.
- **Three shared queues, one per external dependency**, each with its own worker pool:

| Queue | Tasks | Limited by | Pool |
|---|---|---|---|
| `extraction_queue` | `process_document` (Document Intelligence layout **+ the Azure OpenAI classification call**, and the spelling check's one call when it has candidates) | `AZURE_DOCUMENT_INTELLIGENCE_MAX_CALLS_PER_SECOND` (10; Azure allows 15) — and, for the classification call, the Azure OpenAI caps | threads |
| `vision_queue` | visual review, signature detection/comparison, field validation + issuer check | `AZURE_OPENAI_MAX_REQUESTS_PER_MINUTE`, `AZURE_OPENAI_MAX_TOKENS_PER_MINUTE` | threads |
| `forensics_queue` (+ legacy `celery`) | metadata forensics, ELA/copy-move/ghost text, duplicate hashing, cross-document check, risk scoring, bulk-zip ingestion | **nothing** — local CPU (exception: when the ghost-content check finds a deleted text block — rare — it makes a few Document Intelligence Read and Azure OpenAI calls for the reviewer's hint, through the same global rate limiters) | **prefork**, one process per CPU of the container |

  Platform jobs run on a fourth, separate **`housekeeping_queue`**: `log_queue_metrics` (every minute), `requeue_stuck_documents` (every 5 minutes) and `reconcile_usage_stats` (nightly). Its only consumer is the beat container (`start-beat.sh`: beat + one `solo` worker slot). Because the every-minute job no longer lands on `forensics_queue`, the forensics workers see work only when documents arrive, so they can **scale to zero** when idle (KEDA on the forensics lists, `minReplicas: 0`; a Redis list-length scaler activates on the first waiting task). `housekeeping_queue` is not one of the three processing queues in the monitor.
- **Stuck-document recovery** (`app/tasks/stuck_documents_task.py`). Queued tasks live only in Redis. If Redis restarts without its data, or is unreachable at upload time (`enqueue_document_pipeline` then logs and returns instead of failing the upload; the document is already stored), a document would stay `pending` (or `processing`) forever. Every 5 minutes, **only while the extraction queue is empty and idle**, the job re-queues the whole pipeline for documents `pending` > `STUCK_DOCUMENT_PENDING_MINUTES` (10) or `processing` with no update for > `STUCK_DOCUMENT_PROCESSING_MINUTES` (30). A document's first task is `process_document` on the extraction queue, so an empty, idle extraction queue means that task is not coming; a long backlog therefore never causes duplicate work. Each re-queue writes a `document_requeued` audit row; after `STUCK_DOCUMENT_MAX_REQUEUES` (3) the document is marked `failed` with an explanation. Re-running is safe because every step writes idempotently (below). The run logs a WARNING `stuck_documents_requeued {json}` line when it acted. Not covered: a document whose extraction finished but whose other queued checks were lost; those checks stay missing until the document is re-uploaded.
- **Fair-share per company** (`app/tasks/fairshare.py`). Each task is published with a priority computed from how much work *its company* already has outstanding (queued + running) on that queue: `priority = min(9, outstanding // FAIR_SHARE_BUCKET_SIZE)` (bucket default 5). Celery's Redis transport keeps one list per level

( `<queue>` , `<queue>:1` … `<queue>:9` ) and workers always drain level 0 first, so a company with little outstanding work is served before the tail of another company's big batch. Equal share, no fixed tiers: priorities are recomputed for every task from live counters, and a company's new tasks move forward again as its backlog drains. Counters are incremented on publish and decremented in `task_postrun` ; a redelivered task (worker crash) is counted once; the per-minute metrics job resets a queue's counters whenever that queue is empty (self-healing). If Redis is unreachable a task is published without priority (FIFO) — scheduling degrades, processing doesn't stop.

- **Fair dispatch per worker**: `worker_prefetch_multiplier=1` + `task_acks_late` — a worker holds only the task it is running.
- **Idempotent writes** ( `app/services/check_store.py` ): exactly one `document_checks` row per (document, check_type), written by `INSERT … ON CONFLICT DO UPDATE` ; likewise one page hash per (document, page), one assessment per (case, evidence fingerprint), one signature match per (reference, document, scope) — all enforced by unique constraints. A task redelivered after a worker crash overwrites instead of duplicating; the full run history stays in the append-only audit log.
- **No transaction held during slow work.** Tasks commit (ending their read transaction) before every slow step — blob download, Azure call, CPU forensics — and task sessions keep loaded attributes after a commit ( `expire_on_commit=False` ) so touching them doesn't silently reopen one. Through PgBouncer in transaction mode an open transaction pins a server connection; the first Linux load test showed 32 threads holding transactions across Azure calls exhausting the pool and stalling the API. The upload endpoint likewise ends its read transaction before reading the body and writing the blob.
- **Independently scalable workers**: `start-worker.sh extraction|vision|forensics` , or the `*-worker` services in `docker-compose.yml` ( `--profile workers` , `--scale forensics-worker=N` ). The forensics worker runs prefork with one process per CPU of **its container's cgroup limit** ( `cpus:` / `FORENSICS_WORKER_CPUS` ), not the host's core count — `nproc` ignores container limits and would oversubscribe (2 replicas each starting 4 processes on a 4-core host). Pool sizes: `EXTRACTION_WORKER_CONCURRENCY` , `VISION_WORKER_CONCURRENCY` , `FORENSICS_WORKER_CONCURRENCY` (0 = per CPU of the limit).
- **Monitoring**: Platform › Processing queues ( `GET /platform/queues` ) — waiting (all priority levels), running, oldest waiting task, average / p95 / max wait, run time, consumers online, limiter counters (throttled calls, seconds waited, Azure 429s) and **outstanding work per company** (the fair-share counters). A beat task logs one structured line per queue every minute, `queue_metrics {"queue": …, "waiting": …, "running": …, "oldest_waiting_seconds": …, "avg_wait_seconds": …, "p95_wait_seconds": …, "started_in_window": …, "window_seconds": 300}` , and a **WARNING** `queue_alert {…same fields…, "alert": "oldest_waiting_exceeded", "threshold_seconds": …}` line when a queue's oldest waiting task is older than `QUEUE_ALERT_OLDEST_WAITING_SECONDS` (default 300; 0 = off). Shipping these logs to a monitoring platform and alerting on `queue_alert` is a deployment step (not built yet: local dev only). For example, on Azure Container Apps with a Log Analytics workspace, a log alert on `ContainerAppConsoleLogs_CL | where Log_s has "queue_alert"` (production-deployment.md).

## Azure rate limits

**Global limiter** ( `app/services/rate_limiter.py` ): Redis sliding windows shared by every worker, applied per Azure call (Celery's own `rate_limit` is per worker). A 429 that still gets through pauses every worker for the Retry-After period (the SDKs retry first).

**This deployment's quota** (from the `x-ratelimit-*` headers of `gpt-4.1-mini` ): 100 requests/min, **100,000 tokens/min**. `backend/.env` caps at **80 %** ( `80` RPM, `80000` TPM). This was 85 % until 2026-10-03. It was lowered because the real-quota load test saw one 429 in 123 calls at full saturation (two independent sliding windows can race). The load-test figures below were measured at 85 %.

**Token reservations are calibrated against real usage** ( `scripts/calibrate_llm_tokens.py` ). Azure charges each request its prompt tokens + its full `max_tokens` up front. Every call now logs its real `usage` next to the reservation

( `llm_usage …` log line, Redis `fddt:llm_usage:<call type>` ). Measured on the 12 sample documents (English + Arabic, 96 calls):

| Call type | Real prompt (avg) | Old estimate | New estimate | Real max output | `max_tokens` old → new |
|---|---|---|---|---|---|
| classification + extraction | 2,105 | 1,270 (**0.60×**) | 2,762 (1.31×) | 1,207 | 4,000 → 4,000 |
| visual review (2 per page) | 2,904 | 1,564 (**0.54×**) | 3,622 (1.25×) | 204 | 1,500 → 640 |
| signature/stamp detection | 1,418 | 1,392 | 1,967 (1.39×) | 144 | 800 → 480 |
| signature comparison | 1,055 | 2,649 (2.5×) | 1,558 (1.48×) | 47 | 800 → 320 |
| issuer entity match | 400 | 353 | 768 (1.92×) | 155 | 4,000 → 512 |

- The old estimator (4 chars/token, flat 1,100 tokens/image) **under-reserved the two most frequent calls by 36–46 %**: at full load the limiter would have let traffic exceed the real quota.
- New model: text at **1.93 chars/token** (the most token-dense observed — Arabic; English prompts are reserved a little high, never low) + images at **1.62 tokens per 32-px patch** (how gpt-4.1-family models bill images; full pages and small crops now cost what they really cost) + **5 % margin**. Every measured call is reserved at ≥ 1.05× its real prompt.
- Output reservations were sized from measured outputs with ample headroom for the four fixed-shape calls; classification keeps 4,000 because its output grows with the number of fields on a document.
- **Real-Azure check**: 40 vision calls from 8 threads through the limiter at the 85 % caps → **40 × HTTP 200, 0 × HTTP 429, 0 SDK retries**; the token limiter held 15–20 calls back and settled at 19 calls/min.
- Re-run the calibration when the model, the prompts or the page render size change.

**Throughput ceiling at the current quota** (what the limiter reserves per one-page document: classification 6.8k + visual review 2 × 4.3k + signature detection ~3 × 2.4k + issuer match 1.3k ≈ 23.9k tokens): **≈ 3.6 documents/minute (~210/hour) for the whole platform.** The earlier "5–6/minute" figure was based on the under-estimates. The `max_tokens` reductions raised the real ceiling by ~26 % (from ≈ 2.8/min). Raising the deployment's TPM is the main lever beyond this.

## Database connections

### Without PgBouncer (direct)

Each process keeps a SQLAlchemy pool **per role it uses** (app + platform): up to `DB_POOL_SIZE + DB_MAX_OVERFLOW` = 5 + 10 = **15 per role**, opened on demand.

| Process | Concurrency that can hold a connection | Max server connections |
|---|---|---|
| API (uvicorn worker) | FastAPI sync endpoints run in a 40-thread pool → capped by the pool | 2 roles × 15 = **30** |
| extraction / vision worker (threads, T threads) | T | ≈ min(T, 15) app + a few platform |
| forensics worker process (prefork) | 1 task at a time | ≈ 1–2 |

Example: 2 API replicas × 4 uvicorn workers (8 × 30 = 240) + extraction 16 + vision 16 + 2 forensics replicas × 2 processes (≈ 8) ≈ **280** — beyond Postgres' default `max_connections = 100` . Hence:

### With PgBouncer (transaction pooling) — the recommended setup

Set `DATABASE_POOLER_HOST` / `DATABASE_POOLER_PORT`: the two app roles connect to PgBouncer (`pgbouncer/pgbouncer.ini`, service `pgbouncer` in `docker-compose.yml`) and **each process keeps no pool of its own** (`NullPool`), so idle slack isn't held twice. Client connections are cheap; PgBouncer multiplexes them onto a small set of server connections:

| PgBouncer setting | Value | Meaning |
|---|---|---|
| `pool_mode` | `transaction` | a server connection is lent per transaction |
| `default_pool_size` | 20 | server connections per (database, user): `fddt_app`, `fddt_platform` |
| `reserve_pool_size` | 5 (after 3 s) | burst headroom per pool |
| `max_db_connections` | 60 | hard cap of server connections to the database |
| `max_client_conn` | 1000 | client connections accepted (API + workers) |

**Postgres `max_connections` required**: PgBouncer's 60 + direct owner connections for migrations/seed (≈ 2) + admin/monitoring (≈ 5) + superuser reserve (3) ⇒ **≥ 70; the default 100 is enough.** Raise `default_pool_size` / `max_db_connections` (and `max_connections` with them) only if PgBouncer's `SHOW POOLS` shows clients waiting (`cl_waiting`).

**Tenant context is safe with transaction pooling** because the company is set with `set_config('app.current_company_id', …, is_local => true)` at the start of every transaction and dies with it. `tests/test_pgbouncer_rls.py` proves it on a single-server-connection pool (forced reuse): a *session*-level setting **does** leak to the next client (negative control — the test can detect leaks), the transaction-local one never does, including 32 concurrent clients × 25 transactions alternating companies. Never set tenant state with plain `SET` / `set_config(..., false)`.

Production: `userlist.txt` holds local dev passwords only — use SCRAM secrets from a vault.

## Load tests

`backend/loadtest/` drives the real API, real Celery workers, Postgres (RLS) and Redis; only Document Intelligence, Azure OpenAI and Blob Storage are stubbed (they simulate latency and Azure's own quotas, answering 429 above them). `docker-compose.loadtest.yml` runs it all in **Linux containers through PgBouncer**.

### Linux, final run (forensics 2 replicas × 2 prefork processes; Azure queues 16 threads; Azure OpenAI quota simulated at 10× so the run takes minutes)

| Check | Result |
|---|---|
| 1 — 27 concurrent single uploads, 3 companies | interleaved (`ABCABCABCACBB…`), Kendall τ vs submission 1.0 / 0.86 / 0.95 (extraction / vision / forensics) |
| 2 — A uploads 100 docs, B uploads 1 right after | **B waited 0.0–0.2 s on forensics (A's median 30 s) and 1.2 s on visual review (A's median 44 s)**; started after 43–45 of A's 100 instead of all 100 |
| 3 — no 429 storms | Document Intelligence peaked at exactly 10 calls/s (quota 15), **0 × 429**; Azure OpenAI **0 × 429**, token limiter held 242 calls back |
| 4 — monitor reflects reality | peaks 20 / 146 / 299 waiting, tracked every 2 s for the whole run |
| Drain | everything drained in 92 s; forensics finished before vision (97 s vs 130 s) |

### Linux, real quota (85 RPM / 85k TPM app caps vs 100 / 100k simulated Azure), 30-document batch

| Queue | Last task started | Oldest wait |
|---|---|---|
| **forensics** | **35 s** | 16 s |
| extraction | 69 s | 50 s |
| vision | 315 s | 288 s |

At the real quota **forensics is the fastest-draining queue by far** and the Azure OpenAI quota is the long pole — as it should be. Fair-share: B started after 10 of A's 30 on forensics and vision. One simulated 429 occurred in 123 calls while the token limiter was fully saturated (103 calls held back) — a boundary race between the app's window and the stub's independent window; the real SDK retries such a 429 after Retry-After (the stub doesn't), and the real-Azure burst at the same caps gave none. For more margin set the caps to 80 % of quota.

### Comparison with the first (Windows, strict FIFO) run

| | Windows threads, FIFO | Linux prefork, fair-share |
|---|---|---|
| B's wait behind A's 100-doc batch (forensics / vision) | 144 s / 70 s, after **100/100** | 0.2 s / 1.2 s, after 43–45/100 |
| Forensics oldest waiting | 264 s | 47 s (10× load), 16 s (real quota) |
| 429s | 0 | 0 |

## Open findings

- `process_document` **couples Document Intelligence and the Azure OpenAI classification in one task.** When the token limiter is saturated, extraction threads sit blocked on the *OpenAI* quota, so OCR for a newly arrived document waits for a free thread (B waited 14 s / 50 s on extraction in the two runs while A's OCR barely waited). Fix: move classification + extraction into its own `vision_queue` task (needs the OCR word boxes persisted between the two). Not done in this change.
- **A rate-limited task holds its worker thread while it waits for the limiter.** Fair-share decides the dequeue order, but a newly prioritised task still needs a free thread. Sizing the Azure pools to just saturate the caps (not far above) keeps that wait short.
- **On one machine, forensics competes with the API for CPU.** Prefork saturates its CPU allowance by design; run workers and the API on separate nodes (or cap `FORENSICS_WORKER_CPUS`) so uploads stay fast.

# 01 — Upload & intake

**What it does:** accepts one PDF at a time into a case, validates it, stores the original immutably, records its hash, and fans out the automated pipeline. It detects nothing itself; it is the chain-of-custody starting point.

**Code:** `api/documents.py` ( `upload_document` ), `services/upload_validation.py` , `services/storage_service.py` , `services/usage_service.py` , `models/document.py` , frontend `pages/NewCasePage.tsx` + `components/upload/FileDropzone.tsx` + `lib/uploadLimits.ts` .

## Flow

1. The UI creates the case ( `POST /cases` ), then uploads each selected file with its own `POST /cases/{case_id}/documents` (multipart field `file` ) using `XMLHttpRequest` for progress. Files upload in parallel; a failed file can be retried without recreating the case, dismissed using the remove icon ( `[X]` ), or discarded completely via `Reset form` which cleanly restores the intake form and unlocks case type selection.
2. The endpoint 404s on an unknown case (or one of another company, or — for a `user` — one they did not submit; platform admins get 403), then **validates the file synchronously** (next section). A rejected file is never stored, never gets a `documents` row or audit event, never counts toward usage and never queues a task, so it costs no worker slot and no Azure call. A valid file's `sha256(content)` is computed.
3. It writes the bytes to Blob Storage at `companies/{company_id}/cases/{case_id}/documents/{document_id}_{sha256}{original_extension}` with `overwrite=False` . The path embeds the hash and the document's own id, so collisions are impossible even if a filename repeats. A storage failure is a **502**.
4. It inserts a `documents` row ( `processing_status = pending` , `content_type = application/pdf` ) and flushes, writes the audit event `document_uploaded` with `{original_filename, file_hash, file_size_bytes}` , bumps the company's usage counters ( `documents_uploaded` , `files_stored` , `storage_bytes` ) in the same transaction, and commits.
5. It enqueues six Celery tasks — each as `(document_id, company_id)` with a fair-share priority — and returns **201**:

| Task | Queue | Module | Depends on | Notes |
|---|---|---|---|---|
| `process_document` | `extraction_queue` | `tasks/document_processing.py` | — | OCR + classify + extract, then the font and spelling checks (they need its OCR words); chains into `run_document_checks` |
| `run_metadata_forensics` | `forensics_queue` | `tasks/metadata_forensics_task.py` | file only | |
| `run_tampering_checks` | `forensics_queue` | `tasks/tampering_checks_task.py` | file only | ELA + copy-move share one render |
| `run_duplicate_check_task` | `forensics_queue` | `tasks/duplicate_check_task.py` | file only | compares within the company only |
| `run_visual_inconsistency_review_task` | `vision_queue` | `tasks/visual_inconsistency_task.py` | file only | also carries the AI-generation question |

| Task | Queue | Module | Depends on | Notes |
|---|---|---|---|---|
| `run_signature_detection` | `vision_queue` | `tasks/signature_detection_task.py` | file only | |

6. The response contains a fresh signed download URL (if signing fails it falls back to the durable URL; the upload itself has already succeeded).

Afterwards the UI offers an **optional** step to draw a reference signature (see 09).

## Upload validation

`services/upload_validation.py`, called before anything is stored. **Only PDF documents are accepted** (decision of 2026-10-03: every forensic check is PDF-based, so an image would only ever get OCR and no tamper analysis). The checks run in this order and the first failure wins. Each rejection returns `detail: {"code", "message", ...}`, and the UI shows `message` as-is next to the file.

| # | Check | HTTP | `code` | Message / extra fields |
|---|---|---|---|---|
| 1 | 0 bytes | 400 | `file_empty` | "The file is empty (0 bytes)." |
| 2 | Over **the company's** per-file limit (`companies.max_file_size_mb`; default **10 MB** = 10,485,760 bytes for a new company) | 413 | `file_too_large` | States both sizes and the limit in effect for that company, e.g. "The file is 12.0 MB; the maximum allowed size is 10.0 MB."; `size_bytes`, `max_size_bytes` |
| 3 | Content is not a PDF, judged by the **header bytes** (`%PDF-`), never the name or declared MIME type | 415 | `unsupported_file_type` | "This file is a JPEG image. Only PDF files are accepted — please save or scan the document as a PDF." (the type is named when recognised: JPEG, PNG, TIFF, GIF, BMP images, ZIP/Office files) |
| 4 | A PDF whose name carries another type's extension (`.png`, `.docx`, …) | 415 | `file_type_mismatch` | `detected_type`, `extension` |
| 5 | The PDF needs a password to open | 422 | `file_password_protected` | "This file is password-protected. Please remove the password and re-upload." |
| 6 | The PDF can't be parsed locally (PyMuPDF) | 422 | `file_corrupted` | "The file appears to be corrupted or unreadable." |

Details:

- **Magic bytes.** `%PDF-` anywhere in the first 1024 bytes (the PDF spec allows junk before the header; Acrobat and MuPDF accept it). An image renamed to `.pdf` is refused as an image.
- **Extensions.** Only a *recognised* file-type extension (images, Office, text, archives, …) is compared with the content. An unrecognised suffix (the ".2026" in "Invoice 12.03.2026") and a name with no extension are accepted on the content alone; `.PDF` in upper case is fine.
- **Parse.** PyMuPDF — the same library the pipeline renders pages with — opens the file and parses every page's content stream. MuPDF silently "repairs" a truncated file, so two cases count as corrupted: a file it repairs into **0**

**pages**, and a file it had to repair that has **no `%%EOF` marker in its last 1024 bytes** (a cut-off file can otherwise yield a few pages and silently lose the rest). A merely sloppy PDF with wrong cross-reference offsets still ends in `%%EOF` and is accepted. All 38 sample PDFs pass, including the 12 tampered samples (incremental saves).

- **Password policy: reject-and-ask** ( 🔶 decision, 2026-10-03). The server never asks for or applies a password, so the system never handles other people's document passwords. A PDF with only an **owner** password (print/copy restrictions; it opens without a password) is accepted. To revisit: the alternative is a password field on upload plus server-side decryption.
- **Stored `content_type`** is always `application/pdf` (the detected type), not the browser-declared one.
- The endpoint reads at most limit + 1 bytes into memory and parses in a worker thread, so the event loop isn't blocked. Uvicorn still receives (and spools to a temp file) the *whole* request body before the endpoint runs — cap the request size at the ingress / reverse proxy in production too (production deployment).
- **Per-company limits:** the size limits are not constants. Each company has `max_file_size_mb` and `max_zip_size_mb` on its row (10 / 300 for a new company, from `DEFAULT_MAX_FILE_SIZE_MB` / `DEFAULT_MAX_ZIP_SIZE_MB`), changed only by a platform admin (Platform › Companies › Edit; `PATCH /platform/companies/{id}`, audited as `company_updated` with old → new values; any positive whole number of MB, no upper ceiling). `services/upload_limits.py` is the single source the single-file endpoint, the bulk-zip endpoint and the bulk ingestion task all read, fresh on every request, so a change applies to the company's next upload without a new sign-in.
- **Frontend:** `lib/uploadLimits.ts` refuses files that are not named/typed as PDF, empty files and files over the company's limit before sending (the server check is the one that counts). The limit comes from `GET /auth/me/upload-limits` ( `hooks/useUploadLimits.ts` ); no size is hardcoded in the frontend. The file picker only offers PDFs and the drop zone states the rules ("PDF only · max file size: 20.0 MB each · not password-protected").
- Tests: `tests/test_upload_validation.py` .

## Output shape

`DocumentResponse` ( `schemas/document.py` ):

```
{
  "id": "uuid", "case_id": "uuid",
  "original_filename": "Invoice.pdf",
  "content_type": "application/pdf",       // detected from the file's header bytes
  "file_size_bytes": 184223,
  "file_hash": "e3b0c4…",                  // SHA-256 hex, 64 chars
  "file_url": "https://…?sv=…&sig=…",      // short-lived SAS URL
  "uploaded_at": "2026-09-20T17:08:40Z",
  "processing_status": "pending"           // pending | processing | complete | failed
}
```

`documents.blob_storage_path` stores the **durable** (unsigned) blob URL. The container is private, so anything that needs the file mints a SAS URL with `StorageService.get_download_url()` (15 min default; 30 min for OCR); `ensure_company_blob()` refuses to sign a path under another company's prefix.

## Invariants

- **Originals are write-once.** `StorageService` has `upload()` and `download_bytes()` but no overwrite/delete path. Every later stage only *reads* the original.
- **`file_hash` is the integrity anchor.** It is recomputed and compared when a report is generated ( `HashVerification` in the report appendix).

- **Byte-identical re-uploads are not blocked.** `file_hash` is indexed but there is no uniqueness constraint; exact duplicates are caught by the *perceptual* check (10), not at upload.

## Known limitations

- No page-count limit and no antivirus scan (type, size, corruption and password checks exist; see above).
- Any company user can create a case, but only its owner or a reviewer of the company can add to it.
- Documents stored before the PDF-only rule may be images; they were OCR'd but no forensic check runs on them, and `pipeline_status` only requires forensic checks for PDFs, so such a case can still be scored.
- One task failing to enqueue (Redis down) after the DB commit leaves a `pending` document that never progresses; there is no sweeper.
- If every file of a new case is rejected, the case itself (created first) remains, with no documents.

## Bulk upload

A zip of many cases goes through the same validation and the same store → record → queue code ( `services/document_intake.py` ), one case at a time. See 01a.

## Risk rules fed

None directly. It produces the documents every rule evaluates.

# 1a. Bulk upload (a zip of cases)

**Code:** `backend/app/api/bulk_uploads.py` (endpoints), `backend/app/services/bulk_upload_service.py` (zip rules, plan, ingestion), `backend/app/tasks/bulk_upload_task.py` (Celery task), `backend/app/services/document_intake.py` (the store → record → queue path shared with single uploads). **Screens:** `/cases/bulk` (upload) and `/bulk-uploads/:id` (live summary).

Many cases in one go: one zip file, **one top-level folder per case**, with that case's documents directly inside the folder. The zip is only a way to get cases in. Once its cases are created and their documents queued, its job is done. Every document then goes through exactly the same per-document pipeline as a single upload (01 onwards), with the same fair-share priority. Nothing in the pipeline reports back to the zip or waits for it.

```
claims.zip
├── case-001/   invoice.pdf  receipt.pdf
├── case-002/   فاتورة.pdf
└── case-003/   quotation.pdf
```

## Limits

| Limit | Default | Setting |
|---|---|---|
| Zip size | **per company**: `companies.max_zip_size_mb` (300 MB for a new company) | Platform › Companies › Edit; new-company default `DEFAULT_MAX_ZIP_SIZE_MB` |
| Each document | **per company**: `companies.max_file_size_mb` (10 MB for a new company), PDF only, plus every single-upload check | Platform › Companies › Edit; new-company default `DEFAULT_MAX_FILE_SIZE_MB` |
| Zip entries (files + folders) | 25,000, a safety cap against zips of millions of tiny entries (a 300 MB zip of small born-digital PDFs holds about 11,000) | `BULK_UPLOAD_MAX_ENTRIES` |
| Cases per zip | **no hard cap**; a soft warning above 100 | `BULK_UPLOAD_CASE_WARNING_THRESHOLD` |

Both size limits are read from the uploading company's row (`services/upload_limits.py`) by the upload request and again by the ingestion task, the same source the single-file upload uses, and every rejection names the limit in effect for that company. The frontend shows the company's limits (from `GET /auth/me/upload-limits`), checks them before sending, and counts the case folders from the zip's central directory to show the warning.

## Two stages

**1. The upload request** (`POST /bulk-uploads`, 202) returns as soon as the zip is stored.

- The body is the zip itself (`Content-Type: application/zip`), not multipart. A zip over the limit is refused **from its `Content-Length` before any of the body is read**. A body without a length is cut off as soon as it passes the limit.
- The body is streamed to a temporary file (it spills to disk above 8 MB) and hashed on the way.
- Only the **central directory** is read. No document is decompressed. Zip-level rejections, each with `detail: {code, message}`:

| Code | HTTP | When |
|---|---|---|
| `zip_empty` | 400 | 0 bytes |

| Code | HTTP | When |
|---|---|---|
| `zip_too_large` | 413 | over the company's zip limit (states the actual size and that limit) |
| `not_a_zip` | 415 | the header bytes are not a zip |
| `zip_corrupted` | 422 | the central directory can't be read (truncated or damaged) |
| `zip_too_many_entries` | 422 | over `BULK_UPLOAD_MAX_ENTRIES` |
| `zip_no_case_folders` | 422 | no folder holds any file |

- It builds the **plan**: one `bulk_upload_cases` row per case folder. Every failure that needs no decompression is already marked, so the uploader sees it in the response, before anything is processed: a subfolder inside a case folder, an empty case folder, and 0-byte, oversized or zip-password-protected files.
- It stores the zip ( `companies/{company_id}/bulk-uploads/{id}_{sha256}.zip` ), writes the `bulk_uploads` row and its plan rows plus a `bulk_upload_received` audit event, counts the zip's storage, and queues **one** `ingest_bulk_upload` task.

**2. Ingestion** ( `ingest_bulk_upload` , on `forensics_queue` : local CPU, no Azure call) goes case by case, in natural folder order ( `case-2` before `case-10` ):

1. Extract each document of the folder, never reading more than the company's per-file limit + 1 byte whatever the zip claims (zip-bomb safe). Then run the **same checks as a single upload** ( `validate_upload` : size, PDF header bytes, extension, parse, password).
2. Store the accepted documents in Blob Storage like any single upload.
3. In **one short transaction**: create the case (submitted by the uploader, case type from the upload, `bulk_upload_id` , `reference_label` = folder name), its documents, their audit events and usage counters, and the folder's `bulk_upload_cases` row.
4. **Queue that case's documents immediately**, before the next folder is even extracted. The first case can be processed, scored and even reviewed while later cases are still being ingested.

No DB transaction is held across the zip download, extraction, validation or blob uploads.

## Partial success

| Situation | Result |
|---|---|
| One bad file in a case (corrupted, password-protected, too large, empty, not a PDF, wrong extension) | That file is rejected with the single-upload code and message. The rest of the case goes ahead |
| Every file in a case rejected | No case is created: `case_no_valid_documents` |
| A case folder contains a subfolder holding files | The whole folder is rejected ( `case_nested_folder` , naming the subfolder). Files are never guessed into a case. An *empty* subfolder is ignored |
| An empty case folder | `case_no_documents` |
| Other case folders | Unaffected |

## Folder and file names

- **Non-ASCII names (Arabic etc.)** are read correctly whether the zip tool sets the UTF-8 flag, writes UTF-8 *without* the flag (common, and shown as mojibake by naive readers), or uses the Info-ZIP Unicode Path extra field. A legacy zip whose names are in a regional code page (e.g. CP720 from an old Windows "Send to → Compressed folder") can't be detected reliably and keeps its CP437 reading.
- **Folder names are labels, not identities.** Two folders that look alike ( `Invoice` and `invoice` ), or the same folder names in a later zip, become separate cases. Identity is always the generated case id / case number. Names are NFC-normalised, so the same name typed on macOS and on Windows is one folder.
- **One wrapper folder** around everything (right-click → Compress on a folder of case folders: `claims/case-001/…` ) is detected and stripped once. The summary screen says so.
- **OS clutter** ( `__MACOSX/` , `.DS_Store` , `Thumbs.db` , `desktop.ini` , dotfiles) is skipped. Loose files at the zip root belong to no case and are listed as ignored.

## Summary screen

`GET /bulk-uploads/{id}` lists every case folder: created or failed (and why, per file) and, for created cases, a **live status** derived on every read from the case and its documents:

`validating` (not ingested yet) → `queued` (no document picked up by a worker) → `processing` → `done` , or `flagged` (medium/high risk tier). A submitter sees `done` instead of `flagged` and never sees the tier, the same rule as the case queue. The screen polls every 3 s until `settled` .

Visibility follows the case rules: a `user` sees only their own uploads, reviewers see their company's, and platform admins get read-only, audited access ( `company_id` query parameter). Every company role may bulk upload; platform admins may not (403).

## Resuming after a crash

Each case is committed together with its `bulk_upload_cases` row, so a redelivered task (the worker died) or a retried one (Blob Storage hiccup: up to 3 retries) skips folders already handled and never duplicates a case. If ingestion fails for good, the upload is marked `failed` with a message. Cases already created stay created and keep processing.

## Sizing (measured)

`python -m scripts.bulk_upload_benchmark --profile scanned|digital` builds a zip close to 300 MB and times both stages (in-memory storage and SQLite, so this is the CPU floor):

| Profile | Cases / documents in ~295 MB | Upload request (inspect) | Ingestion | First document queued |
|---|---|---|---|---|
| Scanned (2 × ~2 MB per case) | 73 / 146 | 21 ms | 3.9 s | 0.8 s |
| Born-digital (3 × ~40 KB per case) | 2,744 / 8,232 | 185 ms | 132 s (≈ 48 ms per case) | 1.4 s |

A real 300 MB zip therefore holds anything from about 70 cases (scanned evidence) to about 3,000 (born-digital invoices). Unzipping and validating in the request would hold it for seconds to minutes, which is why ingestion is a background task.

## Known limitations

- Every case in a zip gets the same case type.
- The zip is kept in Blob Storage as the record of the submission (and counted in the company's storage). There is no retention or cleanup job.
- A worker crash between storing a case's documents and committing the case leaves orphaned blobs, the same as a single upload that fails after storage.
- Ingestion of one zip is sequential on one `forensics_queue` worker. Very large born-digital zips take minutes to ingest fully, while their first cases are already processing.

# 02 — Extraction & normalization

**What it does:** turns an uploaded PDF into structured, comparable data — a document type and a set of normalized fields — with no per-document-type template.

**Where it runs:** `process_document` is on `extraction_queue`, in a session bound to the document's company. Both Azure calls pass through the global Redis rate limiter: Document Intelligence at `AZURE_DOCUMENT_INTELLIGENCE_MAX_CALLS_PER_SECOND` (10), the Azure OpenAI call within `AZURE_OPENAI_MAX_REQUESTS_PER_MINUTE` / `AZURE_OPENAI_MAX_TOKENS_PER_MINUTE` (each call reserves its estimated prompt tokens + `max_tokens` 4,000). The task commits before each slow step, so it never holds a database transaction across an Azure call (processing-queues.md).

**Code:** `tasks/document_processing.py` ( `process_document` ), `services/ocr_service.py` , `services/classification_service.py` , `services/extraction_service.py` , `services/llm_service.py` , `services/field_locator_service.py` .

## Algorithm

1. **Status:** `documents.processing_status = processing` .
2. **OCR / layout.** `OCRService.analyze_url(sas_url)` calls Azure Document Intelligence **`prebuilt-layout`** on a 30-minute SAS URL (Azure fetches the file itself). Printed Arabic is supported; **handwritten Arabic is not reliable** (an Azure limitation). Returns an `OCRResult` :

   ```
   @dataclass OCRResult:
       text: str                              # full text, reading order
       tables: list[OCRTable]                 # row_count, column_count, cells
       key_value_pairs: dict[str, str]
       pages: list[OCRPage]                   # page_number, words[], lines[]
   @dataclass OCRWord: text, x, y, width, height   # normalized 0-1 page fractions
                       font_family, font_confidence, italic_or_handwritten, confidence  # word-level
   extras
   ```

3. **Classify + extract in one call.** `extraction_service.classify_and_extract(llm, ocr_text)` → `LLMService.classify_and_extract` , an Azure OpenAI chat completion at `temperature=0` with **strict JSON-schema output**, returning a `DocumentAnalysis` (below). The label list is passed in from `classification_service.DOCUMENT_TYPE_LABELS` , so adding a type is a one-line change.
4. **Locate fields on the page (best effort).** `field_locator_service.attach_field_locations(pages, extracted_fields)` matches each extracted value back onto the OCR words and stores a normalized `bounding_box` on the field. It is text matching, so the position is approximate; ambiguous values (a total that also appears in a summary line) are disambiguated by nearby keywords ("total", "subtotal", "VAT", "date"); a field that cannot be found simply has no box. **A failure here never fails extraction.** Then the deterministic post-processing ( `line_item_parsing.enrich_extracted_fields` , which is idempotent): line items read from the OCR layout, `pdf_info` (Producer, Creator, CreationDate, ModDate, and `image_count` — raster images the pages draw), and the clean-up in `services/extraction_postprocess.py` :
   - **Multi-column tables** ( `parse_multi_column_tables` ). A table with separate amount columns (Price / Discount / Net / VAT / Total) is read column by column: each row keeps its value per column in `columns` (OCR's "D.DD" read as 0.00, "131,900,00" as 131,900.00), `line_total` is the Net (else Total) column, and the column total rows ("Total Net Price", "VAT", "Total Price") go to `column_totals` . These rows replace the model's lines for the same rows — the model mixes columns. Field validation sums each column on its own (03).

- **Form template date.** A "Date of issue" that sits beside document-control markers (Rev. No, Issue No, Document/Form No — at least two) is the date the blank *form* was issued, not the document's date. It is labelled `form_template_date` (ISO `value`, `raw_text`, a `note`; the model's own name kept as `field_name_as_extracted`). A plain "Date of issue" (a certificate's) is left alone.
- **Signature ink read as a name.** OCR reads a signature as a word ("Prepared by: Princess" + the scribble read as "Dalab"). In person-role fields (prepared/signed/approved by, principal, …), words that OCR marked handwritten **and** set ≥ 1.8× the page's median word height, or that lie inside a detected signature region (when detection has already finished), are removed. The original stays in `value_as_read`.
- **Several invoices in one file** (`services/multi_invoice.py`). A page whose invoice number differs from the previous page's starts a new invoice. With two or more, each invoice's pages — the PDF's own text layer, else OCR (a free-tier OCR reads only the first two pages) — are sent to the extraction model separately and kept in `extracted_fields.invoices` (`page`, `pages`, `invoice_number`, `core_fields`, `line_items`). The document-level fields stay the first invoice's.

5. **Checks that need the OCR words** run here, before the document is marked complete, each best effort (its row is written inside a savepoint; a failure is stored as a failed check and never fails the extraction): font consistency (06a, PDFs only) and spelling (06b, English words, every document).
6. **Persist.** `document_type`, `ocr_text`, `extracted_fields`; `processing_status = complete`; audit `document_processing_completed` `{document_type, document_type_confidence, fields_located}`; then `run_document_checks.delay(document_id, company_id)` (on `vision_queue`).
7. **On any exception:** `processing_status = failed`, `processing_error` (≤ 4000 chars), audit `document_processing_failed`, and the case-level completion check and scoring are still triggered so a failed document cannot block the case forever.

## Labels

`school_document, vendor_invoice, commercial_invoice, procurement_documentation, quotation, travel_invoice, payment_evidence, other`.

`get_document_role()` maps them to a **role** used by the cross-document check: `payment_evidence` → `evidence`, `other` → `other`, anything else → `claim`, `None` → `pending`. (A claim and its own evidence are *expected* to differ on issuer and date.) The frontend mirrors this in `types/case.ts`'s `getDocumentRole`; the two are kept in sync **by hand**.

## Output shape

The LLM returns a `DocumentAnalysis` (Pydantic, `services/llm_service.py`):

```
class DocumentAnalysis:
    document_type: str
    document_type_confidence: float
    issuer: FieldValue            # value: str|None, confidence: float, uncertain: bool
    reference_number: FieldValue
    date: DateFieldValue          # value = ISO 8601 (YYYY-MM-DD) or None; raw_text kept
    amount: AmountFieldValue      # value: float|None, currency: str|None, raw_text
    subtotal: AmountFieldValue
    tax_amount: AmountFieldValue
    tax_rate: NumericFieldValue
    additional_fields: list[AdditionalField]   # {field_name, value, confidence, uncertain}
```

Amounts are **normalized at extraction time** to plain floats (Western digits, no separators or symbols) with a separate ISO currency code, and dates to ISO 8601, regardless of the document's own script or numeral system — so an Arabic-

Indic-numeral invoice and a Western-numeral receipt compare directly. `raw_text` keeps what was actually seen, for display only; **logic must read `.value`, never `.raw_text`**.

What is stored in `documents.extracted_fields` (JSONB):

```
{
  "document_type_confidence": 0.97,
  "core_fields": {
    "issuer":           {"value": "Northgate Engineering", "confidence": 0.95, "uncertain": false,
                         "bounding_box": {"page":1,"x":0.05,"y":0.06,"width":0.30,"height":0.02}},
    "reference_number": {"value": "INV-NGT-2026-1049", "confidence": 0.99, "uncertain": false,
"bounding_box": {…}},
    "date":             {"value": "2026-08-25", "raw_text": "٢٥ أغسطس ٢٠٢٦", "confidence": 0.9,
"uncertain": false, …},
    "amount":           {"value": 197500.0, "currency": "AED", "raw_text": "197,500.00", …},
    "subtotal": {…}, "tax_amount": {…}, "tax_rate": {…}
  },
  "additional_fields": [
    {"field_name": "Payment terms", "value": "30 days", "confidence": 0.8, "uncertain": false,
"bounding_box": {…}}
  ]
}
```

`additional_fields` is how the pipeline stays template-free under Azure's strict-schema rule (which cannot express an open-ended object): arbitrary fields are a list of `{field_name, value}` pairs.

## Known limitations

- **Low-confidence routing is not implemented.** `uncertain` is shown on the case detail page but nothing acts on it.
- The extraction model sees **text only** (OCR output), never the page image, so it cannot use visual layout cues beyond what OCR text order conveys.
- Only the first attempt runs; there is no retry of a failed extraction.
- Field locations are approximate and documents extracted before the locator existed have none (`backend/backfill_field_locations.py` re-runs OCR only, no LLM).
- LLM output is validated against a JSON schema but its *content* is not verified against the source.

## Risk rules fed

None directly. Extraction feeds 03, 04 and 05.

# 03 — Field validation

**What it detects:** internally inconsistent documents — the kind of arithmetic or date mistake that falsified or hand-edited documents often contain. It is **pure rule logic over one document's own extracted fields**: no I/O, no model, no database.

**Code:** `services/field_validation_service.py` ( `validate_fields` ), wrapped by `tasks/document_checks.py` ( `run_document_checks` ), which stores a `document_checks` row with `check_type = field_validation` . Highlight regions come from `services/field_regions.py` and `services/field_exception_service.py` .

## Sub-checks

`validate_fields(extracted_fields) -> {"result": "pass"|"flag", "details": {...}}` runs these independent sub-checks; the overall result is `flag` if **any** sub-check is `flag` .

| Sub-check | Flags when | Skipped when |
|---|---|---|
| `date_in_future` | the document's `date` core field is after today | no parseable date |
| `line_item_arithmetic` | a line's quantity × unit price (− its discount) ≠ the line total printed | no line with quantity, unit price and total |
| `subtotal_line_item_consistency` | the line items that count toward the total don't add up to the `subtotal` — or, with no subtotal shown, to the `amount` (less `tax_amount` when tax is shown). Not summed: rows that are totals themselves ("Total Fees", "Grand total", "Subtotal", "Amount due"), and installment rows ("Term I/II/III", "Installment 1/2", "Payment 1") that split another line — their sum equals that line (CASE-346FA23E: Term I + II + III = Tuition Fees, so the payable items are 38,834 against a total of 34,012) | no line totals, or no subtotal/total |
| ( `subtotal_line_item_consistency` , multi-column tables) | when the line items come from a table with separate amount columns (Price / Discount / Net / VAT / Total — `line_item_parsing.parse_multi_column_tables` , which keeps each row's values per column and reads OCR's "D.DD" as 0.00 and "131,900,00" as 131,900.00), **each column is summed on its own** and compared with its own total row: Net with "Total Net Price" (else the subtotal), VAT with "VAT" (else the tax amount), Total with "Total Price" (else the total); a Net column with no Total column beside it is compared with the total. Never one row's Net Price with another's Total Price (CASE-D87E2E82: 120,007.50 / 6,000.37 / 126,007.87, all matching). When only the VAT column can be compared, the plain check runs instead | — |
| `installment_consistency` | two or more installment rows come within 20 % of another line (the largest, at least as big as each installment) without adding up to it — a part of the schedule was changed. Passes, with the sum spelled out, when they add up exactly | no installment rows splitting another line (terms that are the charges themselves are not installments) |
| `date_sequence` | the dates are out of order: the page was printed (browser print header, first line: "8/18/23, 12:54 PM"; month/day first, day/month when that is impossible) more than a day before its own date, or the PDF was created more than a day before the page was printed. Passes with the sequence ("dated 17 Aug 2023 → printed 18 Aug 2023 → PDF created 21 Aug 2023: in order") | no print header |

| Sub-check | Flags when | Skipped when |
|---|---|---|
| `fake_scan_watermark` | a scanner app's line ("Scanned with CamScanner", "Adobe Scan", Microsoft Lens…) on a file with **no raster image** ( `pdf_info.image_count` = 0): nothing was scanned, the line was typed in | no watermark; image count not recorded (documents extracted before it was) |
| `stamp_authenticity` | a detected stamp is live text and/or vector lines in the file ( `material` recorded by signature/stamp detection — `services/forensics/pdf_facts.py` ), not an image of an ink stamp. `synthetic_stamp_unsigned` : such a stamp and no signature anywhere | no stamp; detection not finished; material not recorded |
| `tax_invoice_trn` | titled TAX INVOICE / فاتورة ضريبية but no TRN shown: no labelled number (TRN, Tax Registration, VAT No, الرقم الضريبي; Western or Arabic-Indic digits) and no UAE TRN (100…) anywhere. Whether a shown TRN is valid is `iban_trn_validation` 's question | not titled a tax invoice |
| `multiple_invoices` | the file holds several invoices ( `services/multi_invoice.py` : a new invoice number starts a new invoice; each invoice's pages — text layer first, OCR for scanned pages — extracted on their own into `extracted_fields.invoices` ) and two are dated in the same month | one invoice in the file |
| `per_invoice_checks` | an invoice of a multi-invoice file fails its own line-item / subtotal / total-and-tax / tax-rate check. A kind of problem the document's own sub-check already flags is listed but not flagged again | one invoice in the file |
| `total_tax_consistency` | `amount` ≠ `subtotal + tax_amount` (or ≠ `subtotal` when no tax line is shown) | no subtotal |
| `tax_rate_consistency` | `tax_amount` ≠ taxable base × `tax_rate` (base = subtotal, or total less tax) | no rate or no tax amount |
| `amount_in_words_consistency` | the amount written in words ≠ the total (or the subtotal) in figures | no amount in words, or it can't be read |
| `reference_number_format` | the reference number is not a plausible identifier (regex `^[A-Za-z0-9](?:[A-Za-z0-9\-/_. #]*[A-Za-z0-9])?$` ) | — |
| `stamp_issuer_consistency` | the text read on the document's stamp does not name the issuer (rapidfuzz `token_set_ratio` < 80, per writing system, with addresses, P.O. boxes, branch numbers, legal forms and UAE place names left out, abbreviations expanded — MBZ = Mohamed Bin Zayed, MBR = Mohammed Bin Rashid, Intl = International, Pvt = Private — and generic words dropped: School, Campus, LLC, Branch, Private) | detection not finished; no stamp text read; no issuer; stamp and issuer share no script |

| Sub-check | Flags when | Skipped when |
|---|---|---|
| `document_date_vs_file_creation` | the printed `date` is more than 1 day after the PDF's own CreationDate **and** the file was modified on or after that date (an older file edited and re-dated). A file created before its date and never modified since — generated in advance — passes | no date; no CreationDate in `pdf_info` (documents extracted before it was recorded); a CreationDate at or before 2000-01-01 (unset clock) |

## Calculation checks

Extraction returns structured `line_items` (description, quantity, unit price, discount, line total, and whether the line counts toward the total) and `amount_in_words` (the words and the number they spell), stored in `documents.extracted_fields`. The model is told to copy every number **exactly as printed and never recompute** one: the discrepancies are the point. On a real scanned invoice it returned `3 × 6,000 = 16,600` as printed, and all three errors on that page were flagged.

- **Amounts are compared to the cent** (one cent of rounding per term, plus half a cent per unit when a unit price is multiplied out), not by a percentage. A 1 % allowance on a 55,500 total would hide a 400 discrepancy.
- **Excluded lines:** a line the document marks as already paid or informational doesn't count toward the total. If including it is what makes the sum match, the check passes rather than flagging the model's judgment.
- **Amount in words:** English words are read by code ( `services/amount_words.py` ), not taken from the model, which tends to "correct" words that contradict the figures. Other languages (Arabic) use the model's reading. Words that leave out the fils/cents still match.
- Each wrong line total and the amount-in-words line are located on the page ( `services/field_locator_service.py` ) and drawn as field-exception boxes.

Line items are also read deterministically from the OCR layout where the model is unreliable ( `services/line_item_parsing.py` ): one-row fee tables, `<rate> PER <unit> X <qty> = <total>` lines, and "Description | Amount" tables whose rows OCR stacked into one cell ( `12,000 9,000 9,000 6,000` beside four description rows, or several descriptions in one cell, one per line) — the n-th description is paired with the n-th amount, only when the counts agree.

Documents extracted before structured line items existed fall back to the old heuristic for `subtotal_line_item_consistency` : additional fields whose name suggests a per-line total (name contains `item` , value hint `total` / `amount` ), compared with the subtotal only.

## Output shape (real example)

```
{
  "result": "flag",
  "details": {
    "date_in_future": {"status": "pass", "reason": "Document date 2026-08-25 is not in the future."},
    "total_tax_consistency": {
      "status": "flag",
      "reason": "Total 197500.00 does not match subtotal 180000.00 + tax 9000.00 (expected 189000.00).",
      "regions": [
        {"field": "amount", "label": "Total amount", "value": "197,500.00 AED",
         "caption": "Field exception: Total amount (197,500.00 AED vs expected 189,000.00)",
         "bounding_box": {"page": 1, "x": 0.7821, "y": 0.5081, "width": 0.1321, "height": 0.0119}}
      ]
    },
    "reference_number_format": {"status": "pass", "reason": "…"},
    "subtotal_line_item_consistency": {"status": "pass", "reason": "Subtotal matches the sum of 3 line-item
field(s)."}
  }
}
```

Unlike the forensic checks, `details` is a **dict keyed by sub-check**, not a list of findings. Each sub-check is `{status, reason, regions?}` where `status` is `pass`, `flag` or `skipped` (no value to check, or unparseable); `regions` (only on `flag`) are drawn as **purple field exception** boxes over the PDF. If a field's position could not be located, the exception is still reported without a box. Rows stored before locations existed are re-derived at read time (`with_field_regions`) without modifying the stored row.

## Known limitations

- Depends entirely on extraction quality; a wrong extracted value produces a wrong flag (or a miss).
- Amounts must agree to the cent (see above). A document that rounds each line differently from its printed total by more than a cent per term will flag.
- Line items and the amount in words come from the extraction model. A line it fails to list, or a number it misreads, produces a miss or a wrong flag; the reason text always shows the numbers used.
- `date_in_future` compares against `date.today()` on the worker's machine, not the case's submission date, so a worker with a wrong clock can mis-flag.

## Risk rules fed

| `rule_id` | Sub-check | Weight | Severity |
|---|---|---|---|
| `field.date_in_future` | `date_in_future` | 25 | high |
| `field.line_item_arithmetic_mismatch` | `line_item_arithmetic` | 25 | high |
| `field.subtotal_line_item_mismatch` (v3: totals and installments not summed) | `subtotal_line_item_consistency` | 15 | medium |
| `field.installment_sum_mismatch` | `installment_consistency` | 15 | medium |

| rule_id | Sub-check | Weight | Severity |
|---|---|---|---|
| field.date_sequence_inconsistent | date_sequence | 15 | medium |
| field.fake_scan_watermark | fake_scan_watermark | 25 | high |
| signature.synthetic_stamp | stamp_authenticity | 25 | high |
| signature.synthetic_stamp_unsigned | synthetic_stamp_unsigned | 5 | low |
| field.tax_invoice_without_trn | tax_invoice_trn | 15 | medium |
| field.multiple_invoices_same_period | multiple_invoices | 10 | medium |
| field.per_invoice_mismatch | per_invoice_checks | 15 | medium |
| field.total_tax_mismatch | total_tax_consistency | 20 | high |
| field.tax_rate_mismatch (v3: rate read from the printed "VAT (5%)" label when not extracted; a rate above 0 with 0.00 tax on a non-zero base is flagged — no longer "explained" by a 0.00 line) | tax_rate_consistency | 15 | medium |
| field.amount_in_words_mismatch | amount_in_words_consistency | 25 | high |
| field.reference_number_malformed | reference_number_format | 6 | low |
| metadata.document_date_after_file_creation | document_date_vs_file_creation | 25 | high |
| signature.stamp_issuer_mismatch (v2: names normalised) | stamp_issuer_consistency | 15 | medium |

(Weights are the template defaults; each company's Reviewer L2s can change them for their company — see 11.)

# 04 — Cross-document consistency

**What it detects:** documents in the same case that disagree on facts they should share — most importantly, an invoice total that does not match the payment that supposedly settled it.

**Code:** `services/cross_document_service.py` ( `find_cross_document_mismatches` ), run by `tasks/document_checks.py` ( `run_cross_document_checks` ). Persisted to `cross_document_findings` (a **case-level** table — `document_checks` has no case concept).

## When it runs

`run_document_checks` (and the failure path of `process_document` ) calls `_maybe_enqueue_cross_document_check` , which enqueues `run_cross_document_checks` once **every** document in the case has a terminal `processing_status` ( `complete` or `failed` ). The check itself:

1. loads the case's documents and keeps only those that are `complete` ;
2. **deletes any existing findings for the case** and recomputes (this table is *not* append-only, unlike `audit_log` ), so a re-run cannot accumulate stale rows;
3. if fewer than **2** completed documents exist, writes no findings;
4. writes one `cross_document_findings` row per mismatch, audit event `cross_document_check_completed {finding_count}` (written even when zero), then requests scoring.

`pipeline_status` waits for a `cross_document_check_completed` event newer than the newest upload before a multi-document case can be scored.

## Algorithm

Every unordered pair of completed documents ( `itertools.combinations` ) is compared on three fields, reading the **normalized** `.value` s (see 02):

| Field | Mismatch when | Notes |
|---|---|---|
| `amount` | currencies are both present and differ; **or** `abs(a − b) > max(0.01, 1% of the larger)` | Skipped if either side has no amount |
| `date` | both parse to a date and the dates differ | ISO dates from extraction; fallback formats exist |
| `issuer` | `rapidfuzz.fuzz.WRatio` (with `default_process` ) < **85** | Case/punctuation-insensitive; skipped if either name is empty |

### Severity depends on the pair's roles

`get_document_role()` classifies each document as `claim` (invoice/quotation/…), `evidence` ( `payment_evidence` ) or `other` . A claim and its own evidence are **expected** to differ on issuer (vendor vs. bank) and date (invoice date vs. payment date), so:

| Field | claim ↔ evidence | any other pair |
|---|---|---|
| `amount` | **high** (always) | **high** |
| `date` , `issuer` | **low**, with an "expected — treat as context" sentence appended | **medium** |

The scoring rules only fire on `medium` / `high`, so an expected claim-vs-evidence date/issuer difference stays visible to a reviewer but adds no risk points.

## Output shape

A row in `cross_document_findings` (model `models/cross_document_finding.py`):

```
{
  "case_id": "uuid",
  "field_name": "amount",                          // amount | date | issuer
  "finding_type": "cross_document_consistency",
  "severity": "high",                              // low | medium | high
  "description": "Amount differs between 'Invoice.pdf' (9030.00 USD) and 'Evidence.pdf' (7250.00 USD).",
  "document_ids": ["<uuid a>", "<uuid b>"]
}
```

The API returns these as `CrossDocumentFindingSummary`, and derives **one highlight region per involved document** at read time (`field_exception_service.cross_document_regions`), each captioned with what the *other* document said, so the reviewer sees both sides without switching pages.

## Known limitations

- Only three fields. Other shared facts (reference numbers, line items, tax IDs) are not compared.
- Pairwise on *all* pairs: a case with three inconsistent documents yields three findings per field.
- A document that failed extraction is excluded, so a case whose only mismatching document failed will not show the mismatch. `pipeline_status` still lets it score.
- Amount tolerance is a flat 1%; there is no per-currency rounding logic and no currency conversion (a currency mismatch is reported, not reconciled).

## Risk rules fed

| `rule_id` | Field | Fires on severities | Weight | Severity |
|---|---|---|---|---|
| `cross_doc.amount_mismatch` | `amount` | medium, high | 35 | high |
| `cross_doc.date_mismatch` | `date` | medium, high | 12 | medium |
| `cross_doc.issuer_mismatch` | `issuer` | medium, high | 15 | medium |

A case-level rule fires **once per case**, with `count` = number of matching findings.

# 05 — Issuer verification

**What it detects:** a document issued by someone who is not a known, active vendor/school/issuer — including fabricated or look-alike issuer names.

**Code:** `services/issuer_service.py` (`match_issuer`), `models/issuer_registry.py`, run by `tasks/document_checks.py` (`run_document_checks`), which stores a `document_checks` row with `check_type = issuer_verification`. Registry administration: `api/settings.py` `/settings/issuers` and the *Settings › Issuer Registry* page (a company's Reviewer L2s; platform admins on a company's behalf).

**The registry is per company.** Each company maintains its own list of known issuers; a document is only ever matched against **its own company's** registry (`issuer_registry.company_id`, enforced by the tenant-bound session and Row-Level Security).

## Algorithm

`match_issuer(db, company_id, issuer_name, threshold=None, llm_service=None) -> IssuerMatchResult`:

1. Empty/blank issuer name → no match.
2. Load every **active** `issuer_registry` row **of the document's company** (deactivated issuers no longer count as known). If none → no match.
3. Build candidate strings: each row contributes its `name` **and**, if present, its `name_arabic`, so an Arabic-script extracted name can match a row whose primary name is English.
4. **Fuzzy pass:** `rapidfuzz.process.extractOne(issuer_name, candidates, scorer=fuzz.WRatio, processor=utils.default_process)` — case-, punctuation- and whitespace-insensitive. Match if the best score ≥ threshold (`ISSUER_FUZZY_MATCH_THRESHOLD`, default **85.0**).
5. **LLM fallback** (only if the fuzzy pass fails and an `LLMService` was passed — the production task passes one): `judge_entity_match(name, candidates)` asks Azure OpenAI whether the extracted name plausibly refers to the same real-world entity as any candidate — this covers transliteration and cross-script cases rapidfuzz cannot score. The model must return one of the candidates **verbatim** or `null`. An LLM failure or missing configuration is swallowed and reported as the fuzzy-only result. The call runs on `vision_queue` through the global Azure OpenAI rate limiter (`max_tokens` 512).

```
@dataclass IssuerMatchResult:
    matched: bool
    best_match_name: str | None
    best_match_score: float | None       # rapidfuzz score, rounded to 2 dp
    threshold: float
    matched_via: Literal["fuzzy", "llm"] | None
```

## Output shape (real example)

`document_checks.result` for an issuer that did not match:

```
{
  "result": "flag",                                // pass | flag
  "details": {
    "issuer_name": "بنك أبوظبي الأول | First Abu Dhabi Bank",
    "threshold": 85.0,
    "match_score": 27.94,
    "matched_via": null,                           // "fuzzy" | "llm" | null
    "matched_registry_name": "Al Nukhba Technical Systems Est."   // the closest, even when not a match
  }
}
```

`details` is a **flat dict**, not a findings list. `matched_registry_name` names the closest registry entry even when the check flags, so a reviewer can see what it was nearest to.

## Registry data

`issuer_registry` columns: `company_id`, `name`, `name_arabic`, `tax_id`, `type` (enum), `is_active`. Registry actions (create / update / deactivate / reactivate) each write an audit event in the company's log; **rows are never deleted**. A **new company's registry starts empty** — its Reviewer L2s fill it in. The migrations seeded a handful of **fake** test issuers into the "Default Company" created when the single-tenant database was migrated. Names must be unique case-insensitively among a company's active issuers (409 otherwise).

## Known limitations

- **A new company's registry is empty, and the Default Company's seeded one is fake.** Until a company's registry holds its real vendors/schools, almost every one of its documents will flag.
- **Names only:** `tax_id` is stored and editable but not compared to the extracted document.
- The LLM fallback is a judgment call that can be wrong in either direction, and it can only choose among registry strings — it cannot verify that the issuer exists in the real world.
- No logo/letterhead matching.
- A single extracted issuer string is checked; multi-party documents (issuer + bank) may extract the "wrong" party as the issuer.

## Risk rules fed

| `rule_id` | Condition | Weight | Severity |
|---|---|---|---|
| `issuer.not_in_registry` | `issuer_verification` result = `flag` | 15 | medium |

# 06 — Metadata forensics

**What it detects:** signs a PDF was edited, re-exported or assembled after it was created: editing software fingerprints, inconsistent or missing timestamps, incremental-save revisions, XMP edit history that contradicts the file, orphaned objects, embedded JavaScript, and a stripped metadata block.

**Code:** `services/forensics/metadata_forensics.py` ( `analyze_pdf_metadata` ), run by `tasks/metadata_forensics_task.py` ( `run_metadata_forensics` ); stores `check_type = metadata_forensics` .

## What is read

Using **pikepdf** on the PDF's own object model (never a page render):

- the **Info dictionary** ( `/Title /Author /Creator /Producer /CreationDate /ModDate` );
- the separate **XMP metadata stream**, including custom namespaces and the `xmpMM:History` revision chain;
- the **document ID chain** ( `/ID` );
- the file's real **incremental-update / xref-revision structure** ( `_walk_xref_chain` );
- **reachability**: which objects are unreachable from `/Root` / `/Info` (orphans);
- JavaScript / `OpenAction` , optional content groups, and digital-signature fields.

## Findings

Every finding is `{finding, severity, description, data?}` . `severity` is `info | low | medium | high` . `info` findings are context (they show the raw metadata) and never affect the result.

| `finding` | Severity | Meaning |
|---|---|---|
| `pdf_unreadable` | high | The file could not be opened as a PDF |
| `metadata_entirely_stripped` | high | No Info and no XMP metadata at all |
| `editing_software_detected` | high | Producer/Creator/ `softwareAgent` matches an image/graphics editor ( `METADATA_FORENSICS_EDITING_SOFTWARE_NAMES` : photoshop, gimp, illustrator, affinity, coreldraw, paint.net, pixlr, canva, inkscape) |
| `pdf_editor_detected` | medium | Producer/Creator names an online or desktop PDF editor ( `METADATA_FORENSICS_PDF_EDITOR_NAMES` : iLovePDF, Smallpdf, Sejda, PDFescape, PDF-XChange Editor, PhantomPDF / Foxit PDF Editor, Nitro Pro, PDFelement, Soda PDF, pdfFiller, DocHub, PDF Candy). One finding per tool. Print drivers of the same vendors are not listed |
| `mod_date_after_creation_date` | high | ModDate later than CreationDate by more than `METADATA_FORENSICS_MOD_DATE_THRESHOLD_SECONDS` (default 1 s — only jitter tolerance; being modified at all is the anomaly) |
| `info_xmp_creation_date_mismatch` | high | Info and XMP creation dates differ by > 1 minute |

| finding | Severity | Meaning |
|---|---|---|
| info_xmp_mod_date_mismatch | high | Info and XMP modification dates differ by > 1 minute |
| info_xmp_producer_mismatch | medium | Info /Producer ≠ XMP producer |
| incremental_updates_present | medium (1 update) / high (> 1) | The file has been saved incrementally |
| history_editing_tool_anomaly | medium | An XMP history entry's softwareAgent names an image editor or a PDF editor |
| history_scanned_document_edited | high | An XMP history entry's action is editedScannedDoc — Acrobat's "Edit scanned document": the scan was converted to editable text and edited |
| history_edit_after_final_date | high | An XMP history event is later than the file's modified date (+1 min) |
| history_shorter_than_revisions | medium | Fewer history entries than xref revisions |
| history_absent_despite_revisions | medium | Revisions exist but the history is empty |
| orphaned_objects | medium (orphans found) / info | Content objects present but unreachable from /Root or /Info — typical of deleted content. Only content counts: streams, fonts, images and form XObjects, pages, annotations ( data.kinds ). A bare array, number or plain dictionary (macOS/iOS Quartz leaves its page box [0 0 612 792] ), or an empty stream ( /Length ≤ 16 or nothing once decoded — PDFium leaves an empty Form XObject per page) is not counted ( data.trivial_ignored ) |
| javascript_or_openaction | medium (flagged) / info | Embedded JS or auto-run action |
| optional_content_groups | low / info | Layers present |

| finding | Severity | Meaning |
|---|---|---|
| editable_text_over_scan | medium | Page structure ( services/forensics/page_structure.py , from the drawing order): an image covering ≥ 90 % of the page drawn **before** visible live text (≥ 15 runs spanning ≥ 50 % of the page height) — a scan converted to editable text, **whatever the fonts are called** (a converter's -NNNN subsets, listed in data.converter_font_pages , or ordinary ABCDEF+ subsets as Acrobat's editor leaves them). Does not depend on XMP, so it still holds when the metadata is stripped. An image drawn *over* the text (a flattened page pasted on top) is not this structure. A normal searchable scan keeps its OCR text invisible (render mode 3). Such pages also get the ghost-content check (07a) |
| producer_scrubbed | medium | The XMP packet was written by Adobe's own library ( x:xmptk "Adobe XMP Core …", read from the raw stream), yet Info Producer/Creator, XMP Producer/CreatorTool and the edit history are all empty. Adobe software always records its name and saves there, so they were removed afterwards. A fully empty file is metadata_entirely_stripped instead |
| web_page_origin | low / medium | The producer is a browser's or an HTML engine's PDF output (PDFium, Skia/PDF, Chrome, wkhtmltopdf, Puppeteer, Playwright, WeasyPrint, Prince…): the document was built as a web page and printed. **Medium** ( metadata.web_page_origin , 10) when it also has no CreationDate or uses ≥ 3 stock colours of a CSS framework (Bootstrap 5, Tailwind — services/forensics/pdf_facts.py ); **low** (shown only) with the engine alone, since school portals print from browsers too |
| digital_signature | info | Signature field present / signed / unsigned / absent |
| info_dictionary , xmp_metadata , xmp_history , revision_count , document_id_chain | info | Raw context |

The check result is flag **if any finding is** medium **or** high , else pass .

All metadata.* rules together add at most the company's metadata cap (default 40) to a case's score — see 11.

## Output shape (real, truncated)

```
{
  "result": "flag",
  "details": [
    {"finding": "orphaned_objects", "severity": "medium",
     "description": "Found 25 object(s) present in the file but not reachable from /Root or /Info — likely
left…",
     "data": {"count": 25, "object_numbers": [2, 3, "…"]}},
    {"finding": "info_dictionary", "severity": "info", "description": "Info dictionary has 6 field(s).",
     "data": {"/Creator": "Adobe Photoshop 2024 (Windows)", "/ModDate": "D:20260902164500Z",
              "/CreationDate": "D:20260801090000Z", "/Producer": "Adobe Photoshop 2024 (Windows)", "…":
"…"}}
  ]
}
```

details is a **list** of findings (the same shape ELA, copy-move, duplicate and visual review use, so one findings-list UI renders them all). These findings carry no page / bounding_box ; metadata is not positional.

## Known limitations

- **PDF only.** Images and Word files are not analyzed.
- Many legitimate tools leave incremental updates or editor fingerprints (a scanned-then-annotated invoice, for example). A finding is a reason to look, not proof.
- A determined forger can rewrite or strip metadata; `metadata_entirely_stripped` is itself scored because clean-room generators rarely strip everything.
- Uploads are PDF-only, so every new document is analysed; a non-PDF stored before that rule is a quiet no-op.
- The visual-review task cross-references this result ( `metadata_forensics_correlation` ) to strengthen or explain its own findings; that cross-reference is deliberately **not scored** to avoid double counting.

## Risk rules fed

| `rule_id` | Finding | Fires on | Weight | Severity |
|---|---|---|---|---|
| `metadata.pdf_unreadable` | `pdf_unreadable` | any | 20 | high |
| `metadata.stripped` | `metadata_entirely_stripped` | any | 12 | medium |
| `metadata.editing_software_detected` | `editing_software_detected` | any | 30 | high |
| `metadata.creation_date_mismatch` | `info_xmp_creation_date_mismatch` | any | 18 | high |
| `metadata.mod_date_mismatch` | `info_xmp_mod_date_mismatch` | any | 18 | high |
| `metadata.producer_mismatch` | `info_xmp_producer_mismatch` | any | 10 | medium |
| `metadata.pdf_editor_producer` | `pdf_editor_detected` | any | 10 | medium |
| `metadata.modified_after_creation` | `mod_date_after_creation_date` | any | 15 | high |
| `metadata.incremental_update_single` | `incremental_updates_present` | medium | 8 | medium |
| `metadata.incremental_update_multiple` | `incremental_updates_present` | high | 20 | high |
| `metadata.history_tool_anomaly` | `history_editing_tool_anomaly` | any | 12 | medium |
| `metadata.history_scanned_doc_edited` | `history_scanned_document_edited` | any | 15 | high |
| `metadata.editable_text_over_scan` (v2: any font naming) | `editable_text_over_scan` | medium, high | 10 | medium |
| `metadata.producer_scrubbed` | `producer_scrubbed` | any | 10 | medium |

| rule_id | Finding | Fires on | Weight | Severity |
|---|---|---|---|---|
| metadata.web_page_origin | web_page_origin | medium, high | 10 | medium |
| metadata.history_edit_after_final | history_edit_after_final_date | any | 22 | high |
| metadata.history_shorter_than_revisions | history_shorter_than_revisions | any | 8 | medium |
| metadata.history_absent_despite_revisions | history_absent_despite_revisions | any | 8 | medium |
| metadata.orphaned_objects (v3: content only, empty streams ignored) | orphaned_objects | medium, high | 8 | medium |
| metadata.javascript_or_openaction | javascript_or_openaction | medium, high | 10 | medium |

Note the two severities: a rule's `severity` (how it is displayed) is independent of the *finding's* severity (which the rule may filter on with `severity_in`).

# 6a. Font consistency

**What it detects:** text set in a different font family from the text around it, the trace a value leaves when it is typed in after the document was produced (an amount retyped in Times on a Helvetica invoice, or pasted in Arial on a Calibri letter before it was printed and scanned).

**Code:** `services/forensics/font_consistency.py` ( `analyze_font_consistency` ), run inside `tasks/document_processing.py` ( `process_document` ) right after OCR and **before the document is marked complete**, so a case is never scored without it. Stores `check_type = font_consistency` .

It runs in the extraction task, not as its own task, because scanned pages need that task's OCR font estimates. A failure is stored as a `failed` check and never fails the extraction.

## Two sources, chosen per page

| Page | Source | How the font is known | Finding severity |
|---|---|---|---|
| Digitally produced (has a text layer) | `text_layer` | The font **name** of every text run, read with PyMuPDF. Exact | `high` |
| Scanned (no text layer) | `ocr` | Azure Document Intelligence Layout's **styleFont** add-on: an estimated similar font family per recognized word ( `services/ocr_service.py` ) | `medium` when 5+ words share the odd font, else `low` (shown, not scored) |
| Neither (scan with the add-on off) | `none` | Not analysed. Fonts there are left to the visual review | |

The text layer is exact, so one run in an out-of-place font is reported and scored, at any size. OCR font recognition is an estimate and noisy. It labels one body font "Segoe UI" on some words and "Tahoma" on others, and guesses wildly on small print and signatures. So on scans:

- families are compared as broad **visual classes** (humanist sans, grotesque sans, geometric sans, serif, slab serif, monospace). Unknown families are not used;
- only labels with confidence **≥ 0.95** on **body-size** text (0.8–1.3 × the page's median word height) that is not italic or handwritten are used;
- **every** word in an out-of-place class is reported and boxed, even a single one. It is **scored** ( `medium` ) only once that class covers **at least 5 words** of the page; below that it is `low` : shown for the reviewer, not scored.

Why the split: on scans of genuine documents, OCR font recognition mislabels a few single words per page (phone numbers, single digits, "Total", "DUE:"), never 5 in one class, while scans with retyped amounts reached 6–8. Scoring single words would mark genuine scans risky; showing them lets the reviewer judge a one-value edit by eye. On a digital PDF every mismatch is exact and always scored.

## The rule

A run is flagged when its family is a **minor** font on the page (under 25 % of the page's characters in that script) and it sits among text of similar size (within 15 % on a text layer, 50 % on a scan) in a more common family:

- **row:** on the same visual row ( `Subtotal:` in Helvetica, `18,450.00 USD` in Times);
- **column:** in a left- or right-aligned column of 3+ runs where most runs are in more common families (a whole line item retyped in Courier between line items in DejaVu Sans).

Not flagged, by design:

- weight/style (Bold, Italic…), subset tags, and metric-compatible substitutes (Arial = Helvetica, Times New Roman = Times, Courier New = Courier);
- symbol/icon fonts and script fonts used for typed signatures;
- text of a different size (a letterhead or heading in a display font);
- text in another writing system: runs are compared only within one script, so Arabic set in an Arabic font beside Latin text in Helvetica is normal;
- a family that sets 25 % or more of the page's text, or most of a column of 4+ runs: that is the document's design.

## Output shape (real, truncated)


```
{
  "result": "flag",
  "details": [
    {"finding": "font_family_inventory", "severity": "info",
     "description": "1 of 1 page(s) analysed (1 scanned, using OCR font recognition); 3 font families: …",
     "data": {"families": {"humanist sans": 905, "grotesque sans": 41, "…": 0},
              "pages": [{"page": 1, "source": "ocr"}]}},
    {"finding": "font_inconsistency", "severity": "medium",
     "description": "Page 1 (scanned): '22,000' looks like a different font (grotesque sans) from the text
around it (humanist sans), per OCR font recognition — …",
     "page": 1, "bounding_box": {"page": 1, "x": 0.3851, "y": 0.4129, "width": 0.0627, "height": 0.0181},
     "data": {"text": "22,000", "font": "Arial, Helvetica, sans-serif", "family": "grotesque sans",
              "expected_family": "humanist sans", "font_size": 10.8, "source": "ocr"}}
  ]
}
```


Each `font_inconsistency` carries `page` and a normalized `bounding_box`. The Case Detail viewer draws it as a solid fuchsia **Font mismatch** box, and so does the exported case report.

## Known limitations

- **PDF only.**
- **Scans:** needs the styleFont add-on (`AZURE_DOCUMENT_INTELLIGENCE_STYLE_FONT`, on by default, billed by Azure per page) and catches only edits spanning several words (see above). Heading-size and small-print text on scans is not compared.
- **Digital PDFs:** an edit pasted in as an image is invisible to it (ELA, copy-move and the visual review cover those). A forger who retypes values in the document's own font, or a design that deliberately mixes families in a small area, defeats it.

## Scans converted to editable text

Acrobat "Edit scanned document" (and similar converters) re-create every word of a scan as live text in subset fonts they generate per guessed style, named `<Family>-NNNN` (`Comic Sans MS-Bold-6003`, `Arial-Bold-6000`). The `-NNNN` number is not part of the family. On a page with at least 60 % of its characters in such subsets, the family comparison is **not** used — the converter's guesses differ word to word on small print, so two subsets side by side (`03` in `Comic Sans MS-BoldItalic-6002` beside `14` in `Arial-Bold-6000`) are not evidence. Instead, a run in a **full** font (no `-NNNN`) on a row of the converter's subsets of similar size is flagged `high`: it was typed in after the conversion. The finding lists the digits the neighbouring subset holds (read from its ToUnicode map) — a subset only contains the glyphs originally printed with it, so `digits_not_in_original` hints at what the value was.

## Size changes inside a number

On every text-layer page, a number whose characters are set ≥ 0.2 pt apart ( `36,000` with `6` and `000` at 11.7 pt inside a 12 pt figure) is noted on the font finding covering it ( `data.size_change` ), or reported on its own as `font_size_inconsistency` ( `high` ). Ordinals like `1st` are not numbers.

## A font embedded more than once

A document exported in one pass embeds each font and style once, as one subset tagged `ABCDEF+` . An edit session that types text in adds **another subset of the same font** (PyMuPDF reports the tags only when asked: `TOOLS.set_subset_fontnames(True)` , set for the read and restored). Fonts are grouped by family and style ignoring the tag; the subset holding the most text is the main copy. Text in an extra copy is flagged `font_subset_split` ( `high` ) when:

- it is on the **same line** as text in the main copy ("AL REEM" beside "SALEH MOHAMED ALHARTHI"), or
- it is an **amount in a column** whose other amounts are in the main copy ("1,450.00" above "4,575.00"), or
- it is an **amount in a copy holding only digits and punctuation** ( `, . 0 1 2 3 4 5 6` ).

An extra copy by itself is not enough: Word puts a stray en-dash, or a second bold-italic heading on another line, in a subset of its own. `data` carries `subset` , `main_subset` , `subset_glyphs` , `digits_only_subset` , `same_row_as` , `same_column_as` . CASE-39CB18BF: 17 spans — the amounts of both columns in a digits-only Times New Roman copy, the totals and amount in words in a third bold copy, the student's first name, ID and class in a second bold copy.

## Risk rules fed

| `rule_id` | Finding | Fires on | Weight | Severity |
|---|---|---|---|---|
| `font.inconsistency` (v3) | `font_inconsistency` , `font_size_inconsistency` , `font_subset_split` | high (text layer) | 40 | high |
| `font.inconsistency_scanned` | `font_inconsistency` | medium (OCR, scans, 5+ words) | 25 | medium |

`low` findings (a scan's odd font on fewer than 5 words) are not scored by any rule.

# 06b — Spelling (English)

**What it detects:** misspelled **ordinary English words** in a document's printed text — "Certifcate", "Univeristy", "Recieved". Genuine issuers print from proofread templates; a retyped or fabricated document often carries a typo. Small issuers make mistakes too, so this is a **weak signal**: every confirmed misspelling is shown (with its box on the page), but the rule that scores it is low-weight.

**Code:** `services/forensics/spelling.py` (`analyze_spelling`), run by `tasks/document_processing.py` (`_run_spelling_check`, right after font consistency — it needs that task's OCR words and extracted fields). Every document, PDF or image. One `document_checks` row: `spelling_consistency`. Dictionaries: `pyspellchecker` (word-frequency lists, bundled — no network).

**English only.** An Arabic pass was built and tried on real documents, but its results were not good enough, so Arabic words are skipped (`CHECKED_LANGUAGES = (ENGLISH,)` in `spelling.py`; adding `ARABIC` turns it back on). Language is decided **per word**, so a mixed English / Arabic document still has its English words checked; the text-layer agreement test below is also computed from English words only.

## How noise is kept out

Two passes. The dictionary pass is cheap and strict about what it lets through; the language model judges only what survives.

### 1. Dictionary pass — what becomes a candidate

An OCR word is checked only if it is:

| Filter | Why |
|---|---|
| Latin letters only (Arabic and mixed-script words skipped); no digits or symbols (`@ # / . _ …`) | codes, IDs, emails, amounts |
| ≥ 4 letters; not an acronym (all caps ≤ 5); not CamelCase | abbreviations, brands |
| not in an extracted field that is a name, address, ID, contact detail or code (`student_name`, `address`, `iban`, `trn`, …) or the reference number | names are not dictionary words |
| not handwriting (OCR styleFont), not ≥ 50 % inside a detected signature or stamp | signature ink, curved stamp text |
| OCR confidence ≥ 0.85 | an uncertain read is more likely an OCR error |

It becomes a **candidate** only if it is:

- **unknown in every accepted variant** — British spellings (`colour`, `organisation`, `programme`, `enrolment` …) and a small regional allow-list (`Sharjah`, `IGCSE`, `KHDA` …);
- **in the PDF's own text layer as read**, on a page whose text layer agrees with OCR on ≥ 60 % of its words (else the layer is not trusted). A word the PDF spells correctly but OCR misread is dropped here. Scans and images have no layer — only the OCR-confidence filter applies;
- **one edit away from a common word** (frequency ≥ 100). An unknown word with no near common word is a name or a foreign word, not a typo.

At most 30 distinct candidates (most frequent first) go on to the model.

### 2. Language-model judgment

One call per document ( `LLMService.judge_spelling` , temperature 0), each candidate with ±6 words of context and its dictionary suggestions. Verdicts: `misspelling` , `valid_word` , `valid_variant` , `proper_noun` , `abbreviation_or_code` , `transliteration_or_foreign` , `ocr_misread` , `unsure` . The prompt tells it that a false alarm is worse than a miss.

**Flagged:** only `misspelling` at **medium or high** confidence, with a correction that differs from the word. Everything else is listed in the scope note ( `data.set_aside` ), not scored.

**No model configured:** nothing is flagged; result `limited` , the unconfirmed words noted. The dictionary alone is too noisy to score.

## Result

```
{"result": "flag" | "pass" | "limited" | "not_applicable",
 "details": [
   {"finding": "spelling_error", "severity": "low", "page": 1,
    "bounding_box": {...}, "description": "Page 1: 'Certifcate' looks misspelled – 'Certificate' (English).
…",
    "data": {"text", "correction", "language": "en", "context", "model_confidence",
             "ocr_confidence", "confirmed_by_text_layer"}},
   …one per occurrence…,
   {"finding": "spelling_scope", "severity": "info",
    "data": {"words_checked", "words_checked_by_language", "unknown_words", "candidates",
             "text_layer_pages", "set_aside": [{"word", "verdict"}], "misspellings"}}]}
```

`not_applicable` : no printed English word was read (e.g. an Arabic-only document).

## Scoring

Rule `content.spelling_error` (migration `a2d6f9c3e7b4` ): any `spelling_error` finding → **weight 5, severity low**, fired **once per document** however many words (one typo is shown, but ten typos do not add up to a high score). Editable in Settings › Risk Rules like any rule.

## Where it shows

- Case page: check "Spelling (English)" — headline "2 misspelled words: 'Certifcate', 'Univeristy'", one item per word with its correction; a **sky-blue box** around each occurrence on the page.
- Report: in the checks table, as a highlight region (sky blue) and as an exception line per word.

# 07 — Error Level Analysis & copy-move detection

**What they detect:** *pixel-level* evidence that part of a page was edited after the rest was last saved (ELA), or that a region was copied from one place on a page to another (copy-move). Together they look for pasted-over totals, cloned signatures/stamps and patched line items.

**Code:** `services/forensics/ela.py` ( `run_ela_check` ), `services/forensics/copy_move.py` ( `run_copy_move_check` ), `services/forensics/pdf_render.py` ( `render_pdf_pages` ), run by `tasks/tampering_checks_task.py` ( `run_tampering_checks` ). Two `document_checks` rows result: `error_level_analysis` and `copy_move_detection` .

## Shared render

`render_pdf_pages(pdf_bytes) -> list[RenderedPage]` rasterizes every page **once** at **200 DPI** with PyMuPDF into BGR `numpy` arrays and hands the same list to both checks (the render is the expensive part, so `run_tampering_checks` does it once). The duplicate-detection, visual-review and signature tasks render the PDF separately with the same function.

```
@dataclass RenderedPage:
    page_number: int          # 1-based
    image: np.ndarray         # BGR uint8
    width: int; height: int
    has_image_content: bool   # page has ≥1 embedded raster image XObject
```

200 DPI is chosen to resemble a real scan: rendering far higher would exaggerate ELA's block artifacts and make its known print-and-rescan weakness worse.

## ELA (Error Level Analysis)

**Idea:** JPEG-recompress the page at a fixed quality and diff it against the original. A region that was edited after the last save recompresses differently from the rest of the page.

**Algorithm** (adapted from an internal reference tool): recompress at JPEG quality **75**; take the non-linear `sqrt(diff)` error with a gain; threshold at the **99.7th percentile** with an absolute floor of 60; group into regions; keep regions between **0.2 %** and **60 %** of the page area. `_detect_anti_forensics` separately flags a page that is *suspiciously clean*: a median-blur noise variance below 5.0 ("possible anti-forensic smoothing") or a Laplacian standard deviation below 1.5 ("possible over-compression") — the signature of someone smoothing or re-compressing a page to defeat ELA.

**Gate:** only pages with `has_image_content` are analyzed. No such page → `not_applicable` .

**Findings:** `recompression_error_region` (medium) — with `page` , `bounding_box` and `data.region_area_fraction` ; `anti_forensic_signal` (high).

## Copy-move detection

**Idea:** find a region duplicated elsewhere on the *same page* — a common way to hide or fabricate content.

**Algorithm** (adapted from an internal reference tool, pipeline unchanged apart from one fixed bug):

1. **BRISK** keypoints on the grayscale page (detector threshold 200), keeping those whose normalized response is above `100 – 90` .

2. **Radius matching** of each keypoint's descriptor against all others (Hamming distance ≤ 20 % of 255 ≈ 51), excluding self-matches.
3. **Displacement filter:** discard matches whose two points are closer than `0.15 × min(page height, width) / 2` — nearby points are not "moved" content. That same distance bounds the grouping in the next step.
4. **Cluster** matches that share a similar displacement and whose endpoints sit near each other; keep clusters of **≥ 30** matches.
5. **De-duplicate** overlapping clusters (IoU > 0.3) and keep regions covering ≥ **2 %** of the page.

It runs on **every** page, image-bearing or not: a duplicated paragraph or logo shows up as a keypoint cluster whether the source is a photo or vector text.

**Safety valves:** if a page yields more than 4000 keypoints after filtering, or more than 4000 matches after the displacement filter, the page is skipped and returns **no clusters** (the clustering loop is $O(n^2)$). A very dense page therefore *cannot* produce a copy-move finding.

**Finding:** `copy_move_cluster` (high) with `page`, `bounding_box` and `data.matched_pairs`. No visualization image is generated; the frontend draws boxes live.

## Output shape (real)

Both checks return `{"result": "pass"|"flag", "details": [Finding, …]}`; ELA can also return `"not_applicable"`. When a page's text is **vector text drawn over an image** (a scan converted to editable text — `services/forensics/page_structure.py`), a clean result is reported as `"limited"` instead of `"pass"`, with an info finding `pixel_analysis_limited`: pixel checks cannot see an edit to that text, so "pass" would give false comfort. A flag stays a flag.

```
// error_level_analysis
{"result": "flag", "details": [{
  "finding": "recompression_error_region", "severity": "medium", "page": 1,
  "description": "Page 1: a localized area shows a higher JPEG recompression error than the rest of the
page",
  "bounding_box": {"page": 1, "x": 0.5526, "y": 0.4951, "width": 0.3688, "height": 0.0338},
  "data": {"region_area_fraction": 0.0094}}]}

// copy_move_detection
{"result": "flag", "details": [{
  "finding": "copy_move_cluster", "severity": "high", "page": 1,
  "description": "Page 1: found a cluster of 58 matching feature pairs, consistent with one region of this
p…",
  "bounding_box": {"page": 1, "x": 0.348, "y": 0.3164, "width": 0.1469, "height": 0.4271},
  "data": {"matched_pairs": 58}}]}
```

The `Finding` dataclass is `{finding, severity, description, page?, bounding_box?, data?}`.

## In the UI

Boxes are drawn live over the PDF by `PdfOverlayViewer` — **ELA solid red, copy-move solid orange** — and in the report's Section 9. Nothing annotated is ever stored by the live UI.

## Known limitations

- **ELA is a signal, not a verdict.** It is weak against print-and-rescan tampering and prone to false positives on legitimately recompressed scans. This is why the seeded weights are moderate.

- **Copy-move flags repeated legitimate structure** (table rows, repeated logos or headers, identical stamps) — it has no notion of "expected" repetition.
- Both are PDF-only and both render at a fixed DPI; very small edits can fall below the region-area floors.
- Copy-move silently produces nothing on very dense pages (the 4000-keypoint / 4000-match caps above).
- Findings on documents with no raster content will never come from ELA ( `not_applicable` ); a pure-text forged PDF is caught, if at all, by metadata, copy-move, visual review and field checks.

On such a page the background itself is searched for traces of text deleted or shortened after the conversion — the ghost-content check, 07a, run by the same task.

## Risk rules fed

| `rule_id` | Check / finding | Weight | Severity |
|---|---|---|---|
| `ela.tamper_region_detected` | ELA / `recompression_error_region` | 25 | medium |
| `ela.anti_forensic_signal` | ELA / `anti_forensic_signal` | 30 | high |
| `copy_move.cluster_detected` | copy-move / `copy_move_cluster` | 35 | high |

# 07a — Deleted / replaced content (ghost text)

**What it detects:** on a scan that was **converted to editable text** (Acrobat "Edit scanned document", iLovePDF and the like), text that was **deleted** or **shortened** after the conversion.

**Code:** `services/forensics/ghost_content.py` (`analyze_ghost_content`, `exclude_signature_regions`), run by `tasks/tampering_checks_task.py` (`run_tampering_checks`, same task as ELA / copy-move since it reads the same bytes). One `document_checks` row: `ghost_content`.

## Why it works

A converter erases the text pixels from the scan and redraws every word as live text on top. The erasure is never perfect: a faint **ghost** of each word stays in the background image.

| What the background shows | Meaning |
|---|---|
| ghost **with** live text on top | normal — the converted word |
| ghost with **no** live text on top | content that was there at conversion and was **deleted** since |
| ghost much **wider** than the live text on top | the line was **shortened or rewritten** |

## When it runs

Only on pages built that way (`page_structure.PageStructure.vector_text_over_image`, from PyMuPDF's drawing order): one image covering **≥ 90 %** of the page (it may overhang the page edges, as converters place it — e.g. `-10,-7,605,849` on a 595 × 842 page) drawn **before** the text, and **≥ 15** visible text runs (render mode ≠ 3) spanning ≥ 50 % of the page height — **whatever the fonts are called** (a converter's `-NNNN` subsets or ordinary `ABCDEF+` subsets).

A plain scan (no text layer), a searchable scan (OCR text invisible, render mode 3) and a born-digital page (no page-sized image) → result `not_applicable`, with the reason in `data.reason` ("no text layer (a plain scan)", …) — shown on the case page and in the report's coverage table ("Not applicable: no text layer (a plain scan)").

Documents processed before this check existed: `python -m scripts.backfill_ghost_content` (all of them, or `--case CASE-…`) runs it on every PDF without a result, and re-scores a case that flags. A converted page whose scan is placed rotated or skewed is also reported `not_applicable` (not handled).

## Algorithm (background image at its native resolution, 8-bit gray)

1. **Residue.** Morphological **black-hat** with a kernel of ½ the median text height: how much darker each pixel is than the paper around it, for marks thinner than a stroke. Broad shading and uneven illumination are part of the "paper".
2. **Not searched:** where the paper itself is not white — shaded cells and bands (often a scanner's halftone dots), photos, logos left in the scan: median tone over ~2 text heights more than 12 levels darker than the page's white. A ghost cannot be told from that texture.
3. **Ghost band.** Residue between the page's noise floor (median over empty 64-px tiles of their 99th percentile, at least 4) and **70** (stronger is ink still in the image — rules, logos, stamps, signatures — removed with a 2-px halo). Ruled lines are removed (bridged across short breaks, then long thin runs; a band as tall as letters is text, not a rule), as are specks < 6 px and components much flatter than a glyph.
4. **Lines.** The ghost mask is smeared horizontally (0.6 × text height) into lines; kept when 0.5–2.5 × the median text height tall, ≥ 25 pt wide, aspect ≥ 1.8 and 3–60 % ink.

5. **What may sit on a ghost:** visible text spans (+3 pt; lines skewed by up to ~6° count), other images (+2 pt — converters lift logos, stamps and signatures into their own images), filled vector shapes, and a 4 % page margin.
6. **Classify each ghost line** (after dropping a line with more ink still in the image than ghost — the faint edge of a signature or of print the converter left as image — and a line ≥ 1.5 × stronger than the page's own erased text, i.e. a smudge):
   - **replaced line** ( `ghost_replaced_line` ): its *uncovered* ghost runs on past the live text on its row (or before it) by > 30 % of the end word and > 15 pt, into empty space (not up to the next word), and that uncovered part itself looks like text (≥ 25 pt, ≥ 50 % of its columns marked, ≥ 4 glyph-height marks). Text the converter could not map to characters is skipped (its width means nothing).
   - **orphan line**: < 20 % covered, no live word on its row within 4 text heights, and text-like (≥ 3 glyph-height marks). Orphan lines less than 1.5 line heights apart, side by side within 3 line heights (a label column and its value column), merge into a **block**; a block needs one line ≥ 40 pt.
7. **Show-through** (when OCR is available): each orphan block's enhanced crop is read as is and mirrored (Azure Document Intelligence Read). If the mirrored reading is real text (≥ 4 characters, confidence ≥ 0.5) and clearly better (× 1.5), it is the back of the page showing through: `ghost_show_through` , info, not scored. The as-is reading is kept as the reviewer's hint — best effort; a heavily compressed scan often gives none.
8. **Vision-model hint** (when an LLM with vision is configured): for each deleted block, the trace and a tiny layout thumbnail of the page (too small to read — the model must not take the surrounding text for the block's) go to the vision model **three times** ( `guess_erased_content` , temperature 0.4). Only what at least two answers agree on is kept ( `data.vision_hint` : `kinds` , `heading` , `legible_words` , `answers_agree` ); confidence is "medium" only when the answers agree on the kind and read something consistently, else "low". On CASE-39CB18BF the answers disagree (bank details / a note about fees), so it shows as a low-confidence guess. Never scored, never evidence.
9. **Signatures and stamps:** a finding lying ≥ 30 % inside a detected signature or stamp region is set aside ( `ghost_near_signature` , info). Applied when the ghost check runs if detection has finished, else when detection finishes ( `refilter_ghost_content` , called from `signature_detection_task` ).

## Findings

| Finding | Severity | Meaning |
|---|---|---|
| `ghost_content_scope` | info | Pages searched (noise floor and erased-text level per page), or why not applicable |
| `ghost_deleted_block` | high | A block of faint text with no live text on top: `data.lines` , `data.box_pt` , `data.hint` |
| `ghost_replaced_line` | medium | A line's trace runs on past its live text: `data.text` , `data.overhang_pt` |
| `ghost_show_through` | info | An orphan block that reads better mirrored — not scored |
| `ghost_near_signature` | info | Set aside: inside a detected signature or stamp — not scored |

Every located finding carries a normalized `bounding_box` and `data.crop_png_base64` — the scan in that box, contrast-stretched (darkest 0.5 % black, the paper's median tone white) so the trace shows. The case page draws deleted / shortened content as a **solid teal** box and shows the crop under the finding; the report draws the same box (Section 9) and adds a page of crops after the annotated page.

`result` is **`flag`** when any finding is medium or high, **`pass`** otherwise, **`not_applicable`** as above. In the report's coverage table a check that was not applicable to any document reads "Not applicable".

## Risk rules fed

| Rule | Finding | Weight | Severity |
|---|---|---|---|
| `content.deleted_ghost_block` | `ghost_deleted_block` (high) | 25 | high |
| `content.replaced_ghost_line` | `ghost_replaced_line` (medium, high) | 10 | medium |

Each fires at most once per document (the reason lists every block / line).

## Reference results

| Case | Result |
|---|---|
| CASE-39CB18BF (Ajyal, `6f277532…`) | **flag**: a 6-line block deleted under the fee table (x 30–270, y 388–480 pt — the bank-transfer details); the "Registration Fees" row's trace runs ~90 pt past its text |
| CASE-1CEDFDBF / CASE-F6F5FE76 (British Orchard, `895a259e…`) | pass — ghosts under the converted text only; the shaded cells, the footer the converter garbled and the stamp give nothing |
| CASE-B70765EC (ILM Academy, `14926b7f…`) | pass — no false orphans from the shaded rows |
| CASE-DE627FA2, CASE-DB43653A (MFP scans), CASE-4B297080 (Word) | not applicable |

## Known limitations

- **The student row of CASE-39CB18BF is not reported as shortened.** Its trace runs only ~6 pt past "PRE-KG-NA" (the words underneath are spaced differently from the live words, but are about as long); converter re-typesetting routinely differs by that much, so it cannot be told apart. The retyped name, ID and class are caught instead by the font check: they sit in a second embedded copy of the bold font (`font_subset_split`, 06a).
- Shaded areas are not searched, so content deleted from a shaded cell is missed.
- The hint OCR rarely reads anything on a heavily compressed scan; the crop is the evidence.
- A scan placed rotated / skewed (as an image transform) is not handled.

# 08 — Visual inconsistency review & AI-generation assessment

**What it detects:** whether a page *looks* edited or synthetic to a vision model — inconsistent fonts, misaligned text, patchy colour/contrast, uneven sharpness or lighting — and whether it appears AI-generated. It is a **secondary, judgment-based** signal that can catch what pixel forensics miss, or corroborate them.

**Code:** `services/visual_inconsistency_service.py` ( `run_visual_inconsistency_review` ), `services/llm_service.py` ( `analyze_page_visual_consistency` , `PageVisualAnalysis` ), run by `tasks/visual_inconsistency_task.py` ; stores `check_type = visual_inconsistency_review` .

## Algorithm

1. Render every page of the PDF ( `render_pdf_pages` , 200 DPI) and encode as PNG, longest side capped at **1536 px** ( `MAX_IMAGE_DIMENSION` ). PNG, not JPEG, so glyph edges stay crisp and compression artifacts are not introduced.
2. Send each page to the vision model **twice, independently** ( `RUNS_PER_PAGE = 2` ), at `temperature = 0.4` — deliberately non-zero, unlike every other call in `LLMService` , so the runs are genuinely independent samples. Each run answers six structured questions: font consistency, text alignment, colour/contrast, resolution/sharpness, shadow/lighting — each a `VisualCategoryFinding {consistent, description, confidence, bounding_box}` — plus the `AIGenerationAssessment {likely_ai_generated, description, confidence}` .
3. **Corroboration rule** (the reliability mechanism): a category is reported as a real finding only if **both runs** flagged it, or **either** run flagged it with `high` confidence. A single low/medium one-off is kept but labelled "treated as noise" at severity `info` .
4. **Severity:** for a confirmed finding, `high` if both runs agreed *and* at least one was high-confidence, else `medium` ; everything not confirmed (both runs consistent, or an uncorroborated one-off) is `info` . The check's `result` is `flag` if any finding is `medium` or `high` , else `pass` .
5. The task appends `metadata_forensics_correlation` (info) cross-referencing the metadata result, so a reviewer sees editing-tool evidence next to visual evidence. It is **not scored** (avoids double counting), as is `experimental_signal_notice` (an info notice always added to the result).

## Findings

| `finding` | Meaning |
|---|---|
| `visual_font_consistency` | Fonts vary within the page |
| `visual_text_alignment` | Text or table alignment is inconsistent |
| `visual_color_contrast_consistency` | Colour/contrast differs across regions |
| `visual_resolution_sharpness_consistency` | Sharpness differs across regions |
| `visual_shadow_lighting_consistency` | Shadows/lighting inconsistent |
| `ai_generation_assessment` | The page may be AI-generated/synthetic |
| `experimental_signal_notice` , `metadata_forensics_correlation` | Info only, never scored |

## Output shape (real, truncated)

```
{
  "result": "flag",                          // flag if any corroborated finding, else pass
  "details": [
    {"finding": "experimental_signal_notice", "severity": "info",
     "description": "This check uses a vision-capable language model's visual judgment, run twice
independently…"},
    {"finding": "visual_font_consistency", "severity": "high", "page": 1,
     "description": "Page 1 — Font consistency: The font weight and style for the item descriptions … This
is a vision-model judgment, not pixel-level analysis — a probabilistic signal, not a definitive finding.",
     "bounding_box": {"page": 1, "x": 0.05, "y": 0.3, "width": 0.9, "height": 0.15},
     "data": {
       "category": "font_consistency",
       "run_agreement": "2-of-2 runs",       // | "1-of-2 runs (high confidence)" | "consistent in both
runs" | "1-of-2 runs, not corroborated" | "not flagged"
       "run_1": {"consistent": false, "confidence": "high", "description": "…", "bounding_box": {"x": 0.05,
"y": 0.3, "width": 0.9, "height": 0.15}},
       "run_2": {"consistent": false, "confidence": "high", "description": "…", "bounding_box": {…}}}}
  ]
}
```

`data` keeps **both raw runs**, so a reviewer or developer can see exactly what was said and why a finding did or did not survive. The `bounding_box` (from the first run that supplied one) is drawn as a **dashed blue "AI-described area (approximate)"** overlay — deliberately styled to look less precise than the red/orange pixel-forensic boxes.

## Known limitations

- **It is a language model's opinion.** Runs vary; corroboration reduces but does not remove hallucination. Boxes are approximate (models are poor at localisation).
- **AI-generation detection is unproven** on scanned/business documents. It is weighted lowest of any visual signal (weight 6) and every description says "experimental, probabilistic".
- **Every page costs two vision calls.** A 10-page PDF is 20 calls; there is no page cap on this task (unlike signature detection, capped at 10 pages).
- Legitimate documents made of mixed sources (a template with a pasted logo, mixed fonts) can flag.
- **Font judgement is unreliable**: the page is downscaled to 1536 px, and a few amounts retyped in a different font are often missed. The font consistency check covers fonts deterministically on digital PDFs, and with OCR font recognition on scans.
- PDF only.

## Risk rules fed

Each rule fires only on `medium` / `high` findings ( `severity_in` ).

| `rule_id` | Finding | Weight | Severity |
|---|---|---|---|
| `visual.font_inconsistency` | `visual_font_consistency` | 12 | medium |
| `visual.alignment_inconsistency` | `visual_text_alignment` | 10 | medium |
| `visual.color_contrast_inconsistency` | `visual_color_contrast_consistency` | 12 | medium |
| `visual.sharpness_inconsistency` | `visual_resolution_sharpness_consistency` | 10 | medium |

| rule_id | Finding | Weight | Severity |
|---|---|---|---|
| visual.lighting_inconsistency | visual_shadow_lighting_consistency | 8 | low |
| ai.generated_content_suspected | ai_generation_assessment | 6 | low |

# 09 — Signature & stamp verification

**What it does:** three separate, deliberately modest things.

1. **Detection** (automatic, every PDF): locates where a handwritten signature or stamp/seal *appears* and whether a document of this kind would normally carry one. Presence and placement only.
2. **Comparison** (on demand): after a reviewer draws a *reference* signature, compares it to the same kind of region on every other document in the case — a qualitative vision-model opinion.
3. **Reuse check** (on demand, no model): notices when the very same signature *image* appears under two documents — which a real hand never produces.

None of these verifies who signed. They give a human reviewer something to look at.

**Code:** `services/signature_detection_service.py`, `services/signature_comparison_service.py`, `tasks/signature_detection_task.py`, `tasks/signature_comparison_task.py`, `api/signatures.py`, `models/signature_reference.py`, `models/signature_match.py`, frontend `components/upload/SignatureReferenceCreator.tsx`.

## 1. Detection

`run_signature_detection` (PDF only) renders the pages and calls `detect_signatures(llm, pages, pdf_bytes)`, analysing at most the first **10** pages. For each page it sends a copy of the render with a light **labelled 0.1 grid** overlay — vision models are poor at estimating "where" as a fraction of a page (observed off by up to ~0.3), so the grid lets them read coordinates off the image — and receives a `PageSignatureDetection`:

```
class PageSignatureDetection:
    signature_expected: bool                    # would this kind of document normally carry one?
    regions: list[DetectedSignatureRegion]      # kind: "signature"|"stamp", description,
                                                # confidence: low|medium|high, bounding_box
```

### Box refinement (accuracy fix)

The model's box is only a first guess, so each region is refined before it is stored:

1. **`embedded_image`** — if the PDF has an embedded raster image near the model's box (the usual case for a pasted or scanned signature/stamp), the image's exact placement rectangle replaces the guess. Boxes and images are paired nearest-first, one-to-one, so a signature and a stamp never claim the same image. Page-sized images (> 35 % of the page) and rotated pages are ignored.
2. **`second_look`** — otherwise the model is shown a zoomed window around its own box (grown by 10 % of the page width and 8 % of its height on each side) and asked again; the answer is mapped back to page coordinates. Skipped if the window is blank.
3. **`none`** — neither worked; the (padded) first-pass box is kept.

Refined boxes are trimmed to the ink inside them (+0.4 % margin). The chosen method is recorded per region in `refinement`. Without this, boxes were seen to land on the printed name under a signature instead of the signature itself.

### Result (stored as a `document_checks` row, `check_type = signature_stamp_detection`)

```
{
  "result": "pass",                       // "flag" only when a signature was expected but none found
  "details": {
    "signature_expected": true,
    "detected": [
      {"kind": "signature", "confidence": "high", "refinement": "embedded_image",
       "description": "Handwritten signature in blue ink under the label …",
       "bounding_box": {"page": 1, "x": 0.130, "y": 0.493, "width": 0.177, "height": 0.053}},
      {"kind": "stamp", "confidence": "high", "refinement": "embedded_image", "description": "Round company
stamp …",
       "bounding_box": {"page": 1, "x": 0.441, "y": 0.485, "width": 0.118, "height": 0.087}}
    ],
    "bounding_box": {"page": 1, "x": 0.130, "y": 0.493, "width": 0.177, "height": 0.053}   // the "primary"
region, or null
  }
}
```

`bounding_box` is the primary region: highest confidence, signature preferred on a tie. `detected` keeps every region with its `kind`; results stored before that field existed have only `bounding_box`. Presence alone is a **pass**. A stamp also carries its legible `text` as the model reads it (organisation, branch, P.O. box, city) — compared with the issuer by field validation's `stamp_issuer_consistency` (03). Each region also records its **`material`** (`services/forensics/pdf_facts.py`): `image` (an embedded picture of it), `scan` (part of a scanned page), or `text` / `vector` / `text_and_vector` — a stamp typed into a born-digital file, flagged by field validation's `stamp_authenticity` (and `synthetic_stamp_unsigned` when there is no signature either). Those stamp sub-checks are shown with this check on the case page and in the report. Detection runs in parallel with extraction; whichever finishes second makes field validation run with the stamps and signatures (`revalidate_fields`), and sets aside ghost-text traces inside a signature or stamp (`refilter_ghost_content`).

## 2. Reference creation

`POST /cases/{case_id}/documents/{document_id}/signature-references` with a `SignatureReferenceCreate`:

```
{"person_name": "Sultan Al-Dhaheri",       // typed by the uploader/reviewer; NEVER auto-filled
 "bounding_box": {"page": 1, "x": 0.10, "y": 0.49, "width": 0.22, "height": 0.05},
 "is_library": true}                        // default true
```

Allowed for the case's owner (who sets the reference while uploading) and any reviewer of the company; anyone else gets a 404, and platform admins are read-only. The endpoint renders the PDF page, crops the normalized box, uploads a PNG to `companies/{company_id}/cases/{case_id}/signatures/{crop_id}.png`, inserts the `signature_references` row and enqueues `run_signature_comparison` (`vision_queue`). If the crop fails (render error) the row is still created with a null image URL and comparisons record `cannot_determine`. The box is drawn in the UI on a `react-pdf` viewer (`SignatureReferenceCreator`), offered after upload. References, like everything else, belong to one company: the library flag never makes a reference visible to another company.

## 3. Comparison (`run_signature_comparison`)

For the new reference, the task:

1. **Infers the reference's kind** (`signature` or `stamp`) by overlapping its box with the source document's detected regions (IoU > 0.1). Unknown if nothing overlaps.

2. For each **other** document in the case, picks the detected region of the **same kind** (a signature is never compared with a stamp); documents with no region of that kind are skipped. If the kind is unknown, the document's primary region is used. Documents whose detection has not finished are skipped and picked up when it does (the detection task re-triggers this one; it is idempotent and skips already-compared pairs).
3. Crops the target region (fresh blob name per attempt).
4. **Reuse check** (signature references only): see below. A match is stored and the model is **not** called.
5. Otherwise calls `LLMService.compare_signatures(ref_image, target_image)` — a strict-JSON vision call at `temperature = 0` returning `{reasoning, result}` (`reasoning` first, so the model commits its thinking before naming a verdict). Advisory wording only.
6. Stores a `signature_matches` row, audits `signature_comparison_completed` `{comparisons_attempted, comparisons_stored, reference_kind, identical_reuse_found}`, and re-scores the case.

If the vision model is not configured, non-reuse targets get `cannot_determine`; the reuse check still runs because it needs no model.

### The reuse check (classical CV, no model)

A model asked "do these look alike?" calls two identical images "consistent" — reassurance, when the real signal is the opposite: nobody signs pixel-for-pixel the same twice, so an exact match means the same image file was inserted into both documents. That can be legitimate (a stored e-signature image), so on its own it is a modest flag.

`find_signature_reuse` trims both crops to their ink, requires similar proportions (± 6 %), resamples to 256 px wide, blurs, and takes the **maximum normalized cross-correlation** allowing ±4 px of shift. Measured on real files: identical copies score 0.999, a 60 % rescale 0.97, JPEG recompression 0.999, a wobbled re-signing 0.40, a different signature 0.17. The threshold is **0.93**.

On a match it reads the **printed line beneath each signature** from the PDF's own text layer (`printed_label_below`, the first text line within 7 % of page height below the box) and compares them with `rapidfuzz.fuzz.token_set_ratio` (< 60 = different people):

| Verdict | Meaning |
|---|---|
| `reused_different_signer` | identical image **and** the printed signer text differs |
| `identical_reuse` | identical image; printed text similar or unreadable (e.g. a scan) |

It never runs for **stamp** references — a company's rubber stamp is legitimately identical every time.

## Output shape

Verdicts (`signature_match_result` enum) and their UI labels:

| `result` | Source | Label |
|---|---|---|
| `consistent` | vision model | Visually consistent with reference on file |
| `possibly_consistent` | vision model | Possibly consistent with reference on file |
| `inconsistent` | vision model | Visual appearance inconsistent with reference |
| `cannot_determine` | vision model / failure | Cannot determine — insufficient image quality |
| `identical_reuse` | pixel check | Pixel-identical to reference — same image appears to be reused |
| `reused_different_signer` | pixel check | Pixel-identical to reference, but printed signer differs |

`SignatureMatchResponse` ( `schemas/signature.py` ):


```
{"id": "uuid", "document_id": "uuid", "case_id": "uuid", "signature_reference_id": "uuid",
 "reference_person_name": "Sultan Al-Dhaheri", "comparison_scope": "in_case",
 "result": "reused_different_signer", "result_label": "Pixel-identical to reference, but printed signer
differs",
 "reasoning": "The signature here is pixel-for-pixel identical to the reference … reference: 'م . سلطان
الظاهري'; this document: 'سعيد المنصوري' …",
 "compared_at": "2026-09-20T23:10:15Z"}
```


There is **no numeric score** on the model verdicts, by design.

## Known limitations

- **Weak, advisory signal.** Handwriting varies; a model's visual opinion is not identity matching. Copy must never claim verification.
- **Reviewer-initiated.** With no reference, no comparison happens. A reference is per case; the library is data collection only.
- **Detection depends on a vision model** and, for non-embedded marks, on its localisation.
- The reuse check needs a **text layer** to compare signer names (scans fall back to `identical_reuse` ), and only sees marks that were cropped in the first place.
- The reference's kind is inferred from overlap; a box that overlaps neither region leaves it unknown (and skips the reuse check).
- The task and endpoints are PDF-only.

## Risk rules fed

| `rule_id` | Condition | Weight | Severity |
|---|---|---|---|
| `signature.expected_missing` | detection `flag` (expected but none located) | 10 | low |
| `signature.stamp_issuer_mismatch` (v2: names normalised) | field validation `stamp_issuer_consistency` flag | 15 | medium |
| `signature.synthetic_stamp` | field validation `stamp_authenticity` flag (stamp is text / vector lines) | 25 | high |
| `signature.synthetic_stamp_unsigned` | field validation `synthetic_stamp_unsigned` flag | 5 | low |
| `signature.inconsistent_with_reference` | verdict `inconsistent` | 25 | medium |
| `signature.possibly_consistent_only` | verdict `possibly_consistent` | 5 | low |
| `signature.identical_reuse` | verdict `identical_reuse` | 15 | medium |
| `signature.reused_different_signer` | verdict `reused_different_signer` | 35 | high |

`cannot_determine` and `consistent` fire no default rule (a company's Reviewer L2 can add one for `cannot_determine` ).

# 10 — Duplicate & near-duplicate detection

**What it detects:** a page that looks the same as a page in a document seen before — a resubmitted invoice, the same quotation re-uploaded under a slightly different vendor name, a re-scan, a re-saved copy. It catches what a byte-level SHA-256 comparison ( `documents.file_hash` ) cannot: re-saving, re-compressing or trivially editing a file changes its bytes but not how it *looks*.

**Code:** `services/forensics/duplicate_check.py` ( `run_duplicate_check` ), run by `tasks/duplicate_check_task.py` ( `run_duplicate_check_task` ); model `models/document_page_hash.py` ( `document_page_hashes` ); stores `check_type = duplicate_detection` .

## Algorithm

1. Render each page ( `render_pdf_pages` , 200 DPI) and compute a **perceptual hash**: `imagehash.phash` (64-bit, `hash_size = 8` ) of the RGB page image → a 16-character hex string, one per page ( `compute_page_hashes` ).
2. `find_matches` loads **every stored page hash of every other document of the same company** (joined to filename, case id and case number) and, for each new page, finds the single **closest** prior page whose Hamming distance is ≤ `DUPLICATE_HASH_HAMMING_THRESHOLD` (default **5** of 64 bits — deliberately conservative: near- identical resubmissions, not merely similar layouts such as two invoices from one template). One match per new page at most.
3. `store_page_hashes` writes this document's own hashes **regardless of outcome**, so future uploads can match against it — as an upsert, one row per (document, page), so a redelivered task can't duplicate them. (The hash is a side effect of the check; there is no separate step.)
4. `result` is `flag` if any page matched, else `pass` .

Match details (filename, case number) are captured **at check time**, not resolved later, so a finding stays readable even if the matched document's case is one the current viewer has never opened.

## Output shape (real)

```
{
  "result": "flag",
  "details": [{
    "finding": "near_duplicate_page",
    "severity": "high",
    "page": 1,
    "description": "Page 1 is a near-identical match (perceptual hash Hamming distance 0) to page 1 of
\"Sample15_Arabic_Case_Evidence.pdf\" in case CASE-8D8C00E1 — a possible duplicate or near-duplicate
resubmission.",
    "data": {
      "matched_document_id": "uuid", "matched_document_filename": "Sample15_Arabic_Case_Evidence.pdf",
      "matched_case_id": "uuid", "matched_case_number": "CASE-8D8C00E1",
      "matched_page": 1, "distance": 0
    }
  }]
}
```

Same `{finding, severity, description, page, data}` shape as ELA/copy-move (no `bounding_box` — the whole page is the match), so the shared findings-list UI renders it. In the UI the finding links to the matched case.

`document_page_hashes` : `id` , `company_id` , `document_id` (FK), `page_number` (1-based), `phash` ( `varchar(32)` , indexed), timestamps; unique on ( `document_id` , `page_number` ).

## Field-aware: resubmission or the same template

A page hash sees layout, so different invoices printed on one template all match. Once both documents' fields are extracted (the check runs in parallel with extraction, so this happens after field validation, or straight away when the fields are already there — `refine_duplicate_check`), each page match that is not the very same file is compared on the invoice on that page (`extracted_fields.invoices` for a multi-invoice file): invoice number, date, amount and student / customer.

- all compared fields equal (at least two) → stays `near_duplicate_page`, high: the same document submitted again (`data.verdict` = "resubmission");
- any of them different → `same_template_page`, **low** (shown, not scored): the same template with other values, listed with the differences (`data.field_comparison`);
- fields not available yet → left as it is.

## Known limitations

- **It matches against everything the company ever stored**, including your own test uploads. Re-uploading the same sample file in a new case (or uploading the two halves of one case that share a page) **will flag**. This is correct behaviour, and the most common source of "unexpected" flags during testing.
- **Never across companies.** The same invoice submitted to two different client companies is not detected: matching across tenants would reveal another company's filenames and case numbers. This is a deliberate privacy choice.
- **Same-case matches count** (see the note at the top) — despite the rule's `cross_case` name.
- Full-scan comparison per page: cost grows linearly with the number of stored pages. There is no index (e.g. BK-tree/LSH); fine for thousands of pages, not millions.
- pHash is sensitive to layout, not content: two documents that differ only in a changed amount will hash almost identically (that is the intent, and why it is high severity), whereas a heavy crop or rotation can defeat it.
- No cleanup: hashes of documents whose cases are closed stay forever. Two uploads processed at exactly the same moment by parallel forensics processes may not see each other.
- PDF only (as are uploads).

## Risk rules fed

| `rule_id` | Finding | Weight | Severity |
|---|---|---|---|
| `duplicate.cross_case_match` (v2: resubmissions only) | `near_duplicate_page` | 40 | high |

This is the heaviest single seeded rule: with the default thresholds (medium ≥ 30, high ≥ 60) one duplicate page alone makes a case **medium** risk; with any other signal it approaches **high**.

# 11 — Risk scoring engine

**What it does:** turns every stored check result into a single **explainable** 0–100 score, a tier ( `low` / `medium` / `high` ) and a list of human-readable reasons. It is a **transparent, weighted rules engine — not a model**. Every point in a score traces to a named rule and a sentence a reviewer can read.

**Code:** `services/risk_scoring_service.py` ( `score_case` , `evaluate_rules` , `pipeline_status` ), `tasks/risk_scoring_task.py` ( `score_case_task` ), `services/risk_rule_seed.py` (the default rules and `seed_risk_rules` ), `services/risk_rule_catalog.py` (what a rule editor may build), `api/settings.py` (rule and threshold administration), `api/platform.py` (the platform rule template), models `risk_rules` , `risk_settings` , `risk_rule_templates` , `case_risk_assessments` .

**Rules and thresholds are per company.** Each company has its own `risk_rules` and `risk_settings` , edited by its Reviewer L2s under *Settings › Risk Rules*; a case is always scored with its own company's rules. A new company starts from the platform's **rule template** (see Rule templates).

## When a case is scored

Every pipeline task ends with `request_case_scoring(case_id)` , which enqueues `score_case_task` on `forensics_queue` (and never raises if the broker is down). `score_case(db, company_id, case_id)` then:

1. **Gate —** `pipeline_status(case)` must be `complete` . For every document: extraction terminal; for every `complete` document: `field_validation` and `issuer_verification` terminal; for every **PDF**: `metadata_forensics` , `error_level_analysis` , `copy_move_detection` , `duplicate_detection` , `visual_inconsistency_review` terminal ( `completed` or `failed` ); and, when the case has ≥ 2 documents, a `cross_document_check_completed` event **newer than the newest upload**. Anything missing is listed in `pipeline.pending` and shown to the reviewer. ( `signature_stamp_detection` is intentionally **not** required — see data-flow.) Scoring a half-checked case would under-report and could let it be approved.
2. **Gather evidence:** the **latest completed** check per (document, check_type), the case's `cross_document_findings` , and all `signature_matches` .
3. **Evaluate every current rule of the case's company** (highest `version` per `rule_id` , inactive included but never fired).
4. **Fingerprint** = SHA-256 of the sorted document ids + the sorted `(rule_id, document_id, count)` of fired rules. If it equals the latest assessment's `evidence_fingerprint` , **stop** — no new row.
5. **Score:** `raw_score = Σ weight` of fired rules, except that the **metadata rules ( `metadata.*` ) together add at most the company's metadata cap** ( `risk_settings.metadata_score_cap` , default **40**, editable next to the thresholds; 100 = no cap) — one edit leaves several metadata traces (modified after created, the edit history, a date after the file's creation, the editor's name), which must not max the score on their own. A cap that applied is recorded in the snapshot as `group_caps: {"metadata": {"cap", "points"}}` and shown on the case and in the report. `score = clamp(round(raw_score), 0, 100)` ; `tier` from the company's thresholds ( `score ≥ high → high` , `≥ medium → medium` , else `low` ; defaults **30 / 60**).
6. **INSERT** a `case_risk_assessments` row (one per (case, fingerprint) — a unique constraint, so a redelivered task can't insert it twice), set `cases.risk_tier` , write `risk_assessment_completed` , and — if the case is still `submitted` / `under_automated_review` — transition it to `pending_manual_review` via the workflow service.

The fingerprint deliberately excludes weights, so a re-trigger can never quietly re-score an old case under newly tuned weights: a case is only re-scored when its *evidence* changes.

## Rules

A rule is a row in the company's `risk_rules` — configuration, not code:

| Column | Meaning |
|---|---|
| `rule_id` | Stable key, `family.name` (e.g. `metadata.editing_software_detected`) |
| `category` | `forensics`, `consistency`, `verification` or `duplication` |
| `check_type` | Which check's output it reads (used for display/grouping) |
| `condition` | JSON: **what to match** (below) — created by the catalog, not hand-written |
| `weight` | Points added when it fires (−100…100) |
| `severity` | `info` / `low` / `medium` / `high` — shown beside the reason |
| `reason_template` | Sentence with `{placeholders}` the reviewer reads |
| `is_active` | Off = evaluated but never fires |
| `version`, `effective_from`, `updated_by`, `change_note` | Versioning trail |

### Condition kinds ( `condition.match` )

| Kind | Matches | Fires |
|---|---|---|
| `finding` | Findings in a check's `details` **list** whose `finding` name equals `condition.finding` and (optionally) whose severity ∈ `severity_in` | **Once per document**, `count` = number of matching findings |
| `check_result` | The check's overall `result` equals `condition.result` (e.g. `flag`) | Once per document |
| `sub_check` | One sub-check in a **dict**-shaped `details` (field validation) with `status == flag` | Once per document |
| `cross_document` | A `cross_document_findings` row for `condition.field_name`, optionally filtered by severity | **Once per case**, `count` = matching findings |
| `signature_match` | A `signature_matches` row whose `result` equals `condition.result` | Once per target document |

A rule fires **once per hit, regardless of how many findings a document has** — five separate `orphaned_objects` findings on one document add the weight once, not five times.

### Reason templates

`render_reason` fills the template from the hit's context. Available placeholders: `{document}` (filename), `{count}`, `{page}`, `{description}`, `{reason}`, `{issuer}`, `{person}`, `{field}`, plus any key in a finding's `data`. A placeholder with no value renders as an **empty string** (never an error). The **rendered** text is what is stored and shown.

### The default rules (64)

These are the rules in the platform template, i.e. what a new company starts with. Full tables of `rule_id`, condition and default weight are in each stage's doc: field ( `field.*`, 15) in 03, 04 (3), 05 (1), 06 ( `metadata.*`, 22), 06a (2), 06b ( `content.spelling_error`, 1), 07 (3), 07a ( `content.*` ghost text, 2), 08 (6), 09 (8), 10 (1). Heaviest single rules:

`duplicate.cross_case_match` (40), `copy_move.cluster_detected` / `cross_doc.amount_mismatch` / `signature.reused_different_signer` (35). Because weights sum, a case with an amount mismatch (35) plus a duplicate page (40) reaches 75 → **high**.

### Unscored findings and the coverage warning

Medium/high findings that **no rule covers** are logged as warnings ("No risk rule covers …") so a new finding type cannot silently go unscored. `metadata_forensics_correlation` and `experimental_signal_notice` are excluded on purpose (see 08).

## Rule templates — how a new company gets its rules

- `risk_rule_templates` is a platform-level table (no `company_id`; only the platform database role can reach it). Platform admins edit it under **Platform › Rule templates** (`/platform/risk-rule-templates`): weight, severity, active flag, or a new rule. Each template edit is a platform audit event (`risk_rule_template_created/updated`, old → new values).
- Creating a company (`POST /platform/companies`) calls `seed_risk_rules(db, company_id)`, which copies every **active** template row into that company's `risk_rules` as version 1, plus default thresholds (30 / 60) into `risk_settings`.
- From then on the copy is the company's own. **Editing the template never changes an existing company** — only companies created afterwards. This is the same "no retroactive change" principle as rule versioning below.
- The template was seeded from the Default Company's current rules (38).

## Versioning — history is never rewritten

- **Rules are immutable.** `PATCH /settings/risk-rules/{rule_id}` (weight, severity, `is_active`) INSERTs a new row with `version + 1` in the company's rules; the old row is untouched (the database grants only SELECT + INSERT on `risk_rules`). The current rule is the highest version — there is no mutable "is_current" flag.
- **Creating a rule** (`POST /settings/risk-rules`) inserts version 1; it applies to cases scored from then on. The editor (a Reviewer L2) picks a match kind and parameters from the catalog (`GET /settings/risk-rule-options`); nobody writes JSON, and a choice the engine cannot evaluate is a 422. Wording (`reason_template`) and `condition` cannot be edited after creation.
- **Every assessment freezes what it used**: the rendered reason text, and `risk_rules_version_snapshot = {"rules": [{rule_pk, rule_id, version, weight, severity}, …], "thresholds": {"medium": …, "high": …}}`. Retuning weights or thresholds later cannot change a historical case's score or reasons.

## Output shape

**Stored** — `case_risk_assessments` (insert-only), the full record:

```
{
  "id": "uuid", "case_id": "uuid", "computed_at": "2026-09-20T17:09:12Z",
  "score": 75, "raw_score": 75.0, "tier": "high",
  "triggered_reasons": [                  // sorted by weight desc, then rule_id
    {"rule_id": "duplicate.cross_case_match", "rule_version": 1, "category": "duplication",
     "check_type": "duplicate_detection", "severity": "high", "weight": 40.0,
     "reason": "Page 1 of 'Invoice.pdf' is a near-identical match … to page 1 of 'X.pdf' in case CASE-….",
     "document_id": "uuid", "document_filename": "Invoice.pdf"},   // both null for case-level rules
    {"rule_id": "cross_doc.amount_mismatch", "rule_version": 1, "weight": 35.0, "…": "…"}
  ],
  "risk_rules_version_snapshot": {        // exactly which immutable versions produced the score
    "rules": [{"rule_pk": "uuid", "rule_id": "duplicate.cross_case_match", "version": 1, "weight": 40.0,
"severity": "high"}, "…"],
    "thresholds": {"medium": 30, "high": 60}
  },
  "evidence_fingerprint": "9f2c…"         // SHA-256 used to skip unchanged re-scores
}
```

**Exposed by the API** — `RiskAssessmentSchema` on `GET /cases/{id}` (`assessment`; reviewers and platform admins only, `null` for a submitter) is a **subset**: `{id, score, raw_score, tier, computed_at, triggered_reasons[], thresholds}`, where each reason is a `RiskReasonSchema` with the fields shown above and `thresholds` is copied from the snapshot. The rule-version snapshot and the evidence fingerprint stay internal (the snapshot is printed in the report's Section 6, the fingerprint is never shown).

## What the tier does

| Tier | Effect in the code |
|---|---|
| any | Case flag/badge colour; case moves to `pending_manual_review`; report Executive Summary |
| `low` | Reviewer may approve without a written justification |
| `medium` / `high` | Approving needs a justification of ≥ 10 characters (422 otherwise); rejecting/escalating always possible |
| — | **No** auto-approval, **no** automatic escalation to L2, **no** SLA (none of these exist) |

Submitters see only a neutral "In review" flag, never the tier, score or reasons — so a bad actor cannot learn what to evade.

## Known limitations

- **Weights are judgment, not calibration.** The seeded values were chosen to be sensible, not fitted to data; tune them against real cases (Settings › Risk Rules, per company; or the platform template for future companies). The thresholds default to 30/60.
- Rules only see what checks store. A check that fails (status `failed`) contributes nothing, and the case still scores — a `failed` forensic task under-reports risk silently (the failure is in the audit log and shown as a failed check, but no rule fires on it).
- Additive scoring: many weak signals can outweigh one strong one, and signals are not de-correlated (an edited PDF may trip metadata, ELA and visual rules for one root cause).
- The default rules encode this deployment's judgment about weak signals (ELA, AI-generation and signature checks are deliberately light).
- A case re-scores when evidence changes (for example a signature comparison finishing), so the score a reviewer first saw can change while they are looking at the case.

# 12 — Short check summaries

**What:** the short, readable form of every check — what the case page shows first and what the report puts first, ahead of the full explanation.

**Code:** `services/check_summaries.py` ( `summarize_check` , `summarize_document_checks` , `risk_reason_short` ). Computed when read from the stored results, so cases checked before it existed read the same way; nothing here changes a result or a score.

## Shape

```
{
  "headline": "3 issues: Text retyped, Totals retyped, Amounts retyped",
  "items": [{
    "severity": "high",
    "title": "Amounts retyped",
    "text": "1,450.00 ×2 · 16,450.00 ×2 · 2,300.00 ×2 · 13,350.00 ×4 — in a second copy of Times New Roman
(a digits-only copy); 4,575.00 in the same column is original",
    "detail": "Page 1: '1,450.00' is set in a second embedded copy of … (9 more of the same kind.)",
    "page": 1, "findings": ["font_subset_split"],
    "image_png_base64": null, "hint": null
  }],
  "notes": ["1 of 1 page(s) analysed; 2 font families: times, courier."]
}
```

- **One item per problem**, in one line with the actual values. Findings with one cause are grouped (17 font findings → 3 items); `detail` is the first full explanation, plus how many more of the same kind.
- **Notes** are info lines (what was searched, what passed, why a check was limited).
- **Headline**: "N issues: …" when something is scored; "Limited — …", "Not applicable — …", "Not checked — …", "Failed to run: …" or "No issues found" otherwise.
- The stamp sub-checks of field validation (stamp names the issuer, stamp typed into the file, no signature either) are shown with **signature / stamp detection**, not field validation.
- Passed sub-checks worth reading are spelled out as notes ("Dates in order: dated 17 Aug 2023 → printed 18 Aug 2023 → PDF created 21 Aug 2023", "Installments add up: …"); the rest are listed as "Passed: …".
- A check whose only findings are low (shown, not scored) reads **Review** in the report, not Flag.
- A not-applicable check gives its stored reason ("Not applicable — born-digital, no images").

## Where it is used

| Place | Short first | Full text |
|---|---|---|
| Case page — each check card | headline under the title; items with **Why?** | "Technical details" (raw findings) |
| Case page — explainable findings, decision dialog | rule title + short line | **Why?** (the stored reason) |
| Case queue / risk banner | top 3 rule titles (+N more) | — |
| Report §3 Explainable Findings | **title — short line** | the stored reason, grey |
| Report §4 All Checks | headline + one bullet per item / note | — |

| Place | Short first | Full text |
|---|---|---|
| Report §5 Exceptions | **title — short line** (one row per item, all its highlights) | first explanation, grey |

API: `DocumentCheckSummary.summary`; `RiskReasonSchema.title` / `.short`.

# Database schema

PostgreSQL, 21 application tables plus Alembic's `alembic_version` (current head `a2d6f9c3e7b4`). All primary keys are UUIDs generated by the application. The entity-relationship diagram is in erd.md; tenant isolation is described in multi-tenancy.md.

### Conventions

- **Multi-tenant.** Every tenant-owned table has `company_id` (FK → `companies.id`). Row-Level Security is enabled on them with a `tenant_isolation` policy for `fddt_app` (`company_id = fddt_current_company_id()`, the transaction-local `app.current_company_id`) and a `platform_all` policy for `fddt_platform`. No role has BYPASSRLS.
- **Insert-only tables** (the app roles are granted only SELECT + INSERT): `audit_log`, `case_risk_assessments`, `case_reports`, `risk_rules` (rule edits insert a new version).
- **JSON columns** (`jsonb`) hold check output and snapshots; their shapes are documented in the pipeline docs.
- **Bounding boxes** anywhere in JSON use one convention: `{page (1-based), x, y, width, height}` as 0-1 fractions of the page, origin top-left.
- **Enums** are native Postgres enums; their values are listed under each table and again in the index at the bottom.

| Table | Tenancy | `fddt_app` | Purpose |
|---|---|---|---|
| `companies` | RLS (own row) | SELECT | One row per client company (tenant). `is_active = false` suspends it: its users can't sign in and existing tokens stop working. |
| `company_usage_stats` | tenant · RLS | SELECT, INSERT, UPDATE | Per-company, per-UTC-day usage counters (cases, documents, files, storage bytes) for the platform billing dashboard; bumped in the same transaction as each upload/case/report and reconciled nightly. |
| `users` | tenant · RLS | SELECT | Accounts (email + bcrypt-hashed password). Company users have a `company_id` and a ranked role (`user` / `reviewer_l1` / `reviewer_l2`); platform admins have `role = platform_admin` and no company (CHECK constraint). |
| `cases` | tenant · RLS | SELECT, INSERT, UPDATE | The unit of work: one submission of one or more documents. Carries workflow `status`, the owning reviewer tier `assigned_tier` and the latest `risk_tier`. |
| `documents` | tenant · RLS | SELECT, INSERT, UPDATE | One row per uploaded PDF. The original bytes live in Blob Storage (immutable); this row holds the SHA-256, the extraction results (`extracted_fields`, `ocr_text`) and `processing_status`. |
| `document_checks` | tenant · RLS | SELECT, INSERT, UPDATE | One row per (document, check type) — the latest run of each automated check (extraction stages, every forensic check, signature detection…), upserted so a redelivered task never duplicates it. `result` is the check's raw JSON output; the run history is in `audit_log`. |
| `document_page_hashes` | tenant · RLS | SELECT, INSERT, UPDATE | Perceptual hash (pHash) of every rendered PDF page, one per (document, page), used to find duplicate/near-duplicate pages within the same company. |
| `cross_document_findings` | tenant · RLS | SELECT, INSERT, UPDATE, DELETE | Case-level reconciliation results: a shared field (amount, date, issuer) that disagrees between documents in the same case. |

| Table | Tenancy | `fddt_app` | Purpose |
|---|---|---|---|
| `issuer_registry` | tenant · RLS | SELECT, INSERT, UPDATE | A company's known vendors/schools/etc. that extracted issuer names are fuzzy-matched against. Rows are deactivated, never deleted. A new company starts empty. |
| `risk_rules` | tenant · RLS | SELECT, INSERT | A company's configurable scoring rules. Rows are immutable and versioned: an edit inserts `version + 1`; the current rule is the highest version. Seeded from `risk_rule_templates` when the company is created. |
| `risk_rule_templates` | platform-only | — | PLATFORM-LEVEL (no `company_id`): the default rule set copied into every NEW company's `risk_rules`. Edited by platform admins; edits never change existing companies. Only `fddt_platform` can access it. |
| `risk_settings` | tenant · RLS | SELECT, INSERT, UPDATE | One row per company holding its `medium` / `high` score thresholds and the metadata score cap (`metadata_score_cap`, default 40: the most points all `metadata.*` rules add together). |
| `case_risk_assessments` | tenant · RLS | SELECT, INSERT | One row per scoring run of a case (insert-only history; unique per (case, evidence fingerprint)): score, tier, frozen reason text and a snapshot of the exact rule versions and thresholds used. |
| `risk_scores` | tenant · RLS | SELECT | LEGACY. Created by the initial migration before the versioned engine existed; nothing in the application reads or writes it. `case_risk_assessments` replaced it. |
| `case_actions` | tenant · RLS | SELECT, INSERT | Reviewer-facing history of human actions on a case (approve / reject / escalate …) with the actor's role at the time. Complements, and is also mirrored into, `audit_log`. |
| `case_reports` | tenant · RLS | SELECT, INSERT | History of generated per-case forensic PDF reports (insert-only). The PDF lives in Blob Storage under `companies/{company}/cases/{case}/reports/`; this row stores its SHA-256, size and the assessment it reflects. |
| `audit_log` | tenant · RLS | SELECT, INSERT | Append-only system of record: every automated check result, human action and settings change is inserted here and never updated or deleted. `company_id` NULL = a platform-level event (company created, template edits, usage reconciliation, platform-admin access), visible only to platform admins; a company user's *user created/updated* goes into that company's log. |
| `signature_references` | tenant · RLS | SELECT, INSERT, UPDATE | A signature/stamp region drawn on a document, cropped and stored as a PNG, used as the reference for comparisons. |
| `signature_matches` | tenant · RLS | SELECT, INSERT, UPDATE | Result of comparing one reference against the signature region of another document in the same case (vision-model verdict or pixel-identical reuse verdict); one row per (reference, document, scope). |
| `bulk_upload_cases` | tenant · RLS | SELECT, INSERT, UPDATE | |
| `bulk_uploads` | tenant · RLS | SELECT, INSERT, UPDATE | |

## companies

One row per client company (tenant). `is_active = false` suspends it: its users can't sign in and existing tokens stop working.

| Column | Type | Null | Default | Notes |
|---|---|---|---|---|
| `id` | uuid | no | | PK |
| `name` | varchar(255) | no | | unique |
| `is_active` | boolean | no | `true` | |
| `created_at` | timestamptz | no | `now()` | |
| `updated_at` | timestamptz | no | `now()` | |
| `max_file_size_mb` | integer | no | | |
| `max_zip_size_mb` | integer | no | | |

- **Index** `companies_name_key` on (name) — unique
- **Check** `ck_companies_max_file_size_mb_positive` : `CHECK ((max_file_size_mb > 0))`
- **Check** `ck_companies_max_zip_size_mb_positive` : `CHECK ((max_zip_size_mb > 0))`
- **Row-Level Security** on: `platform_all` (ALL to fddt_platform); `tenant_isolation` (ALL to fddt_app)
- **Grants** `fddt_app` : SELECT · `fddt_platform` : SELECT, INSERT, UPDATE, DELETE

## company_usage_stats

Per-company, per-UTC-day usage counters (cases, documents, files, storage bytes) for the platform billing dashboard; bumped in the same transaction as each upload/case/report and reconciled nightly.

| Column | Type | Null | Default | Notes |
|---|---|---|---|---|
| `company_id` | uuid | no | | PK, FK → `companies.id` |
| `usage_date` | date | no | | PK |
| `cases_created` | integer | no | `0` | |
| `documents_uploaded` | integer | no | `0` | |
| `files_stored` | integer | no | `0` | |
| `storage_bytes` | bigint | no | `'0'::bigint` | |
| `updated_at` | timestamptz | no | `now()` | |

- **Row-Level Security** on: `platform_all` (ALL to fddt_platform); `tenant_isolation` (ALL to fddt_app)
- **Grants** `fddt_app` : SELECT, INSERT, UPDATE · `fddt_platform` : SELECT, INSERT, UPDATE, DELETE

## users

Accounts (email + bcrypt-hashed password). Company users have a `company_id` and a ranked role ( `user` / `reviewer_l1` / `reviewer_l2` ); platform admins have `role = platform_admin` and no company (CHECK constraint).

| Column | Type | Null | Default | Notes |
|---|---|---|---|---|
| `id` | uuid | no | | PK |
| `created_at` | timestamptz | no | | |
| `updated_at` | timestamptz | no | | |
| `email` | varchar(320) | no | | unique |
| `hashed_password` | varchar(255) | no | | |
| `full_name` | varchar(255) | yes | | |
| `role` | enum `user_role` | no | | |
| `is_active` | boolean | no | | |
| `company_id` | uuid | yes | | FK → `companies.id` |

- **Index** `ix_users_company_id` on (company_id)
- **Index** `ix_users_email` on (email)
- **Index** `users_email_key` on (email) — unique
- **Check** `ck_users_platform_admin_has_no_company` : `CHECK (((role = 'platform_admin'::user_role) = (company_id IS NULL)))`
- **Enum** `user_role` : `user` , `reviewer_l1` , `reviewer_l2` , `admin` , `platform_admin`
- **Row-Level Security** on: `platform_all` (ALL to fddt_platform); `tenant_isolation` (ALL to fddt_app)
- **Grants** `fddt_app` : SELECT · `fddt_platform` : SELECT, INSERT, UPDATE, DELETE

## cases

The unit of work: one submission of one or more documents. Carries workflow `status` , the owning reviewer tier `assigned_tier` and the latest `risk_tier` .

| Column | Type | Null | Default | Notes |
|---|---|---|---|---|
| `id` | uuid | no | | PK |
| `created_at` | timestamptz | no | | |
| `updated_at` | timestamptz | no | | |
| `case_number` | varchar(64) | no | | unique |
| `submitted_by_user_id` | uuid | no | | FK → `users.id` |
| `status` | enum `case_status` | no | | |
| `risk_tier` | enum `risk_tier` | yes | | |
| `external_ref_id` | varchar(255) | yes | | |
| `case_type` | enum `case_type` | no | | |
| `assigned_tier` | enum `case_tier` | no | `'l1'::case_tier` | |
| `company_id` | uuid | no | | FK → `companies.id` |
| `bulk_upload_id` | uuid | yes | | FK → `bulk_uploads.id` |

| Column | Type | Null | Default | Notes |
|---|---|---|---|---|
| `reference_label` | varchar(255) | yes | | |

- **Index** `cases_case_number_key` on (case_number) — unique
- **Index** `ix_cases_bulk_upload_id` on (bulk_upload_id)
- **Index** `ix_cases_case_number` on (case_number)
- **Index** `ix_cases_company_id` on (company_id)
- **Enum** `case_status`: `submitted`, `under_automated_review`, `pending_manual_review`, `auto_approved`, `escalated`, `under_investigation`, `approved`, `rejected`, `closed`
- **Enum** `risk_tier`: `low`, `medium`, `high`
- **Enum** `case_type`: `school_document`, `vendor_invoice`, `commercial_invoice`, `procurement_documentation`, `quotation`, `travel_reimbursement`, `other`
- **Enum** `case_tier`: `l1`, `l2`
- **Row-Level Security** on: `platform_all` (ALL to fddt_platform); `tenant_isolation` (ALL to fddt_app)
- **Grants** `fddt_app`: SELECT, INSERT, UPDATE · `fddt_platform`: SELECT, INSERT, UPDATE, DELETE

## documents

One row per uploaded PDF. The original bytes live in Blob Storage (immutable); this row holds the SHA-256, the extraction results (`extracted_fields`, `ocr_text`) and `processing_status`.

| Column | Type | Null | Default | Notes |
|---|---|---|---|---|
| `id` | uuid | no | | PK |
| `created_at` | timestamptz | no | | |
| `updated_at` | timestamptz | no | | |
| `case_id` | uuid | no | | FK → `cases.id` |
| `uploaded_by_user_id` | uuid | no | | FK → `users.id` |
| `original_filename` | varchar(512) | no | | |
| `blob_storage_path` | varchar(1024) | no | | |
| `file_hash` | varchar(64) | no | | |
| `content_type` | varchar(255) | yes | | |
| `file_size_bytes` | integer | yes | | |
| `document_type` | varchar(128) | yes | | |
| `processing_status` | enum `document_processing_status` | no | | |
| `extracted_fields` | jsonb | yes | | |
| `ocr_text` | text | yes | | |
| `processing_error` | text | yes | | |
| `company_id` | uuid | no | | FK → `companies.id` |

- **Index** `ix_documents_case_id` on (case_id)
- **Index** `ix_documents_company_id` on (company_id)
- **Index** `ix_documents_file_hash` on (file_hash)
- **Enum** `document_processing_status`: `pending`, `processing`, `complete`, `failed`
- **Row-Level Security** on: `platform_all` (ALL to fddt_platform); `tenant_isolation` (ALL to fddt_app)
- **Grants** `fddt_app`: SELECT, INSERT, UPDATE · `fddt_platform`: SELECT, INSERT, UPDATE, DELETE

## document_checks

One row per (document, check type) — the latest run of each automated check (extraction stages, every forensic check, signature detection…), upserted so a redelivered task never duplicates it. `result` is the check's raw JSON output; the run history is in `audit_log`.

| Column | Type | Null | Default | Notes |
|---|---|---|---|---|
| `id` | uuid | no | | PK |
| `created_at` | timestamptz | no | | |
| `updated_at` | timestamptz | no | | |
| `document_id` | uuid | no | | FK → `documents.id` |
| `check_type` | enum `document_check_type` | no | | |
| `status` | enum `document_check_status` | no | | |
| `result` | jsonb | yes | | |
| `confidence` | double precision | yes | | |
| `error_message` | varchar(2048) | yes | | |
| `company_id` | uuid | no | | FK → `companies.id` |

- **Index** `ix_document_checks_company_id` on (company_id)
- **Index** `ix_document_checks_document_id` on (document_id)
- **Index** `uq_document_checks_document_check_type` on (document_id, check_type) — unique
- **Enum** `document_check_type`: `ocr_layout`, `classification`, `field_extraction`, `issuer_verification`, `signature_stamp_detection`, `metadata_forensics`, `error_level_analysis`, `copy_move_detection`, `duplicate_detection`, `ai_content_detection`, `field_validation`, `visual_inconsistency_review`, `font_consistency`, `ghost_content`, `spelling_consistency`
- **Enum** `document_check_status`: `pending`, `running`, `completed`, `failed`
- **Row-Level Security** on: `platform_all` (ALL to fddt_platform); `tenant_isolation` (ALL to fddt_app)
- **Grants** `fddt_app`: SELECT, INSERT, UPDATE · `fddt_platform`: SELECT, INSERT, UPDATE, DELETE

## document_page_hashes

Perceptual hash (pHash) of every rendered PDF page, one per (document, page), used to find duplicate/near-duplicate pages within the same company.

| Column | Type | Null | Default | Notes |
|---|---|---|---|---|
| `id` | uuid | no | | PK |
| `created_at` | timestamptz | no | | |
| `updated_at` | timestamptz | no | | |
| `document_id` | uuid | no | | FK → `documents.id` |
| `page_number` | integer | no | | |
| `phash` | varchar(32) | no | | |
| `company_id` | uuid | no | | FK → `companies.id` |

- **Index** `ix_document_page_hashes_company_id` on (company_id)
- **Index** `ix_document_page_hashes_document_id` on (document_id)
- **Index** `ix_document_page_hashes_phash` on (phash)
- **Index** `uq_document_page_hashes_document_page` on (document_id, page_number) — unique
- **Row-Level Security** on: `platform_all` (ALL to fddt_platform); `tenant_isolation` (ALL to fddt_app)
- **Grants** `fddt_app` : SELECT, INSERT, UPDATE · `fddt_platform` : SELECT, INSERT, UPDATE, DELETE

## cross_document_findings

Case-level reconciliation results: a shared field (amount, date, issuer) that disagrees between documents in the same case.

| Column | Type | Null | Default | Notes |
|---|---|---|---|---|
| `id` | uuid | no | | PK |
| `created_at` | timestamptz | no | | |
| `updated_at` | timestamptz | no | | |
| `case_id` | uuid | no | | FK → `cases.id` |
| `field_name` | varchar(128) | no | | |
| `finding_type` | varchar(128) | no | | |
| `severity` | enum `finding_severity` | no | | |
| `description` | varchar(2048) | no | | |
| `document_ids` | jsonb | yes | | |
| `company_id` | uuid | no | | FK → `companies.id` |

- **Index** `ix_cross_document_findings_case_id` on (case_id)
- **Index** `ix_cross_document_findings_company_id` on (company_id)
- **Enum** `finding_severity` : `info` , `low` , `medium` , `high`
- **Row-Level Security** on: `platform_all` (ALL to fddt_platform); `tenant_isolation` (ALL to fddt_app)
- **Grants** `fddt_app` : SELECT, INSERT, UPDATE, DELETE · `fddt_platform` : SELECT, INSERT, UPDATE, DELETE

## issuer_registry

A company's known vendors/schools/etc. that extracted issuer names are fuzzy-matched against. Rows are deactivated, never deleted. A new company starts empty.

| Column | Type | Null | Default | Notes |
|---|---|---|---|---|
| `id` | uuid | no | | PK |
| `created_at` | timestamptz | no | | |
| `updated_at` | timestamptz | no | | |
| `name` | varchar(255) | no | | |
| `tax_id` | varchar(64) | yes | | |
| `type` | enum `issuer_registry_type` | no | | |
| `name_arabic` | varchar(255) | yes | | |
| `is_active` | boolean | no | `true` | |
| `company_id` | uuid | no | | FK → `companies.id` |
| `country` | varchar(2) | yes | | |

- **Index** `ix_issuer_registry_company_id` on (company_id)
- **Index** `ix_issuer_registry_name` on (name)
- **Index** `ix_issuer_registry_name_arabic` on (name_arabic)
- **Enum** `issuer_registry_type` : `vendor` , `school` , `government` , `other`
- **Row-Level Security** on: `platform_all` (ALL to fddt_platform); `tenant_isolation` (ALL to fddt_app)
- **Grants** `fddt_app` : SELECT, INSERT, UPDATE · `fddt_platform` : SELECT, INSERT, UPDATE, DELETE

## risk_rules

A company's configurable scoring rules. Rows are immutable and versioned: an edit inserts `version + 1` ; the current rule is the highest version. Seeded from `risk_rule_templates` when the company is created.

| Column | Type | Null | Default | Notes |
|---|---|---|---|---|
| `id` | uuid | no | | PK |
| `created_at` | timestamptz | no | | |
| `updated_at` | timestamptz | no | | |
| `rule_id` | varchar(64) | no | | |
| `category` | varchar(128) | no | | |
| `condition` | jsonb | no | | |
| `weight` | double precision | no | | |
| `severity` | varchar(32) | no | | |
| `reason_template` | varchar(1024) | no | | |

| Column | Type | Null | Default | Notes |
|---|---|---|---|---|
| `is_active` | boolean | no | | |
| `check_type` | varchar(64) | no | | |
| `version` | integer | no | | |
| `effective_from` | timestamptz | no | | |
| `updated_by` | uuid | yes | | FK → `users.id` |
| `change_note` | varchar(1024) | yes | | |
| `company_id` | uuid | no | | FK → `companies.id` |

- **Index** `ix_risk_rules_company_id` on (company_id)
- **Index** `ix_risk_rules_rule_id` on (rule_id)
- **Index** `uq_risk_rules_company_rule_id_version` on (company_id, rule_id, version) — unique
- **Row-Level Security** on: `platform_all` (ALL to fddt_platform); `tenant_isolation` (ALL to fddt_app)
- **Grants** `fddt_app` : SELECT, INSERT · `fddt_platform` : SELECT, INSERT

## risk_rule_templates

PLATFORM-LEVEL (no `company_id`): the default rule set copied into every NEW company's `risk_rules`. Edited by platform admins; edits never change existing companies. Only `fddt_platform` can access it.

| Column | Type | Null | Default | Notes |
|---|---|---|---|---|
| `id` | uuid | no | | PK |
| `rule_id` | varchar(64) | no | | unique |
| `category` | varchar(128) | no | | |
| `check_type` | varchar(64) | no | | |
| `condition` | jsonb | no | | |
| `weight` | double precision | no | | |
| `severity` | varchar(32) | no | | |
| `reason_template` | varchar(1024) | no | | |
| `is_active` | boolean | no | `true` | |
| `updated_by` | uuid | yes | | FK → `users.id` |
| `created_at` | timestamptz | no | `now()` | |
| `updated_at` | timestamptz | no | `now()` | |

- **Index** `risk_rule_templates_rule_id_key` on (rule_id) — unique
- **Grants** `fddt_app` : — · `fddt_platform` : SELECT, INSERT, UPDATE

## risk_settings

One row per company holding its `medium` / `high` score thresholds and the metadata score cap ( `metadata_score_cap` , default 40: the most points all `metadata.*` rules add together).

| Column | Type | Null | Default | Notes |
|---|---|---|---|---|
| `id` | uuid | no | | PK |
| `created_at` | timestamptz | no | | |
| `updated_at` | timestamptz | no | | |
| `medium_threshold` | integer | no | | |
| `high_threshold` | integer | no | | |
| `updated_by` | uuid | yes | | FK → `users.id` |
| `company_id` | uuid | no | | FK → `companies.id` , unique |
| `metadata_score_cap` | integer | no | `40` | |

- **Index** `ix_risk_settings_company_id` on (company_id)
- **Index** `uq_risk_settings_company_id` on (company_id) — unique
- **Row-Level Security** on: `platform_all` (ALL to fddt_platform); `tenant_isolation` (ALL to fddt_app)
- **Grants** `fddt_app` : SELECT, INSERT, UPDATE · `fddt_platform` : SELECT, INSERT, UPDATE, DELETE

## case_risk_assessments

One row per scoring run of a case (insert-only history; unique per (case, evidence fingerprint)): score, tier, frozen reason text and a snapshot of the exact rule versions and thresholds used.

| Column | Type | Null | Default | Notes |
|---|---|---|---|---|
| `id` | uuid | no | | PK |
| `case_id` | uuid | no | | FK → `cases.id` |
| `raw_score` | double precision | no | | |
| `score` | integer | no | | |
| `tier` | enum `risk_tier` | no | | |
| `triggered_reasons` | jsonb | no | | |
| `risk_rules_version_snapshot` | jsonb | no | | |
| `evidence_fingerprint` | varchar(64) | no | | |
| `computed_at` | timestamptz | no | | |
| `company_id` | uuid | no | | FK → `companies.id` |

- **Index** `ix_case_risk_assessments_case_id` on (case_id)
- **Index** `ix_case_risk_assessments_company_id` on (company_id)
- **Index** `ix_case_risk_assessments_computed_at` on (computed_at)

- **Index** `ix_case_risk_assessments_evidence_fingerprint` on (evidence_fingerprint)
- **Index** `uq_case_risk_assessments_case_evidence` on (case_id, evidence_fingerprint) — unique
- **Enum** `risk_tier`: `low`, `medium`, `high`
- **Row-Level Security** on: `platform_all` (ALL to fddt_platform); `tenant_isolation` (ALL to fddt_app)
- **Grants** `fddt_app`: SELECT, INSERT · `fddt_platform`: SELECT, INSERT

## risk_scores

LEGACY. Created by the initial migration before the versioned engine existed; nothing in the application reads or writes it. `case_risk_assessments` replaced it.

| Column | Type | Null | Default | Notes |
|---|---|---|---|---|
| `id` | uuid | no | | PK |
| `created_at` | timestamptz | no | | |
| `updated_at` | timestamptz | no | | |
| `case_id` | uuid | no | | FK → `cases.id` |
| `score` | integer | no | | |
| `tier` | enum `risk_score_tier` | no | | |
| `triggered_reasons` | jsonb | no | | |
| `rules_snapshot` | jsonb | no | | |
| `company_id` | uuid | no | | FK → `companies.id` |

- **Index** `ix_risk_scores_case_id` on (case_id)
- **Index** `ix_risk_scores_company_id` on (company_id)
- **Enum** `risk_score_tier`: `low`, `medium`, `high`
- **Row-Level Security** on: `platform_all` (ALL to fddt_platform); `tenant_isolation` (ALL to fddt_app)
- **Grants** `fddt_app`: SELECT · `fddt_platform`: SELECT, INSERT, UPDATE, DELETE

## case_actions

Reviewer-facing history of human actions on a case (approve / reject / escalate …) with the actor's role at the time. Complements, and is also mirrored into, `audit_log`.

| Column | Type | Null | Default | Notes |
|---|---|---|---|---|
| `id` | uuid | no | | PK |
| `created_at` | timestamptz | no | | |
| `updated_at` | timestamptz | no | | |
| `case_id` | uuid | no | | FK → `cases.id` |
| `actor_user_id` | uuid | yes | | FK → `users.id` |
| `action_type` | enum `case_action_type` | no | | |

| Column | Type | Null | Default | Notes |
|---|---|---|---|---|
| `notes` | varchar(4096) | yes | | |
| `actor_role` | varchar(32) | yes | | |
| `company_id` | uuid | no | | FK → `companies.id` |

- **Index** `ix_case_actions_case_id` on (case_id)
- **Index** `ix_case_actions_company_id` on (company_id)
- **Enum** `case_action_type` : `assign` , `comment` , `approve` , `reject` , `escalate` , `reopen`
- **Row-Level Security** on: `platform_all` (ALL to fddt_platform); `tenant_isolation` (ALL to fddt_app)
- **Grants** `fddt_app` : SELECT, INSERT · `fddt_platform` : SELECT, INSERT, UPDATE, DELETE

## case_reports

History of generated per-case forensic PDF reports (insert-only). The PDF lives in Blob Storage under `companies/{company}/cases/{case}/reports/` ; this row stores its SHA-256, size and the assessment it reflects.

| Column | Type | Null | Default | Notes |
|---|---|---|---|---|
| `id` | uuid | no | | PK |
| `case_id` | uuid | no | | FK → `cases.id` |
| `generated_by_user_id` | uuid | no | | FK → `users.id` |
| `generated_at` | timestamptz | no | | |
| `blob_url` | text | no | | |
| `file_size_bytes` | integer | no | | |
| `page_count` | integer | no | | |
| `report_sha256` | varchar(64) | no | | |
| `risk_assessment_id` | uuid | yes | | FK → `case_risk_assessments.id` |
| `company_id` | uuid | no | | FK → `companies.id` |

- **Index** `ix_case_reports_case_id` on (case_id)
- **Index** `ix_case_reports_company_id` on (company_id)
- **Index** `ix_case_reports_generated_at` on (generated_at)
- **Row-Level Security** on: `platform_all` (ALL to fddt_platform); `tenant_isolation` (ALL to fddt_app)
- **Grants** `fddt_app` : SELECT, INSERT · `fddt_platform` : SELECT, INSERT

## audit_log

Append-only system of record: every automated check result, human action and settings change is inserted here and never updated or deleted. `company_id` NULL = a platform-level event (company created, template edits, usage reconciliation, platform-admin access), visible only to platform admins; a company user's *user created/updated* goes into that company's log.

| Column | Type | Null | Default | Notes |
|---|---|---|---|---|
| `id` | uuid | no | | PK |
| `created_at` | timestamptz | no | | |
| `case_id` | uuid | yes | | FK → `cases.id` |
| `document_id` | uuid | yes | | FK → `documents.id` |
| `actor_user_id` | uuid | yes | | FK → `users.id` |
| `event_type` | varchar(128) | no | | |
| `event_data` | jsonb | yes | | |
| `company_id` | uuid | yes | | FK → `companies.id` |

- **Index** `ix_audit_log_case_id` on (case_id)
- **Index** `ix_audit_log_company_id` on (company_id)
- **Index** `ix_audit_log_created_at` on (created_at)
- **Index** `ix_audit_log_document_id` on (document_id)
- **Index** `ix_audit_log_event_type` on (event_type)
- **Row-Level Security** on: `platform_all` (ALL to fddt_platform); `tenant_isolation` (ALL to fddt_app)
- **Grants** `fddt_app` : SELECT, INSERT · `fddt_platform` : SELECT, INSERT

## signature_references

A signature/stamp region drawn on a document, cropped and stored as a PNG, used as the reference for comparisons.

| Column | Type | Null | Default | Notes |
|---|---|---|---|---|
| `person_name` | text | no | | |
| `signature_image_url` | text | yes | | |
| `bounding_box` | jsonb | yes | | |
| `source_document_id` | uuid | no | | FK → `documents.id` |
| `source_case_id` | uuid | no | | FK → `cases.id` |
| `created_by` | uuid | no | | FK → `users.id` |
| `is_library` | boolean | no | `false` | |
| `id` | uuid | no | | PK |
| `created_at` | timestamptz | no | | |
| `updated_at` | timestamptz | no | | |
| `company_id` | uuid | no | | FK → `companies.id` |

- **Index** `ix_signature_references_company_id` on (company_id)
- **Index** `ix_signature_references_source_case_id` on (source_case_id)
- **Index** `ix_signature_references_source_document_id` on (source_document_id)
- **Row-Level Security** on: `platform_all` (ALL to fddt_platform); `tenant_isolation` (ALL to fddt_app)

- **Grants** `fddt_app`: SELECT, INSERT, UPDATE · `fddt_platform`: SELECT, INSERT, UPDATE, DELETE

## signature_matches

Result of comparing one reference against the signature region of another document in the same case (vision-model verdict or pixel-identical reuse verdict); one row per (reference, document, scope).

| Column | Type | Null | Default | Notes |
|---|---|---|---|---|
| `id` | uuid | no | | PK |
| `document_id` | uuid | no | | FK → `documents.id` |
| `case_id` | uuid | no | | FK → `cases.id` |
| `signature_reference_id` | uuid | no | | FK → `signature_references.id` |
| `comparison_scope` | enum `comparison_scope` | no | | |
| `result` | enum `signature_match_result` | no | | |
| `reasoning` | text | yes | | |
| `compared_at` | timestamptz | no | | |
| `company_id` | uuid | no | | FK → `companies.id` |

- **Index** `ix_signature_matches_case_id` on (case_id)
- **Index** `ix_signature_matches_company_id` on (company_id)
- **Index** `ix_signature_matches_document_id` on (document_id)
- **Index** `ix_signature_matches_signature_reference_id` on (signature_reference_id)
- **Index** `uq_signature_matches_ref_doc_scope` on (signature_reference_id, document_id, comparison_scope) — unique
- **Enum** `comparison_scope`: `in_case`, `library`
- **Enum** `signature_match_result`: `consistent`, `possibly_consistent`, `inconsistent`, `cannot_determine`, `identical_reuse`, `reused_different_signer`
- **Row-Level Security** on: `platform_all` (ALL to fddt_platform); `tenant_isolation` (ALL to fddt_app)
- **Grants** `fddt_app`: SELECT, INSERT, UPDATE · `fddt_platform`: SELECT, INSERT, UPDATE, DELETE

## bulk_upload_cases

| Column | Type | Null | Default | Notes |
|---|---|---|---|---|
| `id` | uuid | no | | PK |
| `company_id` | uuid | no | | FK → `companies.id` |
| `bulk_upload_id` | uuid | no | | FK → `bulk_uploads.id` |
| `folder_index` | integer | no | | |
| `folder` | varchar(1024) | no | | |
| `status` | varchar(16) | no | | |
| `error_code` | varchar(64) | yes | | |

| Column | Type | Null | Default | Notes |
|---|---|---|---|---|
| `error_message` | text | yes | | |
| `case_id` | uuid | yes | | FK → `cases.id` |
| `case_number` | varchar(64) | yes | | |
| `files` | jsonb | no | | |
| `created_at` | timestamptz | no | `now()` | |
| `updated_at` | timestamptz | no | `now()` | |

- **Index** `bulk_upload_cases_bulk_upload_id_folder_index_key` on (bulk_upload_id, folder_index) — unique
- **Index** `ix_bulk_upload_cases_bulk_upload_id` on (bulk_upload_id)
- **Index** `ix_bulk_upload_cases_company_id` on (company_id)
- **Row-Level Security** on: `platform_all` (ALL to fddt_platform); `tenant_isolation` (ALL to fddt_app)
- **Grants** `fddt_app`: SELECT, INSERT, UPDATE · `fddt_platform`: SELECT, INSERT, UPDATE, DELETE

## bulk_uploads

| Column | Type | Null | Default | Notes |
|---|---|---|---|---|
| `id` | uuid | no | | PK |
| `company_id` | uuid | no | | FK → `companies.id` |
| `uploaded_by_user_id` | uuid | no | | FK → `users.id` |
| `original_filename` | varchar(512) | no | | |
| `case_type` | enum `case_type` | no | | |
| `blob_storage_path` | varchar(1024) | no | | |
| `file_hash` | varchar(64) | no | | |
| `zip_size_bytes` | bigint | no | | |
| `status` | enum `bulk_upload_status` | no | | |
| `error_code` | varchar(64) | yes | | |
| `error_message` | text | yes | | |
| `case_folder_count` | integer | yes | | |
| `cases_created` | integer | no | `0` | |
| `cases_failed` | integer | no | `0` | |
| `documents_accepted` | integer | no | `0` | |
| `documents_rejected` | integer | no | `0` | |
| `zip_details` | jsonb | yes | | |
| `started_at` | timestamptz | yes | | |
| `finished_at` | timestamptz | yes | | |

| Column | Type | Null | Default | Notes |
|---|---|---|---|---|
| `created_at` | timestamptz | no | `now()` | |
| `updated_at` | timestamptz | no | `now()` | |

- **Index** `ix_bulk_uploads_company_id` on (company_id)
- **Index** `ix_bulk_uploads_uploaded_by_user_id` on (uploaded_by_user_id)
- **Enum** `case_type`: `school_document`, `vendor_invoice`, `commercial_invoice`, `procurement_documentation`, `quotation`, `travel_reimbursement`, `other`
- **Enum** `bulk_upload_status`: `queued`, `ingesting`, `complete`, `failed`
- **Row-Level Security** on: `platform_all` (ALL to fddt_platform); `tenant_isolation` (ALL to fddt_app)
- **Grants** `fddt_app`: SELECT, INSERT, UPDATE · `fddt_platform`: SELECT, INSERT, UPDATE, DELETE

## Enum index

| Enum | Values |
|---|---|
| `bulk_upload_status` | `queued`, `ingesting`, `complete`, `failed` |
| `case_action_type` | `assign`, `comment`, `approve`, `reject`, `escalate`, `reopen` |
| `case_status` | `submitted`, `under_automated_review`, `pending_manual_review`, `auto_approved`, `escalated`, `under_investigation`, `approved`, `rejected`, `closed` |
| `case_tier` | `l1`, `l2` |
| `case_type` | `school_document`, `vendor_invoice`, `commercial_invoice`, `procurement_documentation`, `quotation`, `travel_reimbursement`, `other` |
| `comparison_scope` | `in_case`, `library` |
| `document_check_status` | `pending`, `running`, `completed`, `failed` |
| `document_check_type` | `ocr_layout`, `classification`, `field_extraction`, `issuer_verification`, `signature_stamp_detection`, `metadata_forensics`, `error_level_analysis`, `copy_move_detection`, `duplicate_detection`, `ai_content_detection`, `field_validation`, `visual_inconsistency_review`, `font_consistency`, `ghost_content`, `spelling_consistency` |
| `document_processing_status` | `pending`, `processing`, `complete`, `failed` |
| `finding_severity` | `info`, `low`, `medium`, `high` |

| Enum | Values |
| --- | --- |
| `issuer_registry_type` | `vendor`, `school`, `government`, `other` |
| `risk_score_tier` | `low`, `medium`, `high` |
| `risk_tier` | `low`, `medium`, `high` |
| `signature_match_result` | `consistent`, `possibly_consistent`, `inconsistent`, `cannot_determine`, `identical_reuse`, `reused_different_signer` |
| `user_role` | `user`, `reviewer_l1`, `reviewer_l2`, `admin`, `platform_admin` |

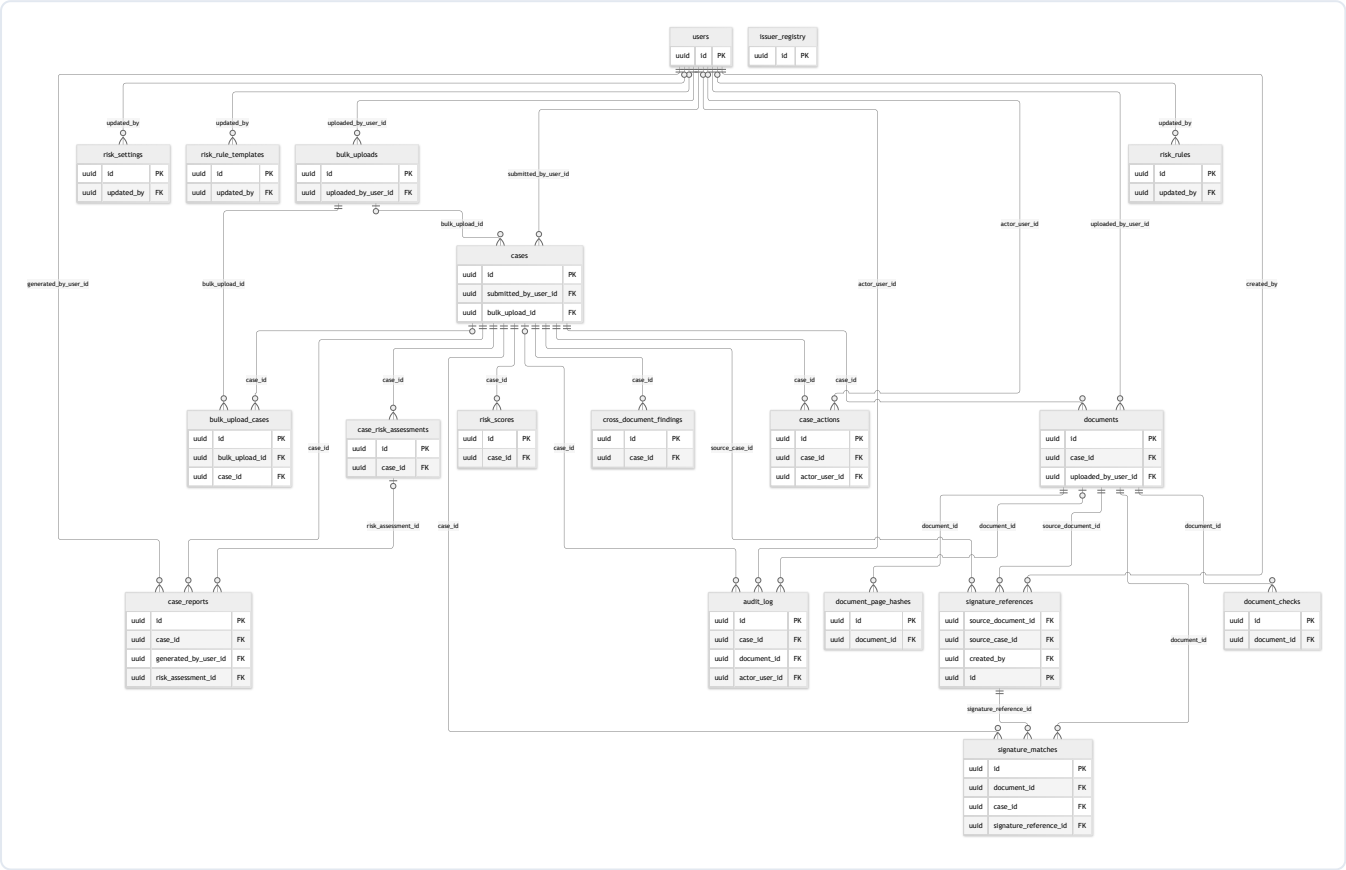
## Schema notes

- **`user_role` still contains `admin`.** The single-tenant `admin` role was migrated to `platform_admin`; Postgres cannot drop an enum value, so `admin` remains in the type but no row uses it and the application never assigns it.
- **`cases.case_number` and `users.email`** carry a `UNIQUE` constraint *plus* a separate non-unique index from the original migrations; uniqueness is enforced either way.
- **`risk_scores`** exists but nothing in the application uses it. It is safe to leave; dropping it needs a migration.
- **No `notifications` table.** Notifications are not part of the current build.
- **Enum values added by later migrations** use `ALTER TYPE … ADD VALUE`; a downgrade leaves them in place.
- **New tenant-owned tables** need `TenantScopedMixin`, RLS with a `tenant_isolation` and a `platform_all` policy, and explicit grants in their migration — there are no default privileges, so a table without them is unreachable by the app.

# Entity-relationship diagram

Every tenant-owned table also references `companies` through `company_id`. Those edges are left out of the first diagram so it stays readable; the second diagram shows them.

## Domain relationships




## Tenancy — what belongs to a company

Every table below references `companies.id` through the listed column (a table, not a diagram: drawn, the fan-out from `companies` is too wide to read).

| Table | Column | Nullable | Row-Level Security |
|---|---|---|---|
| `company_usage_stats` | `company_id` | no | on |
| `users` | `company_id` | yes | on |
| `cases` | `company_id` | no | on |
| `documents` | `company_id` | no | on |
| `document_checks` | `company_id` | no | on |
| `document_page_hashes` | `company_id` | no | on |
| `cross_document_findings` | `company_id` | no | on |
| `issuer_registry` | `company_id` | no | on |

| Table | Column | Nullable | Row-Level Security |
|---|---|---|---|
| `risk_rules` | `company_id` | no | on |
| `risk_settings` | `company_id` | no | on |
| `case_risk_assessments` | `company_id` | no | on |
| `risk_scores` | `company_id` | no | on |
| `case_actions` | `company_id` | no | on |
| `case_reports` | `company_id` | no | on |
| `audit_log` | `company_id` | yes | on |
| `signature_references` | `company_id` | no | on |
| `signature_matches` | `company_id` | no | on |
| `bulk_upload_cases` | `company_id` | no | on |
| `bulk_uploads` | `company_id` | no | on |

`users.company_id` and `audit_log.company_id` are nullable: NULL marks a platform admin and a platform-level event respectively. `risk_rule_templates` has no `company_id` — it is platform-level and only reachable by the platform database role.

# API reference

The request/response detail is **not hand-written here** — it is generated from the route definitions, so it cannot drift from the code:

| What | Where |
|---|---|
| Interactive docs (Swagger UI) | `http://localhost:8000/docs` while the backend runs |
| ReDoc | `http://localhost:8000/redoc` |
| Machine-readable schema | `http://localhost:8000/openapi.json` |
| Snapshot committed to the repo | `openapi.json` (OpenAPI 3.1, 37 paths / 46 operations / 68 schemas) |

Every operation carries a `summary`, a `description` (rules, side effects, role requirements) and its error responses; every request field has a description. Import `openapi.json` into Postman, Insomnia or a client generator if you prefer.

## Regenerating the snapshot

The snapshot is produced from the app object itself, no server needed:

```
cd backend
python -c "import json; from app.main import app; json.dump(app.openapi(),
open('../docs/api/openapi.json','w',encoding='utf-8'), indent=2, ensure_ascii=False)"
```

Regenerate it whenever a route or schema changes. A quick consistency check is to compare it with the live `/openapi.json`.

## Trying it in Swagger UI

1. `POST /auth/login` with `{"email": "admin@example.com", "password": "ChangeMe123!"}` (the seeded **platform admin**, see local setup) and copy `access_token`. To try company endpoints, log in as a company user created by the platform admin instead.
2. Click **Authorize** (top right), paste the token (Swagger adds `Bearer` for you).
3. Every other call now sends it.

## Conventions

- **Auth:** `Authorization: Bearer <JWT>`. Tokens live 8 h by default (`JWT_ACCESS_TOKEN_EXPIRE_MINUTES`) and carry `role`, `company_id` and `is_platform_admin`. The user is re-read from the database on every request; a token whose company or platform flag no longer matches, or whose company is suspended, is refused. A missing/invalid/expired token or an inactive user is **401**; a valid user with the wrong role is **403**.
- **Two mandatory checks on every endpoint:** the role check and the tenant check. A company user's request is confined to their own company — by the application and by PostgreSQL Row-Level Security.
- **Company roles (ranked):** `user` (submitter) < `reviewer_l1` < `reviewer_l2`. Each role can do everything the roles below it can; routes name the lowest role they admit. On a case escalated to L2 (`assigned_tier` = `l2`) a `reviewer_l1` can still `GET` it but gets **403** from approve/reject/escalate.
- `platform_admin` belongs to no company and is not on that ladder. It manages companies, users, the rule template, billing and the queue monitor, and has **read-only, audited** access to any company's data by passing

`company_id` (query parameter) on the company endpoints that accept it — `GET /cases`, `/audit-log`, `/settings/*` — or simply opening a case by id. It can never approve, reject, escalate, upload, create cases or generate reports (403), but may edit a company's issuer registry, risk rules and thresholds on its behalf (audited in that company's log).

- **A resource of another company is a 404**, and so is a case that is not yours for a `user` — ids cannot be probed.
- **Ids** are UUIDs. **Timestamps** are ISO-8601 with timezone.
- **Bounding boxes** are always `{page (1-based), x, y, width, height}` as 0–1 fractions of the page, origin top-left.
- **Errors** are `{"detail": "<human-readable message>"}` (or FastAPI's standard list for **422** validation errors). **Upload rejections** are structured: `{"detail": {"code": …, "message": …, …}}` (see 01-upload-intake). Status codes used: 400, 401, 403, 404, 409 (state conflict or duplicate), 413 (upload too large), 415 (not a PDF / wrong extension), 422 (validation, business rule, corrupted or password-protected PDF), 502 (Azure Blob Storage failure).
- **There is no `/api/v1` prefix in the backend.** `API_V1_PREFIX` is declared in settings but never used. In development the Vite dev server proxies `/api/*` → `http://127.0.0.1:8000/*`, so the browser calls `/api/cases` while the backend sees `/cases`. Production serves the SPA and the API behind one ingress the same way (production deployment).
- **Pagination:** only `GET /audit-log` is paged (`limit` 1–500, default 100, and `offset`; response is `{total, items}`). Other lists return everything.
- **No dedicated "checks" endpoints.** Per-document check results, extracted fields, cross-document findings, forensic findings, the risk assessment and reviewer actions are all embedded in `GET /cases/{case_id}`. Signature comparison results have their own endpoint.
- **Asynchronous work:** uploads and signature-reference creation return before processing finishes. Poll `GET /cases/{case_id}` (`pipeline.complete`, `documents[].processing_status`) and `GET /cases/{case_id}/signature-matches`.

## Endpoint index

`Access` shows who may call each endpoint. "own case" means a `user` may act only on a case they submitted (anyone else gets a 404), while reviewers (L1 or L2) may act on any case of their company. "reviewer" means `reviewer_l1` or above. "PA read" means a platform admin may read it (audited). See the security notes in architecture/overview.md.

### Auth — tag `auth`

| Method & path | Summary | Access |
|---|---|---|
| `POST /auth/login` | Log in and obtain a JWT | public |
| `GET /auth/me` | Get the current user (with company) | any |
| `GET /auth/me/upload-limits` | The caller's company upload limits (max file / zip size), read fresh each time | company users (404 for platform admins) |

### Cases — tag `cases`

| Method & path | Summary | Access |
|---|---|---|
| `POST /cases` | Create a case | company users |
| `GET /cases` | List cases (the case queue); filters `status`, `case_type`, `assigned_tier`, `actionable`; each item has `can_act` | company users (`user` sees only own); PA read with `company_id` |

| Method & path | Summary | Access |
|---|---|---|
| `GET /cases/{case_id}` | Get case detail (documents, checks, findings, assessment, actions) | company users ( `user` sees a reduced view); PA read |
| `GET /cases/{case_id}/audit-log` | Get a case's activity timeline | own case for `user`, reviewers; PA read |

## Reviewer decisions — tag `case-actions`

| Method & path | Summary | Access |
|---|---|---|
| `POST /cases/{case_id}/approve` | Approve a case (justification required above low risk) | reviewer; L2 case: `reviewer_l2` |
| `POST /cases/{case_id}/reject` | Reject a case (reason required) | reviewer; L2 case: `reviewer_l2` |
| `POST /cases/{case_id}/escalate` | Escalate a case to the L2 tier (reason required; one-way; 409 if already L2) | reviewer on an L1 case |

## Documents — tag `documents`

| Method & path | Summary | Access |
|---|---|---|
| `POST /cases/{case_id}/documents` | Upload a document (multipart field `file`, one per request; **PDF only**, within the company's own size limit (10 MB by default; set per company by a platform admin), not password-protected; rejections return `detail: {code, message}`, see 01-upload-intake) | own case, or reviewer |
| `POST /bulk-uploads` | Bulk upload: one zip (raw body, `Content-Type: application/zip`; query `case_type`, `filename`), one folder per case, within the company's zip limit. 202 with the per-case plan; ingestion runs in the background (01a) | any company role |
| `GET /bulk-uploads` | Recent bulk uploads | own (user) / company (reviewers) / platform admin per company |
| `GET /bulk-uploads/{id}` | Bulk upload summary with each case's live status | same as above |
| `GET /cases/{case_id}/documents/{document_id}/file-url` | Get a fresh signed URL for the original file | own case, or reviewer; PA read |

## Signatures — tag `signatures`

| Method & path | Summary | Access |
|---|---|---|
| `POST /cases/{case_id}/documents/{document_id}/signature-references` | Create a signature reference from a drawn box | own case, or reviewer |
| `GET /cases/{case_id}/signature-references` | List a case's references | own case, or reviewer; PA read |

| Method & path | Summary | Access |
|---|---|---|
| `GET /cases/{case_id}/signature-matches` | List a case's comparison results | own case, or reviewer; PA read |

### Reports — tag `case-reports`

| Method & path | Summary | Access |
|---|---|---|
| `POST /cases/{case_id}/reports` | Generate a case report (PDF) | reviewer |
| `GET /cases/{case_id}/reports` | List a case's generated reports | reviewer; PA read |

> Organisation-wide MIS/compliance reporting ( `app/api/reports.py` ) is a stub with no routes.

### Audit — tag `audit`

| Method & path | Summary | Access |
|---|---|---|
| `GET /audit-log` | Search the company's audit log ( `event_type` , `case_id` , `q` , `limit` , `offset` ); never shows platform-access rows to a company user | reviewer; PA with `company_id` (that company's log incl. platform-access rows about it) or without (the platform-level log) |
| `GET /audit-log/event-types` | List distinct audit event types | reviewer; PA |

### Settings — tag `settings`

Issuer registry, risk rules and thresholds are **per company**: a company's `reviewer_l2` , or a platform admin passing `company_id` (audited). Users are platform-admin only.

| Method & path | Summary | Access |
|---|---|---|
| `GET /settings/issuers` | List issuers ( `include_inactive` ) | `reviewer_l2` ; PA |
| `POST /settings/issuers` | Create an issuer | `reviewer_l2` ; PA |
| `PATCH /settings/issuers/{issuer_id}` | Update or deactivate an issuer | `reviewer_l2` ; PA |
| `GET /settings/risk-rules` | List current risk rules | `reviewer_l2` ; PA |
| `POST /settings/risk-rules` | Create a risk rule | `reviewer_l2` ; PA |
| `GET /settings/risk-rule-options` | Options for the Add-rule form | `reviewer_l2` ; PA |
| `GET /settings/risk-rules/{rule_id}/history` | A rule's version history | `reviewer_l2` ; PA |
| `PATCH /settings/risk-rules/{rule_id}` | Change weight / severity / active flag (inserts a new version) | `reviewer_l2` ; PA |

| Method & path | Summary | Access |
|---|---|---|
| `GET /settings/risk-thresholds` | Get the tier thresholds | `reviewer_l2` ; PA |
| `PUT /settings/risk-thresholds` | Set the tier thresholds | `reviewer_l2` ; PA |
| `GET /settings/users` | List users (filter `company_id` ) | PA only |
| `POST /settings/users` | Create a user (company + role, or a platform admin) | PA only |
| `PATCH /settings/users/{user_id}` | Change role / company / active flag (moving company invalidates the user's token) | PA only |

### Platform — tag `platform` (platform admins only)

| Method & path | Summary |
|---|---|
| `GET /platform/companies` | List companies |
| `POST /platform/companies` | Create a company (seeds its risk rules from the template and default thresholds) |
| `PATCH /platform/companies/{company_id}` | Rename / suspend / reactivate a company, or change its upload limits ( `max_file_size_mb` , `max_zip_size_mb` ; audited old → new) |
| `GET /platform/usage` | Billing & usage per company ( `period` or a custom date range) |
| `POST /platform/usage/reconcile` | Recompute the usage counters now |
| `GET /platform/queues` | Processing-queue monitor (depth, waits, workers, limiter counters, outstanding per company) |
| `GET /platform/risk-rule-templates` | List the platform rule template |
| `POST /platform/risk-rule-templates` | Add a template rule (affects companies created afterwards) |
| `PATCH /platform/risk-rule-templates/{rule_id}` | Change a template rule (never changes existing companies) |

### Health — tag `health`

| Method & path | Summary | Access |
|---|---|---|
| `GET /health` | Liveness check (does not touch DB/Redis/Azure) | public |

## Not in the API (by design or not yet built)

No password reset or change-password, no self-service sign-up, no logout/refresh, no company-level user management (users are created by platform admins), no document delete (originals are immutable), no case delete, no notification endpoints, no organisation-wide reporting, no external-integration hooks.

# API endpoint reference

FDDT — Fraud Document Detection Tool API · version 0.1.0 · OpenAPI 3.1.0

47 operations across 38 paths and 69 data models. This reference is generated from the API definition itself (docs/api/openapi.json).

## Conventions

- **Authentication:** endpoints marked Bearer JWT need the header `Authorization: Bearer <token>`. Obtain a token from `POST /auth/login` (email and password).
- **Roles** (ranked, each includes everything below it): `user` (submitter) < `reviewer_l1` < `reviewer_l2` < `admin`. "Reviewer/admin" means `reviewer_l1` or above. On a case escalated to L2 (`assigned_tier` = `l2`) only `reviewer_l2` and `admin` may approve, reject or escalate; a `reviewer_l1` gets 403 there but can still read the case. Role rules for each endpoint are stated in its description.
- **Errors:** every operation lists its documented error responses. 422 is a request-validation error.
- **Formats:** requests and responses are JSON (file uploads are `multipart/form-data`); identifiers are UUIDs; timestamps are ISO 8601.

## Endpoint index

| Method | Path | Summary |
|---|---|---|
| **Health** | | |
| GET | `/health` | Liveness check |
| **Authentication** | | |
| POST | `/auth/login` | Log in and obtain a JWT |
| GET | `/auth/me` | Get the current user |
| GET | `/auth/me/upload-limits` | Get my company's upload limits |
| POST | `/auth/me/password` | Change my password |
| **Cases** | | |
| POST | `/cases` | Create a case |
| GET | `/cases` | List cases (the case queue) |
| GET | `/cases/{case_id}` | Get case detail |
| GET | `/cases/{case_id}/audit-log` | Get a case's activity timeline |
| **Documents** | | |
| POST | `/cases/{case_id}/documents` | Upload a document to a case |
| GET | `/cases/{case_id}/documents/{document_id}/file-url` | Get a fresh signed URL for a document |
| **Case actions (review decisions)** | | |

| Method | Path | Summary |
|---|---|---|
| POST | `/cases/{case_id}/approve` | Approve a case |
| POST | `/cases/{case_id}/reject` | Reject a case |
| POST | `/cases/{case_id}/escalate` | Escalate a case |
| **Signature references & matches** | | |
| POST | `/cases/{case_id}/documents/{document_id}/signature-references` | Create a signature reference |
| GET | `/cases/{case_id}/signature-references` | List a case's signature references |
| GET | `/cases/{case_id}/signature-matches` | List a case's signature comparison results |
| **Audit log** | | |
| GET | `/audit-log` | Search the audit log |
| GET | `/audit-log/event-types` | List audit event types |
| **Case reports** | | |
| POST | `/cases/{case_id}/reports` | Generate a case report (PDF) |
| GET | `/cases/{case_id}/reports` | List a case's generated reports |
| **Settings (admin)** | | |
| GET | `/settings/issuers` | List issuers |
| POST | `/settings/issuers` | Create an issuer |
| PATCH | `/settings/issuers/{issuer_id}` | Update or deactivate an issuer |
| GET | `/settings/risk-rules` | List current risk rules |
| POST | `/settings/risk-rules` | Create a risk rule |
| GET | `/settings/risk-rule-options` | Get the options for the Add-rule form |
| GET | `/settings/risk-rules/{rule_id}/history` | Get a risk rule's version history |
| PATCH | `/settings/risk-rules/{rule_id}` | Change a risk rule's weight, severity or active flag |
| GET | `/settings/risk-thresholds` | Get the risk tier thresholds |
| PUT | `/settings/risk-thresholds` | Set the risk tier thresholds |
| GET | `/settings/users` | List users |
| POST | `/settings/users` | Create a user |
| PATCH | `/settings/users/{user_id}` | Change a user's role, company or active flag |
| POST | `/settings/users/{user_id}/reset-password` | Reset user password (platform admin only) |
| **Other** | | |
| POST | `/bulk-uploads` | Bulk upload: a zip of cases |
| GET | `/bulk-uploads` | List bulk uploads |

| Method | Path | Summary |
|---|---|---|
| GET | `/bulk-uploads/{bulk_upload_id}` | Bulk upload summary (live per-case status) |
| GET | `/platform/companies` | List companies |
| POST | `/platform/companies` | Create a company |
| PATCH | `/platform/companies/{company_id}` | Rename, suspend/reactivate, or change a company's upload limits |
| GET | `/platform/risk-rule-templates` | List the platform risk-rule template |
| POST | `/platform/risk-rule-templates` | Add a rule to the platform template |
| PATCH | `/platform/risk-rule-templates/{rule_id}` | Edit a platform template rule |
| GET | `/platform/usage` | Billing & usage by company |
| POST | `/platform/usage/reconcile` | Reconcile usage counters now |
| GET | `/platform/queues` | Processing queue monitor |

## Health

### GET `/health`

**Liveness check** Public

Unauthenticated. Returns `{status: ok, environment}`; does not touch the database, Redis or Azure.

#### Responses

| Status | Description | Body |
|---|---|---|
| `200` | Successful Response | map of string |

## Authentication

### POST `/auth/login`

**Log in and obtain a JWT** Public

Exchanges email + password for a bearer token (HS256; lifetime from JWT_ACCESS_TOKEN_EXPIRE_MINUTES). The token carries `role`, `company_id` (null for a platform admin) and `is_platform_admin`. An unknown email, a wrong password, a deactivated account and a suspended company all return the same 401 so accounts cannot be enumerated. Throttled (app/services/login_throttle.py): after LOGIN_MAX_FAILED_ATTEMPTS failures an email address is refused for LOGIN_LOCKOUT_MINUTES, and each client IP gets LOGIN_MAX_ATTEMPTS_PER_IP_PER_MINUTE attempts per minute; both answer 429 with a Retry-After header.

#### Request body

LoginRequest

#### Responses

| Status | Description | Body |
|---|---|---|
| `200` | Successful Response | TokenResponse |

| Status | Description | Body |
|---|---|---|
| `401` | Incorrect email or password (or the account/company is inactive). | — |
| `422` | Validation Error | HTTPValidationError |
| `429` | Too many sign-in attempts for this email address or from this IP. | — |

### GET `/auth/me`

**Get the current user** Bearer JWT

Returns the authenticated user's profile, role and company. The role and company are re-read from the database on every request rather than trusted from the token, so a demotion, deactivation or company suspension takes effect immediately.

#### Responses

| Status | Description | Body |
|---|---|---|
| `200` | Successful Response | CurrentUserResponse |
| `401` | Missing, invalid or expired bearer token, or the user is inactive. | — |

### GET `/auth/me/upload-limits`

**Get my company's upload limits** Bearer JWT

The per-file and per-zip limits the caller's company is held to (set per company by a platform admin). Read fresh on every request, so upload screens always show the limit that will actually be enforced. Platform admins belong to no company: 404.

#### Responses

| Status | Description | Body |
|---|---|---|
| `200` | Successful Response | UploadLimitsResponse |
| `401` | Missing, invalid or expired bearer token, or the user is inactive. | — |
| `404` | Platform admins have no company and therefore no upload limits. | — |

### POST `/auth/me/password`

**Change my password** Bearer JWT

Any signed-in user (every role) changes their own password by giving the current one. The new password (8-128 characters, different from the current one) is stored hashed and the change is recorded in the audit log (never the password). A forgotten password cannot be reset here: a platform admin resets it ( `POST /settings/users/{id}/reset-password` ). Tokens already issued stay valid until they expire.

#### Request body

ChangePasswordRequest

#### Responses

| Status | Description | Body |
|---|---|---|
| `204` | Successful Response | — |
| `400` | The current password is wrong, or the new one equals it. | — |

| Status | Description | Body |
|---|---|---|
| `401` | Missing, invalid or expired bearer token, or the user is inactive. | — |
| `422` | The new password is shorter than 8 or longer than 128 characters. | — |

## Cases

### POST /cases

**Create a case** Bearer JWT

Opens an empty case (status `submitted`) owned by the caller, in the caller's company; any company role may create one (platform admins may not — 403). The case number is `CASE-` plus the first 8 hex characters of the case UUID. Attach documents afterwards with POST /cases/{case_id}/documents. Writes a `case_created` audit event.

#### Request body

CaseCreateRequest

#### Responses

| Status | Description | Body |
|---|---|---|
| `201` | Successful Response | CaseResponse |
| `401` | Missing, invalid or expired bearer token, or the user is inactive. | — |
| `403` | Platform admins cannot create cases. | — |
| `422` | Validation Error | HTTPValidationError |

### GET /cases

**List cases (the case queue)** Bearer JWT

Only the caller's company. Reviewers see every case of their company; a `user` sees only the cases they submitted. A platform admin must pass `company_id` (one company at a time, audited) — there is no cross-company case list. For `reviewer_l2` and platform admins, open cases escalated to L2 sort first; otherwise newest first. Optional filters: `status`, `case_type`, `assigned_tier`, and `actionable=true` (only cases the caller may act on — for a `reviewer_l1` that excludes L2 cases). Each item carries `can_act`. For submitters `risk_tier` is null and `flag` is a neutral 'In review' marker, so scoring signals are never disclosed to them.

#### Parameters

| Name | In | Type | Required | Description |
|---|---|---|---|---|
| `status` | query | CaseStatus (nullable) | no | |
| `case_type` | query | CaseType (nullable) | no | |
| `assigned_tier` | query | CaseTier (nullable) | no | |
| `actionable` | query | boolean | no | Only cases the caller may approve/reject/escalate. |

| Name | In | Type | Required | Description |
|---|---|---|---|---|
| `company_id` | query | string (uuid) (nullable) | no | Platform admins only (required for them): the company to act on. Company users are always confined to their own company. |

### Responses

| Status | Description | Body |
|---|---|---|
| `200` | Successful Response | array of CaseListItem |
| `400` | A platform admin did not pass `company_id`. | — |
| `401` | Missing, invalid or expired bearer token, or the user is inactive. | — |
| `404` | `company_id` names an unknown company (or, for a company user, not theirs). | — |
| `422` | Validation Error | HTTPValidationError |

## GET `/cases/{case_id}`

### Get case detail

Bearer JWT

Everything the case detail page needs in one call: documents (with signed download URLs, extracted fields and per-document check results), cross-document findings, forensic findings, the latest risk assessment, pipeline progress (`pipeline.complete` / `pipeline.pending`) and reviewer actions. A submitter does not receive the risk assessment and sees only approve/reject actions. A case of another company, or owned by someone else, is a 404, not a 403, so ids cannot be probed. A platform admin may read any company's case; each such access is written to that company's audit log.

### Parameters

| Name | In | Type | Required | Description |
|---|---|---|---|---|
| `case_id` | path | string (uuid) | yes | |

### Responses

| Status | Description | Body |
|---|---|---|
| `200` | Successful Response | CaseDetail |
| `401` | Missing, invalid or expired bearer token, or the user is inactive. | — |
| `404` | No such case (or, for a `user`, not one they submitted). | — |
| `422` | Validation Error | HTTPValidationError |

## GET `/cases/{case_id}/audit-log`

### Get a case's activity timeline

Bearer JWT

Chronological feed for the case detail page's activity timeline — System Specification section 3.5. Reads the same append-only `audit_log` rows every check task and case/document endpoint already writes; this adds no new events, just a way to read the ones that exist.

### Parameters

| Name | In | Type | Required | Description |
|---|---|---|---|---|
| `case_id` | path | string (uuid) | yes | |

### Responses

| Status | Description | Body |
|---|---|---|
| `200` | Successful Response | array of AuditLogEntrySchema |
| `401` | Missing, invalid or expired bearer token, or the user is inactive. | — |
| `404` | No such case (or, for a `user`, not one they submitted). | — |
| `422` | Validation Error | HTTPValidationError |

## Documents

**POST** `/cases/{case_id}/documents`

### Upload a document to a case

Bearer JWT

One file per request (multipart field `file`). The original bytes are stored immutably in Blob Storage at `companies/{company_id}/cases/{case_id}/documents/{document_id}_{sha256}{ext}`, a `documents` row is written with the file's SHA-256, then the Celery pipeline is enqueued (extraction, metadata forensics, ELA/copy-move, duplicate detection, visual review, signature detection). The response returns immediately with `processing_status: pending`; poll GET /cases/{case_id} for progress. Before anything is stored or queued the file is validated: at most the company's per-file limit (`max_file_size_mb`, set by a platform admin; 10 MB by default), not empty, a PDF by its header bytes (not its name), a matching extension, and locally parseable (not corrupted, not password-protected). A rejection returns `detail: {code, message, ...}` with one of the codes `file_empty`, `file_too_large`, `unsupported_file_type`, `file_type_mismatch`, `file_corrupted`, `file_password_protected`. Each pipeline step is its own task on a shared, FIFO queue (see app/tasks/celery_app.py). A `user` may upload only to a case they submitted; reviewers to any case of their company; platform admins never (403).

### Parameters

| Name | In | Type | Required | Description |
|---|---|---|---|---|
| `case_id` | path | string (uuid) | yes | |

### Request body

Body_upload_document_cases__case_id__documents_post (multipart/form-data)

### Responses

| Status | Description | Body |
|---|---|---|
| `201` | Successful Response | DocumentResponse |
| `400` | `file_empty`: the file is 0 bytes. | — |
| `401` | Missing, invalid or expired bearer token, or the user is inactive. | — |

| Status | Description | Body |
|---|---|---|
| `403` | Platform admins have read-only access. | — |
| `404` | No such case (or, for a `user`, not one they submitted). | — |
| `413` | `file_too_large`: over the size limit (`size_bytes`, `max_size_bytes`). | — |
| `415` | `unsupported_file_type` (not a PDF by content; images are refused) or `file_type_mismatch` (extension names a different type than the content). | — |
| `422` | `file_corrupted` (cannot be parsed) or `file_password_protected`. | — |
| `502` | Blob Storage rejected the upload. | — |

### GET `/cases/{case_id}/documents/{document_id}/file-url`

**Get a fresh signed URL for a document** Bearer JWT

Mint a fresh signed URL for a document's original file — the URL handed back at upload time expires after a few minutes, which a reviewer working through the post-upload screen can easily outlast.

#### Parameters

| Name | In | Type | Required | Description |
|---|---|---|---|---|
| `case_id` | path | string (uuid) | yes | |
| `document_id` | path | string (uuid) | yes | |

#### Responses

| Status | Description | Body |
|---|---|---|
| `200` | Successful Response | DocumentFileUrlResponse |
| `401` | Missing, invalid or expired bearer token, or the user is inactive. | — |
| `404` | No such document in this case (or, for a `user`, not their case). | — |
| `422` | Validation Error | HTTPValidationError |
| `502` | Blob Storage failed to sign the URL. | — |

## Case actions (review decisions)

### POST `/cases/{case_id}/approve`

**Approve a case** Bearer JWT

Company reviewers only (platform admins: 403). Requires the automated pipeline to have finished and a risk assessment to exist (409 otherwise). Above `low` risk a written justification (`note`, at least 10 characters) is required (422). Approving an already-decided case is a 409. Writes a `case_actions` row and an audit event.

### Parameters

| Name | In | Type | Required | Description |
|---|---|---|---|---|
| `case_id` | path | string (uuid) | yes | |

### Request body

ApproveRequest

### Responses

| Status | Description | Body |
|---|---|---|
| `200` | Successful Response | CaseDecisionResponse |
| `401` | Missing, invalid or expired bearer token, or the user is inactive. | — |
| `403` | Requires a company reviewer role; a `reviewer_l1` also gets 403 on a case escalated to L2. | — |
| `404` | No such case in your company. | — |
| `409` | Case already decided, checks still running, or an invalid state transition. | — |
| `422` | Case is above low risk and no justification of at least 10 characters was given. | — |

## POST `/cases/{case_id}/reject`

**Reject a case** Bearer JWT

Company reviewers only (platform admins: 403). A `reason` is required. Rejecting an already-decided case is a 409. Writes a `case_actions` row and an audit event.

### Parameters

| Name | In | Type | Required | Description |
|---|---|---|---|---|
| `case_id` | path | string (uuid) | yes | |

### Request body

RejectRequest

### Responses

| Status | Description | Body |
|---|---|---|
| `200` | Successful Response | CaseDecisionResponse |
| `401` | Missing, invalid or expired bearer token, or the user is inactive. | — |
| `403` | Requires a company reviewer role; a `reviewer_l1` also gets 403 on a case escalated to L2. | — |
| `404` | No such case in your company. | — |
| `409` | Invalid state transition (e.g. already decided). | — |
| `422` | Validation Error | HTTPValidationError |

### POST `/cases/{case_id}/escalate`

**Escalate a case** Bearer JWT

Company reviewers only (platform admins: 403). Moves the case to the L2 tier ( `assigned_tier` = `l2` ); `status` is unchanged. A `reviewer_l1` then keeps read access but can no longer act on it. There is no de-escalation — an L2 reviewer approves or rejects it. A `reason` is required. Escalating an already-escalated or decided case is a 409.

#### Parameters

| Name | In | Type | Required | Description |
|---|---|---|---|---|
| `case_id` | path | string (uuid) | yes | |

#### Request body

EscalateRequest

#### Responses

| Status | Description | Body |
|---|---|---|
| `200` | Successful Response | CaseDecisionResponse |
| `401` | Missing, invalid or expired bearer token, or the user is inactive. | — |
| `403` | Requires a company reviewer role; a `reviewer_l1` also gets 403 on a case escalated to L2. | — |
| `404` | No such case in your company. | — |
| `409` | Invalid state transition (e.g. already decided). | — |
| `422` | Validation Error | HTTPValidationError |

## Signature references & matches

### POST `/cases/{case_id}/documents/{document_id}/signature-references`

**Create a signature reference** Bearer JWT

Create a signature reference from a reviewer-drawn bounding box.

Crops the signature region from the document's PDF, uploads it to Blob Storage, inserts the signature_references row, then immediately enqueues the in-case comparison Celery task for this reference.

`person_name` is required and must be typed by the reviewer — the system does not attempt to auto-extract it (System Specification §2.3).

#### Parameters

| Name | In | Type | Required | Description |
|---|---|---|---|---|
| `case_id` | path | string (uuid) | yes | |
| `document_id` | path | string (uuid) | yes | |

#### Request body

SignatureReferenceCreate

### Responses

| Status | Description | Body |
|---|---|---|
| `201` | Successful Response | SignatureReferenceResponse |
| `401` | Missing, invalid or expired bearer token, or the user is inactive. | — |
| `403` | Platform admins have read-only access. | — |
| `404` | No such case (or, for a `user`, not one they submitted), or the document is not in this case. | — |
| `422` | Validation Error | HTTPValidationError |

## GET `/cases/{case_id}/signature-references`

**List a case's signature references** Bearer JWT

All signature references created for this case. Used by the upload screen to list what references were set during this session.

### Parameters

| Name | In | Type | Required | Description |
|---|---|---|---|---|
| `case_id` | path | string (uuid) | yes | |

### Responses

| Status | Description | Body |
|---|---|---|
| `200` | Successful Response | array of SignatureReferenceResponse |
| `401` | Missing, invalid or expired bearer token, or the user is inactive. | — |
| `404` | No such case (or, for a `user`, not one they submitted). | — |
| `422` | Validation Error | HTTPValidationError |

## GET `/cases/{case_id}/signature-matches`

**List a case's signature comparison results** Bearer JWT

All in-case signature match results for this case, enriched with the reference person_name. Filtered to comparison_scope='in_case' only — library-scope matches are not generated yet (System Specification §2.3).

### Parameters

| Name | In | Type | Required | Description |
|---|---|---|---|---|
| `case_id` | path | string (uuid) | yes | |

### Responses

| Status | Description | Body |
|---|---|---|
| `200` | Successful Response | array of SignatureMatchResponse |
| `401` | Missing, invalid or expired bearer token, or the user is inactive. | — |

| Status | Description | Body |
|---|---|---|
| `404` | No such case (or, for a `user`, not one they submitted). | — |
| `422` | Validation Error | HTTPValidationError |

## Audit log

### GET `/audit-log`

**Search the audit log** Bearer JWT

Newest-first, paged read of the append-only `audit_log`, confined to the caller's company. Filters: `event_type`, `case_id` and free-text `q` (event type, case number, actor name/email). Company reviewers, and platform admins (`company_id` = one company's log, audited; omitted = the platform-level log).

#### Parameters

| Name | In | Type | Required | Description |
|---|---|---|---|---|
| `event_type` | query | string (nullable) | no | |
| `case_id` | query | string (uuid) (nullable) | no | |
| `q` | query | string (nullable) | no | Matches event type, case number, actor name/email |
| `company_id` | query | string (uuid) (nullable) | no | Platform admins only. |
| `limit` | query | integer | no | |
| `offset` | query | integer | no | |

#### Responses

| Status | Description | Body |
|---|---|---|
| `200` | Successful Response | AuditPage |
| `401` | Missing, invalid or expired bearer token, or the user is inactive. | — |
| `403` | Requires a reviewer (L1 or L2) or platform admin role. | — |
| `422` | Validation Error | HTTPValidationError |

### GET `/audit-log/event-types`

**List audit event types** Bearer JWT

Distinct `event_type` values present in the caller's company's audit log (platform admins: the chosen company's, or the platform log), for the filter dropdown.

#### Parameters

| Name | In | Type | Required | Description |
|---|---|---|---|---|
| `company_id` | query | string (uuid) (nullable) | no | Platform admins only. |

### Responses

| Status | Description | Body |
|---|---|---|
| `200` | Successful Response | array of string |
| `401` | Missing, invalid or expired bearer token, or the user is inactive. | — |
| `403` | Requires a reviewer (L1 or L2) or platform admin role. | — |
| `422` | Validation Error | HTTPValidationError |

## Case reports

**POST** `/cases/{case_id}/reports`

### Generate a case report (PDF) Bearer JWT

Company reviewers only. Renders the per-case forensic report PDF, stores it in Blob Storage under the company's `companies/{company_id}/cases/{case_id}/reports/` prefix, records a `case_reports` row (with the file's SHA-256) and returns it with a signed download URL. Every call creates a new report; earlier ones are kept.

### Parameters

| Name | In | Type | Required | Description |
|---|---|---|---|---|
| `case_id` | path | string (uuid) | yes | |

### Responses

| Status | Description | Body |
|---|---|---|
| `201` | Successful Response | CaseReportResponse |
| `401` | Missing, invalid or expired bearer token, or the user is inactive. | — |
| `403` | Requires a reviewer (L1 or L2) or `admin` role. | — |
| `404` | No such case. | — |
| `422` | Validation Error | HTTPValidationError |
| `502` | Blob Storage failed while saving or signing the report. | — |

**GET** `/cases/{case_id}/reports`

### List a case's generated reports Bearer JWT

Report history for the case, newest first.

### Parameters

| Name | In | Type | Required | Description |
|---|---|---|---|---|
| `case_id` | path | string (uuid) | yes | |

### Responses

| Status | Description | Body |
|---|---|---|
| `200` | Successful Response | array of CaseReportResponse |

| Status | Description | Body |
|---|---|---|
| `401` | Missing, invalid or expired bearer token, or the user is inactive. | — |
| `403` | Requires a reviewer (L1 or L2) or `admin` role. | — |
| `404` | No such case. | — |
| `422` | Validation Error | HTTPValidationError |
| `502` | Blob Storage failed while signing a download URL. | — |

## Settings (admin)

### GET `/settings/issuers`

**List issuers** Bearer JWT

The company's issuer registry, ordered by name. Pass `include_inactive=false` to hide deactivated entries. Reviewer L2 (own company) or platform admin ( `company_id` ).

#### Parameters

| Name | In | Type | Required | Description |
|---|---|---|---|---|
| `include_inactive` | query | boolean | no | |
| `company_id` | query | string (uuid) (nullable) | no | Platform admins only (required for them): the company to act on. Company users are always confined to their own company. |

#### Responses

| Status | Description | Body |
|---|---|---|
| `200` | Successful Response | array of IssuerResponse |
| `401` | Missing, invalid or expired bearer token, or the user is inactive. | — |
| `403` | Requires the Reviewer L2 role (or platform admin). | — |
| `422` | Validation Error | HTTPValidationError |

### POST `/settings/issuers`

**Create an issuer** Bearer JWT

Adds a known vendor/school/etc. to the company's registry used by issuer verification. Names are unique (case-insensitively) among the company's active issuers.

#### Parameters

| Name | In | Type | Required | Description |
|---|---|---|---|---|
| `company_id` | query | string (uuid) (nullable) | no | Platform admins only (required for them): the company to act on. Company users are always confined to their own company. |

#### Request body

IssuerCreate

### Responses

| Status | Description | Body |
|---|---|---|
| `201` | Successful Response | IssuerResponse |
| `401` | Missing, invalid or expired bearer token, or the user is inactive. | — |
| `403` | Requires the Reviewer L2 role (or platform admin). | — |
| `409` | An active issuer with this name already exists. | — |
| `422` | Validation Error | HTTPValidationError |

## PATCH `/settings/issuers/{issuer_id}`

**Update or deactivate an issuer** Bearer JWT

Partial update: only the fields sent are changed. Issuers are never deleted; set `is_active=false` to retire one. A request that changes nothing is a no-op. An issuer of another company is a 404.

### Parameters

| Name | In | Type | Required | Description |
|---|---|---|---|---|
| `issuer_id` | path | string (uuid) | yes | |
| `company_id` | query | string (uuid) (nullable) | no | Platform admins only (required for them): the company to act on. Company users are always confined to their own company. |

### Request body

IssuerUpdate

### Responses

| Status | Description | Body |
|---|---|---|
| `200` | Successful Response | IssuerResponse |
| `401` | Missing, invalid or expired bearer token, or the user is inactive. | — |
| `403` | Requires the Reviewer L2 role (or platform admin). | — |
| `404` | No such issuer. | — |
| `422` | Validation Error | HTTPValidationError |

## GET `/settings/risk-rules`

**List current risk rules** Bearer JWT

The CURRENT version of every rule (active or not) of the company, by category.

### Parameters

| Name | In | Type | Required | Description |
|---|---|---|---|---|
| `company_id` | query | string (uuid) (nullable) | no | Platform admins only (required for them): the company to act on. Company users are always confined to their own company. |

### Responses

| Status | Description | Body |
|---|---|---|
| `200` | Successful Response | array of RiskRuleResponse |
| `401` | Missing, invalid or expired bearer token, or the user is inactive. | — |
| `403` | Requires the Reviewer L2 role (or platform admin). | — |
| `422` | Validation Error | HTTPValidationError |

## POST `/settings/risk-rules`

**Create a risk rule** Bearer JWT

A new rule at version 1, for this company only. It applies to the company's cases scored from now on; nothing already scored changes (assessments freeze the versions that produced them).

### Parameters

| Name | In | Type | Required | Description |
|---|---|---|---|---|
| `company_id` | query | string (uuid) (nullable) | no | Platform admins only (required for them): the company to act on. Company users are always confined to their own company. |

### Request body

RiskRuleCreate

### Responses

| Status | Description | Body |
|---|---|---|
| `201` | Successful Response | RiskRuleResponse |
| `401` | Missing, invalid or expired bearer token, or the user is inactive. | — |
| `403` | Requires the Reviewer L2 role (or platform admin). | — |
| `409` | A rule with this `rule_id` already exists in the company. | — |
| `422` | The chosen match/parameters cannot be evaluated by the engine. | — |

## GET `/settings/risk-rule-options`

**Get the options for the Add-rule form** Bearer JWT

Everything the Add-rule form offers, so it never has to hardcode (or guess) what the checks can produce.

### Responses

| Status | Description | Body |
|---|---|---|
| `200` | Successful Response | object |
| `401` | Missing, invalid or expired bearer token, or the user is inactive. | — |
| `403` | Requires the Reviewer L2 role (or platform admin). | — |

## GET `/settings/risk-rules/{rule_id}/history`

### Get a risk rule's version history

Bearer JWT

Every version of one of the company's rules, newest first — the per-rule audit trail (who changed what, when). The same edits are also in `audit_log`.

#### Parameters

| Name | In | Type | Required | Description |
|---|---|---|---|---|
| `rule_id` | path | string | yes | |
| `company_id` | query | string (uuid) (nullable) | no | Platform admins only (required for them): the company to act on. Company users are always confined to their own company. |

#### Responses

| Status | Description | Body |
|---|---|---|
| `200` | Successful Response | array of RiskRuleResponse |
| `401` | Missing, invalid or expired bearer token, or the user is inactive. | — |
| `403` | Requires the Reviewer L2 role (or platform admin). | — |
| `404` | No such rule. | — |
| `422` | Validation Error | HTTPValidationError |

## PATCH `/settings/risk-rules/{rule_id}`

### Change a risk rule's weight, severity or active flag

Bearer JWT

Rules are versioned: an edit inserts a new immutable row (`version` + 1) and leaves the old one untouched, so every historical assessment stays truthful. Only `weight`, `severity` and `is_active` can be changed; at least one must differ. Only the company's own rule is affected.

#### Parameters

| Name | In | Type | Required | Description |
|---|---|---|---|---|
| `rule_id` | path | string | yes | |
| `company_id` | query | string (uuid) (nullable) | no | Platform admins only (required for them): the company to act on. Company users are always confined to their own company. |

#### Request body

RiskRuleUpdate

#### Responses

| Status | Description | Body |
|---|---|---|
| `200` | Successful Response | RiskRuleResponse |
| `400` | Nothing to change. | — |

| Status | Description | Body |
|---|---|---|
| `401` | Missing, invalid or expired bearer token, or the user is inactive. | — |
| `403` | Requires the Reviewer L2 role (or platform admin). | — |
| `404` | No such rule. | — |
| `409` | The rule was changed concurrently; reload and retry. | — |
| `422` | Validation Error | HTTPValidationError |

## GET `/settings/risk-thresholds`

### Get the risk tier thresholds

Bearer JWT

The company's scores at which a case becomes `medium` and `high` risk (defaults 30 and 60). The row is created on first read if it does not exist.

#### Parameters

| Name | In | Type | Required | Description |
|---|---|---|---|---|
| `company_id` | query | string (uuid) (nullable) | no | Platform admins only (required for them): the company to act on. Company users are always confined to their own company. |

#### Responses

| Status | Description | Body |
|---|---|---|
| `200` | Successful Response | RiskThresholdsResponse |
| `401` | Missing, invalid or expired bearer token, or the user is inactive. | — |
| `403` | Requires the Reviewer L2 role (or platform admin). | — |
| `422` | Validation Error | HTTPValidationError |

## PUT `/settings/risk-thresholds`

### Set the risk tier thresholds

Bearer JWT

Safe for history: each assessment embeds the thresholds it used.

#### Parameters

| Name | In | Type | Required | Description |
|---|---|---|---|---|
| `company_id` | query | string (uuid) (nullable) | no | Platform admins only (required for them): the company to act on. Company users are always confined to their own company. |

#### Request body

RiskThresholds

#### Responses

| Status | Description | Body |
|---|---|---|
| `200` | Successful Response | RiskThresholdsResponse |

| Status | Description | Body |
|---|---|---|
| `401` | Missing, invalid or expired bearer token, or the user is inactive. | — |
| `403` | Requires the Reviewer L2 role (or platform admin). | — |
| `422` | Validation Error | HTTPValidationError |

## GET `/settings/users`

**List users** Bearer JWT

Every account on the platform, ordered by company then email; filter with `company_id`. Platform admin only.

### Parameters

| Name | In | Type | Required | Description |
|---|---|---|---|---|
| `company_id` | query | string (uuid) (nullable) | no | |

### Responses

| Status | Description | Body |
|---|---|---|
| `200` | Successful Response | array of UserAdminResponse |
| `401` | Missing, invalid or expired bearer token, or the user is inactive. | — |
| `403` | Requires the platform admin role. | — |
| `422` | Validation Error | HTTPValidationError |

## POST `/settings/users`

**Create a user** Bearer JWT

Creates an account and assigns it to a company + role. Company roles ( `user` , `reviewer_l1` , `reviewer_l2` ) need a `company_id` ; a `platform_admin` must have none. Platform admin only.

### Request body

UserCreate

### Responses

| Status | Description | Body |
|---|---|---|
| `201` | Successful Response | UserAdminResponse |
| `401` | Missing, invalid or expired bearer token, or the user is inactive. | — |
| `403` | Requires the platform admin role. | — |
| `404` | No such company. | — |
| `409` | An account with that email already exists. | — |
| `422` | Company/role combination is invalid. | — |

## PATCH `/settings/users/{user_id}`

### Change a user's role, company or active flag

Bearer JWT

Partial update of `role`, `company_id` and/or `is_active`. A platform admin cannot change their own role or deactivate themselves (this also guarantees a platform admin always remains). Changing a user's company or role invalidates their existing tokens (they must sign in again). There is no self-service signup and no password-reset endpoint. Platform admin only.

#### Parameters

| Name | In | Type | Required | Description |
|---|---|---|---|---|
| `user_id` | path | string (uuid) | yes | |

#### Request body

UserAdminUpdate

#### Responses

| Status | Description | Body |
|---|---|---|
| `200` | Successful Response | UserAdminResponse |
| `400` | Attempted to change your own role or deactivate yourself. | — |
| `401` | Missing, invalid or expired bearer token, or the user is inactive. | — |
| `403` | Requires the platform admin role. | — |
| `404` | No such user (or company). | — |
| `422` | Company/role combination is invalid. | — |

## POST `/settings/users/{user_id}/reset-password`

### Reset user password (platform admin only)

Bearer JWT

Resets the password for any user account. The new password is stored hashed. Only platform admins have access to this action. Action is logged in the audit trail.

#### Parameters

| Name | In | Type | Required | Description |
|---|---|---|---|---|
| `user_id` | path | string (uuid) | yes | |

#### Request body

UserPasswordReset

#### Responses

| Status | Description | Body |
|---|---|---|
| `200` | Successful Response | UserAdminResponse |
| `401` | Missing, invalid or expired bearer token, or the user is inactive. | — |
| `403` | Requires the platform admin role. | — |
| `404` | No such user. | — |

| Status | Description | Body |
|---|---|---|
| `422` | Validation Error | HTTPValidationError |

## Other

**POST** `/bulk-uploads`

### Bulk upload: a zip of cases

Bearer JWT

Request body: the zip file itself ( `Content-Type: application/zip` ), not multipart. Query: `case_type` (applied to every case) and `filename` (the zip's name, for display). The zip holds one top-level folder per case, each containing that case's documents directly. The response (202) returns once the zip is stored: it lists every case folder found, with structural failures already marked (subfolder in a case folder, empty folder, empty or file over the company's per-file limit). Documents are then extracted, validated with the same checks as a single upload, and each valid case is created and queued, one case at a time. Poll GET /bulk-uploads/{id}. Partial success: a bad file is rejected on its own and a case with no valid file fails on its own. Zip rejections return `detail: {code, message, ...}` with `zip_empty` , `zip_too_large` (over the company's zip limit, `max_zip_size_mb` , 300 MB by default, decided from Content-Length before the body is read), `not_a_zip` , `zip_corrupted` , `zip_too_many_entries` or `zip_no_case_folders` .

#### Parameters

| Name | In | Type | Required | Description |
|---|---|---|---|---|
| `case_type` | query | CaseType | yes | Case type for every case in the zip. |
| `filename` | query | string | yes | The zip's file name. |

#### Responses

| Status | Description | Body |
|---|---|---|
| `202` | Successful Response | BulkUploadDetail |
| `400` | `zip_empty` . | — |
| `401` | Missing, invalid or expired bearer token, or the user is inactive. | — |
| `403` | Platform admins have read-only access. | — |
| `413` | `zip_too_large` ( `size_bytes` , `max_size_bytes` ). | — |
| `415` | `not_a_zip` . | — |
| `422` | `zip_corrupted` , `zip_too_many_entries` , `zip_no_case_folders` . | — |
| `502` | Blob Storage rejected the upload. | — |

**GET** `/bulk-uploads`

### List bulk uploads

Bearer JWT

Newest first. A `user` sees only their own; reviewers see their company's. Platform admins pass `company_id` (audited).

### Parameters

| Name | In | Type | Required | Description |
|---|---|---|---|---|
| `limit` | query | integer | no | |
| `company_id` | query | string (uuid) (nullable) | no | Platform admins only (required for them): the company to act on. Company users are always confined to their own company. |

### Responses

| Status | Description | Body |
|---|---|---|
| `200` | Successful Response | array of BulkUploadSummary |
| `422` | Validation Error | HTTPValidationError |

## GET `/bulk-uploads/{bulk_upload_id}`

**Bulk upload summary (live per-case status)** Bearer JWT

Every case folder of the zip with its outcome (created / failed and why, per file) and, for created cases, live progress (queued → processing → done / flagged). Poll until `settled`. Another user's upload (for a `user`) or another company's is a 404.

### Parameters

| Name | In | Type | Required | Description |
|---|---|---|---|---|
| `bulk_upload_id` | path | string (uuid) | yes | |
| `company_id` | query | string (uuid) (nullable) | no | Platform admins only (required for them): the company to act on. Company users are always confined to their own company. |

### Responses

| Status | Description | Body |
|---|---|---|
| `200` | Successful Response | BulkUploadDetail |
| `422` | Validation Error | HTTPValidationError |

## GET `/platform/companies`

**List companies** Bearer JWT

### Responses

| Status | Description | Body |
|---|---|---|
| `200` | Successful Response | array of CompanyResponse |
| `401` | Missing, invalid or expired bearer token, or the user is inactive. | — |
| `403` | Requires the platform admin role. | — |

### POST `/platform/companies`

**Create a company** Bearer JWT

Creates a client company and copies every active platform rule template into its own risk rules, plus default tier thresholds (its own independent copy, which its Reviewer L2s can then tune; later template edits don't affect it). The issuer registry starts empty. Users are added separately (Settings > Users).

#### Request body

CompanyCreate

#### Responses

| Status | Description | Body |
|---|---|---|
| `201` | Successful Response | CompanyResponse |
| `401` | Missing, invalid or expired bearer token, or the user is inactive. | — |
| `403` | Requires the platform admin role. | — |
| `409` | A company with this name already exists. | — |
| `422` | Validation Error | HTTPValidationError |

### PATCH `/platform/companies/{company_id}`

**Rename, suspend/reactivate, or change a company's upload limits** Bearer JWT

`is_active=false` suspends the company: its users can no longer sign in, and existing tokens stop working on the next request. Data is kept. `max_file_size_mb` / `max_zip_size_mb` change the company's upload limits (any positive whole number of MB; no upper ceiling), effective on its next upload with no new sign-in. Every change is written to the platform audit log as `company_updated` with old → new values.

#### Parameters

| Name | In | Type | Required | Description |
|---|---|---|---|---|
| `company_id` | path | string (uuid) | yes | |

#### Request body

CompanyUpdate

#### Responses

| Status | Description | Body |
|---|---|---|
| `200` | Successful Response | CompanyResponse |
| `401` | Missing, invalid or expired bearer token, or the user is inactive. | — |
| `403` | Requires the platform admin role. | — |
| `404` | No such company. | — |
| `409` | Name taken. | — |
| `422` | Validation Error | HTTPValidationError |

### GET `/platform/risk-rule-templates`

**List the platform risk-rule template** Bearer JWT

The default rule set copied into every NEW company's own risk rules at creation time. Editing it never changes companies that already exist.

#### Responses

| Status | Description | Body |
|---|---|---|
| `200` | Successful Response | array of RuleTemplateResponse |
| `401` | Missing, invalid or expired bearer token, or the user is inactive. | — |
| `403` | Requires the platform admin role. | — |

### POST `/platform/risk-rule-templates`

**Add a rule to the platform template** Bearer JWT

Same rule builder as a company's Add-rule form. Applies to companies created from now on.

#### Request body

RiskRuleCreate

#### Responses

| Status | Description | Body |
|---|---|---|
| `201` | Successful Response | RuleTemplateResponse |
| `401` | Missing, invalid or expired bearer token, or the user is inactive. | — |
| `403` | Requires the platform admin role. | — |
| `409` | A template rule with this rule_id already exists. | — |
| `422` | Validation Error | HTTPValidationError |

### PATCH `/platform/risk-rule-templates/{rule_id}`

**Edit a platform template rule** Bearer JWT

Changes weight, severity, wording or active flag. Existing companies keep their own copies unchanged; only companies created afterwards start from the new values.

#### Parameters

| Name | In | Type | Required | Description |
|---|---|---|---|---|
| `rule_id` | path | string | yes | |

#### Request body

RuleTemplateUpdate

#### Responses

| Status | Description | Body |
|---|---|---|
| `200` | Successful Response | RuleTemplateResponse |
| `401` | Missing, invalid or expired bearer token, or the user is inactive. | — |

| Status | Description | Body |
|---|---|---|
| `403` | Requires the platform admin role. | — |
| `404` | No such template rule. | — |
| `422` | Validation Error | HTTPValidationError |

### GET `/platform/usage`

**Billing & usage by company** Bearer JWT

One row per company for the period: cases created, documents uploaded, files stored (documents + generated reports) and bytes stored during the period, plus each company's current all-time storage. Read from incrementally-maintained daily counters (company_usage_stats), not live scans of the case tables; a nightly job reconciles them against the source data. Periods use UTC days.

#### Parameters

| Name | In | Type | Required | Description |
|---|---|---|---|---|
| `period` | query | string: `"this_month"`, `"last_month"`, `"last_30_days"`, `"this_year"`, `"all_time"`, `"custom"` | no | |
| `start` | query | string (date) (nullable) | no | custom period only |
| `end` | query | string (date) (nullable) | no | custom period only |

#### Responses

| Status | Description | Body |
|---|---|---|
| `200` | Successful Response | UsageResponse |
| `401` | Missing, invalid or expired bearer token, or the user is inactive. | — |
| `403` | Requires the platform admin role. | — |
| `422` | Validation Error | HTTPValidationError |

### POST `/platform/usage/reconcile`

**Reconcile usage counters now** Bearer JWT

Recomputes every company's daily counters from the cases/documents/reports tables and corrects any drift (the same job runs nightly). Returns what was corrected; a correction is also written to the platform audit log.

#### Responses

| Status | Description | Body |
|---|---|---|
| `200` | Successful Response | ReconcileResponse |
| `401` | Missing, invalid or expired bearer token, or the user is inactive. | — |
| `403` | Requires the platform admin role. | — |

### GET `/platform/queues`

**Processing queue monitor** Bearer JWT

Per queue (extraction / vision / forensics): tasks waiting and running, age of the oldest waiting task, average / p95 / max wait over the window, average run time, the rate limit in force and the limiter's counters (calls throttled, Azure 429s), and the worker processes consuming it. Wait times are measured from enqueue to start.

#### Parameters

| Name | In | Type | Required | Description |
|---|---|---|---|---|
| `window_minutes` | query | integer | no | |

#### Responses

| Status | Description | Body |
|---|---|---|
| `200` | Successful Response | object |
| `401` | Missing, invalid or expired bearer token, or the user is inactive. | — |
| `403` | Requires the platform admin role. | — |
| `422` | Validation Error | HTTPValidationError |

## Data models

### ApproveRequest

| Field | Type | Required | Description |
|---|---|---|---|
| `note` | string (nullable) | no | Justification. Optional for low-risk cases; at least 10 characters (after trimming) is required to approve a medium- or high-risk case. |

### AuditEventResponse

| Field | Type | Required | Description |
|---|---|---|---|
| `id` | string (uuid) | yes | |
| `created_at` | string (date-time) | yes | |
| `event_type` | string | yes | |
| `actor_name` | string (nullable) | yes | |
| `actor_email` | string (nullable) | yes | |
| `case_id` | string (uuid) (nullable) | yes | |
| `case_number` | string (nullable) | yes | |
| `document_id` | string (uuid) (nullable) | yes | |
| `document_filename` | string (nullable) | yes | |
| `event_data` | object (nullable) | yes | |

## AuditLogEntrySchema

One `audit_log` row scoped to a case — System Specification section 3.5's "review history/timeline ... built from the same audit log." Every event_type here is a real one already written by app/services/ audit_service.py's call sites (case creation, uploads, and each Celery check task) — nothing here is synthesized for display.

| Field | Type | Required | Description |
|---|---|---|---|
| `id` | string (uuid) | yes | |
| `event_type` | string | yes | |
| `actor_name` | string (nullable) | yes | |
| `document_id` | string (uuid) (nullable) | yes | |
| `event_data` | object (nullable) | yes | |
| `created_at` | string (date-time) | yes | |

## AuditPage

| Field | Type | Required | Description |
|---|---|---|---|
| `items` | array of AuditEventResponse | yes | |
| `total` | integer | yes | |

## Body_upload_document_cases__case_id__documents_post

| Field | Type | Required | Description |
|---|---|---|---|
| `file` | string (binary) | yes | |

## BoundingBoxSchema

Normalized bounding box — page-fraction (0-1) coordinates, same convention as every other bounding box in this codebase (ELA, copy-move, visual inconsistency). `page` is 1-based.

| Field | Type | Required | Description |
|---|---|---|---|
| `page` | integer | yes | |
| `x` | number | yes | |
| `y` | number | yes | |
| `width` | number | yes | |
| `height` | number | yes | |

## BulkUploadCaseSchema

| Field | Type | Required | Description |
|---|---|---|---|
| `index` | integer | yes | |
| `folder` | string | yes | |
| `status` | string: `"pending"`, `"created"`, `"failed"` | yes | |
| `error_code` | string (nullable) | no | |
| `error_message` | string (nullable) | no | |
| `case_id` | string (uuid) (nullable) | no | |
| `case_number` | string (nullable) | no | |
| `files` | array of BulkUploadFileSchema | yes | |
| `live_status` | string: `"validating"`, `"failed"`, `"queued"`, `"processing"`, `"done"`, `"flagged"` | yes | |
| `case_status` | CaseStatus (nullable) | no | |
| `documents_total` | integer | no | |
| `documents_finished` | integer | no | |
| `flag` | CaseFlagSchema (nullable) | no | |

## BulkUploadDetail

| Field | Type | Required | Description |
|---|---|---|---|
| `id` | string (uuid) | yes | |
| `original_filename` | string | yes | |
| `case_type` | CaseType | yes | |
| `status` | BulkUploadStatus | yes | |
| `error_code` | string (nullable) | yes | |
| `error_message` | string (nullable) | yes | |
| `zip_size_bytes` | integer | yes | |
| `case_folder_count` | integer (nullable) | yes | |
| `cases_created` | integer | yes | |
| `cases_failed` | integer | yes | |
| `documents_accepted` | integer | yes | |
| `documents_rejected` | integer | yes | |
| `uploaded_by` | UserSummary (nullable) | yes | |
| `created_at` | string (date-time) | yes | |
| `started_at` | string (date-time) (nullable) | yes | |
| `finished_at` | string (date-time) (nullable) | yes | |
| `wrapper_folder` | string (nullable) | no | |
| `ignored_entries` | array of string | no | |
| `ignored_entry_count` | integer | no | |
| `warnings` | array of string | no | |
| `progress` | BulkUploadProgress | yes | |
| `settled` | boolean | yes | |
| `cases` | array of BulkUploadCaseSchema | yes | |

## BulkUploadFileSchema

| Field | Type | Required | Description |
|---|---|---|---|
| `name` | string | yes | |
| `status` | string: `"pending"`, `"accepted"`, `"rejected"`, `"skipped"` | yes | |
| `code` | string (nullable) | no | |
| `message` | string (nullable) | no | |
| `size_bytes` | integer (nullable) | no | |
| `document_id` | string (uuid) (nullable) | no | |

## BulkUploadProgress

| Field | Type | Required | Description |
|---|---|---|---|
| `validating` | integer | no | |
| `failed` | integer | no | |
| `queued` | integer | no | |
| `processing` | integer | no | |
| `done` | integer | no | |
| `flagged` | integer | no | |

## BulkUploadStatus

Allowed values: `"queued"`, `"ingesting"`, `"complete"`, `"failed"`

## BulkUploadSummary

| Field | Type | Required | Description |
|---|---|---|---|
| `id` | string (uuid) | yes | |
| `original_filename` | string | yes | |
| `case_type` | CaseType | yes | |
| `status` | BulkUploadStatus | yes | |
| `error_code` | string (nullable) | yes | |
| `error_message` | string (nullable) | yes | |
| `zip_size_bytes` | integer | yes | |
| `case_folder_count` | integer (nullable) | yes | |
| `cases_created` | integer | yes | |
| `cases_failed` | integer | yes | |
| `documents_accepted` | integer | yes | |
| `documents_rejected` | integer | yes | |
| `uploaded_by` | UserSummary (nullable) | yes | |
| `created_at` | string (date-time) | yes | |
| `started_at` | string (date-time) (nullable) | yes | |
| `finished_at` | string (date-time) (nullable) | yes | |

## CaseActionSchema

A reviewer decision/note on the case (approve/reject/escalate).

| Field | Type | Required | Description |
|---|---|---|---|
| `id` | string (uuid) | yes | |
| `action_type` | string | yes | |
| `actor_name` | string (nullable) | yes | |
| `actor_role` | string (nullable) | no | Display label of the actor's role when they acted, e.g. 'Reviewer L1'. |
| `notes` | string (nullable) | yes | |
| `created_at` | string (date-time) | yes | |

## CaseCreateRequest

| Field | Type | Required | Description |
|---|---|---|---|
| `case_type` | CaseType | yes | Case-level category chosen at submission. Independent of each document's own `document_type`, which the classification step fills in later. |

## CaseDecisionResponse

| Field | Type | Required | Description |
|---|---|---|---|
| `case_id` | string (uuid) | yes | |
| `status` | CaseStatus | yes | |
| `assigned_tier` | CaseTier | yes | |
| `action` | CaseActionSchema | yes | |

## CaseDetail

| Field | Type | Required | Description |
|---|---|---|---|
| `id` | string (uuid) | yes | |
| `case_number` | string | yes | |
| `case_type` | CaseType | yes | |
| `status` | CaseStatus | yes | |
| `assigned_tier` | CaseTier | yes | |
| `risk_tier` | RiskTier (nullable) | yes | |
| `submitted_by` | UserSummary (nullable) | no | |
| `created_at` | string (date-time) | yes | |
| `document_count` | integer | yes | |
| `flag` | CaseFlagSchema | yes | |
| `can_act` | boolean | yes | Whether the caller may approve/reject/escalate this case given its tier. False for submitters, and for a `reviewer_l1` on an L2-assigned case (read-only). Does not reflect status — a decided case is still `can_act` but every action is a 409. |
| `documents` | array of CaseDocumentSummary | yes | |
| `cross_document_findings` | array of CrossDocumentFindingSummary | yes | |
| `forensic_findings` | array of ForensicFindingSummary | yes | |
| `assessment` | RiskAssessmentSchema (nullable) | yes | |
| `pipeline` | PipelineStatusSchema | yes | |
| `actions` | array of CaseActionSchema | yes | |

## CaseDocumentSummary

A document as shown on the case-detail page. `file_url` here is a short-lived signed URL (see StorageService.get_download_url) — the storage container is private, so the durable URL alone isn't browser-fetchable.

`document_type` and `extracted_fields` are filled in by the OCR + classification + extraction Celery pipeline (app/tasks/document_ processing.py) — both stay null until `processing_status` reaches "complete". `extracted_fields` shape: {"document_type_confidence": float, "core_fields": {name: {value, confidence, uncertain}}, "additional_fields": [{field_name, value, confidence, uncertain}]}.

`checks` is filled in afterward by app/tasks/document_checks.py (field_validation, issuer_verification, and later the forensic checks) — empty until those complete.

| Field | Type | Required | Description |
|---|---|---|---|
| `id` | string (uuid) | yes | |
| `original_filename` | string | yes | |
| `document_type` | string (nullable) | yes | |
| `processing_status` | string | yes | |
| `extracted_fields` | object (nullable) | yes | |
| `processing_error` | string (nullable) | yes | |
| `content_type` | string (nullable) | yes | |
| `file_size_bytes` | integer (nullable) | yes | |
| `file_hash` | string | yes | |
| `file_url` | string | yes | |
| `uploaded_at` | string (date-time) | yes | |
| `checks` | array of DocumentCheckSummary | yes | |

## CaseFlagSchema

Wire shape of app/services/case_flag_service.py's CaseFlag: the case's real risk tier (low/medium/high), or "pending" until scored.

| Field | Type | Required | Description |
|---|---|---|---|
| `flag` | string: `"low"`, `"medium"`, `"high"`, `"pending"` | yes | |
| `label` | string | yes | |
| `description` | string | yes | |
| `score` | integer (nullable) | no | |

### CaseListItem

| Field | Type | Required | Description |
|---|---|---|---|
| `id` | string (uuid) | yes | |
| `case_number` | string | yes | |
| `case_type` | CaseType | yes | |
| `status` | CaseStatus | yes | |
| `assigned_tier` | CaseTier | yes | |
| `risk_tier` | RiskTier (nullable) | yes | |
| `submitted_by` | UserSummary (nullable) | no | |
| `created_at` | string (date-time) | yes | |
| `document_count` | integer | yes | |
| `flag` | CaseFlagSchema | yes | |
| `can_act` | boolean | yes | Whether the caller may approve/reject/escalate this case given its tier. False for submitters, and for a `reviewer_l1` on an L2-assigned case (read-only). Does not reflect status — a decided case is still `can_act` but every action is a 409. |

## CaseReportResponse

One generated report. `download_url` is a short-lived signed URL (the container is private — see app/services/storage_service.py), so it is minted fresh on every response rather than stored.

| Field | Type | Required | Description |
|---|---|---|---|
| `id` | string (uuid) | yes | |
| `case_id` | string (uuid) | yes | |
| `generated_by_name` | string (nullable) | yes | |
| `generated_at` | string (date-time) | yes | |
| `file_size_bytes` | integer | yes | |
| `page_count` | integer | yes | |
| `report_sha256` | string | yes | |
| `risk_tier` | string (nullable) | yes | |
| `risk_score` | integer (nullable) | yes | |
| `download_url` | string | yes | |

## CaseResponse

| Field | Type | Required | Description |
|---|---|---|---|
| `id` | string (uuid) | yes | |
| `case_number` | string | yes | |
| `case_type` | CaseType | yes | |
| `status` | CaseStatus | yes | |
| `assigned_tier` | CaseTier | yes | |
| `risk_tier` | RiskTier (nullable) | yes | |
| `submitted_by_user_id` | string (uuid) | yes | |
| `created_at` | string (date-time) | yes | |

## CaseStatus

Allowed values: `"submitted"`, `"under_automated_review"`, `"pending_manual_review"`, `"auto_approved"`, `"escalated"`, `"under_investigation"`, `"approved"`, `"rejected"`, `"closed"`

## CaseTier

Which reviewer tier owns the case — orthogonal to `status`. Every case starts at `l1`; escalating it moves it to `l2` (one-directional, there is no de-escalation). On an `l2` case a `reviewer_l1` keeps read access but every action endpoint returns 403; `reviewer_l2` and `admin` can act on cases of either tier.

Allowed values: `"l1"`, `"l2"`

## CaseType

Case-level category, selected at submission time (System Specification section 3.6's document-type coverage list). This is separate from a document's own `document_type`, which is filled in later by the classification step — a case can hold documents of more than one type (e.g. an invoice plus its approval voucher).

Allowed values: `"school_document"`, `"vendor_invoice"`, `"commercial_invoice"`, `"procurement_documentation"`, `"quotation"`, `"travel_reimbursement"`, `"other"`

## ChangePasswordRequest

Change your own password. The current password is required; a forgotten password is reset by a platform admin instead (there is no self-service reset).

| Field | Type | Required | Description |
|---|---|---|---|
| `current_password` | string | yes | Your current password. |
| `new_password` | string | yes | New password (8-128 chars); stored hashed. |

## CompanyCreate

| Field | Type | Required | Description |
|---|---|---|---|
| `name` | string | yes | |
| `max_file_size_mb` | integer (nullable) | no | Max document size in MB (per company). |
| `max_zip_size_mb` | integer (nullable) | no | Max bulk-upload zip size in MB (per company). |

## CompanyResponse

| Field | Type | Required | Description |
|---|---|---|---|
| `id` | string (uuid) | yes | |
| `name` | string | yes | |
| `is_active` | boolean | yes | |
| `created_at` | string (date-time) | yes | |
| `user_count` | integer | yes | |
| `max_file_size_mb` | integer | yes | Largest document this company may upload, in MB. |
| `max_zip_size_mb` | integer | yes | Largest bulk-upload zip this company may upload, in MB. |

## CompanyUpdate

| Field | Type | Required | Description |
|---|---|---|---|
| `name` | string (nullable) | no | |
| `is_active` | boolean (nullable) | no | |
| `max_file_size_mb` | integer (nullable) | no | Max document size in MB (per company). |
| `max_zip_size_mb` | integer (nullable) | no | Max bulk-upload zip size in MB (per company). |

## CompanyUsageRow

| Field | Type | Required | Description |
|---|---|---|---|
| `company_id` | string (uuid) | yes | |
| `company_name` | string | yes | |
| `is_active` | boolean | yes | |
| `cases_created` | integer | yes | |
| `documents_uploaded` | integer | yes | |
| `files_stored` | integer | yes | |
| `storage_bytes` | integer | yes | |
| `total_storage_bytes` | integer | yes | |
| `total_files_stored` | integer | yes | |

## CrossDocumentFindingSummary

One `cross_document_findings` row (System Specification section 3.1, "Cross-document validation & reconciliation") — a shared field (amount/date/issuer) that disagreed between two of this case's documents. Case-level, not per-document — see app/services/ cross_document_service.py and app/tasks/document_checks.py.

| Field | Type | Required | Description |
|---|---|---|---|
| `id` | string (uuid) | yes | |
| `field_name` | string | yes | |
| `finding_type` | string | yes | |
| `severity` | string | yes | |
| `description` | string | yes | |
| `document_ids` | array of string (nullable) | yes | |
| `created_at` | string (date-time) | yes | |
| `regions` | array of object | no | |

## CurrentUserResponse

| Field | Type | Required | Description |
|---|---|---|---|
| `id` | string (uuid) | yes | |
| `email` | string (email) | yes | |
| `full_name` | string (nullable) | yes | Display name, if one was set. |
| `role` | UserRole | yes | `user` (submitter), `reviewer_l1`, `reviewer_l2` (company roles) or `platform_admin`. |
| `role_label` | string | yes | Display label, e.g. "Reviewer L2". |
| `is_active` | boolean | yes | Deactivated accounts are rejected on every request. |
| `is_platform_admin` | boolean | yes | True for platform admins, who belong to no company. |
| `company_id` | string (uuid) (nullable) | yes | The user's company; null for a platform admin. |
| `company_name` | string (nullable) | yes | Display name of the user's company. |

## DocumentCheckSummary

One `document_checks` row (System Specification sections 2.2/3.1/3.2 — field_validation, issuer_verification, and later the forensic checks). `result` holds whatever shape that check_type produces — see the individual check services (e.g. app/services/field_validation_service.py) for what's inside — the frontend's Checks panel renders it generically rather than assuming a fixed schema per check_type.

| Field | Type | Required | Description |
|---|---|---|---|
| `id` | string (uuid) | yes | |
| `check_type` | string | yes | |
| `status` | string | yes | |
| `result` | object (nullable) | yes | |
| `error_message` | string (nullable) | yes | |
| `created_at` | string (date-time) | yes | |
| `summary` | object (nullable) | no | |

## DocumentFileUrlResponse

A fresh short-lived signed URL for one document's original file.

| Field | Type | Required | Description |
|---|---|---|---|
| `file_url` | string | yes | |

## DocumentResponse

| Field | Type | Required | Description |
|---|---|---|---|
| `id` | string (uuid) | yes | |
| `case_id` | string (uuid) | yes | |
| `original_filename` | string | yes | |
| `content_type` | string (nullable) | yes | |
| `file_size_bytes` | integer (nullable) | yes | |
| `file_hash` | string | yes | |
| `file_url` | string | yes | |
| `uploaded_at` | string (date-time) | yes | |
| `processing_status` | string | yes | |

## EscalateRequest

| Field | Type | Required | Description |
|---|---|---|---|
| `reason` | string | yes | Why the case needs an L2 reviewer. Required and must not be blank. |

## ForensicFindingSummary

One medium/high-severity finding from a flagged metadata_forensics/ error_level_analysis/copy_move_detection check — the per-document counterpart to CrossDocumentFindingSummary, both shown together in the case detail "Explainable findings" panel. See app/services/ case_flag_service.py's get_forensic_findings for what feeds this and why it's a temporary stand-in for the real risk-scoring engine.

| Field | Type | Required | Description |
|---|---|---|---|
| `document_id` | string (uuid) | yes | |
| `document_filename` | string | yes | |
| `check_type` | string | yes | |
| `check_type_label` | string | yes | |
| `finding` | string | yes | |
| `severity` | string | yes | |
| `description` | string | yes | |

## HTTPValidationError

| Field | Type | Required | Description |
|---|---|---|---|
| `detail` | array of ValidationError | no | |

## IssuerCreate

| Field | Type | Required | Description |
|---|---|---|---|
| `name` | string | yes | English/Latin issuer name. |
| `name_arabic` | string (nullable) | no | Arabic-script name, matched separately during issuer verification. |
| `tax_id` | string (nullable) | no | Tax/VAT registration number, if known. |
| `type` | IssuerType | no | Kind of issuer (vendor, school, ...). |

## IssuerResponse

| Field | Type | Required | Description |
|---|---|---|---|
| `id` | string (uuid) | yes | |
| `name` | string | yes | |
| `name_arabic` | string (nullable) | yes | |
| `tax_id` | string (nullable) | yes | |
| `type` | IssuerType | yes | |
| `is_active` | boolean | yes | |
| `created_at` | string (date-time) | yes | |
| `updated_at` | string (date-time) | yes | |

## IssuerType

Allowed values: `"vendor"`, `"school"`, `"government"`, `"other"`

## IssuerUpdate

| Field | Type | Required | Description |
|---|---|---|---|
| `name` | string (nullable) | no | |
| `name_arabic` | string (nullable) | no | |
| `tax_id` | string (nullable) | no | |
| `type` | IssuerType (nullable) | no | |
| `is_active` | boolean (nullable) | no | |

## LoginRequest

| Field | Type | Required | Description |
|---|---|---|---|
| `email` | string (email) | yes | Account email (matched exactly as stored). |
| `password` | string | yes | Account password. |

### PipelineStatusSchema

Whether every automated check has finished. `pending` names what is still outstanding — shown to the reviewer when Approve is disabled.

| Field | Type | Required | Description |
|---|---|---|---|
| `complete` | boolean | yes | |
| `pending` | array of string | yes | |

### ReconcileResponse

| Field | Type | Required | Description |
|---|---|---|---|
| `rows_checked` | integer | yes | |
| `rows_corrected` | integer | yes | |
| `drift` | array of object | yes | |

### RejectRequest

| Field | Type | Required | Description |
|---|---|---|---|
| `reason` | string | yes | Why the case is rejected. Required and must not be blank. |

### RiskAssessmentSchema

| Field | Type | Required | Description |
|---|---|---|---|
| `id` | string (uuid) | yes | |
| `score` | integer | yes | |
| `raw_score` | number | yes | |
| `tier` | RiskTier | yes | |
| `computed_at` | string (date-time) | yes | |
| `triggered_reasons` | array of RiskReasonSchema | yes | |
| `thresholds` | map of integer | yes | |
| `group_caps` | map of map of number | no | |

## RiskReasonSchema

One fired rule, as frozen at scoring time (rendered text, weight and rule version) — see app/models/case_risk_assessment.py.

| Field | Type | Required | Description |
|---|---|---|---|
| `rule_id` | string | yes | |
| `rule_version` | integer | yes | |
| `category` | string | yes | |
| `check_type` | string | yes | |
| `severity` | string | yes | |
| `weight` | number | yes | |
| `reason` | string | yes | |
| `document_id` | string (uuid) (nullable) | yes | |
| `document_filename` | string (nullable) | yes | |
| `title` | string | no | |
| `short` | string | no | |

## RiskRuleCreate

A brand-new rule (version 1). The admin picks a rule TYPE and its parameters from the catalog (GET /settings/risk-rule-options); the API builds and validates the engine `condition` from them — nobody writes JSON, and a choice the engine can't evaluate is rejected.

| Field | Type | Required | Description |
|---|---|---|---|
| `rule_id` | string | yes | Stable key, lower-case with a dot: e.g. custom.editing_tool_seen |
| `category` | string: `"forensics"`, `"consistency"`, `"verification"`, `"duplication"` | yes | |
| `match` | string: `"finding"`, `"check_result"`, `"sub_check"`, `"cross_document"`, `"signature_match"` | yes | |
| `check_type` | string (nullable) | no | |
| `finding` | string (nullable) | no | |
| `severity_in` | array of string: `"low"`, `"medium"`, `"high"` (nullable) | no | |
| `sub_check` | string (nullable) | no | |
| `field_name` | string (nullable) | no | |
| `signature_result` | string (nullable) | no | |
| `weight` | number | yes | |
| `severity` | string: `"low"`, `"medium"`, `"high"` | yes | |
| `reason_template` | string | yes | |
| `is_active` | boolean | no | |
| `change_note` | string (nullable) | no | |

## RiskRuleResponse

One (current or historical) version of a rule.

| Field | Type | Required | Description |
|---|---|---|---|
| `id` | string (uuid) | yes | |
| `rule_id` | string | yes | |
| `category` | string | yes | |
| `check_type` | string | yes | |
| `weight` | number | yes | |
| `severity` | string | yes | |
| `reason_template` | string | yes | |
| `is_active` | boolean | yes | |
| `version` | integer | yes | |
| `effective_from` | string (date-time) | yes | |
| `updated_by_name` | string (nullable) | yes | |
| `change_note` | string (nullable) | yes | |

## RiskRuleUpdate

An edit. Never mutates the existing row: the API inserts version+1.

| Field | Type | Required | Description |
|---|---|---|---|
| `weight` | number (nullable) | no | Points added to the case score when the rule fires. |
| `severity` | string: `"low"`, `"medium"`, `"high"` (nullable) | no | Severity shown beside the rule's reason. |
| `is_active` | boolean (nullable) | no | Inactive rules are evaluated but never fire. |
| `change_note` | string (nullable) | no | Why the change was made. |

## RiskThresholds

| Field | Type | Required | Description |
|---|---|---|---|
| `medium_threshold` | integer | yes | Score at or above which a case is `medium` risk. |
| `high_threshold` | integer | yes | Score at or above which a case is `high` risk. Must exceed the medium threshold. |
| `metadata_score_cap` | integer (nullable) | no | Most points the metadata rules ( `metadata.*` ) may add together — one edit leaves several metadata traces. 100 = no cap. Omitted: unchanged. |

## RiskThresholdsResponse

| Field | Type | Required | Description |
|---|---|---|---|
| `medium_threshold` | integer | yes | Score at or above which a case is `medium` risk. |
| `high_threshold` | integer | yes | Score at or above which a case is `high` risk. Must exceed the medium threshold. |
| `metadata_score_cap` | integer | yes | |
| `updated_at` | string (date-time) | yes | |
| `updated_by_name` | string (nullable) | yes | |

## RiskTier

Allowed values: `"low"`, `"medium"`, `"high"`

## RuleTemplateResponse

| Field | Type | Required | Description |
|---|---|---|---|
| `id` | string (uuid) | yes | |
| `rule_id` | string | yes | |
| `category` | string | yes | |
| `check_type` | string | yes | |
| `condition` | object | yes | |
| `weight` | number | yes | |
| `severity` | string | yes | |
| `reason_template` | string | yes | |
| `is_active` | boolean | yes | |
| `updated_at` | string (date-time) | yes | |

## RuleTemplateUpdate

| Field | Type | Required | Description |
|---|---|---|---|
| `weight` | number (nullable) | no | |
| `severity` | string: `"low"`, `"medium"`, `"high"` (nullable) | no | |
| `reason_template` | string (nullable) | no | |
| `is_active` | boolean (nullable) | no | |

## SignatureMatchResponse

One signature_matches row returned by the API, enriched with the reference's person_name so the frontend can show "compared against [name]" without a separate reference lookup.

| Field | Type | Required | Description |
|---|---|---|---|
| `id` | string (uuid) | yes | |
| `document_id` | string (uuid) | yes | |
| `case_id` | string (uuid) | yes | |
| `signature_reference_id` | string (uuid) | yes | |
| `reference_person_name` | string | yes | |
| `comparison_scope` | string | yes | |
| `result` | string | yes | |
| `result_label` | string | yes | |
| `reasoning` | string (nullable) | yes | |
| `compared_at` | string (date-time) | yes | |

## SignatureReferenceCreate

Request body for POST /cases/{case_id}/documents/{document_id}/ signature-references.

`person_name` is required and must be provided by the reviewer — the system never attempts to auto-extract or pre-fill it (System Specification §2.3).

| Field | Type | Required | Description |
|---|---|---|---|
| `person_name` | string | yes | Name of the person whose signature this is, typed by the reviewer. Required — the system does not attempt to auto-extract this field. |
| `bounding_box` | BoundingBoxSchema | yes | |
| `is_library` | boolean | no | Defaults to True: every reference is saved to the permanent signature library. False is still accepted (case-only reference). Either way, at this phase a reference is only compared against other documents in its own case — no cross-case/library-wide comparison logic runs yet. |

## SignatureReferenceResponse

One signature_references row returned by the API.

| Field | Type | Required | Description |
|---|---|---|---|
| `id` | string (uuid) | yes | |
| `person_name` | string | yes | |
| `signature_image_url` | string (nullable) | yes | |
| `bounding_box` | object (nullable) | yes | |
| `source_document_id` | string (uuid) | yes | |
| `source_case_id` | string (uuid) | yes | |
| `created_by` | string (uuid) | yes | |
| `is_library` | boolean | yes | |
| `created_at` | string (date-time) | yes | |

## TokenResponse

| Field | Type | Required | Description |
|---|---|---|---|
| `access_token` | string | yes | JWT. Send as `Authorization: Bearer <token>`. |
| `token_type` | string | no | Always `bearer`. |

## UploadLimitsResponse

The calling company's upload limits, as enforced on its next upload.

| Field | Type | Required | Description |
|---|---|---|---|
| `max_file_size_mb` | integer | yes | Largest document, in MB (1 MB = 1,048,576 bytes). |
| `max_zip_size_mb` | integer | yes | Largest bulk-upload zip, in MB. |
| `max_file_size_bytes` | integer | yes | |
| `max_zip_size_bytes` | integer | yes | |

## UsageResponse

| Field | Type | Required | Description |
|---|---|---|---|
| `period` | string: `"this_month"`, `"last_month"`, `"last_30_days"`, `"this_year"`, `"all_time"`, `"custom"` | yes | |
| `start` | string (date) (nullable) | yes | First UTC day included (null = from the beginning). |
| `end` | string (date) (nullable) | yes | Last UTC day included (null = up to today). |
| `companies` | array of CompanyUsageRow | yes | |
| `totals` | UsageTotals | yes | |

## UsageTotals

| Field | Type | Required | Description |
|---|---|---|---|
| `cases_created` | integer | yes | |
| `documents_uploaded` | integer | yes | |
| `files_stored` | integer | yes | |
| `storage_bytes` | integer | yes | |
| `total_storage_bytes` | integer | yes | |
| `total_files_stored` | integer | yes | |

## UserAdminResponse

| Field | Type | Required | Description |
|---|---|---|---|
| `id` | string (uuid) | yes | |
| `email` | string (email) | yes | |
| `full_name` | string (nullable) | yes | |
| `role` | UserRole | yes | |
| `is_active` | boolean | yes | |
| `created_at` | string (date-time) | yes | |
| `company_id` | string (uuid) (nullable) | yes | Null for platform admins. |
| `company_name` | string (nullable) | yes | |

## UserAdminUpdate

| Field | Type | Required | Description |
|---|---|---|---|
| `role` | UserRole (nullable) | no | New role. Cannot be changed on your own account. |
| `company_id` | string (uuid) (nullable) | no | Move the user to another company (company roles only). |
| `is_active` | boolean (nullable) | no | False deactivates the account. Not allowed on yourself. |

## UserCreate

An account created by an administrator (there is no self-service sign-up). The password is set here and only ever stored hashed.

| Field | Type | Required | Description |
|---|---|---|---|
| `email` | string (email) | yes | Login email. Unique regardless of letter case. |
| `full_name` | string (nullable) | no | Display name (optional). |
| `role` | UserRole | no | `user`, `reviewer_l1`, `reviewer_l2` or `platform_admin`. |
| `company_id` | string (uuid) (nullable) | no | Required for company roles; must be empty for `platform_admin`. |
| `password` | string | yes | Initial password (8-128 chars); stored hashed. |

## UserPasswordReset

| Field | Type | Required | Description |
|---|---|---|---|
| `password` | string | yes | New password (8-128 chars); stored hashed. |

## UserRole

Allowed values: `"user"`, `"reviewer_l1"`, `"reviewer_l2"`, `"platform_admin"`

## UserSummary

| Field | Type | Required | Description |
|---|---|---|---|
| `id` | string (uuid) | yes | |
| `email` | string | yes | |
| `full_name` | string (nullable) | yes | |

## ValidationError

| Field | Type | Required | Description |
|---|---|---|---|
| `loc` | array of string \| integer | yes | |
| `msg` | string | yes | |
| `type` | string | yes | |

# Frontend screens

React 19 + TypeScript SPA ( `frontend/src` ), routed with React Router 7 ( `App.tsx` ). Server state is TanStack Query ( `retry: false` , `refetchOnWindowFocus: false` ); the auth token lives in `localStorage` ( `hooks/useAuth.tsx` ). In development the Vite server proxies `/api/*` to `127.0.0.1:8000` and strips the prefix ( `vite.config.ts` ), so `API_BASE_URL` defaults to `/api` (override with `VITE_API_BASE_URL` ). Every page except Login is loaded on demand ( `React.lazy` in `App.tsx` ), so the first download is the ~360 kB main bundle plus the page being opened.

Platform admins pick which company they are looking at with the **company picker** ( `CompanyPicker` , `hooks/useActingCompany.tsx` , remembered per browser) on the case queue, dashboard, audit history and the company settings pages; an amber note reminds them the access is read-only and recorded in the platform audit log.

`ProtectedRoute` redirects to `/login` if there is no token or the token does not resolve to a user. `RoleRoute` (in `components/AdminRoute.tsx` ) shows an "access required" panel to the wrong role. **Both are conveniences — the backend independently 401/403s every call**, so a user who reaches a URL still cannot read or change anything they are not entitled to.

## Route map

| Route | Screen | Access |
|---|---|---|
| `/login` | LoginPage | public |
| `/` , `/cases` , `/review-queue` | CaseQueuePage | any logged-in user (platform admin: per company, read-only) |
| `/cases/new` | NewCasePage | company users (hidden for platform admins) |
| `/cases/bulk` | BulkUploadPage | company users (platform admins see it read-only, without the upload form) |
| `/bulk-uploads/:bulkUploadId` | BulkUploadDetailPage | the uploader; reviewers of the company; platform admins (per company, read-only) |
| `/cases/:caseId` | CaseDetailPage | any (reduced view for `user` ; read-only decision panel for a Reviewer L1 on an L2 case; "Support view" for a platform admin) |
| `/dashboard` | DashboardPage | company users |
| `/my-cases` | MyCasesPage | company users |
| `/audit-history` | AuditHistoryPage | Reviewer L1 and above; platform admins |
| `/settings` → `/settings/issuer-registry` | SettingsIssuerRegistryPage | Reviewer L2; platform admins (per company) |
| `/settings/risk-rules` | SettingsRiskRulesPage | Reviewer L2; platform admins (per company) |
| `/settings/users` | SettingsUsersPage | platform admins only |
| `/platform` → `/platform/companies` | PlatformCompaniesPage | platform admins only |
| `/platform/rule-templates` | PlatformRuleTemplatesPage | platform admins only |

| Route | Screen | Access |
|---|---|---|
| `/platform/usage` | PlatformUsagePage | platform admins only |
| `/platform/queues` | PlatformQueuesPage | platform admins only |
| `*` | redirect to `/` | — |

The top `Nav` ( `design-system/Nav.tsx` ) shows each item only to who may use it ( `NAV_VISIBLE` ): *Dashboard* and *My cases* to company users, *Audit history* to Reviewer L1+ and platform admins, *Settings* to Reviewer L2 and platform admins, *Platform* to platform admins. It shows the company name (or "Platform") next to the user, and hides *New upload* from platform admins. Route guards ( `RoleRoute minRole=… platformAdmin="allow" | "only"` ) mirror this.

---

## LoginPage — `/login`

**Purpose:** email/password sign-in. **Endpoints:** `POST /auth/login` , then `GET /auth/me` (via `useAuth` ). **Notes:** stores the JWT in `localStorage` . Deliberately has no "trust this device", SSO button or compliance badges.

## CaseQueuePage — `/` , `/cases` , `/review-queue`

**Purpose:** the case list / reviewer queue. Shows counts (open, awaiting review, cleared, escalated), a table with case number, type, status, flag (risk tier), escalation badge, document count and submitter. Reviewer L1 gets tabs *My queue* ( `can_act` ) / *Escalated to L2 · view only*; Reviewer L2 gets *All cases* / *Escalated queue (L2)*; rows the user can't act on show a "View only" pill. **Endpoints:** `GET /cases` (once; refetched by other screens via the `["cases", token]` query key). **Behaviour:** search box + status / case-type / flag filters applied **in the browser**. The status column is a UI mapping of `CaseStatus` → `Analyzing` ( `submitted` , `under_automated_review` ), `Awaiting Review` ( `pending_manual_review` and any other open status), `Approved` ( `approved` , `auto_approved` , `closed` ), `Rejected` . **"Escalated" is a tier filter (L2), not a status.** For Reviewer L2 the API sorts open L2 cases first, then newest first. **Roles:** every company user, but a `user` only receives their own cases and no risk tier (the flag reads "In review"). A platform admin sees one company at a time, chosen with the company picker — never a mixed list.

## NewCasePage — `/cases/new`

**Purpose:** submit a case: choose a case type, attach PDFs, upload. **Endpoints:** `POST /cases` , `POST /cases/{id}/documents` (once per file, XHR with progress), and, for the optional signature step, `GET …/documents/{id}/file-url` and `POST …/documents/{id}/signature-references` (through `SignatureReferenceCreator` ). **Flow:** React Hook Form + Zod validate the case type. The drop zone accepts **PDF only, within the company's per-file limit, not password-protected** (it shows the limit, from `GET /auth/me/upload-limits` ); a file that is not a PDF, is empty or is over that limit is refused in the browser before anything is sent ( `lib/uploadLimits.ts` ). The case is created once (so a retry after a failed upload reuses it); files upload in parallel; each shows its own progress, or the server's specific rejection message (e.g. "This file is password-protected. Please remove the password and re-upload."). After upload the user can open **Set reference signature** for any uploaded document (drawing a box over a `react-pdf` page) — optional; without it no comparison runs. Then the case detail page is opened. **Roles:** any company user may create a case, upload to their own case and set reference signatures on it; reviewers may do so on any case of their company. A stranger's request is a 404; platform admins can't upload.

## BulkUploadPage — `/cases/bulk`

**Purpose:** submit many cases at once as one zip with one folder per case (pipeline 01a). **Endpoints:** `POST /bulk-uploads` (the zip as the raw body, XHR with progress), `GET /bulk-uploads` (recent uploads). **Flow:** pick the case type for every case in the zip, then drop one `.zip`. The browser refuses a non-zip, empty or over-limit zip (the company's own zip limit, shown on the page) before sending ( `lib/uploadLimits.ts` ). It also counts the case folders from the zip's central directory and warns above 100, without blocking. After upload it opens the summary page. Linked from NewCasePage ("Bulk upload (zip of cases)").

## BulkUploadDetailPage — `/bulk-uploads/:bulkUploadId`

**Purpose:** the live per-case summary of one bulk upload. **Endpoint:** `GET /bulk-uploads/{id}`, polled every 3 s until `settled`. **Shows:** zip-level status and any failure, the >100-case warning, the wrapper-folder and ignored-files notes, an overall progress bar with counts per live status, and a table of every case folder: case number (a link once created), documents processed, live status (Checking files / Queued / Processing / Done / Flagged / Not created) and risk flag. Each rejected file is listed under its case with the reason. Filters: All / Needs attention / In progress / Done. Folder and file names render with `dir="auto"` (Arabic).

## CaseDetailPage — `/cases/:caseId`

**Purpose:** everything about one case in three columns ( `lg:grid-cols-12` ): **documents** (3), the **selected document + its checks** (5), and **risk + decision** (4), plus the activity timeline below. **Endpoints:** `GET /cases/{id}` (polls every **5 s** while `pipeline.complete` is false or there is no assessment), `GET /cases/{id}/audit-log`, `GET /cases/{id}/signature-matches` (polled every 10 s), and, through child components, the decision endpoints and report endpoints. **Contents:**

- *Left:* the case flag card and the document list (type and processing status).
- *Middle:* `PdfOverlayViewer` showing the selected PDF with live overlays from that document's checks and cross-document findings; the document's **core** extracted fields (issuer — the only one with an "(uncertain)" marker — reference number, date, amounts; `additional_fields` are not listed here); `DocumentChecksPanel` (one expandable card per check — its one-line headline always visible, the short problem lines with **Why?** when expanded — plus a cross-document card and the signature comparison card).
- *Right:* score/tier and the explainable findings ( `SeverityFinding` per triggered reason, from `assessment.triggered_reasons` : its short `title` and `short` line, the stored reason behind **Why?**), `CaseDecisionPanel`, and `CaseReportExport` (reviewers only).
- *Bottom:* `ActivityTimeline` (the case's audit trail, newest first). **Roles:** everyone can open a case they may see (a foreign case is a 404 → "Case not found"). A `user` receives no assessment, no tier and only approve/reject actions; decision and report controls are reviewer only. A platform admin gets an amber **"Support view — read-only"** banner and no decision or report controls. The header shows an "Escalated · L2" badge on an L2 case; when the API returns `can_act: false` (a Reviewer L1 on an L2 case) the decision panel is a read-only card saying the case is with Reviewer L2, while the report export stays available.

## DashboardPage — `/dashboard`

**Purpose:** at-a-glance operational summary. **Endpoints:** `GET /cases` only. **Everything is computed in the browser** from that list: KPI cards (counts by flag: high/medium/low/pending, escalated open cases, cleared), a risk-distribution panel, the top-5 open cases the user can act on ( `can_act` ; escalated first, then highest risk score, then newest), and a "Case Summary" strip (total cases, documents analyzed, escalated, cleared). **Notes:** this is *not* an MIS/compliance dashboard (no fraud rate, resolution time, volumes over time, export). It contains no charts library.

## MyCasesPage — `/my-cases`

**Purpose:** the signed-in user's own cases, with status filters (Under review / Cleared / Action required) and search. **Endpoints:** `GET /cases`; client filters to cases the user submitted. **Notes:** a submitter sees decision outcomes here but not risk detail.

## AuditHistoryPage — `/audit-history`

**Purpose:** read the company's append-only `audit_log` (platform admins: a chosen company's log, or — with the *Platform-level log* checkbox — the platform events: companies, users, template edits, usage reconciliations, platform access). **Endpoints:** `GET /audit-log` (paged; `event_type`, `q` with a debounce, `limit` / `offset`) and `GET /audit-log/event-types` (for the filter). Event descriptions are rendered by `lib/audit.ts` (`describeAuditEvent`), which turns each `event_type` + `event_data` into a sentence. **Roles:** Reviewer L1 and above, and platform admins (`RoleRoute minRole="reviewer_l1" platformAdmin="allow"`; the API also 403s others). Company users never see platform-access rows.

## SettingsIssuerRegistryPage — `/settings/issuer-registry`

**Purpose:** manage the company's issuer registry (name, Arabic name, tax id, type, active). **Endpoints:** `GET/POST /settings/issuers`, `PATCH /settings/issuers/{id}` (deactivate/reactivate — rows are never deleted). **Roles:** Reviewer L2 (own company); platform admins pick a company. Uses `SettingsShell` (tab bar, company picker) and `Modal`.

## SettingsRiskRulesPage — `/settings/risk-rules`

**Purpose:** view and tune the company's scoring rules and thresholds. **Endpoints:** `GET /settings/risk-rules`, `PATCH /settings/risk-rules/{rule_id}` (weight / severity / active — inserts a new version), `GET …/{rule_id}/history` (per-rule change trail), `GET/PUT /settings/risk-thresholds`, and via `AddRuleModal` `GET /settings/risk-rule-options` + `POST /settings/risk-rules`. **Notes:** edits apply to the company's cases scored from then on; historical assessments keep the versions they used. **Roles:** Reviewer L2 (own company); platform admins pick a company.

## SettingsUsersPage — `/settings/users`

**Purpose:** manage every account on the platform (there is no self-service sign-up and no company-level user management). Lives under the *Platform* tabs (`PlatformShell`). **Endpoints:** `GET/POST /settings/users` (via `AddUserModal`), `PATCH /settings/users/{id}` (role, company, active). The table has a Company column and a company filter; *Add user* asks for the company and role. The role picker lists User / Reviewer L1 / Reviewer L2 / Platform Admin (`ROLE_OPTIONS` in `types/auth.ts`; API values `user` / `reviewer_l1` / `reviewer_l2` / `platform_admin`). Moving a user to another company invalidates their current token. A platform admin cannot change their own role or deactivate themselves. **Roles:** platform admins only.

## PlatformCompaniesPage — `/platform/companies`

**Purpose:** list, create, rename, suspend and reactivate companies, and set each company's upload limits. **Endpoints:** `GET/POST /platform/companies`, `PATCH /platform/companies/{id}`. **Notes:** creating a company seeds its risk rules from the platform template and default thresholds; its issuer registry starts empty. Suspending blocks sign-in and existing tokens for all its users. The table shows each company's limits ("10 MB / file · 300 MB / zip"). **Edit** opens a form with the name, **Max file size (MB)** and **Max zip size (MB)**, pre-filled; only changed values are sent, each change is platform-audited (`company_updated`, old → new), and it applies to the company's next upload without a new sign-in.

**Form validation (every form):** a problem is shown in red directly under the field ( `FieldError` + red border), never only as a hover tooltip. Submit buttons stay enabled, so clicking with a problem marks the field instead of silently doing nothing.

## PlatformRuleTemplatesPage — `/platform/rule-templates`

**Purpose:** edit the default rule set that every **new** company starts from. **Endpoints:** `GET/POST /platform/risk-rule-templates` , `PATCH /platform/risk-rule-templates/{rule_id}` (re-uses `AddRuleModal` ). **Notes:** template edits never change an existing company's rules.

## PlatformUsagePage — `/platform/usage`

**Purpose:** billing & usage, one row per company: cases created, documents uploaded, files stored, storage added in the period, total storage/files. **Endpoints:** `GET /platform/usage` (this month / last month / last 30 days / this year / all time / custom UTC range), `POST /platform/usage/reconcile` (*Reconcile now*).

## PlatformQueuesPage — `/platform/queues`

**Purpose:** live processing-queue monitor (refreshes every 5 s): waiting / running per queue, oldest waiting task, average / p95 / max wait, run time, consumers online, rate-limiter counters (throttled calls, seconds waited, Azure 429s) and outstanding work per company (the fair-share counters). **Endpoints:** `GET /platform/queues` . See processing-queues.md.

# Frontend components

Shared building blocks, grouped by folder. All are function components styled with Tailwind; props are listed as they exist in the code. The `token` prop appears on several components because they call the API themselves (through `@/api/*`) rather than receiving data from a page.

## The PDF overlay engine

**`PdfOverlayViewer`** — **`components/case/PdfOverlayViewer.tsx`**

Renders **one page at a time** of a case document with bounding boxes drawn **live on top**. Nothing annotated is ever generated or stored; the boxes vanish the moment `overlays` is empty. Wrapped in an error boundary (react-pdf can throw synchronously on an expired or blocked URL, which would otherwise unmount the whole case page). Page width is measured from the card and capped at 560 px so it never spills into the adjacent column. Uses `react-pdf` (pdf.js) with a Vite-bundled worker.

| Prop | Type | Meaning |
|---|---|---|
| `fileUrl` | `string` | A signed SAS URL for the PDF (from `CaseDocumentSummary.file_url`) |
| `overlays` | `OverlayBox[]` | Boxes to draw |

```
interface OverlayBox { box: BoundingBox; color: OverlayColor; label: string }
type OverlayColor = "destructive" | "warning" | "ai" | "field"
// BoundingBox = { page (1-based), x, y, width, height }  — 0-1 page fractions, origin top-left
```

The colours are **fixed per source**, so a reviewer learns the legend once:

| Colour | Style | Source |
|---|---|---|
| `destructive` | solid red | ELA recompression region |
| `warning` | solid orange | Copy-move cluster |
| `ai` | **dashed** blue, "approximate" | Vision-model judgment (visual review, signature comparison) — styled to look less precise |
| `field` | solid purple | Rule-based field exception (failed field validation, cross-document mismatch) |
| `font` | solid fuchsia | Text set in an out-of-place font, or in a second embedded copy of a font (font consistency) |
| `ghost` | solid teal | Deleted / shortened content: the faint trace of erased text in a converted scan (ghost content) — only `ghost_deleted_block` / `ghost_replaced_line`, not its info notes |
| `spelling` | solid sky blue | A misspelled English word (spelling check, `spelling_error`) |

Because boxes are page fractions, they are resolution-independent and land correctly regardless of the rendered size.
**Used in:** `CaseDetailPage` (middle column).

`getCheckOverlays(checks, crossDocumentFindings?, documentId?) → OverlayBox[]` — `DocumentChecksPanel.tsx`

The **single function that turns stored check rows into boxes**, shared by the viewer and the findings list so the highlighted regions and the listed findings can never disagree (both read the same `document_checks` rows). It reads ELA/copy-move/font/ghost-content/spelling findings' `bounding_box`, flagged field-validation sub-checks' `regions`, visual-review boxes, and cross-document regions (only those belonging to `documentId`, so each document shows its own side of a mismatch).

## Case components — `components/case/`

| Component | Props | Purpose / where used |
|---|---|---|
| `DocumentChecksPanel` | `checks: DocumentCheck[]`, `crossDocumentFindings: CrossDocumentFinding[]`, `hasEnoughDocumentsForCrossCheck: boolean`, `signatureMatches?: SignatureMatch[]` | One expandable card per check with its findings, a synthetic cross-document card, and the signature-comparison card (badge colours: consistent green, possibly-consistent/identical-reuse amber, inconsistent/different-signer red, cannot-determine grey). Each card shows the check's **headline** under its title ( `check.summary.headline` , from the API — 12); expanded, one line per problem ( `SummaryItem` : title — values, a **Why?** toggle for the full explanation, the ghost-text trace image and the vision guess), short notes, and the raw findings under **Technical details** (which handle the differing `details` shapes: list of findings vs. dict of sub-checks vs. flat issuer dict). The stamp sub-checks of field validation (stamp names the issuer, stamp typed into the file, no signature) are shown with signature / stamp detection, which turns Flagged when one flags. Duplicate findings link to the matched case. *CaseDetailPage.* |
| `CaseDecisionPanel` | `caseDetail: CaseDetail`, `role: UserRole \| undefined`, `token: string` | Approve / reject / escalate with confirmation dialogs. Approve is disabled until `pipeline.complete` and an assessment exists; above low risk it requires a ≥ 10-character justification. **These are conveniences — every rule is enforced by the API.** Invalidates the `case` , `caseAuditLog` and `cases` queries on success. Escalate requires a reason and moves the case to L2. Non-reviewers get a read-only "Review status" card with the decision history instead; a Reviewer L1 on an L2 case ( `caseDetail.can_act` false) gets a read-only "Escalated · L2" card. The decision history shows each actor's role label ( `actor_role` ). *CaseDetailPage.* |
| `CaseReportExport` | `caseId: string`, `token: string` | "Export report" button ( `POST /cases/{id}/reports` ; opens the new PDF in a tab, with an "Open PDF" link as a popup-blocker fallback) and a collapsible history of previous reports. Reviewers only (hidden in a platform admin's support view). *CaseDetailPage.* |
| `ActivityTimeline` | `entries: AuditLogEntry[]`, `isLoading: boolean` | The case's audit trail, **newest first** (the API returns oldest first). Sentences come from `lib/audit.ts` `describeAuditEvent` . *CaseDetailPage.* |
| `CaseBadges` | exports `CaseFlagBadge({flag})` , `TierBadge({tier})` , `RiskBadge` , `riskStripeClass(tier)` | The coloured risk-tier pill and "Escalated · L2" marker. *Queue, Dashboard, MyCases, Detail.* |

## Upload components — `components/upload/`

| Component | Props | Purpose |
|---|---|---|
| `FileDropzone` | `files: FileWithProgress[]`, `onFilesAdded(files: File[])`, `onFileRemoved(index: number)`, `disabled?` | Drag-and-drop / click-to-browse list with per-file progress and error. `FileWithProgress = { file: File; progress?: number (0-100); error?: string }`. The picker only offers PDFs (`accept` from `lib/uploadLimits.ts`) and the zone states the rules ("PDF only · up to 10.0 MB each · not password-protected"); the page refuses non-PDF, empty and over-limit files before upload, and shows the server's specific rejection message per file. *NewCasePage.* |
| `SignatureReferenceCreator` | `caseId`, `documentId`, `documentFilename`, `token`, `onCreated(ref: SignatureReference)`, `onClose()` | Modal that fetches a signed URL, renders the PDF with `react-pdf`, lets the reviewer **drag a box over a signature** (pointer coordinates and the box are both relative to the rendered page wrapper, so they always agree), takes a required `person_name`, then `POST`s the reference. Error-boundary wrapped. Available to whoever uploaded (the case owner) and to the company's reviewers. *NewCasePage.* |

## Settings components — `components/settings/`

| Component | Props | Purpose |
|---|---|---|
| `SettingsShell` | `active: SettingsTab` (`"issuers" \| "risk-rules" \| "users"`), `title?`, `description`, `actions?`, `children`; also exports `StatusLine({error, ok})` | Shared page frame and tab bar for the company settings screens (Issuer Registry, Risk Rules); shows the company picker to platform admins ("Changes apply to this company only and are recorded in its audit log.") |
| `Modal` | `title`, `description?`, `children`, `onClose`, `busy?`, `wide?` | Generic dialog used by the settings pages |
| `AddRuleModal` | `onClose()`, `onCreated(rule: RiskRule)` (token comes from `useAuth`) | Builds a risk rule from `GET /settings/risk-rule-options` (no hand-written JSON), then `POST /settings/risk-rules` — or, on the Rule templates page, `POST /platform/risk-rule-templates` |
| `AddUserModal` | `onClose()`, `onCreated(user: AdminUser)` | `POST /settings/users` with company + role (or a platform admin); can generate a random 14-character initial password |

## Platform and tenancy components

| Component | Props | Purpose |
|---|---|---|
| `PlatformShell` (`components/platform/`) | `active` tab, `title`, `description`, `actions?`, `children` | Page frame and tab bar of the platform-admin area: Companies, Users, Rule templates, Billing & usage, Processing queues |
| `CompanyPicker` (`components/`) | `note?` | Company selector for platform admins on company-scoped pages, with an amber read-only/audited note. Default note: "Read-only support access. Every view is recorded in the platform audit log (the company does not see it)." |

| Component | Props | Purpose |
|---|---|---|
| `useActingCompany()` (`hooks/useActingCompany.tsx`) | — | `{isPlatformAdmin, companyId, platformCompanyParam, …}`: which company a platform admin is acting on (remembered per browser); company users always get their own. Pages pass `platformCompanyParam` as `company_id` to the API |

## Design system — `design-system/`

Small presentational pieces reused across pages: `Nav` (top navigation; props `active: NavItemId`, `onNewUploadClick?`; items filtered by `NAV_VISIBLE` — rank via `hasRank` for company users, explicit `is_platform_admin` checks for the Platform area; shows the company name next to the user), `PageHeader`, `StatCard` (KPI tile), `InfoChip`, `SeverityFinding` (`title`, `description`, `detail?` (behind a **Why?** toggle), `pointDelta`, `tone` — one triggered risk reason with its weight), `AuditRow`. The static HTML mockups these were built against are in `stitch_docauth_document_review_platform/`.

## UI primitives — `components/ui/`

`badge`, `button`, `card`, `input`, `label`, `progress`, `select` — shadcn/ui-style wrappers over Radix + Tailwind (class-variance-authority, `cn()` from `lib/utils`).

## Routing and auth

- `ProtectedRoute` — token + resolved user required, else redirect to `/login`.
- `RoleRoute({minRole, platformAdmin, label})` (file `AdminRoute.tsx`) — renders `<Outlet/>` when the user's company role ranks at least `minRole` (`hasRank`), or when the user is a platform admin and `platformAdmin` is `"allow"` / `"only"` (`"only"` = platform admins exclusively); else an "access required" panel.
- `types/auth.ts` — `UserRole` (`user`, `reviewer_l1`, `reviewer_l2`, `platform_admin`), the rank table mirrored from the backend, `hasRank(role, minimum)` (always false for `platform_admin`, which is not on the ladder), `ROLE_LABELS` ("Reviewer L1" … "Platform Admin") and `ROLE_OPTIONS` (role pickers). Gate UI on `hasRank` / `is_platform_admin`, never on lists of role names.
- `hooks/useAuth.tsx` — `AuthProvider` / `useAuth()` exposing `token`, `user`, `isLoadingUser`, `login`, `logout`; token in `localStorage`; a token the backend rejects is cleared.

## Data layer

- `api/client.ts` — `apiFetch<T>(path, {token, …})` (JSON, Bearer header, `ApiError(status, message, code?)`), `parseErrorDetail(body)` (reads `detail` whether it is a string or a `{code, message}` object) and `API_BASE_URL` (`VITE_API_BASE_URL` or `/api`). Uploads use `XMLHttpRequest` instead, purely for progress events.
- `api/auth.ts`, `api/cases.ts`, `api/settings.ts`, `api/platform.ts` — one function per endpoint.
- `lib/uploadLimits.ts` — `ACCEPTED_UPLOAD_TYPES` (PDF), `clientUploadProblem(file, maxBytes)`, `clientZipProblem(file, maxBytes)`, `estimateZipCaseCount(file)`. No size is hardcoded: the limits are per company and come from `hooks/useUploadLimits.ts` (`GET /auth/me/upload-limits`, re-read whenever an upload screen opens).
- `components/ui/field-error.tsx` — `FieldError` (a red message directly under a field, always visible, never a hover tooltip) and `invalidFieldClass` (red border). Forms keep their submit button enabled; a click with a problem marks the field(s) instead of doing nothing, and forms set `noValidate` so the browser's own hover bubbles never replace these messages.

- Every page except Login is a lazy route chunk ( `App.tsx` ).
- `types/` — hand-written TypeScript mirrors of the backend schemas ( `types/case.ts` includes `SignatureMatchResult` , `getDocumentRole` — **kept in sync with the backend by hand**, so a new backend enum value needs a matching edit here, as the signature verdicts did).
- Forms use React Hook Form + Zod; the Zod schemas are hand-mirrored from the Pydantic models.

# Per-case forensic report (PDF)

A reviewer (L1 or L2) of the case's company can export a standalone, printable PDF describing everything the system found about one case — what was checked, what was flagged, why the score is what it is, who did what, and how far to trust it. It is designed to be read by someone who has **not** used the application.

**Trigger:** `POST /cases/{case_id}/reports` (button *Export report* on the case detail page, `CaseReportExport`). **History:** `GET /cases/{case_id}/reports`. Reviewers only; a platform admin can list a company's reports (read-only, audited) but cannot generate one. **Code:** `services/case_report_service.py` (orchestration + storage), `services/case_report_data.py` (gathering — `collect_report_data → ReportData`), `services/case_report_pdf.py` (rendering — `build_report_pdf(ReportData) → bytes`, pure, no DB or network).

## How it is generated

Generation is **synchronous** (a few seconds for a typical case; there is no Celery job because the codebase has no async-export pattern; move `generate_case_report` into a task if reports get heavy).

1. `collect_report_data` **reads** what the pipeline already stored: `document_checks`, `case_risk_assessments`, `cross_document_findings`, `case_actions`, `signature_matches` and the append-only `audit_log`. It **never re-runs a check, re-scores a case, regenerates a reason (the reason text is the one frozen at scoring time) or writes its own audit trail.**
2. It downloads each **original** from Blob Storage purely to (a) **re-hash** it, proving the chain of custody still matches the recorded SHA-256, and (b) render the pages that carry findings. The original is only ever read.
3. `build_report_pdf` builds the PDF in memory with PyMuPDF (HTML "story" pages + annotated page renders).
4. The PDF is uploaded to `companies/{company_id}/cases/{case_id}/reports/{report_id}.pdf`, and a `case_reports` row is inserted with `report_sha256`, `page_count`, `file_size_bytes` and the `risk_assessment_id` it reflects; the company's `files_stored` / `storage_bytes` usage counters go up in the same transaction.
5. Audit event `case_report_generated {report_id, page_count, risk_assessment_id, report_sha256}`.

Every generation adds a **new** row; earlier reports stay downloadable. A report is a point-in-time record: it reflects the evidence, assessment and rule versions as they were then, so two reports for the same case can legitimately differ. Storage failure is a 502 and nothing is recorded.

### Live overlays vs. the report's annotated pages

These are different mechanisms, on purpose. The live UI draws boxes **on top of** a rendered page in the browser and stores nothing. The report **burns** findings into *its own* rendered page images. That does not violate the immutable-original rule: the original blob is only read, and the annotated renders exist only inside the new report. Both use the same colour convention (red = ELA, orange = copy-move, dashed blue = model judgment, purple = rule-based field exception, fuchsia = font mismatch, teal = ghost text — deleted or shortened content).

### Short line first

Every finding is written twice (12): a **short line** with the values ("Amounts retyped — 1,450.00 · 16,450.00 … in a second copy of Times New Roman") and the check's full explanation. Sections 3, 4 and 5 show the short line first, in bold, with the full explanation under it in grey. Findings with one cause are grouped (17 font findings → 3 exception rows, each linking all its highlights).

## The nine sections

| # | Section | What it contains | Data source ( `ReportData` field → tables) |
|---|---|---|---|
| 1 | **Case Details** | Case number, type, status, submitter, submission time, document list and types, who generated the report and when (with their role label), the recorded decision / escalation, each "by Name (Reviewer L1/L2)" from `case_actions.actor_role` | `case_*` scalars, `documents`, `decision: DecisionInfo`, `escalation: EscalationInfo` ← `cases`, `case_actions`, `users` |
| 2 | **Executive Summary** | Short: tier and score, the decision, how many documents/exceptions, pointers to Sections 3, 4, 5 and 9. Notes when the case had not finished analysis ( `pipeline_pending` ) | `risk: RiskSummary`, `pipeline_pending`, `exceptions` ← `case_risk_assessments`, `pipeline_status` |
| 3 | **Explainable Findings** | Each triggered risk reason: severity, weight, check, document, page, its short title and line ("Fonts: text retyped — …"), the frozen reason text under it, and region IDs (R1, R2 …) | `findings: list[FindingRow]` ← `case_risk_assessments.triggered_reasons` |
| 4 | **All Checks** | The complete checklist: every check on every document with its result (Pass / Flag / Review — shown but not scored / Limited / N/A …), its headline and one bullet per problem or note, the cross-document table, and the signature comparison | `documents[].check_rows: CheckRow`, `cross_document: list[CrossRow]` (None if < 2 documents) ← `document_checks`, `cross_document_findings`, `signature_matches` |
| 5 | **Exceptions** | Everything that did **not** pass, written as concrete short items ("Total ≠ subtotal + tax — total 197,500 ≠ subtotal 180,000 + tax 9,000") with the full explanation under each, each linked to the highlighted region(s) that show it | `exceptions: list[ExceptionItem]` derived from Section 4's data (failed sub-checks, flagged findings, cross-document mismatches, missing required fields, signature results other than *consistent*) |
| 6 | **Limitations and Assumptions** | What the report is and is not (system-assisted analysis to support human review, not a certified forensic opinion); which checks are deterministic vs. model judgment; **which checks actually ran on this case (coverage)** — Ran / Did not run / Not applicable, with the reason a check stored ("Not applicable: no text layer (a plain scan)", "born-digital, no images"); the rule versions and thresholds behind the score; assumptions specific to this case | `coverage: list[CoverageRow]`, `assumptions`, `risk.rule_versions`, `risk.thresholds` |
| 7 | **Audit Trail** | Chronological events for the case, **oldest first, capped at the first 150**; any later events are counted ("N later event(s) omitted"). Human actors are shown as "Name (Reviewer L1)" etc., using the role recorded in the event at the time (falling back to the user's current role for older events). Platform-admin access rows are never included (they are platform-only) | `audit: list[AuditRow]`, `audit_truncated` ← `audit_log` |
| 8 | **Appendix** | **8.1** extracted fields per document with confidence and whether the field was located; **8.2** SHA-256 of each original with a live **verification** status ( `match` / `mismatch` / `unverified` — the original could not be fetched); **8.3** technical parameters of each check | `documents[].fields`, `documents[].hash: HashVerification`, `technical: list[TechnicalBlock]` |
| 9 | **PDF Highlighted Regions** | One consolidated page per flagged document page: the original page render with every highlight burned in, a legend, and the list of findings on it; when ghost-text highlights carry an image of the erased trace (contrast-stretched), an extra page of those images follows | `documents[].annotations: list[PageAnnotation]` + a re-render of the original page |

### Region IDs

Every highlight gets a report-wide ID **R1, R2, …** (assigned in document order, then page order by `assign_region_ids`). Sections 3 and 5 cite them, and each citation is an internal **PDF link** to the Section 9 page that shows it. Annotation kinds map to the colours above (`AnnotationKind`).

### Wording rules

The report follows the same restraint as the rest of the product: model-judgment items are labelled "approximate", the AI-generation and signature results are described as advisory/experimental, and the limitations section states plainly that **the absence of a flagged finding does not guarantee a document is genuine**. It never uses "verified" for a signature.

## Output record

`case_reports` row / `CaseReportResponse` (`schemas/case_report.py`):

```
{"id": "uuid", "case_id": "uuid", "generated_by_name": "Seed Admin",
 "generated_at": "2026-09-20T18:02:11Z", "file_size_bytes": 412331, "page_count": 14,
 "report_sha256": "…64 hex…", "risk_tier": "high", "risk_score": 75,
 "download_url": "https://…?sv=…&sig=…"}      // short-lived SAS URL, minted on each list/create call
```

## Known limitations

- Only the **first 150 audit events** (oldest first) appear in Section 7; on a long-lived case the *later* events — often the reviewer's decision and the report itself — are omitted from the excerpt and only counted. The full history is in the audit log (`GET /cases/{id}/audit-log`).
- A report of a case whose original is missing or unreadable degrades gracefully: that document's Section 9 page shows a banner explaining the problem, and 8.2 reports `unverified` instead of failing the export.
- Annotated pages exist only for pages that carry findings and whose original could be rendered — PDFs only.
- Generation cost grows with the number of documents and flagged pages, since it is synchronous.
- The report reflects the **latest assessment at generation time**; a case that is re-scored afterwards (for example a signature comparison completing) is not reflected until a new report is generated.

# Local setup

Locally FDDT runs as: **Postgres**, **Redis** and **PgBouncer** (in Docker), the **FastAPI** app, the **Celery workers** (one per queue, or one that serves all three), **Celery beat**, and the **Vite dev server**. Examples are Windows/PowerShell first (the project is developed on Windows); bash equivalents are in comments. For production, see production-deployment.md.

## Prerequisites

| Tool | Version | Notes |
|---|---|---|
| Docker Desktop | any recent | Postgres 16, Redis 7, PgBouncer 1.23 |
| Python | 3.11+ (developed and tested on **3.14**; the Docker image uses 3.12) | |
| Node.js | 20+ (developed on **24**) | |
| Azure resources | — | Storage account, Document Intelligence, Azure OpenAI (vision-capable deployment). Needed for the *pipeline*, not to start the app — see environment-variables.md |

## 1. Start Postgres, Redis and PgBouncer

From the repo root:

```
docker compose up -d        # postgres, redis, pgbouncer
docker compose ps           # postgres and redis "healthy", pgbouncer "running"
```

This creates `docauth-postgres` (port 5432), `docauth-redis` (6379) and `docauth-pgbouncer` (6432) with named volumes ( `postgres_data` , `redis_data` ). The owner user/password/database are all `docauth` . Data persists across restarts; `docker compose down -v` deletes it. PgBouncer is optional locally (see step 2).

## 2. Backend

```
cd backend
python -m venv .venv
.\.venv\Scripts\Activate.ps1          # bash: source .venv/bin/activate
pip install -r requirements.txt       # several minutes (OpenCV, PyMuPDF, pikepdf, Azure SDKs)
copy .env.example .env                # bash: cp .env.example .env
```

Edit `backend/.env` : set `JWT_SECRET_KEY` and the Azure values (storage, Document Intelligence, OpenAI, and the Azure OpenAI rate caps at ~80 % of your deployment's quota). The Postgres/Redis defaults already match `docker-compose.yml` . To route the app through PgBouncer as production does, also set `DATABASE_POOLER_HOST=localhost` (port 6432); leave it unset to connect directly. **Always run backend commands from** `backend/` — the `.env` path is relative to the working directory.

Create the schema, the database roles and the first account:

```
alembic upgrade head        # 14 migrations → 19 tables, RLS policies, the app roles fddt_app /
                            # fddt_platform (passwords from DATABASE_APP_PASSWORD /
DATABASE_PLATFORM_PASSWORD),
                            # the "Default Company" with 38 rules, thresholds 30/60 and a few FAKE test
                            # issuers, and the platform rule template
python seed.py              # creates the PLATFORM ADMIN admin@example.com / ChangeMe123! (idempotent;
                            # re-running resets that password — set SEED_ADMIN_EMAIL / SEED_ADMIN_PASSWORD)
```

The migrations run as the **owner** ( `DATABASE_URL` ), which needs CREATEROLE but not superuser. The app itself connects as `fddt_app` / `fddt_platform` , never as the owner.

The seeded account is a **platform admin**: it belongs to no company and cannot submit or review cases. After logging in, under **Platform**: create a company (or use "Default Company"), then under **Platform › Users** create that company's `user` , `reviewer_l1` and `reviewer_l2` accounts. Sign in as those to exercise the case workflow.

## 3. Run the API

```
uvicorn app.main:app --reload        # http://localhost:8000
```

Check: `http://localhost:8000/health` → `{"status":"ok","environment":"local"}` ; interactive docs at `http://localhost:8000/docs` . Smoke-test login:

```
curl.exe -X POST http://localhost:8000/auth/login -H "Content-Type: application/json" `
     -d '{\"email\":\"admin@example.com\",\"password\":\"ChangeMe123!\"}'
# (use curl.exe, not `curl`: in Windows PowerShell 5.1 `curl` is an alias for Invoke-WebRequest)
# → {"access_token":"eyJ…","token_type":"bearer"}
```

## 4. Run the Celery workers and beat

The pipeline uses three queues (processing-queues.md). Locally the simplest setup is **one worker that serves all three** (second terminal):

```
cd backend
.\.venv\Scripts\Activate.ps1
celery -A app.tasks.celery_app worker --loglevel=info --pool=threads --concurrency=8 `
       -Q extraction_queue,vision_queue,forensics_queue,celery
```

**Windows cannot use the prefork pool** — use `--pool=threads` (or `--pool=solo` , one task at a time). On Linux/macOS run one worker per queue exactly as production does:

```
sh start-worker.sh extraction     # threads, EXTRACTION_WORKER_CONCURRENCY (4)
sh start-worker.sh vision         # threads, VISION_WORKER_CONCURRENCY (4)
sh start-worker.sh forensics      # prefork, one process per CPU (FORENSICS_WORKER_CONCURRENCY=0)
```

or all of them in Linux containers: `docker compose --profile workers up -d` (add `--scale forensics-worker=N` ). The worker lists its tasks ( `process_document` , `run_document_checks` , `run_cross_document_checks` , `run_metadata_forensics` , `run_tampering_checks` , `run_duplicate_check` , `run_visual_inconsistency_review` , `run_signature_detection` , `run_signature_comparison` , `score_case` , `reconcile_usage_stats` , `log_queue_metrics` , `requeue_stuck_documents` ) and then `ready` . **Without a worker, uploads succeed but every document stays `pending` forever** — the API only enqueues.

Optionally run **beat** and the worker for its jobs (per-minute queue metrics line, stuck-document recovery every 5 minutes, nightly usage reconciliation). These jobs run on their own `housekeeping_queue`, so they need this worker; run exactly one of each:

```
celery -A app.tasks.celery_app beat --loglevel=info
celery -A app.tasks.celery_app worker -Q housekeeping_queue -n housekeeping@%h --pool=solo --loglevel=info
# Linux/macOS/containers: both in one process with `sh start-beat.sh` (`--beat` does not work on Windows)
```

> **Restart the workers after every backend code change.** Unlike `uvicorn --reload`, Celery does not auto-reload; it keeps running the old code.

## 5. Run the frontend

```
cd frontend
npm install                # or `npm ci` for an exact lockfile install
npm run dev                # http://localhost:5173
```

No env file is needed: the dev server proxies `/api/*` → `http://127.0.0.1:8000/*` (prefix stripped), so no CORS setup is required. Sign in with `admin@example.com` / `ChangeMe123!` (platform admin) or a company account you created.

Other scripts: `npm run build` (type-check + production build into `dist/`, route-level code splitting), `npm run lint` (oxlint), `npm run preview`.

## 6. Try the pipeline

1. Sign in as a company `user` or reviewer → **New case** → pick a case type → drop PDFs from `sample-documents/` (e.g. the `Sample9`, `Sample11` Arabic pairs, or the PDFs inside `tampered_test_samples/tampered_test_samples.zip`; the `_Invoice` / `_Evidence` pairs share a case). Only PDFs within the company's limit (10 MB by default) are accepted; images, password-protected and corrupted PDFs are refused with a specific message.
2. Open the case; the page polls every 5 s until *Analyzing* becomes a risk tier and *Approve* unlocks.
3. Optionally, on the upload screen, **Set reference signature** to trigger signature comparison.
4. As the platform admin, watch **Platform › Processing queues** and **Billing & usage**.

Re-uploading a sample you already uploaded *in the same company* **will** flag duplicates (10). The `stitch_docauth_document_review_platform/` folder holds static HTML design mockups, not part of the app.

## Tests

```
cd backend
pytest                     # ~480 tests, ~4 minutes; external services are mocked (SQLite)
```

The database-level tests need a real, migrated Postgres and are skipped otherwise:

```
$env:FDDT_RLS_TEST_DATABASE_URL = "postgresql+psycopg2://docauth:docauth@localhost:5432/docauth"
pytest tests/test_rls_postgres.py       # Row-Level Security, role privileges, upserts as the app role
$env:FDDT_PGBOUNCER_HOST = "localhost"
pytest tests/test_pgbouncer_rls.py      # tenant context never leaks through transaction pooling
```

The rate-limiter tests need Redis on `localhost:6379`. `backend/loadtest/` holds the load-test harness (`docker-compose.loadtest.yml`).

## Housekeeping and troubleshooting

| Symptom | Cause / fix |
|---|---|
| Documents stuck at `pending` | No worker is consuming that queue, or Redis is unreachable. Check **Platform › Processing queues** |
| Upload rejected with a message | Working as designed: PDF only, within the company's size limit, not empty, not corrupted, not password-protected (01) |
| Upload returns 503 | `AZURE_STORAGE_CONNECTION_STRING` missing/invalid |
| Document shows `failed` with an Azure error | Check the document's `processing_error`, and the Document Intelligence / OpenAI keys, endpoint and deployment name |
| Visual review / signature checks fail, extraction works | The OpenAI deployment is not vision-capable (must contain `gpt-4o`, `gpt-4.1`, `gpt-4-turbo` or `gpt-4-vision`) |
| Azure 429s / slow vision queue | Raise the deployment's TPM, or lower `AZURE_OPENAI_MAX_*_PER_MINUTE` to ~80 % of the real quota |
| Case stuck at *Analyzing* | A pipeline task has not finished; `GET /cases/{id}` → `pipeline.pending` lists what it is waiting for |
| "permission denied for table …" in a worker log | A write needs a privilege the app role lacks — every tenant table's grants are explicit (see schema) |
| Login works but a company user sees nothing | They belong to another company than the data, or the company is suspended |
| Port 5432/6379/6432 in use | Another Postgres/Redis/PgBouncer; stop it or set `POSTGRES_PORT` / `REDIS_PORT` / `PGBOUNCER_PORT` (and the URLs) |
| `alembic` cannot connect | Run it from `backend/`, with the containers healthy and `DATABASE_URL` matching |
| Backend code change has no effect on processing | Restart the Celery workers |
| Wrong/empty `.env` values | Confirm you started the process from `backend/` |

Useful commands:

```
docker exec docauth-postgres psql -U docauth -d docauth -c "select name, is_active from companies"
python backfill_field_locations.py [CASE-XXXXXXXX]   # re-locate extracted fields (re-runs OCR only, no LLM)
python -m scripts.calibrate_llm_tokens               # re-measure token usage after a model/prompt change
cd ..\docs\tools; ..\..\backend\.venv\Scripts\python gen_db_docs.py   # regenerate the schema docs
```

# Environment variables

The backend reads settings through `backend/app/core/config.py` (Pydantic `BaseSettings`). Values come from the process environment, then from `backend/.env`. The env file path is **relative to the working directory**, so start `uvicorn`, `celery`, `alembic` and `pytest` from `backend/`. Names are case-insensitive. Copy `backend/.env.example` to `backend/.env`; never commit `.env`.

**Required** below means "the pipeline does not work without it", not "the process refuses to start" — most services are built lazily, so a missing value surfaces when first used.

## Backend — required for a working local pipeline

| Variable | Default | Purpose | Where to get it | If missing |
|---|---|---|---|---|
| `DATABASE_URL` | `postgresql+psycopg2://docauth:docauth@localhost:5432/docauth` | The **owner** connection — used only by Alembic and `seed.py` (needs CREATEROLE, not superuser). Its host/port/database are also used for the two app roles below | Matches the Postgres in `docker-compose.yml` (default works as-is) | Migrations fail; API/worker can't build their URLs |
| `DATABASE_APP_USER` / `DATABASE_APP_PASSWORD` | `fddt_app` / `fddt_app_local` | The role every company request and pipeline task connects as (NOBYPASSRLS — Row-Level Security applies). The migration creates it with this password | Choose a long random password **before** running `alembic upgrade head` | Every request fails to connect |
| `DATABASE_PLATFORM_USER` / `DATABASE_PLATFORM_PASSWORD` | `fddt_platform` / `fddt_platform_local` | The role for sign-in, platform-admin screens and usage reconciliation (all rows through the `platform_all` policy; no BYPASSRLS) | Same | Login and platform screens fail |
| `CELERY_BROKER_URL` | `redis://localhost:6379/0` | Celery broker | The Redis in `docker-compose.yml` | Uploads succeed but nothing is processed; documents stay `pending` |
| `CELERY_RESULT_BACKEND` | `redis://localhost:6379/1` | Celery results | Same Redis, DB 1 | Worker errors on result storage |
| `JWT_SECRET_KEY` | `changeme-in-.env` | Signs login tokens (HS256) | Generate a long random string, e.g. `python -c "import secrets; print(secrets.token_urlsafe(48))"` | **Works with an insecure default — always change it**; changing it logs everyone out |
| `AZURE_STORAGE_CONNECTION_STRING` | *(none)* | Blob Storage account for originals, signature crops and reports | Azure portal → Storage account → *Access keys* → connection string | Upload and file-url endpoints return **503** with an explanatory message |

| Variable | Default | Purpose | Where to get it | If missing |
|---|---|---|---|---|
| `AZURE_STORAGE_CONTAINER_NAME` | `documents` | Blob container (must be **private**) | Create it in the storage account | Uploads fail |
| `AZURE_DOCUMENT_INTELLIGENCE_ENDPOINT` | *(none)* | OCR/layout ( `prebuilt-layout` ) | Azure portal → Document Intelligence resource → *Keys and Endpoint* | Every document ends `failed` at extraction |
| `AZURE_DOCUMENT_INTELLIGENCE_KEY` | *(none)* | Same | Same | Same |
| `AZURE_OPENAI_KEY` | *(none)* | Classification, extraction, issuer semantic match, visual review, signature detect/compare | Azure portal (or AI Foundry) → your OpenAI resource → *Keys* | Extraction fails (document `failed` ); visual review/signature checks fail |
| `AZURE_OPENAI_ENDPOINT` | *(none)* | Resource endpoint. Accepts either `https://<res>.openai.azure.com/` or an AI Foundry project URL `https://<res>.services.ai.azure.com/api/projects/<name>` (normalized to the origin) | Same | Same |
| `AZURE_OPENAI_DEPLOYMENT_NAME` | *(none)* | The model **deployment** to call. **Must be vision-capable**: its name must contain `gpt-4o` , `gpt-4.1` , `gpt-4-turbo` or `gpt-4-vision` , or the visual review and signature features raise `LLMConfigurationError` | Name you gave the deployment in Azure | Same |
| `AZURE_OPENAI_MAX_REQUESTS_PER_MINUTE` | `60` | Global cap on Azure OpenAI requests across every worker (Redis limiter). Set to **~80 %** of the deployment's RPM | Azure portal → deployment → *Rate limit*, or the `x-ratelimit-limit-requests` response header | Default 60 may under- or over-shoot your quota (429s or idle capacity) |
| `AZURE_OPENAI_MAX_TOKENS_PER_MINUTE` | `0` (off) | Global token cap (prompt + `max_tokens` reservation per call). Usually the binding limit. Set to **~80 %** of the deployment's TPM | `x-ratelimit-limit-tokens` header / portal | Without it, bursts exceed TPM → 429s |

There is **no local/offline substitute** for storage, OCR or the LLM (no Azurite/mock backend is wired in); a real Azure account is needed to exercise the pipeline. The unit tests mock all three, so they do not call Azure.

## Backend — optional / tuning

| Variable | Default | Purpose |
|---|---|---|
| `ENVIRONMENT` | `local` | Label shown in `/health` and the OpenAPI description |
| `JWT_ALGORITHM` | `HS256` | JWT algorithm |

| Variable | Default | Purpose |
|---|---|---|
| `JWT_ACCESS_TOKEN_EXPIRE_MINUTES` | `480` (8 h) | Login token lifetime |
| `AZURE_OPENAI_API_VERSION` | `2024-10-21` | Azure OpenAI REST API version |
| `AZURE_OPENAI_REQUEST_TIMEOUT_SECONDS` | `60.0` | Per-call LLM timeout. Kept short on purpose: a hung call holds a worker thread (and its rate-limiter reservation) |
| `AZURE_DOCUMENT_INTELLIGENCE_MAX_CALLS_PER_SECOND` | `10.0` | Global cap on Document Intelligence calls (Azure's default S0 limit is 15 TPS) |
| `AZURE_DOCUMENT_INTELLIGENCE_STYLE_FONT` | `true` | Request Layout's font-style add-on ( `styleFont` ), which the font consistency check uses on scanned pages. Billed by Azure as an add-on per page; `false` leaves fonts on scans to the visual review only |
| `DATABASE_POOLER_HOST` / `DATABASE_POOLER_PORT` | *(unset)* / `6432` | When set, the two app roles connect through **PgBouncer** (transaction pooling) and each process keeps no pool of its own. Recommended in production (processing-queues.md) |
| `DB_POOL_SIZE` / `DB_MAX_OVERFLOW` | `5` / `10` | SQLAlchemy pool per role per process when connecting **directly** (no PgBouncer) |
| `DEFAULT_MAX_FILE_SIZE_MB` / `DEFAULT_MAX_ZIP_SIZE_MB` | `10` / `300` | Upload limits a **new** company is created with. Limits are per company ( `companies.max_file_size_mb` / `max_zip_size_mb` , changed by a platform admin under Platform › Companies); changing these defaults never changes an existing company. The frontend reads the company's real limits from `GET /auth/me/upload-limits` |
| `BULK_UPLOAD_MAX_ENTRIES` | `25000` | Safety cap on entries (files + folders) in one zip. A 300 MB zip of small born-digital PDFs holds about 11,000 entries |
| `BULK_UPLOAD_CASE_WARNING_THRESHOLD` | `100` | Above this many cases the UI warns; the zip is still accepted |
| `DEFAULT_COMPANY_NAME` | `Default Company` | Name of the company the multi-tenancy migration assigns pre-existing data to (used once, by the migration) |
| `PLATFORM_ACCESS_AUDIT_DEDUP_SECONDS` | `0` | 0 = every platform-admin read of company data writes its own (platform-only) audit row; > 0 collapses identical reads within that window |
| `EXTRACTION_WORKER_CONCURRENCY` / `VISION_WORKER_CONCURRENCY` | `4` / `4` | Threads per extraction / vision worker ( `start-worker.sh` ). Size to just saturate the Azure caps |

| Variable | Default | Purpose |
|---|---|---|
| `FORENSICS_WORKER_CONCURRENCY` | `0` | Prefork processes per forensics worker; 0 = one per CPU of the container's limit |
| `FAIR_SHARE_BUCKET_SIZE` | `5` | A company's tasks drop one priority level per this many outstanding tasks on a queue |
| `QUEUE_ALERT_OLDEST_WAITING_SECONDS` | `300` | The per-minute `queue_metrics` job logs a WARNING `queue_alert {json}` line when a queue's oldest waiting task is older than this; 0 = off |
| `USAGE_RECONCILIATION_HOUR_UTC` | `2` | Hour (UTC) of the nightly usage-counter reconciliation (Celery beat) |
| `STUCK_DOCUMENT_PENDING_MINUTES` / `STUCK_DOCUMENT_PROCESSING_MINUTES` | `10` / `30` | Stuck-document recovery (every 5 minutes, `app/tasks/stuck_documents_task.py`): a document still `pending` after this long, or `processing` with no update for this long, **while the extraction queue is empty and idle**, lost its tasks (Redis restart/outage) and is queued again. `0` disables that half |
| `STUCK_DOCUMENT_MAX_REQUEUES` | `3` | Re-queues per document (counted from its `document_requeued` audit rows) before it is marked `failed` with an explanation |
| `STUCK_DOCUMENT_BATCH_SIZE` | `200` | Most documents re-queued per run |
| `LOGIN_MAX_FAILED_ATTEMPTS` / `LOGIN_LOCKOUT_MINUTES` | `5` / `15` | Sign-in throttling (`app/services/login_throttle.py`, Redis): after this many failed sign-ins for one email address, it is refused (HTTP 429 + `Retry-After`) for this many minutes. Counted per address (unknown addresses too), cleared by a successful sign-in. `0` disables |
| `LOGIN_MAX_ATTEMPTS_PER_IP_PER_MINUTE` | `20` | Sign-in attempts of any outcome per client IP per minute (best effort: behind the proxy chain a client can influence `X-Forwarded-For`). `0` disables. Both limits fail open if Redis is unreachable |
| `SEED_ADMIN_EMAIL` / `SEED_ADMIN_PASSWORD` | `admin@example.com` / `ChangeMe123!` | Read by `seed.py` (not by the app): the platform admin it creates — **and resets on every run** (`start-api.sh` runs it). Set both in production |
| `CELERY_POOL` | per queue | Read by `start-worker.sh`: overrides the pool type (`threads` for extraction/vision, `prefork` for forensics) |
| `CELERY_BEAT_SCHEDULE_FILE` | `/tmp/celerybeat-schedule` | Read by `start-beat.sh` (beat + the one-slot worker for `housekeeping_queue`) |
| `ISSUER_FUZZY_MATCH_THRESHOLD` | `85.0` | rapidfuzz WRatio score (0–100) an extracted issuer must reach to match the registry (05) |

| Variable | Default | Purpose |
|---|---|---|
| `METADATA_FORENSICS_MOD_DATE_THRESHOLD_SECONDS` | `1.0` | Jitter tolerance before "modified after created" flags (06) |
| `METADATA_FORENSICS_EDITING_SOFTWARE_NAMES` | `photoshop,gimp,illustrator,affinity,coreldraw,paint.net,pixlr,canva,inkscape` | Comma-separated, case-insensitive substrings that mark a Producer/Creator as an editing tool |
| `METADATA_FORENSICS_PDF_EDITOR_NAMES` | `ilovepdf,smallpdf,sejda,pdfescape,pdf-xchange editor,phantompdf,foxit pdf editor,nitro pro,nitro pdf pro,pdfelement,soda pdf,sodapdf,pdffiller,dochub,pdfcandy,pdf candy` | Same, for PDF editors (online and desktop); weighted lower than image editors |
| `DUPLICATE_HASH_HAMMING_THRESHOLD` | `5` | Max pHash Hamming distance (0–64) for a near-duplicate (10) |

Risk **weights, severities and tier thresholds are not environment variables** — they live in the database, per company, and are edited in Settings › Risk Rules (and the platform template under Platform › Rule templates; see 11).

## Backend — declared but UNUSED (do not need a value)

| Variable | Would be for | Status |
|---|---|---|
| `AZURE_AI_VISION_ENDPOINT`, `AZURE_AI_VISION_KEY` | Signature/stamp detection, logo matching | Never read — detection uses the Azure OpenAI vision model |
| `AI_CONTENT_DETECTION_ENDPOINT`, `AI_CONTENT_DETECTION_KEY` | Content Safety / Hive | Never read — see 08 |
| `EMAIL_SERVICE_CONNECTION_STRING`, `EMAIL_FROM_ADDRESS` | Transactional email | Never read — notifications are not built |
| `API_V1_PREFIX` (`/api/v1`) | URL prefix | Never read — routes have no prefix |
| `REDIS_URL` | Redis | Never read — Celery, the rate limiter, the fair-share counters and the queue monitor all use `CELERY_BROKER_URL` |

## docker-compose (optional overrides)

Read by `docker-compose.yml` from the shell or a root `.env`; defaults shown.

| Variable | Default | Purpose |
|---|---|---|
| `POSTGRES_USER` / `POSTGRES_PASSWORD` / `POSTGRES_DB` | `docauth` / `docauth` / `docauth` | Postgres container credentials. If you change them, update `DATABASE_URL` to match |
| `POSTGRES_PORT` | `5432` | Host port |
| `REDIS_PORT` | `6379` | Host port |

| Variable | Default | Purpose |
|---|---|---|
| `PGBOUNCER_PORT` | `6432` | Host port of PgBouncer ( `pgbouncer/pgbouncer.ini` , `pgbouncer/userlist.txt` — dev passwords only) |
| `FORENSICS_WORKER_CPUS` | `2` | CPUs per `forensics-worker` replica ( `--profile workers` ) |

## Frontend

| Variable | Default | Purpose |
|---|---|---|
| `VITE_API_BASE_URL` | `/api` | Base URL the SPA calls (build time). The default relies on a proxy that maps `/api/*` → the API with the prefix stripped — the Vite dev server locally, the ingress in production (production-deployment.md). Setting it to a different origin instead requires CORS on the backend, which currently has **no** CORS middleware |

No other frontend variables exist. Login state is a JWT in `localStorage` .

## Secrets checklist

`AZURE_STORAGE_CONNECTION_STRING` , `AZURE_DOCUMENT_INTELLIGENCE_KEY` , `AZURE_OPENAI_KEY` , `JWT_SECRET_KEY` , the owner database password (in `DATABASE_URL` ), `DATABASE_APP_PASSWORD` , `DATABASE_PLATFORM_PASSWORD` and `SEED_ADMIN_PASSWORD` are the secrets (plus PgBouncer's `userlist.txt` ). Keep them out of the repo ( `.env` is git-ignored) and, on Azure, in Key Vault referenced from the container apps (see production-deployment.md).

# Azure migration notes

> **Status:** background and open items. The concrete production plan — architecture, service tiers, costs and the step-by-step runbook — is production-deployment.md, which chooses **Azure Container Apps**. The repo ships container images ( `backend/Dockerfile` , `frontend/Dockerfile` + `nginx.conf.template` ) but no infrastructure-as-code or CI/CD yet.

## What is already Azure-shaped

| Area | State |
|---|---|
| **File storage** | Already Azure Blob Storage ( `AzureBlobStorageService` ), private container, SAS URLs, write-once uploads. Behind a `StorageService` interface. Nothing to migrate |
| **OCR** | Already Azure Document Intelligence ( `prebuilt-layout` ) |
| **LLM** | Already Azure OpenAI (text + vision) behind `LLMService` ; accepts a classic resource endpoint or an AI Foundry project URL |
| **Configuration** | All secrets, endpoints and thresholds come from environment variables ( `core/config.py` ); no hardcoded local paths. The same image can run anywhere given the right env |
| **Statelessness** | The API and the workers hold no local state. Files go to Blob, everything else to Postgres/Redis. Rendering is done in memory, so no local scratch disk is required |
| **Schema management** | Alembic migrations apply cleanly to an empty database (verified), so a new environment is bootstrapped by `alembic upgrade head` |
| **Managed PostgreSQL** | No role needs SUPERUSER or BYPASSRLS; the migration owner needs only CREATEROLE — which the Flexible Server admin has. The whole chain was verified as a non-superuser owner |
| **Connection pooling** | Tenant context is transaction-local, so the built-in PgBouncer (transaction mode) of Flexible Server works as-is |
| **Images** | `backend/Dockerfile` (API, every worker, beat, migrations — the command differs) and `frontend/Dockerfile` (nginx serving the SPA and proxying `/api` ) |

## What is still local-only (and what to do about it)

| Component today | Local form | Azure target (proposed) | Notes |
|---|---|---|---|
| PostgreSQL | Docker `postgres:16-alpine` + PgBouncer container | **Azure Database for PostgreSQL – Flexible Server** (v16) with the **built-in PgBouncer** (General Purpose tier) | `DATABASE_URL` with `?sslmode=require` ; private access; PITR backups. Run migrations as a one-off job per release. Set `SEED_ADMIN_PASSWORD` (the seed resets it) |
| Redis (broker, fair-share counters, rate limiter) | Docker `redis:7-alpine` , no auth | **Azure Managed Redis**, *Non-Clustered* policy | Azure Cache for Redis no longer accepts new customers and retires 2028-09-30. Use `rediss://…:10000/0` for **both** Celery URLs (database 0 only) |

| Component today | Local form | Azure target (proposed) | Notes |
|---|---|---|---|
| FastAPI app | `uvicorn --reload` on the host | Container App with internal ingress (`backend/Dockerfile`) | `uvicorn … --workers 2`; not `start-api.sh` (it migrates and re-seeds on every start) |
| Celery workers | one local worker | Three Container Apps (`start-worker.sh extraction | vision |
| Frontend | Vite dev server + `/api` proxy | Container App `fddt-web` (`frontend/Dockerfile`: nginx serves the build and proxies `/api/*` to the API's internal FQDN) | One origin, so no CORS. nginx caps request bodies (`UPLOAD_BODY_LIMIT`, default 11m; `BULK_UPLOAD_BODY_LIMIT` for bulk zips, default 301m) |
| Secrets | `backend/.env` file | **Key Vault** referenced by the Container Apps | `JWT_SECRET_KEY`, owner / `fddt_app` / `fddt_platform` passwords, storage connection string, Document Intelligence and OpenAI keys, Redis key, seed password |
| Azure credentials | Long-lived keys / connection string | **Managed identity** + RBAC (Storage Blob Data Contributor, Cognitive Services User) | The code authenticates with keys/connection strings only; switching to `DefaultAzureCredential` is a code change in `storage_service.py`, `ocr_service.py`, `llm_service.py` (each already sits behind an interface). Note the SAS generation currently needs an account key — with managed identity use *user delegation* SAS |
| Health/readiness | `GET /health` (does not touch dependencies) | Probe target | Consider a readiness probe that checks Postgres and Redis |
| Auth | Email/password JWT, in `localStorage` | Same for v1; **Entra ID SSO** is a possible later upgrade | Not started |
| Email/notifications | Not built | Azure Communication Services (planned) | `EMAIL_*` settings are declared but unused |
| CI/CD | None | GitHub Actions or Azure DevOps | Suggested stages: lint (`oxlint`), `tsc -b`, `pytest`, build images, migrate, deploy |

## Things that must change before any production exposure

These are pre-deployment blockers found while documenting, not deployment tasks per se:

1. ~~**Authorization gaps** on upload, file-url and the signature endpoints~~ — **fixed** (details). Still worth a security review of the remaining routes before exposure.
2. ~~**Remove** `/__debug_new_case`~~ — **done** (the public route outside `ProtectedRoute` no longer exists).
3. **TLS + single origin.** Terminate TLS at the Container Apps ingress (or Front Door) and keep the SPA and API on one origin (the nginx proxy) — the backend has no CORS middleware. Blob Storage needs a CORS rule for the app origin (the PDF viewer reads SAS URLs directly).
4. **JWT hardening.** Change `JWT_SECRET_KEY`; consider shorter lifetimes, refresh tokens, and server-side revocation (there is no logout or password reset endpoint). Tokens in `localStorage` are exposed to XSS.

5. ~~**Login abuse.** No rate limiting or lockout on `POST /auth/login`~~ — **fixed**: sign-in throttling in Redis (`services/login_throttle.py`): 5 failed attempts lock an email address for 15 minutes, 20 attempts per IP per minute, both answering 429. Still: no password policy beyond 8+ characters.
6. **Issuer registry.** Each company's registry starts empty (the migrated "Default Company" has fake test issuers); load the client's real data, or most documents will fail issuer verification.
7. **Upload limits.** PDF only (by header bytes), per-company size limits (10 MB / 300 MB zip by default, set by a platform admin), corruption and password-protection checks exist (`services/upload_validation.py`, `services/upload_limits.py`). The nginx proxy's body caps (`UPLOAD_BODY_LIMIT` / `BULK_UPLOAD_BODY_LIMIT`) must cover the largest limit any company is given. Still missing: a page-count limit, a malware scan (e.g. Defender for Storage malware scanning) and a Blob lifecycle/quarantine policy.
8. ~~**Stuck-work recovery.**~~ — **fixed**: a failed publish no longer fails the upload, and the `requeue_stuck_documents` job (every 5 minutes, `tasks/stuck_documents_task.py`) re-queues documents left `pending` / `processing` while the extraction queue is idle, up to 3 times, then marks them failed. Not covered: checks lost for a document whose extraction had already finished. Failed Azure calls are still not retried automatically.
9. **Observability.** Python `logging`, the audit log, the queue monitor and the per-minute `queue_metrics` / `queue_alert` log lines exist. Ship container logs to Log Analytics and create the alerts in production-deployment.md; Application Insights/OpenTelemetry tracing is not wired in.

## Scaling and cost considerations

- **Azure OpenAI is the throughput limit; Document Intelligence is the larger cost.** Per document: one extraction call, an optional issuer-match call, **two vision calls per PDF page** (visual review), plus signature detection and, on demand, comparison calls — about 1 cent of gpt-4.1-mini per 2-page document, against 2 cents of Document Intelligence. Full model: production-deployment.md › Costs. Set the TPM quota for the volume and the app's caps to 80 % of it.
- **Duplicate detection is a linear scan** of every stored page hash of the company per new page. Fine for thousands of pages; move to an index (BK-tree/LSH) or a pre-filter before millions.
- **Worker memory:** a multi-page PDF is rendered to full-resolution arrays (200 DPI); large documents benefit from a memory-limited, low-concurrency worker rather than many parallel tasks.
- **Report generation is synchronous** in the API request; if reports get heavy, move `generate_case_report` into a Celery task and poll (the function is already the unit to move).
- **Pipeline tasks are not retried automatically.** A transient Azure failure marks the document/check `failed`; there is no retry policy or dead-letter handling. Consider Celery `autoretry_for` with backoff on the LLM/OCR calls.
- **Data retention and privacy.** Documents and extracted text (including personal data) are stored indefinitely. Define retention, encryption-at-rest (Azure default), region/residency, and who may read the audit log. Azure OpenAI abuse-monitoring/data-handling terms should be reviewed for the client's data classification.

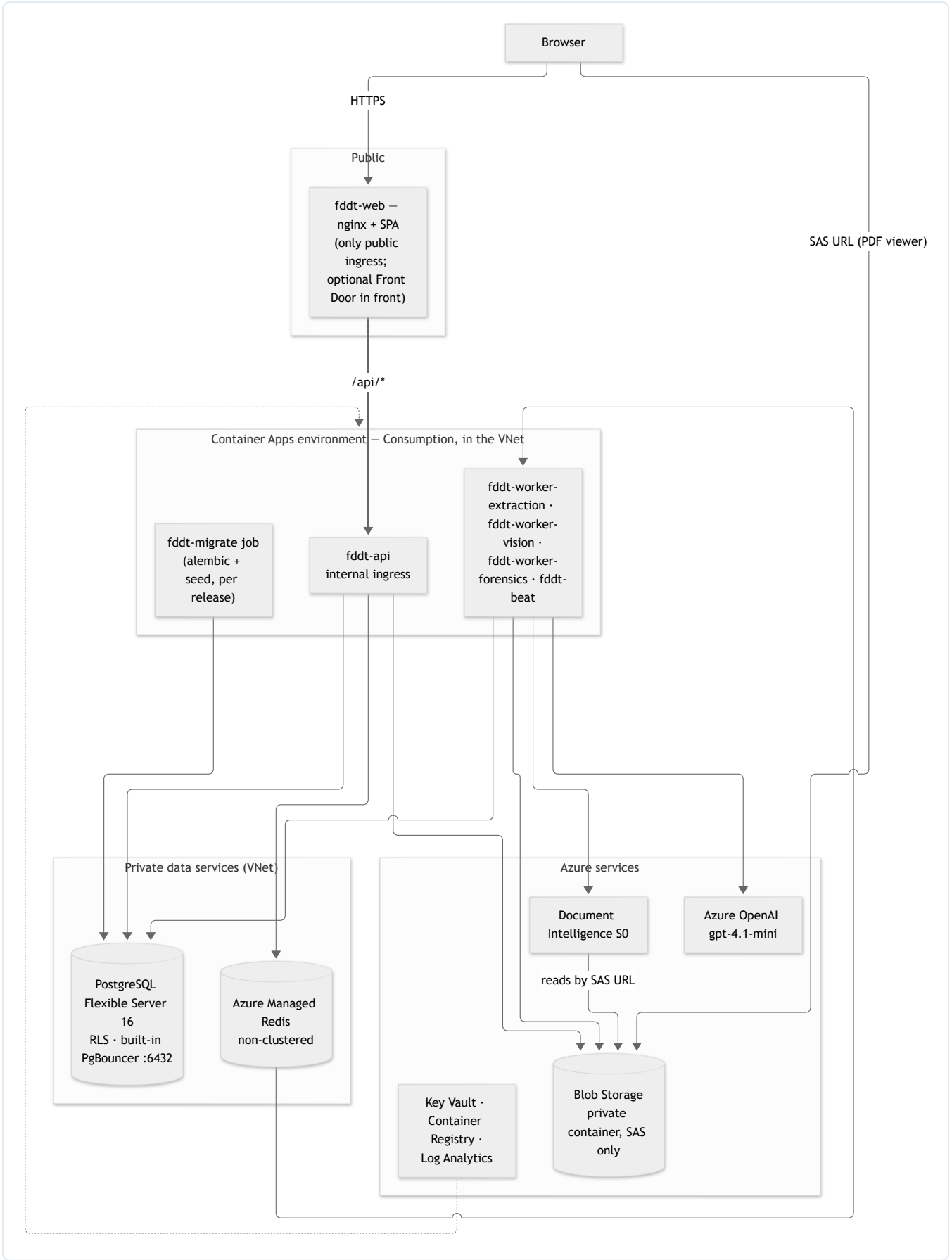
## Suggested order

1. Follow production-deployment.md section 4 (network, registry, PostgreSQL, Redis, Storage, AI resources, Key Vault, environment, migration job, apps, domain).
2. Work through the blockers above that still apply (JWT hardening, upload malware scan).
3. Add CI/CD (suggested stages: `oxlint`, `tsc -b`, `pytest` incl. the Postgres RLS suite, build images, run the migration job, roll the apps) and infrastructure-as-code (Bicep/Terraform) for the resources.
4. Load real issuer data, tune risk weights on real cases, and validate the weak signals (ELA, signature and AI-generation checks) against known-genuine and known-forged documents before relying on them.

# Production deployment (Azure)

> **Status:** this is the deployment plan and runbook. Per the project rules no infrastructure-as-code or CI/CD is committed yet — the repository ships the two container images (`backend/Dockerfile`, `frontend/Dockerfile` + `nginx.conf.template`) and everything below is done with the Azure CLI or portal. Prices are **list prices in USD for UAE North, retrieved 2026-10-03** from the Azure Retail Prices API; they exclude VAT and any EA/CSP discount. Re-check them in the pricing calculator before quoting.
>
> **To deploy, follow azure-production-runbook.md**, the complete copy-paste runbook (network/IP/port plan, NSGs, DNS, YAML for every app, autoscaling, alerts, release and rollback). Section 4 below is the condensed version of it.

# 1. Target architecture



| Component | Azure service | Why |
|---|---|---|
| SPA + reverse proxy | Container App `fddt-web` (frontend image), **public** ingress | One origin for the SPA and `/api`, so no CORS is needed; nginx caps request bodies ( `UPLOAD_BODY_LIMIT` / `BULK_UPLOAD_BODY_LIMIT` , at least the largest per-company limit) |
| API | Container App `fddt-api` (backend image), **internal** ingress only | Never exposed directly; reached only through `fddt-web` |
| Workers | Container Apps `fddt-worker-extraction` , `-vision` , `-forensics` (backend image), no ingress | One per queue so each scales on its own (processing-queues.md) |
| Scheduler | Container App `fddt-beat` , no ingress, **min = max = 1** | Beat plus a one-slot worker for `housekeeping_queue` : per-minute queue metrics, stuck-document recovery every 5 minutes, nightly usage reconciliation. Two beats would run every job twice |
| Migrations | Container Apps **job** `fddt-migrate` (manual trigger) | Runs `alembic upgrade head` + `seed.py` once per release, as the database owner — not on every API start |
| Database | Azure Database for PostgreSQL **Flexible Server 16**, private access (VNet) | Row-Level Security, roles without superuser/BYPASSRLS (verified to work without superuser); built-in PgBouncer on General Purpose tiers |
| Broker / limiter / fair-share | **Azure Managed Redis**, *Non-Clustered* policy | Azure Cache for Redis no longer accepts new customers (since 2026-04-01) and retires 2028-09-30 |
| Files | Storage account (Standard LRS/ZRS, hot), one **private** container | Originals (write-once), signature crops, report PDFs; accessed only through short-lived SAS URLs |
| OCR | Document Intelligence **S0** | `prebuilt-layout` |
| LLM | Azure OpenAI / Foundry, **gpt-4.1-mini** deployment | Classification, extraction, visual review, signatures, issuer match |
| Secrets | Key Vault, referenced as Container Apps secrets | No secret in images or the repo |
| Images | Container Registry (Basic) | Backend and frontend images |
| Observability | Log Analytics workspace (Container Apps logs) + Azure Monitor alerts | `queue_alert` , failed documents, database metrics |
| Edge (optional, not used by default) | Front Door Standard | Global edge, caching of static assets, managed certificate. Container Apps ingress already provides TLS and a free managed certificate, and sign-in throttling is built into the app, so this is optional. WAF custom rules work on Standard; the managed OWASP rule sets need **Front Door Premium** ($330/month base) |

## Region and data residency

- **UAE North** is used for everything that *stores* data (Storage, PostgreSQL, Redis, Document Intelligence processing). The documents are mostly UAE/Saudi.
- **Azure OpenAI gpt-4.1-mini is only offered as a *Global* deployment in UAE North** (no regional or Data Zone SKU there). With a Global deployment, prompts — the OCR text and page images — may be **processed in any**

**Azure region**; data at rest stays in the resource's region. If the client's data classification forbids that, use a region/model combination that offers a regional deployment, or have the client accept Global processing in writing. (In West Europe, a Data Zone EU deployment costs 10 % more: $0.44 / $1.76 per 1M tokens.)

- Review the Azure OpenAI data-handling and abuse-monitoring terms against the client's policy; request modified abuse monitoring if required.

## 2. Service tiers

Three sizes. All three use the same images and the same configuration keys — only replica counts, CPU and SKUs differ, so moving between them is a scaling change, not a redeployment.

| | **S — Pilot** | **M — Production** | **L — Scale** |
|---|---|---|---|
| Volume (2-page documents/month) | ≈ 3,000 | ≈ 20,000 | ≈ 100,000 |
| Companies | 1–3 | up to ~20 | 20+ |
| `fddt-web` | 1 × 0.25 vCPU / 0.5 GiB | 2 × 0.25 / 0.5 (max 4) | 2 × 0.25 / 0.5 (max 6) |
| `fddt-api` | 1 × 0.5 / 1 (max 2) | **1** × 1 / 2 (HTTP autoscale to 4) | 3 × 1 / 2 (max 6) |
| `fddt-worker-extraction` | 1 × 0.5 / 1 | 1 × 1 / 2 | 2 × 1 / 2 |
| `fddt-worker-vision` | 1 × 0.5 / 1 | 1 × 1 / 2 | 4 × 1 / 2 |
| `fddt-worker-forensics` | **0**–2 × 1 / 2 (scale to zero) | **0**–4 × 2 / 4 (scale to zero) | 1–10 × 2 / 4 |
| `fddt-beat` (beat + housekeeping worker) | 1 × 0.5 / 1 | 1 × 0.5 / 1 | 1 × 0.5 / 1 |
| PostgreSQL | Burstable **B2s** (2 vCore, 4 GiB), 64 GB, no HA, direct connections | General Purpose **D2ds_v5** (2 vCore, 8 GiB), 128 GB, **no HA at launch**, built-in PgBouncer | General Purpose **D4ds_v5** (4 vCore, 16 GiB), 512 GB, zone-redundant HA, built-in PgBouncer |
| Redis | Managed Redis **B0**, no HA | Managed Redis **B0**, **no HA** | Managed Redis **B3**, HA |
| Azure OpenAI gpt-4.1-mini Global Standard TPM (target; no quota is requested, the deployment uses the subscription's free capacity) | 100k | 150k | 500k |
| Availability | single zone, no failover | single zone (99.9 % database SLA; a zone failure means a restore, ~1 hour); 2 web replicas | zone-redundant database + Redis (99.99 %) |

Notes on sizing:

- **High availability (HA) on tier M is a client decision.** Without it the database is one server in one zone: Azure restarts it after a crash, and a whole-zone outage means a point-in-time restore (about an hour). Turning HA on later is one command and roughly doubles the database cost (+$176/month): `az postgres flexible-server update -g $RG -n $PG --high-availability ZoneRedundant`. Record the decision with the client.
- **Redis without HA is safe** because Redis holds no durable data, and the stuck-document job (processing-queues.md) re-queues any document whose tasks were lost when Redis restarted. Tier L keeps HA so that a Redis

restart doesn't pause a busy platform for minutes.

- **Forensics memory:** pages are rendered at 200 DPI in memory; give each forensics *process* ~2 GiB ( `start-worker.sh` runs one prefork process per vCPU of the container's limit).
- **Forensics scales to zero** (tiers S/M): KEDA starts a replica as soon as a forensics task waits, and stops it after ~5 idle minutes. The first document after a quiet period waits ~30–60 s longer for its forensic checks. This works because the every-minute platform jobs run on their own `housekeeping_queue` in the beat container, so nothing else wakes the forensics workers.
- **Vision/extraction** pools are I/O-bound threads; their real ceiling is the Azure OpenAI quota. Throughput ≈ (TPM cap) ÷ ~32k tokens *reserved* per 2-page document — e.g. 80k TPM ≈ 2.5 documents/min, 400k TPM ≈ 12/min. No quota is requested: the deployment uses the subscription's free capacity, which is then the ceiling (processing-queues.md). They keep one replica each (no scale-to-zero), so OCR of a new document starts at once; more replicas than the quota can feed don't help.
- **The beat container** runs beat plus a one-slot worker for `housekeeping_queue` (queue metrics every minute, stuck-document recovery every 5 minutes, nightly usage reconciliation). Always exactly one.

## 3. Costs

### Unit prices used (UAE North, USD, list)

| Service | Meter | Price |
|---|---|---|
| Container Apps (Consumption) | vCPU active / idle | $0.000024 / $0.000003 per vCPU-second (≈ $63.07 / $7.88 per vCPU-month) |
| | Memory (**same rate active or idle**: there is no idle discount on memory) | $0.000003 per GiB-second (≈ $7.88 per GiB-month) |
| | Free grant per subscription per month | 180,000 vCPU-s + 360,000 GiB-s + 2M requests (≈ $5.40) |
| | Environment management (only if environment private endpoint or planned maintenance is enabled) | $0.143/hour (≈ $104/month) — **not used** in this design |
| PostgreSQL Flexible Server | Burstable B2s | $0.079/hour |
| | General Purpose Ddsv5 | $0.1085 per vCore-hour (D2ds_v5 $0.217, D4ds_v5 $0.434); 1-year reserved $571 per vCore-year (≈ 40 % less), 3-year $1,141 per vCore (≈ 60 % less) |
| | Storage / backup storage beyond the free 100 % | $0.138 / $0.105 per GB-month (HA doubles compute **and** storage) |
| Azure Managed Redis (Balanced) | B0 / B3 per instance | $0.0726 / $0.09295 per hour (HA = 2 instances). *The UAE North AMR meters look inconsistent (B1 is listed above B3; West Europe lists B0 at $0.019) — confirm in the calculator; the figures here are the higher, conservative ones* |
| Blob Storage | Hot LRS data / writes / reads | $0.0203 per GB-month / $0.06 per 10k / $0.0048 per 10k |
| Document Intelligence S0 | Layout (pre-built) pages | **$10 per 1,000 pages**; commitment tiers from 20k pages for $190/month (overage $9.50/1k) and 100k pages for $900/month ($9/1k) |

| Service | Meter | Price |
|---|---|---|
| | Font-style add-on ( `styleFont` , requested on every page: `AZURE_DOCUMENT_INTELLIGENCE_STYLE_FONT=true` ) | **≈ $6 per 1,000 pages — to confirm** on the Document Intelligence pricing page (add-on capabilities). It feeds the font consistency check on scanned pages |
| Azure OpenAI gpt-4.1-mini (Global) | Input / cached input / output | **$0.40 / $0.10 / $1.60 per 1M tokens**; Batch API half price (not usable for this interactive pipeline) |
| Container Registry | Basic | $0.1666/day (≈ $5/month) |
| Key Vault | Standard operations | $0.0396 per 10,000 |
| Log Analytics | Analytics Logs ingestion (first 5 GB/month free) | $3.29/GB |
| Azure Monitor alerts | Log alert at 5-minute frequency | $1.50/month each |
| Front Door Standard (optional, not in the estimate) | Base fee + requests/data | $35/month + usage |
| NAT Gateway (optional, not in the estimate) | Gateway + static IP + data | ≈ $33/month + $0.045/GB |
| Azure Bastion Standard (on demand only, not in the estimate) | Per hour while it exists | ≈ $0.29/hour (≈ $210/month if left running) |
| Bandwidth | Internet egress | First 100 GB/month free, then $0.075–0.12/GB |

## AI cost per document (measured)

Token use is taken from the app's own measurements of real calls ( `fddt:llm_usage:*` , 12 English + Arabic sample documents):

| Call | Per | Real input tokens | Real output tokens |
|---|---|---|---|
| Classification + extraction | document | 2,105 | 889 |
| Visual review (2 runs) | page | 2 × 2,904 | 2 × 177 |
| Signature/stamp detection | document (~3 calls) | 3 × 1,418 | 3 × 84 |
| Issuer semantic match | document, only if fuzzy match fails (assumed always) | 400 | 126 |
| Signature comparison | on demand only | 1,055 | 47 |
| Spelling judgment (English) — *estimate, not yet measured* | document, only if the dictionary pass leaves candidates (most clean documents: none or 1–3) | ~600 + ~60 per candidate | ~70 per candidate |

| Document size | Azure OpenAI | Document Intelligence layout | Font add-on (to confirm) | **AI cost per document** |
|---|---|---|---|---|
| 1 page | $0.0076 | $0.0100 | $0.006 | **$0.024** |
| 2 pages | $0.0105 | $0.0200 | $0.012 | **$0.043** |

| Document size | Azure OpenAI | Document Intelligence layout | Font add-on (to confirm) | **AI cost per document** |
|---|---|---|---|---|
| 3 pages | $0.0134 | $0.0300 | $0.018 | **$0.061** |
| 5 pages | $0.0192 | $0.0500 | $0.030 | **$0.099** |

Document Intelligence (per page) is the larger part; Azure OpenAI is ~1 cent per document. The spelling judgment (added after these measurements) adds well under a tenth of a cent for a typical document, so it is not in the totals above. Without the font add-on (`AZURE_DOCUMENT_INTELLIGENCE_STYLE_FONT=false`) a 2-page document costs $0.031, but scanned pages then get no font consistency check (fonts on scans are left to the visual review).

## Monthly estimate per tier (2-page documents)

| | **S — Pilot** | **M — Production** | **L — Scale** |
|---|---|---|---|
| Container Apps (range: see below) | $75–130 | $180–300 | $730–1,180 |
| PostgreSQL | $67 | $176 (no HA) | $775 (HA) |
| Managed Redis | $53 | $53 (no HA) | $136 (HA) |
| Blob Storage | $1 | $4 | $19 |
| Container Registry, Key Vault | $6 | $9 | $21 |
| Log Analytics + alerts | $10 | $10–15 | $25–60 |
| **Infrastructure** | **≈ $210–265** | **≈ $430–555** | **≈ $1,705–2,190** |
| Document Intelligence layout | $60 | $400 | $2,000 |
| Document Intelligence font add-on (to confirm) | $36 | $240 | $1,200 |
| Azure OpenAI | $32 | $210 | $1,051 |
| **AI (usage-based)** | **≈ $128** | **≈ $850** | **≈ $4,251** |
| **Total per month** | **≈ $340–395** | **≈ $1,280–1,405** | **≈ $5,955–6,440** |
| Per document | ≈ $0.11–0.13 | ≈ $0.064–0.070 | ≈ $0.060–0.064 |

The previous design (always-active billing, HA on M, 2 forensics + 2 API replicas always on, Front Door, 30 GB of logs) estimated infrastructure at $369 / $1,283 / $2,636. The changes above cut it by 28–43 % / 57–66 % / 17–35 %. The AI line grew only because the font add-on is now counted.

How the figures were computed (730 h/month):

- **Container Apps, low end:** replicas sitting at their minimum with no work are billed the **idle** vCPU rate (Azure applies it automatically when a replica uses < 0.01 vCPU and serves no requests; a Celery worker waiting on Redis normally qualifies). Activity assumed: web/API active 30 % of the month (working hours), extraction/vision active 15 % (M) / 60 % (L) of the time, forensics billed only while it has work (scale to zero; tier L ~1 replica on average), beat idle. **Memory is always billed in full.**
- **Container Apps, high end:** the same, but web, API, extraction, vision and beat billed at the active rate all month (in case Azure does not treat the idle workers as idle).
- **Logs:** the app's own log volume is small (a few lines per task, one metrics line per queue per minute), so S/M most likely stay within the free 5 GB/month; the workspace gets a 1 GB/day cap as a guard. Alerts ~$8/month.
- Blob assumes ~0.5 MB per document retained plus ~4 writes and ~40 reads per document.
- Not included: VAT, support plans, Defender for Cloud, Front Door, NAT Gateway, Bastion, egress beyond 100 GB, signature comparisons, staging (a staging copy of tier S, paused when unused, costs well under tier S).

- **Check it after the first month** in Cost Management (filter by tag `app=fddt`), and adjust the replica sizes to what the apps really use.

### Cost levers

#### Already applied in this design

| Lever | Effect |
|---|---|
| Idle billing + forensics scale-to-zero (S/M), housekeeping jobs on their own queue | Most of the Container Apps saving |
| `fddt-api` minimum 1 replica on M (autoscale to 4); rolling updates still cause no downtime | ≈ −$25–80/month |
| PostgreSQL and Redis without HA on M at launch | −$176 and −$53/month (re-enable when the client needs 99.99 %) |
| Sign-in throttling in the app (Redis) instead of a Front Door WAF | −$35+/month |
| Logs kept in the normal (Analytics) tier but small, with a 1 GB/day cap. *Basic Logs* are not used: the alert rules read the same table, and Basic tables can't drive them | −$60–200/month vs. the old estimate |
| NAT Gateway only if someone must allow-list our outbound IP; Bastion created only for admin sessions, then deleted | Avoids $33 and $210/month |

#### Later, once usage is known

- **PostgreSQL 1-year reservation** after 1–2 months on a stable size: ≈ 40 % off compute (M without HA: −$63/month; L: −$254). It is a commitment, paid even if you resize or delete.
- **Document Intelligence commitment tier** once volume is steady: at 200k pages/month (tier L), the 100k-page commitment ($900) + 100k overage at $9/1k = $1,800 instead of $2,000.
- **Font add-on:** confirm its price. If the font consistency check on scanned pages is not worth it, set `AZURE_DOCUMENT_INTELLIGENCE_STYLE_FONT=false` (−$12 per 1,000 two-page documents).
- **Right-size extraction/vision** after a few weeks of real metrics: 0.5 vCPU / 1 GiB each saves ≈ $8–40/month per app.
- **Azure savings plan for compute**: check whether it covers Container Apps consumption in the client's agreement (≈ 15 % off).
- **Staging:** stop its PostgreSQL server (`az postgres flexible-server stop`; not billed for compute while stopped, up to 7 days at a time) and scale its apps to zero when unused.
- Keep pages per document honest: the cost scales with pages, and every page is OCR'd and reviewed twice.

## 4. Deployment steps

All commands are Azure CLI (bash). Replace `<...>` placeholders. Run them in this order the first time; later releases only repeat steps 10 and 12.

## 4.1 Prerequisites and naming

```
az login
az extension add --name containerapp --upgrade
az provider register --namespace Microsoft.App
az provider register --namespace Microsoft.OperationalInsights

RG=rg-fddt-prod
LOC=uaenorth
PREFIX=fddtprod                    # lowercase, globally unique where needed
az group create -n $RG -l $LOC
```

## 4.2 Logging

```
az monitor log-analytics workspace create -g $RG -n $PREFIX-logs -l $LOC --retention-time 30
LA_ID=$(az monitor log-analytics workspace show -g $RG -n $PREFIX-logs --query customerId -o tsv)
LA_KEY=$(az monitor log-analytics workspace get-shared-keys -g $RG -n $PREFIX-logs --query primarySharedKey
-o tsv)
```

## 4.3 Network

```
az network vnet create -g $RG -n $PREFIX-vnet -l $LOC --address-prefixes 10.20.0.0/16
# Container Apps (workload-profiles environment needs a /27 or larger, delegated)
az network vnet subnet create -g $RG --vnet-name $PREFIX-vnet -n aca --address-prefixes 10.20.0.0/23 \
  --delegations Microsoft.App/environments
# PostgreSQL private access (delegated)
az network vnet subnet create -g $RG --vnet-name $PREFIX-vnet -n pg --address-prefixes 10.20.2.0/24 \
  --delegations Microsoft.DBforPostgreSQL/flexibleServers
# Private endpoints (Redis)
az network vnet subnet create -g $RG --vnet-name $PREFIX-vnet -n pe --address-prefixes 10.20.3.0/24
```

## 4.4 Container registry and images

```
az acr create -g $RG -n ${PREFIX}acr --sku Basic
az acr build -r ${PREFIX}acr -t fddt-backend:<version> ./backend
az acr build -r ${PREFIX}acr -t fddt-web:<version> ./frontend
```

The frontend image is built with the default `VITE_API_BASE_URL=/api`; nginx proxies `/api/*` to the API.

## 4.5 PostgreSQL

```
az postgres flexible-server create -g $RG -n $PREFIX-pg -l $LOC --version 16 \
  --tier GeneralPurpose --sku-name Standard_D2ds_v5 --storage-size 128 \
  --vnet $PREFIX-vnet --subnet pg --private-dns-zone $PREFIX-pg.private.postgres.database.azure.com \
  --admin-user fddtowner --admin-password '<OWNER_PASSWORD>' --backup-retention 35
# Tier S instead: --tier Burstable --sku-name Standard_B2s --storage-size 64
# Tier L (or M once the client wants 99.99 %): add --high-availability ZoneRedundant

az postgres flexible-server db create -g $RG -s $PREFIX-pg -d fddt
# Built-in PgBouncer (General Purpose / Memory Optimized only — NOT available on Burstable)
az postgres flexible-server parameter set -g $RG -s $PREFIX-pg -n pgbouncer.enabled --value true
az postgres flexible-server parameter set -g $RG -s $PREFIX-pg -n metrics.pgbouncer_diagnostics --value on
```

- The admin login (`fddtowner`) is the **owner** role the migrations run as. It has CREATEROLE through `azure_pg_admin` but is not a superuser — exactly what the migrations need (they create `fddt_app` and `fddt_platform` themselves, without BYPASSRLS).
- TLS is enforced by default (`require_secure_transport = on`); always add `?sslmode=require` to `DATABASE_URL`. The app keeps that query string when it derives the two app-role URLs and the pooler URL from it.
- Built-in PgBouncer: port **6432**, transaction pooling (the default), `default_pool_size` 50 per user/database. The app sets the tenant only transaction-locally, so it is safe behind transaction pooling (`tests/test_pgbouncer_rls.py`). It authenticates against the database, so no `userlist.txt` is used in production.

### 4.6 Redis

```
az redisenterprise create -g $RG -n $PREFIX-redis -l $LOC --sku Balanced_B0 \
  --clustering-policy NoCluster --high-availability Disabled \
  --public-network-access Disabled --access-keys-authentication Enabled
# Tier L: --sku Balanced_B3 --high-availability Enabled
az network private-endpoint create -g $RG -n $PREFIX-redis-pe --vnet-name $PREFIX-vnet --subnet pe \
  --private-connection-resource-id $(az redisenterprise show -g $RG -n $PREFIX-redis --query id -o tsv) \
  --group-id redisEnterprise --connection-name redis
REDIS_KEY=$(az redisenterprise database list-keys -g $RG --cluster-name $PREFIX-redis --query primaryKey -o
tsv)
```

(The `az redisenterprise` command group manages Azure Managed Redis; flags vary between CLI versions. `NoCluster` exists in the current API, but some CLI versions only accept `OSSCluster` / `EnterpriseCluster` — in that case create the cache in the **portal** and pick *Non-Clustered*. **The clustering policy cannot be changed after the database is created** — choose it correctly the first time.) Requirements for Celery:

- **Clustering policy *Non-Clustered***. Celery's Redis transport and the fair-share priority lists use multi-key commands across `<queue>`, `<queue>:1…9`; the default *OSS* policy needs a cluster-aware client, and *Enterprise* can reject cross-slot multi-key commands.
- Only database **0** exists — point **both** Celery URLs at `/0` (`rediss://:<KEY>@<name>.<region>.redis.azure.net:10000/0?ssl_cert_reqs=required`; port 10000, TLS). Result keys are prefixed, so sharing the database is safe.
- Add a private DNS zone link for the private endpoint (`privatelink.redis.azure.net`) so the apps resolve it inside the VNet.

### 4.7 Storage

```
az storage account create -g $RG -n ${PREFIX}files -l $LOC --sku Standard_ZRS --kind StorageV2 \
  --min-tls-version TLS1_2 --allow-blob-public-access false --https-only true
az storage container create --account-name ${PREFIX}files -n documents --auth-mode login   # private
az storage cors add --account-name ${PREFIX}files --services b --methods GET HEAD OPTIONS \
  --origins https://<your-app-domain> --allowed-headers '*' --exposed-headers '*' --max-age 3600
```

- The account **must keep its public endpoint reachable**: the browser fetches each PDF directly by its short-lived SAS URL (the case viewer), and Document Intelligence fetches the file by SAS URL too. Access is protected by the SAS signature, and anonymous blob access is disabled. Locking the account to private endpoints would require code changes (stream files through the API; send bytes to Document Intelligence).
- The CORS rule lets the viewer read PDFs from the web app's origin; add the Front Door hostname too if used.
- Consider a lifecycle rule moving blobs older than e.g. 180 days to *Cool* (cheaper storage; the files are immutable).

## 4.8 Document Intelligence and Azure OpenAI

```
az cognitiveservices account create -g $RG -n $PREFIX-docintel -l $LOC --kind FormRecognizer --sku S0 \
  --custom-domain $PREFIX-docintel
az cognitiveservices account create -g $RG -n $PREFIX-openai -l $LOC --kind AIServices --sku S0 \
  --custom-domain $PREFIX-openai
az cognitiveservices account deployment create -g $RG -n $PREFIX-openai --deployment-name gpt-4.1-mini \
  --model-name gpt-4.1-mini --model-version 2025-04-14 --model-format OpenAI \
  --sku-name GlobalStandard --sku-capacity <free>      # thousands of TPM: the subscription's free capacity
(runbook step 10)
```

- The deployment name must contain `gpt-4.1` (or `gpt-4o`, …) — the app checks it is vision-capable.
- Set the app's caps to **80 %** of the granted quota: `AZURE_OPENAI_MAX_TOKENS_PER_MINUTE` = 0.8 × TPM, `AZURE_OPENAI_MAX_REQUESTS_PER_MINUTE` = 0.8 × RPM (read both from the portal or the `x-ratelimit-limit-*` response headers). Document Intelligence S0 allows 15 requests/s; keep `AZURE_DOCUMENT_INTELLIGENCE_MAX_CALLS_PER_SECOND=10`.
- After the first real traffic, run `python -m scripts.calibrate_llm_tokens` once against the production deployment (costs a few cents) to confirm the token reservations.

## 4.9 Key Vault and secrets

```
az keyvault create -g $RG -n $PREFIX-kv -l $LOC --enable-rbac-authorization true
for s in jwt-secret owner-db-url app-db-password platform-db-password redis-url storage-conn \
         docintel-key openai-key seed-admin-password; do
  az keyvault secret set --vault-name $PREFIX-kv -n $s --value '<value>'
done
```

| Secret | Value |
|---|---|
| `jwt-secret` | `python -c "import secrets; print(secrets.token_urlsafe(48))"` |
| `owner-db-url` | `postgresql+psycopg2://fddtowner:<OWNER_PASSWORD>@<pg-host>:5432/fddt?sslmode=require` |
| `app-db-password` / `platform-db-password` | Two long random passwords (they become the passwords of `fddt_app` / `fddt_platform`) |
| `redis-url` | `rediss://:<REDIS_KEY>@<redis-host>:10000/0?ssl_cert_reqs=required` (Celery refuses a `rediss://` URL without `ssl_cert_reqs`; URL-encode the key) |
| `storage-conn` | The storage account connection string (the code signs SAS URLs with the account key) |
| `docintel-key`, `openai-key` | Resource keys |
| `seed-admin-password` | The first platform admin's password |

URL-encode special characters inside passwords that are embedded in URLs.

## 4.10 Container Apps environment, identity and shared configuration

```
az containerapp env create -g $RG -n $PREFIX-env -l $LOC \
  --infrastructure-subnet-resource-id $(az network vnet subnet show -g $RG --vnet-name $PREFIX-vnet -n aca
--query id -o tsv) \
  --logs-workspace-id $LA_ID --logs-workspace-key $LA_KEY --enable-workload-profiles
```

Every backend app uses one **user-assigned managed identity** with *AcrPull* on the registry and *Key Vault Secrets User* on the vault. Secrets are referenced as `--secrets jwt=keyvaultref:https://$PREFIX-kv.vault.azure.net/secrets/jwt-secret,identityref:<identity-id> …` and mapped into environment variables with `secretref:`.

**Backend environment (identical for API, workers, beat and the migration job):**

| Variable | Value |
|---|---|
| `ENVIRONMENT` | `production` |
| `DATABASE_URL` | `secretref:owner-db-url` |
| `DATABASE_APP_PASSWORD` / `DATABASE_PLATFORM_PASSWORD` | `secretref:app-db-password` / `secretref:platform-db-password` |
| `DATABASE_POOLER_HOST` / `DATABASE_POOLER_PORT` | `<pg-host>` / `6432` (tiers M/L). Tier S (Burstable): leave unset and set `DB_POOL_SIZE=3`, `DB_MAX_OVERFLOW=5` |
| `CELERY_BROKER_URL`, `CELERY_RESULT_BACKEND` | both `secretref:redis-url` |
| `JWT_SECRET_KEY` | `secretref:jwt-secret` |
| `AZURE_STORAGE_CONNECTION_STRING`, `AZURE_STORAGE_CONTAINER_NAME` | `secretref:storage-conn`, `documents` |
| `AZURE_DOCUMENT_INTELLIGENCE_ENDPOINT` / `_KEY` | `https://$PREFIX-docintel.cognitiveservices.azure.com/` / `secretref:docintel-key` |
| `AZURE_OPENAI_ENDPOINT` / `_KEY` / `_DEPLOYMENT_NAME` | `https://$PREFIX-openai.openai.azure.com/` / `secretref:openai-key` / `gpt-4.1-mini` |
| `AZURE_OPENAI_MAX_REQUESTS_PER_MINUTE` / `_MAX_TOKENS_PER_MINUTE` | 80 % of the deployment's RPM / TPM |
| `AZURE_DOCUMENT_INTELLIGENCE_MAX_CALLS_PER_SECOND` | `10` |
| `DEFAULT_MAX_FILE_SIZE_MB` / `DEFAULT_MAX_ZIP_SIZE_MB` | `10` / `300` (limits for new companies; each company's own are set under Platform › Companies) |
| `QUEUE_ALERT_OLDEST_WAITING_SECONDS` | `300` |
| `EXTRACTION_WORKER_CONCURRENCY` / `VISION_WORKER_CONCURRENCY` | `4` / `4` (per replica) |
| `FORENSICS_WORKER_CONCURRENCY` | `0` (one process per vCPU of the container) |
| `SEED_ADMIN_EMAIL` / `SEED_ADMIN_PASSWORD` | the first platform admin / `secretref:seed-admin-password` — **migration job only** |

Full reference: environment-variables.md.

### 4.11 Database migration job (every release, before the apps)

```
az containerapp job create -g $RG -n $PREFIX-migrate --environment $PREFIX-env \
  --trigger-type Manual --replica-timeout 1800 --replica-retry-limit 0 \
  --image ${PREFIX}acr.azurecr.io/fddt-backend:<version> --cpu 0.5 --memory 1Gi \
  --command sh --args "-c" "alembic upgrade head && python seed.py" \
  --env-vars <the backend variables above> ...
az containerapp job start -g $RG -n $PREFIX-migrate
az containerapp job execution list -g $RG -n $PREFIX-migrate -o table     # wait for Succeeded
```

- **Take a database backup/point-in-time marker before migrating** (Flexible Server keeps automatic backups; note the time so a PITR restore target is known).
- `seed.py` creates the platform admin and **resets that account's password to `SEED_ADMIN_PASSWORD` on every run**. Remove `&& python seed.py` from the job after the first release (or keep the variable at the real password).
- Do **not** use `start-api.sh` as the API command in production: it migrates and re-seeds on every start, and several API replicas would race.

### 4.12 Applications

```
IMG=${PREFIX}acr.azurecr.io/fddt-backend:<version>
# API — internal ingress only
az containerapp create -g $RG -n fddt-api --environment $PREFIX-env --image $IMG \
  --ingress internal --target-port 8000 --cpu 1 --memory 2Gi --min-replicas 2 --max-replicas 4 \
  --command uvicorn --args app.main:app --host 0.0.0.0 --port 8000 --workers 2 --proxy-headers --forwarded-
allow-ips '*' \
  --env-vars <backend variables>
# Workers — no ingress
az containerapp create -g $RG -n fddt-worker-extraction --environment $PREFIX-env --image $IMG \
  --cpu 1 --memory 2Gi --min-replicas 1 --max-replicas 1 --command sh --args start-worker.sh extraction --
env-vars <…>
az containerapp create -g $RG -n fddt-worker-vision --environment $PREFIX-env --image $IMG \
  --cpu 1 --memory 2Gi --min-replicas 1 --max-replicas 1 --command sh --args start-worker.sh vision --env-
vars <…>
az containerapp create -g $RG -n fddt-worker-forensics --environment $PREFIX-env --image $IMG \
  --cpu 2 --memory 4Gi --min-replicas 0 --max-replicas 4 --command sh --args start-worker.sh forensics --
env-vars <…>
# (scale-to-zero needs the KEDA Redis rules on forensics_queue — see the runbook, step 15.2)
# Beat — exactly one
az containerapp create -g $RG -n fddt-beat --environment $PREFIX-env --image $IMG \
  --cpu 0.5 --memory 1Gi --min-replicas 1 --max-replicas 1 --command sh --args start-beat.sh --env-vars <…>
# Web — the only public app
az containerapp create -g $RG -n fddt-web --environment $PREFIX-env \
  --image ${PREFIX}acr.azurecr.io/fddt-web:<version> --ingress external --target-port 80 \
  --cpu 0.25 --memory 0.5Gi --min-replicas 2 --max-replicas 3 \
  --env-vars API_UPSTREAM=https://fddt-api.internal.<environment-default-domain>
```

(Add `--registry-server`, `--registry-identity` and `--user-assigned` for the managed identity on each command; the replica counts above are tier M — take them from section 2. The CLI parses `--host`-style tokens inside `--args` as its own flags, so in practice create the apps from YAML as the runbook does.)

- `API_UPSTREAM` is the API's **internal FQDN** (`az containerapp show -n fddt-api --query properties.configuration.ingress.fqdn`). nginx strips `/api`, forwards to it, and refuses bodies over `UPLOAD_BODY_LIMIT` (default **11m**) — or, for `/api/bulk-uploads` (streamed, unbuffered), `BULK_UPLOAD_BODY_LIMIT` (default **301m**) — before they reach Uvicorn. Upload limits are per company, so these two must be at least the **largest** per-company limit granted (plus ~1 MB of multipart overhead for single files):

when a platform admin gives a company 20 MB / 500 MB, redeploy `fddt-web` with `--env-vars UPLOAD_BODY_LIMIT=21m BULK_UPLOAD_BODY_LIMIT=501m`.

- Workers need no probes beyond the default; give the API a liveness probe on `GET /health`.
- Rolling update order for a release: build images → run `fddt-migrate` → update `fddt-api` → update the workers and beat → update `fddt-web`. Migrations are additive within a release, so old workers keep working during the roll.
- **Restart the workers whenever their code changes** (a new revision does this).

### 4.13 Domain and TLS

```
az containerapp hostname add -g $RG -n fddt-web --hostname app.<client-domain>
az containerapp hostname bind -g $RG -n fddt-web --hostname app.<client-domain> \
  --environment $PREFIX-env --validation-method CNAME     # free managed certificate
```

Or put Front Door Standard in front (managed certificate on Front Door, origin = `fddt-web`) and restrict `fddt-web` ingress to the Front Door service tag. Add the final origin to the Blob CORS rule (4.7).

### 4.14 First login and smoke test

1. Sign in as the seeded platform admin; change nothing else until the checks below pass.
2. **Platform › Companies** → create the client company (it receives the 38 template rules and 30/60 thresholds; its issuer registry is empty). **Platform › Users** → create its reviewers and users.
3. As a company user, upload one sample PDF; confirm it reaches a risk tier, the PDF renders in the case viewer (Blob CORS works) and **Platform › Processing queues** shows all three queues with consumers.
4. Upload an image and a password-protected PDF — both must be refused with their specific messages.
5. Generate a report as a reviewer and open it.
6. As the company's Reviewer L2, load the company's real **issuer registry** (until then almost every document flags `issuer.not_in_registry`).

## 5. Operations

### Monitoring and alerts

| Alert | Query / metric | Suggested threshold |
|---|---|---|
| Queue backlog | `ContainerAppConsoleLogs_CL \| where Log_s has "queue_alert"` | any row in 5 min (the app logs it when the oldest waiting task > 300 s) |
| Worker errors | `ContainerAppConsoleLogs_CL \| where ContainerAppName_s startswith "fddt-worker" and Log_s has_any ("ERROR", "Traceback")` | > 5 in 15 min |
| Database permission errors | `… Log_s has "permission denied"` | any (a write needs a grant the app role lacks) |
| Azure throttling | `… Log_s has "429"` | > 10 in 15 min — raise TPM or lower the caps |
| Stuck documents re-queued | `… Log_s has "stuck_documents_requeued"` | any — Redis lost queued tasks (restart/outage); the documents were queued again automatically |

| Alert | Query / metric | Suggested threshold |
|---|---|---|
| Sign-in lockouts | `… Log_s has "login_lockout"` | > 10 in 15 min (password guessing) |
| Failed documents | `audit_log` events `document_processing_failed` / `*_failed` (query through the platform audit log) | review daily |
| PostgreSQL | CPU > 80 %, storage > 80 %, `client_connections_waiting` (PgBouncer) > 0 for 5 min | |
| Redis | Used memory > 70 %, server load > 80 % | |
| API | `fddt-api` 5xx rate, replica restarts | |

`queue_metrics {json}` lines (one per queue per minute) can be charted directly in Log Analytics ( `parse_json(extract("queue_metrics (.*)", 1, Log_s))` ).

### Backups and disaster recovery

| Data | Protection | Restore |
|---|---|---|
| PostgreSQL | Automatic backups, 35-day point-in-time restore; zone-redundant HA (L; M once enabled) fails over automatically; without HA a zone outage means a restore | PITR to a new server, repoint `DATABASE_URL` / pooler host |
| Blob Storage | ZRS; enable **soft delete** (14 days) and **versioning** for defence in depth (the app never deletes or overwrites) | Undelete / previous version |
| Redis | Holds only queued tasks, fair-share counters, sign-in throttle and rate-limiter windows | A lost broker loses queued tasks; the stuck-document job queues the affected documents again within ~15 minutes (checks of documents whose extraction had already finished are not re-run) |
| Configuration | Key Vault (soft delete + purge protection on) and the image tags | Redeploy |

Recovery objectives with tier M as designed: RPO ≈ 5 minutes (PostgreSQL log backups), RTO ≈ 1 hour (restore + repoint), excluding in-flight pipeline tasks.

### Routine tasks

- **Secret rotation:** rotate storage/Document Intelligence/OpenAI keys with the two-key pattern; rotating `JWT_SECRET_KEY` signs everyone out. To rotate `fddt_app` / `fddt_platform`, `ALTER ROLE … PASSWORD` as the owner, then update the Key Vault secrets and restart the apps.
- **Usage reconciliation** runs nightly (02:00 UTC); **Billing & usage** feeds client invoicing.
- **Re-calibrate token reservations** ( `scripts.calibrate_llm_tokens` ) after any model, prompt or render change; re-check the 80 % caps after any quota change.
- **Schema docs:** regenerate `docs/database/*.md` with `docs/tools/gen_db_docs.py` after each migration.

## 6. Go-live checklist

**Must do**

- [ ] `JWT_SECRET_KEY`, both app-role passwords, the owner password and `SEED_ADMIN_PASSWORD` are long random values in Key Vault; nothing secret is in an image or the repo.
- [ ] The migration job ran (`alembic upgrade head` → head `c9e3a1b5d7f4` or later) and the seed step was removed or pinned to the real password.
- [ ] `DATABASE_URL` ends in `?sslmode=require`; tiers M/L connect through the built-in PgBouncer (6432).
- [ ] Redis is Non-Clustered and both Celery URLs use `/0` over `rediss://`.
- [ ] gpt-4.1-mini Global Standard deployment uses the free capacity (no quota request); the app caps at 80 % of it.
- [ ] Exactly one `fddt-beat` replica (beat + the `housekeeping_queue` worker).
- [ ] HA decision for PostgreSQL and Redis recorded with the client (tier M launches without HA).
- [ ] Blob container private, anonymous access off, CORS limited to the app origin, soft delete on.
- [ ] Only `fddt-web` has public ingress.
- [ ] Alerts from section 5 created and routed to an on-call address.
- [ ] Data-residency decision for the Global Azure OpenAI deployment recorded with the client.
- [ ] The client's issuer registry loaded; risk weights reviewed with the client.
- [ ] A full browser walkthrough of every role (submitter, Reviewer L1, Reviewer L2, platform admin).

**Known gaps to accept or schedule** (from azure-migration-notes.md)

- No forgot-password flow (users change their own password; admins reset others), tokens in `localStorage`. Sign-in throttling exists (per address and per IP).
- No malware scan and no page-count limit on uploads (type, size, corruption and password are checked).
- No automatic retry of a failed Azure call. (Documents whose queued tasks were lost are re-queued by the stuck-document job.)
- Keys and connection strings, not managed identity, for Storage / Document Intelligence / OpenAI.

## Sources

- Azure Retail Prices API — `https://prices.azure.com/api/retail/prices` (UAE North and West Europe, retrieved 2026-10-03)
- Billing in Azure Container Apps
- PgBouncer in Azure Database for PostgreSQL Flexible Server
- Azure Cache for Redis retirement FAQ
- Azure Managed Redis architecture (clustering policies)
- Azure Managed Redis pricing
- Azure Front Door pricing

# Azure production deployment — step-by-step runbook

This is the **hands-on runbook** a DevOps engineer follows to stand up FDDT in production on Azure, from an empty subscription to a verified go-live. Every step has the exact commands, the values to use, and a ✅ **Verify** check before moving on.

- The *why* (architecture choices, S/M/L service tiers, monthly costs) is in production-deployment.md. This runbook implements that design and adds the network, port, IP, DNS, scaling and release details.
- All settings are explained in environment-variables.md; queue, rate-limit and connection-pool design in processing-queues.md.
- Written against the code at Alembic head `f3b7d1e8a4c5` (2026-10-08). If `alembic heads` shows a newer revision, that is fine. Use whatever it reports in step 13.

> **First time on Azure? Read 0.4 Your workspace and 0.5 Finding things in the Azure portal before step 1.**
>
> Every step starts with a 📍 **Where** box with three lines:
>
> - **Who** does it: you, or the client's admin.
> - **Type in:** where you type the commands. Almost always *Azure Cloud Shell on the client's screen, in* `~/FDDT`.
> - **Check in the portal:** where to click in https://portal.azure.com to see the result.
>
> Portal paths are written as **Resource › menu group › item**. For example, *fddtprod-pg › Settings › Server parameters* means: open the resource `fddtprod-pg`, find the group **Settings** in its left menu, and click **Server parameters**.

---

## Contents


0. Before you start: decisions, access, existing Azure limits
1. Target architecture and traffic flow
2. Network, IP and port reference (DevOps sheet)
3. Step 1 — Variables file
4. Step 2 — Resource group and providers
5. Step 3 — Log Analytics
6. Step 4 — Virtual network, subnets, NSGs, egress IP
7. Step 5 — Managed identity and Key Vault
8. Step 6 — Container Registry and images
9. Step 7 — PostgreSQL Flexible Server
10. Step 8 — Azure Managed Redis
11. Step 9 — Blob Storage
12. Step 10 — Document Intelligence and Azure OpenAI
13. Step 11 — Secrets into Key Vault
14. Step 12 — Container Apps environment and shared YAML helpers
15. Step 13 — Database migration job
16. Step 14 — API
17. Step 15 — Workers and scheduler

18. Step 16 — Web front end
19. Step 17 — Blob CORS for the app origin
20. Step 18 — Custom domain and TLS (DNS records)
21. Step 19 — Optional: Front Door + WAF
22. Step 20 — Monitoring and alerts
23. Step 21 — Admin access to private data (Bastion / jump box)
24. Step 22 — Smoke test and first-day setup
25. Step 23 — Post-go-live hardening
26. Scaling playbook
27. Release, rollback and staging
28. Backups, DR and routine operations
29. Troubleshooting
30. Go-live checklist
31. Handover sheet

---

# 0. Before you start

## 0.1 Decisions to record (fill these in first)

| Decision | Options | Default in this runbook |
|---|---|---|
| **Service tier** | **S** pilot (~3k docs/month) · **M** production (~20k) · **L** scale (~100k). Sizing and costs | `M` |
| Region | One region for everything that stores data | `uaenorth` |
| App hostname | e.g. `fddt.client.com` | `app.<client-domain>` |
| Edge | Container Apps ingress only (free managed TLS) **or** Front Door Standard + WAF (rate limiting on login, edge TLS) | Container Apps only; Front Door in step 19 |
| Who may reach the UI | Whole internet **or** client office IP ranges only | Internet (restrict in step 16.4 if required) |
| Static egress IP | Needed only if a client/partner firewall must allow-list our outbound traffic, or to lock the AI services to our IP. A NAT Gateway costs ~$33/month + data | **No** ( `USE_NAT="no"` ); set `yes` if needed |
| **High availability (HA)** for PostgreSQL and Redis | HA = a standby in another zone (99.99 %). Without it: one zone, 99.9 %, a zone outage means a restore (~1 hour). Redis without HA is safe: lost tasks are re-queued automatically | **Tier M launches without HA** (saves ~$229/month); tier L keeps HA. Get the client's OK in writing |
| Azure OpenAI data processing | gpt-4.1-mini is **Global**-only in UAE North (prompts may be processed in any Azure region). **Get written client sign-off**, or pick a region with a regional/Data Zone deployment | Global, signed off |
| **OCR service** | Azure Document Intelligence, pricing tier **S0 (Standard)**. **Not F0 (Free)**: F0 analyses only the first 2 pages of each document and allows 500 pages per month at 1 request/s, which is not usable for production | **S0** (decided) |

| Decision | Options | Default in this runbook |
|---|---|---|
| **LLM** | Azure OpenAI, model **gpt-4.1-mini**, deployment type **Global Standard**: pay per token, no commitment, and the largest default capacity. Not *Provisioned* (PTU: reserved capacity billed every hour even when idle) and not *Standard/Data Zone* (not offered for this model in UAE North) | **gpt-4.1-mini, Global Standard** (decided) |
| First platform admin | A real mailbox of the operator | `platform-admin@<operator-domain>` |
| On-call alert address | Email / distribution list | `ops@<operator-domain>` |

## 0.2 Access you need

| Permission | Scope | Why |
|---|---|---|
| **Owner**, or **Contributor + User Access Administrator** | The **subscription**. This is the account the client signs in with during the screen share | Creating the resource group, registering providers, creating resources **and** role assignments (AcrPull, Key Vault Secrets User) |
| Permission to create DNS records | The client's DNS zone | CNAME + TXT for the app hostname (step 18) |
| Azure OpenAI access | The subscription | Creating the gpt-4.1-mini deployment (step 10) |

## 0.3 Azure limits: use what the subscription already has (no quota requests)

**No quota is requested from Microsoft for this deployment.** Everything uses the limits the client's subscription already has:

| Limit | What we use | What it means |
|---|---|---|
| **Azure OpenAI gpt-4.1-mini, Global Standard**, tokens per minute (TPM) | **All of the subscription's free TPM** for this model in the region. Step 10 reads the free amount and creates the deployment with it | This number decides how many documents per minute the platform can process (scaling playbook). Unused capacity costs nothing: Global Standard is billed per token used |
| Document Intelligence **S0** | Default 15 requests/s; the app uses at most 10 | Enough for every tier |
| PostgreSQL vCores, Container Apps cores | Subscription defaults | Enough for tiers S and M. For tier L, check them first (below) |

> 📍 **Who:** you, in the client's Cloud Shell · **When:** at the start of the session, after section 0.4 (Cloud Shell must be open) · **Check in the portal:** https://ai.azure.com → **Management center** → **Quota**, filter *UAE North*, *gpt-4.1-mini*, *Global Standard*: the same numbers as the command below, read-only.

Check how much gpt-4.1-mini Global Standard capacity is free in the region (unit: thousands of TPM, the same unit `--sku-capacity` uses in step 10):

```
az cognitiveservices usage list -l uaenorth \
  --query "[?name.value=='OpenAI.GlobalStandard.gpt-4.1-mini'].{used:currentValue, limit:limit}" -o table
```

`limit - used` is the free capacity. For example, `limit 1000, used 0` means 1,000k TPM free. Compare it with the tier's target ( `AOAI_TPM_K` in `deploy.env` : S 100, M 150, L 500):

- **Free ≥ target:** fine. Step 10 uses all of it, so the platform gets more throughput at no extra cost.

- **Free < target:** the deployment still works, but processes fewer documents per minute than the tier was sized for (throughput = 80 % of TPM ÷ 32,000 two-page documents per minute). Tell the client the expected throughput. Documents simply queue longer at peaks; nothing fails.
- **Free = 0** (other deployments in this subscription use it all): step 10 cannot create the deployment. Free some capacity from another gpt-4.1-mini deployment in the region (Foundry → **Deployments** → that deployment → **Edit** → lower its TPM), or use a region where capacity is free.
- If the command shows nothing, the model name differs in that region. Run it without `--query` and look for the `gpt-4.1-mini` *GlobalStandard* row.

Tier L only: in the portal, search **Quotas** → **My quotas**, and filter *Azure Database for PostgreSQL Flexible Server* and *Container Apps* for UAE North. You need about 8 free PostgreSQL vCores and about 60 Container Apps cores. If they are lower, size down to tier M; no request is needed.

## 0.4 Your workspace: Azure Cloud Shell on the client's screen

**The setup:** the client's admin shares their screen (Teams, Zoom, …) and gives you control. They are signed in at **https://portal.azure.com** with **their own account**: they type their own password and approve MFA on their own phone. Never ask for their password. If Azure asks to sign in again, hand control back to them.

You **create** everything by pasting commands into **Azure Cloud Shell**, a Linux terminal inside the portal. It is already signed in as the client and has the Azure CLI, git and openssl, so **nothing is installed and nothing runs on your own PC**. You use the portal pages only to **check** that things look right. Do not change settings by hand in the portal unless a step says so: if the portal and the commands disagree, later steps break in confusing ways.

**Before the session (on your side)**

- **Push the exact version to deploy** to GitHub ( `main` ). Cloud Shell downloads the code from there, not from your PC. Uncommitted changes on your PC will not be deployed.
- The repository is private, so create a **GitHub personal access token** for the clone: github.com → your avatar → **Settings › Developer settings › Personal access tokens › Fine-grained tokens › Generate new token** → *Repository access*: only `FDDT` → *Permissions › Contents*: **Read-only** → expiry **1 day**. Keep it private; delete it after the deployment.
- **The runbook must be readable on the client's screen.** With remote control, *your* clipboard usually does **not** reach *their* computer, so you cannot copy from your own PDF and paste into their Cloud Shell. Once the code is cloned (step 4 below), open this file **inside Cloud Shell** and copy the commands from there. Alternatively, send the client the PDF beforehand and have them open it on their PC.

**Opening Cloud Shell (first time)**

1. In the portal's top bar, click the **>_** icon (*Cloud Shell*), just right of the search bar. If the bar is narrow, it is under the **…** menu.
2. Choose **Bash**. If it opens as *PowerShell*, click **Switch to Bash** in the Cloud Shell toolbar. Never use PowerShell: these commands are bash.
3. On *Getting started*, choose **Mount storage account** → pick the client's subscription → **Apply** → **We will create a storage account for you** → **Next**. This makes a small storage account (cents per month, in a resource group named like `cloud-shell-storage-<region>` ) that **keeps your files** between sessions. **Do not** choose *No storage account required*: your files ( `deploy.env` , `reload.sh` , the YAML) would be deleted every time the session ends.
4. **Get the code** (paste your token when asked for the *password*; the screen shows nothing while you type or paste it):

```
git clone https://github.com/Deepraj-chawda/FDDT.git ~/FDDT
cd ~/FDDT
git checkout main && git log --oneline -1      # the commit you are deploying
```

5. **Open this runbook in Cloud Shell's editor**, to copy the commands from the client's screen: `code ~/FDDT/docs/deployment/azure-production-runbook.md`. Close the editor with **...** (top right of the editor) → **Close Editor**.
6. **Check the account and add the CLI extensions** (one time):

   ```
   az account show -o table                    # the client's subscription
   az extension add --name containerapp --upgrade
   az extension add --name redisenterprise --upgrade
   ```

   If the wrong subscription is shown, run `az account list -o table`, then `az account set --subscription "<name or id>"`.

**Make it comfortable:** maximize Cloud Shell with the □ icon on its toolbar, or open **https://shell.azure.com** in a separate browser tab: same files, full screen. Keep the portal in another tab for the checks.

**How to paste and run**

- Paste in Cloud Shell: **Ctrl + Shift + V**, or **right-click** → **Paste**. Copy from the Cloud Shell editor with Ctrl + C.
- Paste **one code block at a time**. Wait until the prompt (`$`) comes back, and read the output before pasting the next block.
- Lines ending in `\` continue on the next line. Always paste the whole command.
- Output in **red** or starting with `ERROR:` means stop. Check Troubleshooting before continuing. Warnings in yellow are usually fine.
- Some commands run for 5–30 minutes (images, PostgreSQL, Redis) and show nothing while they work.

**Cloud Shell disconnects after ~20 minutes without keyboard input**, even while a long command is running.

- During long commands, click into Cloud Shell and press **Shift** every ~10 minutes.
- If it disconnects anyway, **the resource keeps being created in Azure**. The command was only waiting. Click **Reconnect** and run the block below. Check the resource in the portal (each step's 📍 box says where) until it is ready, then run the **remaining** commands of that step (those after the long one).

**Every time Cloud Shell (re)starts**, run this. Every step saves what it needs in Azure, so this brings all values back:

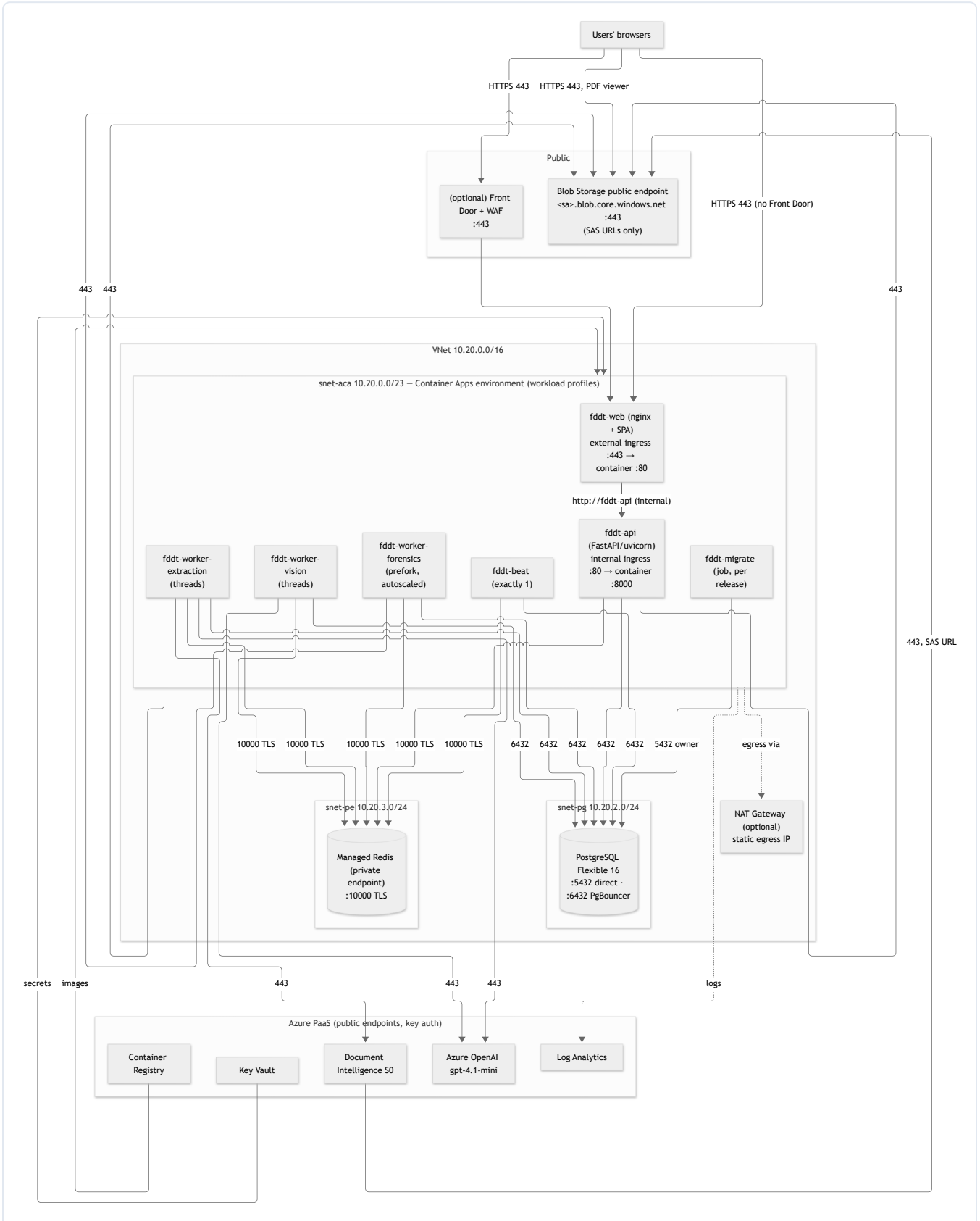
```
cd ~/FDDT
source ~/fddt-deploy/reload.sh        # ~1 minute; re-creates all variables (file created in step 1)
```

## 0.5 Finding things in the Azure portal

1. The client is signed in at **https://portal.azure.com** (the same account Cloud Shell uses).
2. **Make sure you are in the right directory.** If the client's account sees several directories (tenants), click the avatar at the top right → **Switch directory** → pick the one that holds the subscription.
3. **Open a resource:** type its name (e.g. `fddtprod-pg`) in the **search bar at the top** and click it. Or search **Resource groups** → `rg-fddtprod` to list everything this runbook creates.
4. **The left menu** of every resource is grouped (*Overview*, *Activity log*, *Access control (IAM)*, then groups such as **Settings**, **Monitoring**, **Security + networking**). Click a group name to expand it. If you can't see an item, type it into the small **search box at the top of the left menu**.
5. **Did something fail?** The **bell icon** (top right, *Notifications*) and the resource group's **Activity log** show recent operations and their errors.
6. **Copy a value:** most fields have a **copy icon** (two squares) at their right end.
7. Menu names change slightly over time. If a path in this runbook doesn't match exactly, use the left-menu search box with the last word of the path.

Estimated time end to end: **3–4 hours**. PostgreSQL with HA takes ~15–20 min and Managed Redis ~15–30 min to provision. Start them early (steps 7–8) and continue with the other steps while they build.

# 1. Target architecture and traffic flow



**Rules this design enforces**

- `fddt-web` is the **only** public entry point. The API, workers, database and Redis have no public address.
- The SPA and the API share **one origin** (nginx proxies `/api/*`), so the backend needs no CORS.
- Blob Storage keeps its **public endpoint**, but only short-lived SAS URLs work (anonymous access is off). The browser's PDF viewer and Document Intelligence both fetch files by SAS URL; locking the storage account to private endpoints would need code changes.
- Database migrations run **once per release** as a job, never on API start.
- Each queue has its own worker app, so each scales on its own.

---

# 2. Network, IP and port reference

## 2.1 Address plan

| Subnet | CIDR | Usable IPs | Delegation | Holds |
|---|---|---|---|---|
| `snet-aca` | `10.20.0.0/23` | 507 | `Microsoft.App/environments` | Container Apps environment. The minimum is `/27`; `/23` leaves room to scale (every node and internal load balancer takes IPs) |
| `snet-pg` | `10.20.2.0/24` | 251 | `Microsoft.DBforPostgreSQL/flexibleServers` | PostgreSQL primary + HA standby (nothing else may be in this subnet) |
| `snet-pe` | `10.20.3.0/24` | 251 | — | Private endpoints: Redis (and optionally Key Vault later) |
| `AzureBastionSubnet` | `10.20.4.0/26` | 59 | — (name is fixed) | Azure Bastion (optional, step 21) |
| `snet-jump` | `10.20.5.0/28` | 11 | — | Admin jump VM (optional, step 21) |
| *reserved* | `10.20.6.0 – 10.20.255.255` | | | Staging peering, App Gateway, growth |

> If the VNet will ever be peered with the client's network or connected by VPN/ExpressRoute, **agree the range with their network team first**. Change `10.20.0.0/16` in `deploy.env` if it overlaps.

## 2.2 Port and flow matrix

| # | Source | Destination | Port / protocol | Purpose | Enforced by |
|---|---|---|---|---|---|
| 1 | Users (internet / client office) | `fddt-web` public ingress (env static IP) or Front Door | **443/TCP HTTPS** (80 → redirects to 443) | UI and `/api/*` | NSG `nsg-aca` inbound; ingress IP restrictions (optional) |

| # | Source | Destination | Port / protocol | Purpose | Enforced by |
|---|---|---|---|---|---|
| 2 | Users' browsers | `<storage>.blob.core.windows.net` | **443/TCP** | PDF viewer reads documents by SAS URL | Client proxy/firewall must allow it; Blob CORS (step 17) |
| 3 | Container Apps ingress (Envoy) | `fddt-web` container | 80/TCP | nginx target port | Container Apps |
| 4 | `fddt-web` (nginx) | `fddt-api` internal ingress ( `http://fddt-api` ) → container **8000** | 80/TCP inside the environment | `/api/*` reverse proxy | Internal ingress, never public |
| 5 | API, workers, beat | PostgreSQL built-in PgBouncer | **6432/TCP + TLS** | App roles `fddt_app` , `fddt_platform` (tiers M/L) | NSG `nsg-pg` |
| 6 | Migration job, jump VM; **all apps on tier S** | PostgreSQL | **5432/TCP + TLS** | Owner role (migrations/seed), admin; tier S (Burstable) has no PgBouncer | NSG `nsg-pg` |
| 7 | API, workers, beat, KEDA scaler | Azure Managed Redis private endpoint | **10000/TCP TLS** | Celery broker, global rate limiter, fair-share counters, queue monitor | Private endpoint; NSG `nsg-pe` |
| 8 | API, workers | Blob Storage | 443/TCP | Write-once uploads, reads, SAS signing | Storage account (key auth) |
| 9 | Extraction worker (forensics, rarely) | Document Intelligence | 443/TCP | OCR / layout | Resource key (+ optional IP rule, step 23) |
| 10 | Extraction, vision workers (forensics, rarely) | Azure OpenAI | 443/TCP | Classification, visual review, signatures | Resource key (+ optional IP rule) |
| 11 | Document Intelligence service | Blob Storage | 443/TCP | Fetches the file by SAS URL | Storage must stay reachable from Azure public network |
| 12 | Container Apps platform | Container Registry | 443/TCP | Image pulls (managed identity, AcrPull) | RBAC |
| 13 | Container Apps platform | Key Vault | 443/TCP | Secret references (managed identity) | RBAC |
| 14 | Container Apps | Log Analytics | 443/TCP | Console/system logs | — |
| 15 | All VNet resources | Azure DNS `168.63.129.16` | 53/UDP+TCP | Resolves the private DNS zones ( `*.postgres.database.azure.com` , `privatelink.redis.azure.net` ) | Must not be blocked |
| 16 | Operator | Azure Bastion → jump VM | 443 → 22/TCP | Admin access to the database and Redis | Bastion |

**Ports inside containers:** `fddt-web` listens on **80**, `fddt-api` on **8000**. Workers, beat and the job listen on nothing.

## 2.3 Static IPs and hostnames you will hand over

| Item | How to get it (after the step) | Used for |
|---|---|---|
| **Inbound public IP** of the Container Apps environment | `az containerapp env show -g $RG -n $ENV --query properties.staticIp -o tsv` (step 12) | Client firewall allow-lists; an A record if an apex domain is used |
| App default FQDN | `az containerapp show -g $RG -n fddt-web --query properties.configuration.ingress.fqdn -o tsv` | CNAME target (step 18) |
| **Outbound (egress) public IP** (only with `USE_NAT="yes"`) | `az network public-ip show -g $RG -n $PREFIX-egress-ip --query ipAddress -o tsv` (step 4) | Partner allow-lists; AI-service IP rules |
| PostgreSQL private IP | `az network private-dns record-set a list -g $RG -z $PG_DNS_ZONE -o table` | Diagnostics only (always connect by FQDN) |
| Redis private IP | `az network private-endpoint show -g $RG -n $REDIS-pe --query "customDnsConfigs[0].ipAddresses[0]" -o tsv` | Diagnostics only |
| Storage hostname | `$SA.blob.core.windows.net` | Client proxy allow-list (flow 2) |

## 2.4 NSG rules (created in step 4)

**`nsg-aca`** (on `snet-aca`). The default rules already allow VNet↔VNet and the Azure load balancer.

| Priority | Direction | Source | Destination port | Action | Note |
|---|---|---|---|---|---|
| 100 | Inbound | `Internet` (or client office CIDRs, or `AzureFrontDoor.Backend` if step 19) | 443 | Allow | HTTPS |
| 110 | Inbound | same | 80 | Allow | HTTP → HTTPS redirect |
| 65500 | Inbound | * | * | Deny | default |
| (default) | Outbound | * | * | Allow | Needs MCR, ACR, Entra ID, Azure Monitor, Storage, Cognitive Services on 443. If outbound is ever locked down (Azure Firewall/UDR), allow service tags `MicrosoftContainerRegistry`, `AzureFrontDoor.FirstParty`, `AzureContainerRegistry`, `AzureActiveDirectory`, `AzureMonitor`, `Storage`, `CognitiveServicesManagement`, `AzureKeyVault` and FQDNs `*.openai.azure.com`, `*.cognitiveservices.azure.com` on 443 |

**`nsg-pg`** (on `snet-pg`)

| Priority | Direction | Source | Destination port | Action |
|---|---|---|---|---|
| 100 | Inbound | `10.20.0.0/23` (snet-aca) | 5432, 6432 | Allow |
| 110 | Inbound | `10.20.5.0/28` (snet-jump) | 5432, 6432 | Allow |
| 120 | Inbound | `10.20.2.0/24` (snet-pg itself, HA replication) | * | Allow |
| 4000 | Inbound | `VirtualNetwork` | * | Deny |

**`nsg-pe`** (on `snet-pe`, private-endpoint network policies enabled)

| Priority | Direction | Source | Destination port | Action |
|---|---|---|---|---|
| 100 | Inbound | `10.20.0.0/23` | 10000 | Allow |
| 110 | Inbound | `10.20.5.0/28` | 10000 | Allow |
| 4000 | Inbound | `VirtualNetwork` | * | Deny |

---

## Step 1 — Variables file

> 📍 **Who:** you · **Type in:** Cloud Shell, in `~/FDDT` · **Check in the portal:** nothing yet. Search **Subscriptions** and confirm you can see the client's subscription.

All names, sizes and options live in one file, `~/fddt-deploy/deploy.env`. It is outside the repository, so it can never be committed. There are three parts: **(a)** paste the block below, which creates the file with example values; **(b)** open it in the editor and change the *edit these* block; **(c)** load it.

Where to find the values:

- `SUB` (subscription ID): run `az account show --query id -o tsv` in Cloud Shell, or portal → search **Subscriptions** → copy the *Subscription ID*.
- `PREFIX`: a short lowercase name, e.g. `fddtprod`. It becomes part of every resource name, and some of those (registry, Key Vault, storage) must be unique in all of Azure.
- `APP_DOMAIN`: the address users will type, agreed with the client, e.g. `fddt.client.com`.
- If the client already created the resource group for you, set `RG` (in the derived block) to its exact name.

```
mkdir -p ~/fddt-deploy
cat > ~/fddt-deploy/deploy.env <<'EOF'
# ---------- edit these ----------
export SUB="<subscription-id>"
export LOC="uaenorth"
export PREFIX="fddtprod"                # lowercase letters/digits; must be globally unique for
ACR/KV/storage
export TIER="M"                         # S | M | L
export APP_DOMAIN="app.client-domain.com"
export SEED_ADMIN_EMAIL="platform-admin@operator-domain.com"
export ALERT_EMAIL="ops@operator-domain.com"
export OFFICE_CIDRS=""                  # e.g. "203.0.113.0/24 198.51.100.10/32" to restrict the UI; empty
= internet
export USE_NAT="no"                     # "yes" = NAT Gateway with a fixed outbound IP (~$33/month), only
if someone must allow-list it

# ---------- network ----------
export VNET_CIDR="10.20.0.0/16"
export SNET_ACA="10.20.0.0/23"
export SNET_PG="10.20.2.0/24"
export SNET_PE="10.20.3.0/24"
export SNET_BASTION="10.20.4.0/26"
export SNET_JUMP="10.20.5.0/28"

# ---------- derived names (no need to edit) ----------
export RG="rg-$PREFIX"
export LAW="$PREFIX-logs"
export VNET="$PREFIX-vnet"
export UAMI="$PREFIX-id"
export KV="$PREFIX-kv"
export ACR="${PREFIX}acr"
export ACR_SERVER="${PREFIX}acr.azurecr.io"
export PG="$PREFIX-pg"
export PG_HOST="$PREFIX-pg.postgres.database.azure.com"
export PG_DNS_ZONE="$PREFIX-pg.private.postgres.database.azure.com"
export PG_DB="fddt"
export PG_OWNER="fddtowner"
export REDIS="$PREFIX-redis"
export SA="${PREFIX}files"
export SA_CONTAINER="documents"
export DI="$PREFIX-docintel"
export AOAI="$PREFIX-openai"
export AOAI_DEPLOYMENT="gpt-4.1-mini"
export ENV="$PREFIX-env"

# ---------- workspace ----------
# Folder for generated files. (MSYS_NO_PATHCONV and pwd -W only matter if these commands are ever run from
# the bash of Git for Windows instead of Cloud Shell; there they stop /subscriptions/... IDs being
rewritten.)
export MSYS_NO_PATHCONV=1
mkdir -p ~/fddt-deploy/fddt-yaml
export WORK=$(cd ~/fddt-deploy && (pwd -W 2>/dev/null || pwd))

# ---------- tier sizing (from production-deployment.md section 2) ----------
case "$TIER" in
  S) export API_CPU=0.5 API_MEM=1Gi API_MIN=1 API_MAX=2
     export EXT_CPU=0.5 EXT_MEM=1Gi EXT_N=1
     export VIS_CPU=0.5 VIS_MEM=1Gi VIS_N=1
     export FOR_CPU=1   FOR_MEM=2Gi FOR_MIN=0 FOR_MAX=2     # forensics scales to zero when idle
     export WEB_MIN=1 WEB_MAX=2
```

```
    export PG_TIER=Burstable PG_SKU=Standard_B2s PG_STORAGE=64 PG_HA=Disabled
    export REDIS_SKU=Balanced_B0 REDIS_HA=Disabled
    export AOAI_TPM_K=100 ;;
  M) export API_CPU=1   API_MEM=2Gi API_MIN=1 API_MAX=4
    export EXT_CPU=1   EXT_MEM=2Gi EXT_N=1
    export VIS_CPU=1   VIS_MEM=2Gi VIS_N=1
    export FOR_CPU=2   FOR_MEM=4Gi FOR_MIN=0 FOR_MAX=4     # forensics scales to zero when idle
    export WEB_MIN=2 WEB_MAX=4
    export PG_TIER=GeneralPurpose PG_SKU=Standard_D2ds_v5 PG_STORAGE=128 PG_HA=Disabled   # HA: client
decision (0.1)
    export REDIS_SKU=Balanced_B0 REDIS_HA=Disabled
    export AOAI_TPM_K=150 ;;
  L) export API_CPU=1   API_MEM=2Gi API_MIN=3 API_MAX=6
    export EXT_CPU=1   EXT_MEM=2Gi EXT_N=2
    export VIS_CPU=1   VIS_MEM=2Gi VIS_N=4
    export FOR_CPU=2   FOR_MEM=4Gi FOR_MIN=1 FOR_MAX=10
    export WEB_MIN=2 WEB_MAX=6
    export PG_TIER=GeneralPurpose PG_SKU=Standard_D4ds_v5 PG_STORAGE=512 PG_HA=ZoneRedundant
    export REDIS_SKU=Balanced_B3 REDIS_HA=Enabled
    export AOAI_TPM_K=500 ;;
esac
EOF
echo "created ~/fddt-deploy/deploy.env"
```

**(b) Edit the values:** run `code ~/fddt-deploy/deploy.env`. The file opens in Cloud Shell's editor above the terminal. Change the lines in the *edit these* block (and `RG` if the client already made the resource group), save with **Ctrl + S**, and close the editor with **...** → **Close Editor**.

**(c) Load it:**

```
source ~/fddt-deploy/deploy.env
az account set --subscription "$SUB"
export VERSION=$(git rev-parse --short HEAD)    # image tag = the current commit
```

Now create the **re-load script** that brings everything back after Cloud Shell restarts (see 0.4). It asks Azure for every value, so it is safe to run at any step: values for resources that don't exist yet just stay empty.

```
cat > ~/fddt-deploy/reload.sh <<'EOF'
source ~/fddt-deploy/deploy.env
az account set --subscription "$SUB"
q() { az "$@" -o tsv 2>/dev/null; }
export LA_ID=$(q monitor log-analytics workspace show -g $RG -n $LAW --query customerId)
export LA_KEY=$(q monitor log-analytics workspace get-shared-keys -g $RG -n $LAW --query primarySharedKey)
export LAW_RES_ID=$(q monitor log-analytics workspace show -g $RG -n $LAW --query id)
export ACA_SUBNET_ID=$(q network vnet subnet show -g $RG --vnet-name $VNET -n snet-aca --query id)
export EGRESS_IP=$(q network public-ip show -g $RG -n $PREFIX-egress-ip --query ipAddress)
export UAMI_ID=$(q identity show -g $RG -n $UAMI --query id)
export UAMI_PRINCIPAL=$(q identity show -g $RG -n $UAMI --query principalId)
export KV_ID=$(q keyvault show -g $RG -n $KV --query id)
export PG_OWNER_PW=$(q keyvault secret show --vault-name $KV -n pg-owner-password --query value)
export PG_ID=$(q postgres flexible-server show -g $RG -n $PG --query id)
export REDIS_ID=$(q redisenterprise show -g $RG -n $REDIS --query id)
export REDIS_HOST=$(q redisenterprise show -g $RG -n $REDIS --query hostName)
export REDIS_PORT=$(q redisenterprise database show -g $RG --cluster-name $REDIS --query port)
export REDIS_KEY=$(q redisenterprise database list-keys -g $RG --cluster-name $REDIS --query primaryKey)
export SA_CONN=$(q storage account show-connection-string -g $RG -n $SA)
export DI_ENDPOINT=$(q cognitiveservices account show -g $RG -n $DI --query properties.endpoint)
export DI_KEY=$(q cognitiveservices account keys list -g $RG -n $DI --query key1)
export AOAI_ENDPOINT=$(q cognitiveservices account show -g $RG -n $AOAI --query properties.endpoint)
export AOAI_KEY=$(q cognitiveservices account keys list -g $RG -n $AOAI --query key1)
RL=$(q cognitiveservices account deployment show -g $RG -n $AOAI --deployment-name $AOAI_DEPLOYMENT \
     --query "properties.rateLimits[].[key,count,renewalPeriod]")
export AOAI_RPM=$(echo "$RL" | awk '$1=="request"{printf "%d", $2*60/$3}')
export AOAI_TPM=$(echo "$RL" | awk '$1=="token"{printf "%d", $2*60/$3}')
[ -n "$AOAI_RPM" ] && export AOAI_RPM_CAP=$(( AOAI_RPM * 80 / 100 )) AOAI_TPM_CAP=$(( AOAI_TPM * 80 / 100
))
export ENV_ID=$(q containerapp env show -g $RG -n $ENV --query id)
export ENV_DOMAIN=$(q containerapp env show -g $RG -n $ENV --query properties.defaultDomain)
export ENV_IP=$(q containerapp env show -g $RG -n $ENV --query properties.staticIp)
export WEB_FQDN=$(q containerapp show -g $RG -n fddt-web --query properties.configuration.ingress.fqdn)
export AG_ID=$(q monitor action-group show -g $RG -n $PREFIX-oncall --query id)
# Image tag: what is deployed now, otherwise the current commit
DEPLOYED=$(q containerapp show -g $RG -n fddt-api --query "properties.template.containers[0].image")
export VERSION=${DEPLOYED##*:}; [ -z "$DEPLOYED" ] && export VERSION=$(git rev-parse --short HEAD
2>/dev/null)
export BACKEND_IMAGE=$ACR_SERVER/fddt-backend:$VERSION WEB_IMAGE=$ACR_SERVER/fddt-web:$VERSION
[ -f ~/fddt-deploy/helpers.sh ] && source ~/fddt-deploy/helpers.sh
echo "Reloaded: RG=$RG TIER=$TIER VERSION=$VERSION ENV_IP=${ENV_IP:-<not yet>} AOAI
caps=${AOAI_RPM_CAP:-?}/${AOAI_TPM_CAP:-?}"
EOF
echo "created ~/fddt-deploy/reload.sh"
```

✅ **Verify:** `echo $RG $TIER $PG_SKU $VERSION $WORK` prints sensible values ( `$WORK` is `/home/<user>/fddt-deploy` ), and `az account show --query name -o tsv` is the client's subscription.

---

## Step 2 — Resource group and providers

📍 **Who:** you, in the client admin's session. Registering providers and creating a resource group need **subscription-level** rights (*Owner* or *Contributor* on the subscription). If the commands fail with *AuthorizationFailed*, the signed-in account lacks those rights: the client must use an account that has them. **Type in:** Cloud Shell. **Check in the portal:** search **Subscriptions** → the subscription → **Settings › Resource providers** → type `Microsoft.App` in the filter → *Status* = **Registered** (repeat for the others). Then search **Resource groups** → `rg-fddtprod` exists, in *UAE North*, and is empty.

```
for ns in Microsoft.App Microsoft.OperationalInsights Microsoft.ContainerRegistry Microsoft.DBforPostgreSQL
\
          Microsoft.Cache Microsoft.Storage Microsoft.CognitiveServices Microsoft.KeyVault
Microsoft.Network \
          Microsoft.ManagedIdentity Microsoft.Insights; do
  az provider register --namespace $ns
done
az group create -n $RG -l $LOC --tags app=fddt env=prod tier=$TIER
```

✅ **Verify:** `az provider show -n Microsoft.App --query registrationState` → `"Registered"` (may take a few minutes).

---

## Step 3 — Log Analytics

📍 **Who:** you · **Type in:** Cloud Shell · **Check in the portal:** `rg-fddtprod` → `fddtprod-logs` → **Overview**. You will run log queries later in **fddtprod-logs › Logs**: close the *Queries* pop-up, and switch the editor from *Simple mode* to **KQL mode** at the top right.

```
az monitor log-analytics workspace create -g $RG -n $LAW -l $LOC --retention-time 30
# Guard against a runaway log bill: at most 1 GB ingested per day (the app normally logs far less)
az monitor log-analytics workspace update -g $RG -n $LAW --quota 1
export LA_ID=$(az monitor log-analytics workspace show -g $RG -n $LAW --query customerId -o tsv)
export LA_KEY=$(az monitor log-analytics workspace get-shared-keys -g $RG -n $LAW --query primarySharedKey
-o tsv)
export LAW_RES_ID=$(az monitor log-analytics workspace show -g $RG -n $LAW --query id -o tsv)
```

✅ **Verify:** `echo $LA_ID` is a GUID.

---

## Step 4 — Virtual network, subnets, NSGs, static egress IP

📍 **Who:** you · **Type in:** Cloud Shell · **Check in the portal:**

- **fddtprod-vnet › Settings › Subnets**: four subnets with the prefixes from 2.1; the *Delegated to* column shows `Microsoft.App/environments` and `Microsoft.DBforPostgreSQL/flexibleServers`.
- **nsg-aca / nsg-pg / nsg-pe › Settings › Inbound security rules**: the rules from 2.4.
- Only if `USE_NAT="yes"`: **fddtprod-egress-ip › Overview**: *IP address* = the static outbound IP. Write it in the handover sheet.

```
# NSGs
az network nsg create -g $RG -n nsg-aca
az network nsg create -g $RG -n nsg-pg
az network nsg create -g $RG -n nsg-pe

# nsg-aca: public HTTPS/HTTP in (internet, or only the office ranges)
SRC=${OFFICE_CIDRS:-Internet}
az network nsg rule create -g $RG --nsg-name nsg-aca -n allow-https-in --priority 100 --direction Inbound \
  --access Allow --protocol Tcp --source-address-prefixes $SRC --destination-port-ranges 443
az network nsg rule create -g $RG --nsg-name nsg-aca -n allow-http-in --priority 110 --direction Inbound \
  --access Allow --protocol Tcp --source-address-prefixes $SRC --destination-port-ranges 80

# nsg-pg: only Container Apps and the jump subnet reach Postgres / PgBouncer
az network nsg rule create -g $RG --nsg-name nsg-pg -n allow-aca-pg --priority 100 --direction Inbound \
  --access Allow --protocol Tcp --source-address-prefixes $SNET_ACA --destination-port-ranges 5432 6432
az network nsg rule create -g $RG --nsg-name nsg-pg -n allow-jump-pg --priority 110 --direction Inbound \
  --access Allow --protocol Tcp --source-address-prefixes $SNET_JUMP --destination-port-ranges 5432 6432
az network nsg rule create -g $RG --nsg-name nsg-pg -n allow-pg-subnet --priority 120 --direction Inbound \
  --access Allow --protocol '*' --source-address-prefixes $SNET_PG --destination-port-ranges '*'
az network nsg rule create -g $RG --nsg-name nsg-pg -n deny-vnet --priority 4000 --direction Inbound \
  --access Deny --protocol '*' --source-address-prefixes VirtualNetwork --destination-port-ranges '*'

# nsg-pe: only Container Apps and the jump subnet reach Redis
az network nsg rule create -g $RG --nsg-name nsg-pe -n allow-aca-redis --priority 100 --direction Inbound \
  --access Allow --protocol Tcp --source-address-prefixes $SNET_ACA --destination-port-ranges 10000
az network nsg rule create -g $RG --nsg-name nsg-pe -n allow-jump-redis --priority 110 --direction Inbound
\
  --access Allow --protocol Tcp --source-address-prefixes $SNET_JUMP --destination-port-ranges 10000
az network nsg rule create -g $RG --nsg-name nsg-pe -n deny-vnet --priority 4000 --direction Inbound \
  --access Deny --protocol '*' --source-address-prefixes VirtualNetwork --destination-port-ranges '*'

# Optional static egress IP (NAT Gateway): every outbound call from the apps leaves from this one IP
NAT_ARGS=""
if [ "$USE_NAT" = "yes" ]; then
  az network public-ip create -g $RG -n $PREFIX-egress-ip --sku Standard --allocation-method Static
  az network nat gateway create -g $RG -n $PREFIX-nat --public-ip-addresses $PREFIX-egress-ip --idle-
timeout 10
  NAT_ARGS="--nat-gateway $PREFIX-nat"
fi

# VNet + subnets
az network vnet create -g $RG -n $VNET -l $LOC --address-prefixes $VNET_CIDR
az network vnet subnet create -g $RG --vnet-name $VNET -n snet-aca --address-prefixes $SNET_ACA \
  --delegations Microsoft.App/environments --network-security-group nsg-aca $NAT_ARGS
az network vnet subnet create -g $RG --vnet-name $VNET -n snet-pg --address-prefixes $SNET_PG \
  --delegations Microsoft.DBforPostgreSQL/flexibleServers --network-security-group nsg-pg
az network vnet subnet create -g $RG --vnet-name $VNET -n snet-pe --address-prefixes $SNET_PE \
  --network-security-group nsg-pe --private-endpoint-network-policies Enabled
az network vnet subnet create -g $RG --vnet-name $VNET -n snet-jump --address-prefixes $SNET_JUMP

export ACA_SUBNET_ID=$(az network vnet subnet show -g $RG --vnet-name $VNET -n snet-aca --query id -o tsv)
[ "$USE_NAT" = "yes" ] && export EGRESS_IP=$(az network public-ip show -g $RG -n $PREFIX-egress-ip --query
ipAddress -o tsv)
echo "Egress IP: ${EGRESS_IP:-none (no NAT Gateway)}"
```

> The NAT Gateway (~$33/month + data) is what gives a **guaranteed** static outbound IP. It is off by default (`USE_NAT="no"`). It can be added later: create the two resources above, then `az network vnet subnet update -g $RG --vnet-name $VNET -n snet-aca --nat-gateway $PREFIX-nat`.

✅ **Verify:** `az network vnet subnet list -g $RG --vnet-name $VNET -o table` shows four subnets with the right prefixes; note `$EGRESS_IP` in the handover sheet.

---

## Step 5 — Managed identity and Key Vault

> 📍 **Who:** you · **Type in:** Cloud Shell · **Check in the portal:**
>
> - **fddtprod-id › Azure role assignments**: *Key Vault Secrets User* (AcrPull appears after step 6).
> - **fddtprod-kv › Access control (IAM) › Role assignments** tab: the identity, and you as *Key Vault Secrets Officer*.
> - Later, the secrets themselves: **fddtprod-kv › Objects › Secrets**.

One **user-assigned managed identity** is used by every app to pull images and read secrets.

```
az identity create -g $RG -n $UAMI
export UAMI_ID=$(az identity show -g $RG -n $UAMI --query id -o tsv)
export UAMI_PRINCIPAL=$(az identity show -g $RG -n $UAMI --query principalId -o tsv)

az keyvault create -g $RG -n $KV -l $LOC --enable-rbac-authorization true \
  --enable-purge-protection true --retention-days 90
export KV_ID=$(az keyvault show -g $RG -n $KV --query id -o tsv)

# The identity reads secrets; you (the operator) write them
az role assignment create --assignee-object-id $UAMI_PRINCIPAL --assignee-principal-type ServicePrincipal \
  --role "Key Vault Secrets User" --scope $KV_ID
az role assignment create --assignee $(az ad signed-in-user show --query id -o tsv) \
  --role "Key Vault Secrets Officer" --scope $KV_ID
```

✅ **Verify:** `az role assignment list --scope $KV_ID -o table` shows both assignments. Role assignments take **1–5 minutes** to apply. If a later `secret set` fails with *Forbidden*, wait and retry.

---

## Step 6 — Container Registry and images

> 📍 **Who:** you · **Type in:** Cloud Shell, **in** `~/FDDT` (`./backend` and `./frontend` are uploaded and built in Azure: ~5–10 minutes each, with the build log streaming in the terminal; if Cloud Shell disconnects, the build carries on) · **Check in the portal:**
>
> - **fddtprodacr › Services › Repositories**: `fddt-backend` and `fddt-web` → click one → the tag = `$VERSION`.
> - If a build fails: **fddtprodacr › Services › Tasks › Runs** tab → click the run → full log.

```
az acr create -g $RG -n $ACR --sku Basic --admin-enabled false
az role assignment create --assignee-object-id $UAMI_PRINCIPAL --assignee-principal-type ServicePrincipal \
  --role AcrPull --scope $(az acr show -n $ACR --query id -o tsv)

# Build in Azure (no local Docker needed). Run from the repo root.
az acr build -r $ACR -t fddt-backend:$VERSION ./backend
az acr build -r $ACR -t fddt-web:$VERSION ./frontend

export BACKEND_IMAGE=$ACR_SERVER/fddt-backend:$VERSION
export WEB_IMAGE=$ACR_SERVER/fddt-web:$VERSION
```

- One backend image runs the API, all three workers, beat and the migration job. Only the command differs.
- The web image is built with the default `VITE_API_BASE_URL=/api`. Do not change it.

✅ **Verify:** `az acr repository show-tags -n $ACR --repository fddt-backend -o table` lists `$VERSION`.

---

## Step 7 — PostgreSQL Flexible Server

📍 **Who:** you · **Type in:** Cloud Shell. The create command runs **15–20 minutes**: press Shift now and then so Cloud Shell does not time out. If it disconnects, reconnect, run `reload.sh`, wait until the server shows **Ready** in the portal, then run the commands **after** `flexible-server create`. The `PG_OWNER_PW` line must not be re-run: if it was lost, see the note under the code. **Check in the portal:**

- **fddtprod-pg › Overview**: *Status* **Ready**; *Server name* = `$PG_HOST`.
- **fddtprod-pg › Settings › Networking**: *Private access (VNet Integration)*, subnet `snet-pg`.
- **fddtprod-pg › Settings › High availability** (tiers M/L): *Zone redundant*, *Healthy*.
- **fddtprod-pg › Settings › Server parameters**: search `pgbouncer.enabled` → **ON** (tiers M/L).
- **fddtprod-pg › Settings › Databases**: `fddt` is listed.

```
export PG_OWNER_PW=$(openssl rand -base64 48 | tr -dc 'A-Za-z0-9' | head -c 32)   # alphanumeric: no URL-encoding needed
# Save it in Key Vault BEFORE the long create, so a disconnect can't lose it (reload.sh reads it back)
az keyvault secret set --vault-name $KV -n pg-owner-password --value "$PG_OWNER_PW" -o none

HA_ARGS=""; [ "$PG_HA" = "ZoneRedundant" ] && HA_ARGS="--high-availability ZoneRedundant"
az postgres flexible-server create -g $RG -n $PG -l $LOC --version 16 \
  --tier $PG_TIER --sku-name $PG_SKU --storage-size $PG_STORAGE --storage-auto-grow Enabled \
  $HA_ARGS --backup-retention 35 \
  --vnet $VNET --subnet snet-pg --private-dns-zone $PG_DNS_ZONE \
  --admin-user $PG_OWNER --admin-password "$PG_OWNER_PW" --yes

az postgres flexible-server db create -g $RG -s $PG -d $PG_DB

# Built-in PgBouncer: General Purpose / Memory Optimized only (tiers M and L). NOT on Burstable (tier S).
if [ "$TIER" != "S" ]; then
  az postgres flexible-server parameter set -g $RG -s $PG -n pgbouncer.enabled --value true
  az postgres flexible-server parameter set -g $RG -s $PG -n metrics.pgbouncer_diagnostics --value on
fi
# Slow-query log (≥ 1 s) for capacity planning
az postgres flexible-server parameter set -g $RG -s $PG -n log_min_duration_statement --value 1000
```

> If Cloud Shell disconnected during the create, `reload.sh` restores `PG_OWNER_PW` from Key Vault, so the next steps still work. If you ever need to set a new owner password: `az postgres flexible-server update -g $RG -n $PG --admin-password "$PG_OWNER_PW"` after generating and saving a new one as above.

**Settings and why**

| Setting | Value | Why |
|---|---|---|
| Version | 16 | What the app and migrations are tested on |
| Admin user `fddt owner` | **Owner** role the migrations run as | Has CREATEROLE (through `azure_pg_admin`) but is not a superuser. That is all the migrations need: they create `fddt_app` and `fddt_platform` themselves, without BYPASSRLS |
| Extensions | none | The migrations need no extension, so nothing to allow-list in `azure.extensions` |
| `require_secure_transport` | `on` (default) | Leave it. Every URL ends in `?sslmode=require` |
| PgBouncer | port **6432**, transaction mode, `default_pool_size` 50 per (user, db) | Tenant context is transaction-local (`set_config(..., true)`), proven safe by `tests/test_pgbouncer_rls.py` |
| `max_connections` | the SKU default (B2s ≈ 429, D2ds_v5 ≈ 859) | PgBouncer uses at most 2 roles × 50 = 100 server connections + owner/admin. Plenty |
| HA | **Off on S and M** (launch), zone-redundant on L | With HA: automatic failover in 60–120 s. Without: 99.9 %, a zone outage means a point-in-time restore (~1 hour). Turn it on later with `az postgres flexible-server update -g $RG -n $PG --high-availability ZoneRedundant` (doubles the database cost) |
| Backups | 35 days PITR | Restore target for a bad migration |

✅ **Verify:** `az postgres flexible-server show -g $RG -n $PG --query "{state:state,ha:highAvailability.state,fqdn:fullyQualifiedDomainName}"` → `Ready`, HA `Healthy` (M/L), fqdn = `$PG_HOST`. Tiers M/L: `az postgres flexible-server parameter show -g $RG -s $PG -n pgbouncer.enabled --query value` → `"true"`.

---

## Step 8 — Azure Managed Redis

> 📍 **Who:** you · **Type in:** Cloud Shell (creation takes **15–30 minutes**) · **Check in the portal:**
>
> - **fddtprod-redis › Overview**: *Status* **Running**; the *Host name* ends in `.redis.azure.net`.
> - **fddtprod-redis › Settings › Advanced settings** (or the *Properties* on Overview): *Clustering policy* = **Non-clustered**.
> - **fddtprod-redis › Settings › Authentication**: *Access keys authentication* enabled.
> - **fddtprod-redis › Settings › Private endpoint** (or *Networking*): `fddtprod-redis-pe`, *Approved*.
> - **privatelink.redis.azure.net › DNS Management › Recordsets**: an A record pointing to `10.20.3.x`.

Celery's Redis transport and the fair-share priority lists use multi-key commands, so the cache **must** use the **Non-Clustered** policy. **The clustering policy cannot be changed after creation.**

```
az redisenterprise create -g $RG --cluster-name $REDIS -l $LOC --sku $REDIS_SKU \
  --clustering-policy NoCluster --high-availability $REDIS_HA \
  --minimum-tls-version 1.2 --public-network-access Disabled --access-keys-authentication Enabled
```

> If your CLI rejects `NoCluster` (older versions only accept `OSSCluster` / `EnterpriseCluster`), create it in the **portal**: *Azure Managed Redis → Create → Balanced B0 (or B3) → Advanced → Clustering policy: **Non-clustered**, Access keys authentication: Enabled → Networking: Private endpoint*. Then continue below.

Private endpoint + private DNS (so the apps resolve the hostname to a `10.20.3.x` address):

```
export REDIS_ID=$(az redisenterprise show -g $RG -n $REDIS --query id -o tsv)
az network private-dns zone create -g $RG -n privatelink.redis.azure.net
az network private-dns link vnet create -g $RG -z privatelink.redis.azure.net -n link-$VNET -v $VNET -e
false
az network private-endpoint create -g $RG -n $REDIS-pe --vnet-name $VNET --subnet snet-pe \
  --private-connection-resource-id $REDIS_ID --group-id redisEnterprise --connection-name redis
az network private-endpoint dns-zone-group create -g $RG --endpoint-name $REDIS-pe -n default \
  --private-dns-zone privatelink.redis.azure.net --zone-name redis

export REDIS_HOST=$(az redisenterprise show -g $RG -n $REDIS --query hostName -o tsv)
export REDIS_PORT=$(az redisenterprise database show -g $RG --cluster-name $REDIS --query port -o tsv)   #
10000
export REDIS_KEY=$(az redisenterprise database list-keys -g $RG --cluster-name $REDIS --query primaryKey -o
tsv)
```

### Settings and why

| Setting | Value | Why |
|---|---|---|
| Clustering policy | **Non-clustered** | Multi-key commands across `<queue>`, `<queue>:1…9` |
| Port / TLS | **10000**, TLS only | Use `rediss://` |
| Database | **0 only** | **Both** `CELERY_BROKER_URL` and `CELERY_RESULT_BACKEND` point to `/0`. Keys are prefixed, so sharing is safe |
| `ssl_cert_reqs=required` in the URL | **mandatory** | Celery 5.4 refuses a `rediss://` URL without it ("A rediss:// URL must have parameter ssl_cert_reqs"). `required` is accepted by both Celery and redis-py |
| Eviction | leave the default (no eviction of broker keys) | Queued tasks must never be evicted |
| HA | **Off on S and M**, on for L | Redis holds no durable data. If it restarts and loses queued tasks, the stuck-document job (every 5 min) queues the affected documents again. Whether HA can be switched on later depends on the SKU: check *Settings › High availability* in the portal; otherwise it means a new cache (a short maintenance window, nothing to migrate) |

✅ **Verify:** `az redisenterprise show -g $RG -n $REDIS --query "{state:provisioningState,res:resourceState}"` → `Succeeded` / `Running`; `echo $REDIS_HOST:$REDIS_PORT`.

---

## Step 9 — Blob Storage

📍 **Who:** you · **Type in:** Cloud Shell · **Check in the portal** ( `fddtprodfiles` ):

- **Data storage › Containers**: `documents` , *Anonymous access level* = **Private**.
- **Data management › Data protection**: soft delete for blobs and containers (14 days) and versioning ticked.
- **Data management › Lifecycle management**: rule `cool-after-180d` .
- **Security + networking › Networking**: *Public network access* = **Enabled from all networks**. This is intended: see the note below.

```
az storage account create -g $RG -n $SA -l $LOC --sku Standard_ZRS --kind StorageV2 --access-tier Hot \
  --min-tls-version TLS1_2 --allow-blob-public-access false --https-only true \
  --allow-shared-key-access true --public-network-access Enabled --default-action Allow
export SA_CONN=$(az storage account show-connection-string -g $RG -n $SA -o tsv)

az storage container create -n $SA_CONTAINER --connection-string "$SA_CONN" --public-access off

# Defence in depth: soft delete + versioning (the app never deletes or overwrites)
az storage account blob-service-properties update -g $RG --account-name $SA \
  --enable-delete-retention true --delete-retention-days 14 \
  --enable-container-delete-retention true --container-delete-retention-days 14 \
  --enable-versioning true

# Cost: move files untouched for 180 days to Cool
cat > $WORK/lifecycle.json <<'EOF'
{"rules":[{"enabled":true,"name":"cool-after-180d","type":"Lifecycle",
  "definition":{"actions":{"baseBlob":{"tierToCool":{"daysAfterModificationGreaterThan":180}}},
                "filters":{"blobTypes":["blockBlob"]}}}]}
EOF
az storage account management-policy create -g $RG --account-name $SA --policy @$WORK/lifecycle.json
```

**Why the public endpoint stays on:** the case viewer in the browser and Document Intelligence both fetch files by **SAS URL**. Anonymous access is off, so a file can only be read with a valid, short-lived SAS. The code signs SAS URLs with the **account key**, which is why shared-key access stays enabled. CORS is added in step 17 once the app hostname exists.

✅ **Verify:** `az storage container show -n $SA_CONTAINER --connection-string "$SA_CONN" --query properties.publicAccess` → `null` (private).

---

## Step 10 — Document Intelligence and Azure OpenAI

📍 **Who:** you · **Type in:** Cloud Shell · **Check in the portal:**

- **fddtprod-docintel › Resource Management › Keys and Endpoint**: an endpoint and two keys.
- **fddtprod-openai › Resource Management › Keys and Endpoint**: the same for OpenAI.
- The model and its limits: **fddtprod-openai › Overview** → button **Go to Azure AI Foundry portal** → left menu **Deployments** → `gpt-4.1-mini` → *Tokens per Minute Rate Limit* and *Rate limit (requests per minute)*. These are the numbers the commands below read.
- The deployment is sized to the subscription's **free** gpt-4.1-mini Global Standard capacity (0.3). If the create fails with *InsufficientQuota*, the free capacity is 0: see 0.3.

```
# Document Intelligence (OCR / layout)
az cognitiveservices account create -g $RG -n $DI -l $LOC --kind FormRecognizer --sku S0 \
  --custom-domain $DI --yes
export DI_ENDPOINT=$(az cognitiveservices account show -g $RG -n $DI --query properties.endpoint -o tsv)
export DI_KEY=$(az cognitiveservices account keys list -g $RG -n $DI --query key1 -o tsv)

# Azure OpenAI resource (the resource's pricing tier is always S0)
az cognitiveservices account create -g $RG -n $AOAI -l $LOC --kind OpenAI --sku S0 \
  --custom-domain $AOAI --yes

# Free gpt-4.1-mini Global Standard capacity in the region (thousands of TPM): no quota request, we use it
all
read AOAI_USED AOAI_LIMIT <<< "$(az cognitiveservices usage list -l $LOC \
  --query "[?name.value=='OpenAI.GlobalStandard.gpt-4.1-mini'] | [0].[currentValue, limit]" -o tsv)"
export AOAI_CAPACITY_K=$(( ${AOAI_LIMIT%.*} - ${AOAI_USED%.*} ))
echo "free: ${AOAI_CAPACITY_K}k TPM (tier $TIER was sized for ${AOAI_TPM_K}k)"
# If the client runs other gpt-4.1-mini apps in this region and wants to keep some capacity for them,
# set a smaller number here by hand, e.g.: export AOAI_CAPACITY_K=150

# The deployment: model gpt-4.1-mini, deployment type Global Standard
az cognitiveservices account deployment create -g $RG -n $AOAI --deployment-name $AOAI_DEPLOYMENT \
  --model-name gpt-4.1-mini --model-version 2025-04-14 --model-format OpenAI \
  --sku-name GlobalStandard --sku-capacity $AOAI_CAPACITY_K
export AOAI_ENDPOINT=$(az cognitiveservices account show -g $RG -n $AOAI --query properties.endpoint -o
tsv)
export AOAI_KEY=$(az cognitiveservices account keys list -g $RG -n $AOAI --query key1 -o tsv)

# Read the granted limits (each has its own renewal period in seconds) and set the app caps to 80 %
RL=$(az cognitiveservices account deployment show -g $RG -n $AOAI --deployment-name $AOAI_DEPLOYMENT \
     --query "properties.rateLimits[].[key,count,renewalPeriod]" -o tsv)
echo "$RL"
export AOAI_RPM=$(echo "$RL" | awk '$1=="request"{printf "%d", $2*60/$3}')
export AOAI_TPM=$(echo "$RL" | awk '$1=="token"{printf "%d", $2*60/$3}')
export AOAI_RPM_CAP=$(( AOAI_RPM * 80 / 100 ))
export AOAI_TPM_CAP=$(( AOAI_TPM * 80 / 100 ))
echo "quota: $AOAI_RPM RPM, $AOAI_TPM TPM  ->  app caps: $AOAI_RPM_CAP RPM, $AOAI_TPM_CAP TPM"
```

The last line must show real numbers. With 150k TPM free it shows about `150 RPM, 150000 TPM -> 120, 120000`. If it shows zeros, compare with the limits on the deployment's page in the Foundry portal (see the 📍 box) and set the two `AOAI_*` values by hand. Write the TPM in the handover sheet: it decides the platform's throughput.

**Why 80 %:** the app's global limiter (Redis sliding windows, shared by every worker) keeps traffic under the quota so Azure never throttles. At 85 % a load test saw one 429 in 123 calls, because two independent sliding windows can race. At 80 % there were none.

- The **deployment name must contain** `gpt-4.1` (or `gpt-4o`). The app checks that the model is vision-capable by name.
- Document Intelligence uses pricing tier **S0** (`--sku S0`), never F0 (see 0.1). S0 allows **15 requests/s**; the app caps at **10** (`AZURE_DOCUMENT_INTELLIGENCE_MAX_CALLS_PER_SECOND`).
- The model deployment is **Global Standard** (`--sku-name GlobalStandard`), pay per token. The account's own `--sku S0` is just the Azure OpenAI resource tier; it is always S0.

✅ **Verify:** `curl -s -o /dev/null -w "%{http_code}\n" "${AOAI_ENDPOINT%/}/openai/deployments/$AOAI_DEPLOYMENT/chat/completions?api-version=2024-10-21" -H "api-key: $AOAI_KEY" -H "Content-Type: application/json" -d '{"messages":[{"role":"user","content":"ping"}],"max_tokens":5}'` → `200`.

## Step 11 — Secrets into Key Vault

📍 **Who:** you · **Type in:** Cloud Shell · **Check in the portal: fddtprod-kv › Objects › Secrets**: 11 secrets. To check one, click its name → the current version → **Show Secret Value**. Have your **team password manager open**: the last command prints the first admin's password once.

```
export JWT_SECRET=$(openssl rand -base64 96 | tr -dc 'A-Za-z0-9' | head -c 64)
export APP_DB_PW=$(openssl rand -base64 48 | tr -dc 'A-Za-z0-9' | head -c 32)
export PLATFORM_DB_PW=$(openssl rand -base64 48 | tr -dc 'A-Za-z0-9' | head -c 32)
export SEED_ADMIN_PW=$(openssl rand -base64 36 | tr -dc 'A-Za-z0-9' | head -c 20)
export REDIS_KEY_ENC=$(printf '%s' "$REDIS_KEY" | sed -e 's/%/%25/g' -e 's/+/%2B/g' -e 's#/#%2F#g' -e
's/=/%3D/g')   # URL-encode

kvset() { az keyvault secret set --vault-name $KV -n "$1" --value "$2" -o none && echo "set $1"; }
kvset jwt-secret            "$JWT_SECRET"
kvset owner-db-url          "postgresql+psycopg2://$PG_OWNER:$PG_OWNER_PW@$PG_HOST:5432/$PG_DB?
sslmode=require"
kvset app-db-password       "$APP_DB_PW"
kvset platform-db-password  "$PLATFORM_DB_PW"
kvset redis-url             "rediss://:$REDIS_KEY_ENC@$REDIS_HOST:$REDIS_PORT/0?ssl_cert_reqs=required"
kvset redis-key             "$REDIS_KEY"
kvset storage-conn          "$SA_CONN"
kvset docintel-key          "$DI_KEY"
kvset openai-key            "$AOAI_KEY"
kvset seed-admin-password   "$SEED_ADMIN_PW"
echo "First platform admin: $SEED_ADMIN_EMAIL / $SEED_ADMIN_PW   <- store in the team password manager NOW"
```

| Secret | Used by | Notes |
|---|---|---|
| `jwt-secret` | API | Rotating it signs every user out |
| `owner-db-url` | all backend apps | Migration job connects with it; the apps take host/port/db from it and connect as the app roles |
| `app-db-password`, `platform-db-password` | all backend apps | The migration creates `fddt_app` / `fddt_platform` **with these passwords**, so set them before the first migration |
| `redis-url` | all backend apps | Key is URL-encoded; `/0`; `ssl_cert_reqs=required` |
| `redis-key` | KEDA scale rules | Raw key |
| `storage-conn`, `docintel-key`, `openai-key` | API + workers | |
| `seed-admin-password` | migration job only | `seed.py` **resets** the admin's password to this value on every run |

✅ **Verify:** `az keyvault secret list --vault-name $KV --query "[].name" -o tsv | sort` lists 11 names (the 10 above + `pg-owner-password` from step 7).

# Step 12 — Container Apps environment and shared YAML helpers

📍 **Who:** you · **Type in:** Cloud Shell (the environment takes ~5–10 minutes) · **Check in the portal:**

- **fddtprod-env › Overview**: *Static IP* (the **inbound public IP**) and *Default domain*. Copy both into the handover sheet.
- **fddtprod-env › Settings › Workload profiles**: *Consumption*.
- **fddtprod-env › Settings › Networking**: VNet `fddtprod-vnet`, subnet `snet-aca`.

## 12.1 Environment

```
ZR=""; [ "$TIER" != "S" ] && ZR="--zone-redundant"
az containerapp env create -g $RG -n $ENV -l $LOC --enable-workload-profiles \
  --infrastructure-subnet-resource-id $ACA_SUBNET_ID --internal-only false \
  --logs-destination log-analytics --logs-workspace-id $LA_ID --logs-workspace-key $LA_KEY $ZR

export ENV_ID=$(az containerapp env show -g $RG -n $ENV --query id -o tsv)
export ENV_DOMAIN=$(az containerapp env show -g $RG -n $ENV --query properties.defaultDomain -o tsv)
export ENV_IP=$(az containerapp env show -g $RG -n $ENV --query properties.staticIp -o tsv)
echo "Inbound IP: $ENV_IP   default domain: $ENV_DOMAIN"
```

- `--internal-only false` makes the environment **external**: it gets one public static IP, but only apps with `external: true` ingress (just `fddt-web`) are reachable on it.
- All apps run on the **Consumption** workload profile (max 4 vCPU / 8 GiB per replica), which covers every tier.

## 12.2 Shared YAML helpers

Every backend app gets the same secrets and environment variables, so they are generated by shell functions. This block saves them to `~/fddt-deploy/helpers.sh` (so `reload.sh` brings them back) and loads them.

```
cat > ~/fddt-deploy/helpers.sh <<'HELPERS'
export KV_URL=https://$KV.vault.azure.net/secrets
SECRETS="jwt-secret owner-db-url app-db-password platform-db-password redis-url redis-key storage-conn
docintel-key openai-key seed-admin-password"

secrets_yaml() {
  for s in $SECRETS; do
    printf '      - name: %s\n        keyVaultUrl: %s/%s\n        identity: %s\n' "$s" "$KV_URL" "$s"
"$UAMI_ID"
  done
}
env_v() { printf '          - name: %s\n            value: "%s"\n' "$1" "$2"; }
env_s() { printf '          - name: %s\n            secretRef: %s\n' "$1" "$2"; }

backend_env() {
  env_v ENVIRONMENT production
  env_s DATABASE_URL owner-db-url
  env_s DATABASE_APP_PASSWORD app-db-password
  env_s DATABASE_PLATFORM_PASSWORD platform-db-password
  if [ "$TIER" = "S" ]; then          # Burstable: no PgBouncer, small direct pools
    env_v DB_POOL_SIZE 3
    env_v DB_MAX_OVERFLOW 5
  else                                # built-in PgBouncer, no per-process pool
    env_v DATABASE_POOLER_HOST "$PG_HOST"
    env_v DATABASE_POOLER_PORT 6432
  fi
  env_s CELERY_BROKER_URL redis-url
  env_s CELERY_RESULT_BACKEND redis-url
  env_s JWT_SECRET_KEY jwt-secret
  env_s AZURE_STORAGE_CONNECTION_STRING storage-conn
  env_v AZURE_STORAGE_CONTAINER_NAME "$SA_CONTAINER"
  env_v AZURE_DOCUMENT_INTELLIGENCE_ENDPOINT "$DI_ENDPOINT"
  env_s AZURE_DOCUMENT_INTELLIGENCE_KEY docintel-key
  env_v AZURE_DOCUMENT_INTELLIGENCE_MAX_CALLS_PER_SECOND 10
  env_v AZURE_OPENAI_ENDPOINT "$AOAI_ENDPOINT"
  env_s AZURE_OPENAI_KEY openai-key
  env_v AZURE_OPENAI_DEPLOYMENT_NAME "$AOAI_DEPLOYMENT"
  env_v AZURE_OPENAI_MAX_REQUESTS_PER_MINUTE "$AOAI_RPM_CAP"
  env_v AZURE_OPENAI_MAX_TOKENS_PER_MINUTE "$AOAI_TPM_CAP"
  env_v DEFAULT_MAX_FILE_SIZE_MB 10
  env_v DEFAULT_MAX_ZIP_SIZE_MB 300
  env_v EXTRACTION_WORKER_CONCURRENCY 4
  env_v VISION_WORKER_CONCURRENCY 4
  env_v FORENSICS_WORKER_CONCURRENCY 0
  env_v FAIR_SHARE_BUCKET_SIZE 5
  env_v QUEUE_ALERT_OLDEST_WAITING_SECONDS 300
  env_v USAGE_RECONCILIATION_HOUR_UTC 2
}

# Common head of every backend app / job YAML (identity, environment, registry, Key Vault secrets)
yaml_head() {
cat <<EOF
location: $LOC
identity:
  type: UserAssigned
  userAssignedIdentities:
    $UAMI_ID: {}
properties:
  environmentId: $ENV_ID
  workloadProfileName: Consumption
```

```
  configuration:
    registries:
      - server: $ACR_SERVER
        identity: $UAMI_ID
    secrets:
$(secrets_yaml)
EOF
}
HELPERS
source ~/fddt-deploy/helpers.sh
```

> The generated YAML files contain **no secret values**, only Key Vault references. They are safe to keep, but the project keeps IaC out of the repo for now, so they stay in `~/fddt-deploy/fddt-yaml` on your PC.

✅ **Verify:** `yaml_head | head -20` and `backend_env | head -12` print well-indented YAML with your values; `echo $ENV_IP` is a public IP.

---

## Step 13 — Database migration job

> 📍 **Who:** you · **Type in:** Cloud Shell · **Check in the portal: fddt-migrate › Execution history** → the newest row's *Status* is **Succeeded** (click **Refresh** at the top; it takes 1–3 minutes) → click the execution → **Console logs** (opens a log query; logs arrive 2–5 minutes late).

Runs `alembic upgrade head`, creates/refreshes the first platform admin (`seed.py`), and prints the revision. It runs **once per release, before the apps are updated**, and never on API start (several API replicas would race).

```
export MIGRATE_CMD="alembic upgrade head && python seed.py && alembic current"

{ yaml_head; cat <<EOF
    triggerType: Manual
    replicaTimeout: 1800
    replicaRetryLimit: 0
    manualTriggerConfig:
      parallelism: 1
      replicaCompletionCount: 1
  template:
    containers:
      - name: migrate
        image: $BACKEND_IMAGE
        command: ["sh", "-c"]
        args: ["$MIGRATE_CMD"]
        resources:
          cpu: 0.5
          memory: 1Gi
        env:
$(backend_env)
$(env_v SEED_ADMIN_EMAIL "$SEED_ADMIN_EMAIL")
$(env_s SEED_ADMIN_PASSWORD seed-admin-password)
EOF
} > $WORK/fddt-yaml/migrate.yaml

az containerapp job create -g $RG -n fddt-migrate --yaml $WORK/fddt-yaml/migrate.yaml
az containerapp job start -g $RG -n fddt-migrate
# Poll until Succeeded (typically 1–2 minutes)
az containerapp job execution list -g $RG -n fddt-migrate \
  --query "[0].{name:name,status:properties.status,start:properties.startTime}" -o table
```

✅ **Verify:**

- The execution status is **Succeeded**.
- Logs (portal → *fddt-migrate* → *Execution history* → *Console logs*, or the query below; logs arrive 2–5 minutes late) end with `f3b7d1e8a4c5 (head)` or a later head.

```
az monitor log-analytics query -w $LA_ID --analytics-query \
 "ContainerAppConsoleLogs_CL | where ContainerGroupName_s startswith 'fddt-migrate' | project
TimeGenerated, Log_s | order by TimeGenerated asc" -o table
```

If it fails: *password authentication failed* means the owner URL is wrong. *could not translate host name* means the private DNS zone is not linked (step 7). *permission denied to create role* means the server's admin user is not the one in `owner-db-url`.

---

## Step 14 — API ( `fddt-api` )

📍 **Who:** you · **Type in:** Cloud Shell · **Check in the portal** ( `fddt-api` ):

- **Application › Revisions and replicas**: the active revision shows *Running status* **Running** and *Health* **Healthy**, with `API_MIN` replicas.
- **Monitoring › Log stream**: live output; you should see `Uvicorn running on http://0.0.0.0:8000` .
- **Application › Containers → Environment variables** tab: the variables from step 12. Secrets show as *Reference a secret*.
- **Settings › Secrets**: 10 secrets, each *Key Vault reference*.
- **Networking › Ingress**: *Accepting traffic from* = **Limited to Container Apps Environment**; *Insecure connections* = **Allowed**; *Target port* 8000.
- **Application › Scale**: min/max replicas and the `http-concurrency` rule.

Internal ingress only. `allowInsecure: true` lets nginx in `fddt-web` call it as `http://fddt-api` inside the environment. The traffic never leaves the environment.

```
{ yaml_head; cat <<EOF
    activeRevisionsMode: Single
    ingress:
      external: false
      targetPort: 8000
      transport: http
      allowInsecure: true
  template:
    terminationGracePeriodSeconds: 60
    containers:
      - name: api
        image: $BACKEND_IMAGE
        command: ["uvicorn"]
        args: ["app.main:app", "--host", "0.0.0.0", "--port", "8000", "--workers", "2",
               "--proxy-headers", "--forwarded-allow-ips", "*", "--timeout-keep-alive", "75"]
        resources:
          cpu: $API_CPU
          memory: $API_MEM
        env:
$(backend_env)
        probes:
          - type: Startup
            httpGet: { path: /health, port: 8000 }
            periodSeconds: 5
            failureThreshold: 10
          - type: Liveness
            httpGet: { path: /health, port: 8000 }
            periodSeconds: 15
            failureThreshold: 3
          - type: Readiness
            httpGet: { path: /health, port: 8000 }
            periodSeconds: 10
            failureThreshold: 3
    scale:
      minReplicas: $API_MIN
      maxReplicas: $API_MAX
      rules:
        - name: http-concurrency
          http:
            metadata:
              concurrentRequests: "30"
EOF
} > $WORK/fddt-yaml/api.yaml

az containerapp create -g $RG -n fddt-api --yaml $WORK/fddt-yaml/api.yaml
```

- **Two uvicorn workers per replica** (1 vCPU). The API scales **out** on HTTP concurrency (30 concurrent requests per replica) between `API_MIN` and `API_MAX`.
- `/health` does not touch the database or Redis, so a database failover never restarts the API.

✅ **Verify:** `az containerapp revision list -g $RG -n fddt-api --query "[].{rev:name,health:properties.healthState,running:properties.runningState}" -o table` → `Healthy` / `Running`. If it shows a Key Vault error, the identity's *Key Vault Secrets User* role has not applied yet. Wait 5 minutes and run `az containerapp update -g $RG -n fddt-api --yaml $WORK/fddt-yaml/api.yaml`.

---

## Step 15 — Workers and scheduler

> 📍 **Who:** you · **Type in:** Cloud Shell · **Check in the portal** (each of `fddt-worker-extraction`, `fddt-worker-vision`, `fddt-worker-forensics`, `fddt-beat`):
>
> - **Application › Revisions and replicas**: **Running**. Workers have no ingress, so the *Health* column may stay empty. That is normal.
> - **Monitoring › Log stream**: a worker prints `Connected to rediss://…` and `… ready.`; `fddt-beat` prints `beat: Starting...` and `housekeeping@… ready.`, then a `log_queue_metrics` task every minute.
> - `fddt-worker-forensics` showing **0 replicas** while no documents are being processed is **normal** (scale to zero). Upload a document and it starts within ~30–60 s.
> - `fddt-worker-forensics` **› Application › Scale**: the two Redis rules `backlog-level0` and `backlog-level9`.

No ingress. `terminationGracePeriodSeconds: 300` matters: on scale-in or a new revision, Celery gets SIGTERM, stops taking new tasks and finishes the running ones. Tasks are acknowledged late, so one killed mid-run is redelivered. With Redis that redelivery happens only after Celery's 1-hour visibility timeout, so a generous grace period avoids that delay.

### 15.1 Extraction and vision workers (fixed replicas, bounded by Azure quota)

```
worker_yaml() {   # $1=queue $2=cpu $3=mem $4=min $5=max
{ yaml_head; cat <<EOF
    activeRevisionsMode: Single
  template:
    terminationGracePeriodSeconds: 300
    containers:
      - name: worker-$1
        image: $BACKEND_IMAGE
        command: ["sh"]
        args: ["start-worker.sh", "$1"]
        resources:
          cpu: $2
          memory: $3
        env:
$(backend_env)
    scale:
      minReplicas: $4
      maxReplicas: $5
EOF
}; }

worker_yaml extraction $EXT_CPU $EXT_MEM $EXT_N $EXT_N > $WORK/fddt-yaml/worker-extraction.yaml
worker_yaml vision     $VIS_CPU $VIS_MEM $VIS_N $VIS_N > $WORK/fddt-yaml/worker-vision.yaml
az containerapp create -g $RG -n fddt-worker-extraction --yaml $WORK/fddt-yaml/worker-extraction.yaml
az containerapp create -g $RG -n fddt-worker-vision     --yaml $WORK/fddt-yaml/worker-vision.yaml
```

**Do not autoscale these two on queue length.** Their throughput is capped by the Azure OpenAI and Document Intelligence quotas through the global limiter. Extra replicas would only sit waiting on the limiter. Size them so the threads just saturate the caps (see the scaling playbook).

### 15.2 Forensics worker (CPU-bound, autoscaled on queue backlog, scale to zero)

Forensics has no external rate limit, so it is the queue that benefits from scaling out, and from scaling **to zero** when there is no work (`FOR_MIN=0` on tiers S/M). That works because the every-minute platform jobs run on their own `housekeeping_queue` in the beat container (15.3), so nothing wakes the forensics workers except real work. The Redis

rules below start the first replica as soon as one forensics task waits, and the platform stops the last replica after ~5 minutes without work. The first document after a quiet period waits ~30–60 s longer for its forensic checks. `start-worker.sh forensics` runs **one prefork process per vCPU of the container's limit** (it reads the cgroup quota). Give it about **2 GiB per vCPU**: pages are rendered at 200 DPI in memory.

KEDA's Redis scaler watches list lengths. Celery's Redis transport keeps one list per priority level (`forensics_queue`, `forensics_queue:1` … `forensics_queue:9`), and Container Apps scales on the **largest** of all rules. Fair-share puts a big batch's tail at level 9 and new or small companies at level 0, so two rules cover the real cases.

```
worker_yaml forensics $FOR_CPU $FOR_MEM $FOR_MIN $FOR_MAX > $WORK/fddt-yaml/worker-forensics.yaml
cat >> $WORK/fddt-yaml/worker-forensics.yaml <<EOF
      rules:
        - name: backlog-level0
          custom:
            type: redis
            metadata:
              address: "$REDIS_HOST:$REDIS_PORT"
              listName: "forensics_queue"
              listLength: "20"
              enableTLS: "true"
            auth:
              - secretRef: redis-key
                triggerParameter: password
        - name: backlog-level9
          custom:
            type: redis
            metadata:
              address: "$REDIS_HOST:$REDIS_PORT"
              listName: "forensics_queue:9"
              listLength: "20"
              enableTLS: "true"
            auth:
              - secretRef: redis-key
                triggerParameter: password
EOF
az containerapp create -g $RG -n fddt-worker-forensics --yaml $WORK/fddt-yaml/worker-forensics.yaml
```

`listLength: 20` adds one replica per ~20 waiting forensics tasks (each document creates about 4 of them). Lower it to scale out sooner. For fixed replicas instead, delete the `rules:` block and set `FOR_MIN = FOR_MAX` (never `FOR_MIN=0` without the rules: nothing would start the worker).

### 15.3 Beat (scheduler + housekeeping worker) — exactly one replica

```
{ yaml_head; cat <<EOF
    activeRevisionsMode: Single
  template:
    containers:
      - name: beat
        image: $BACKEND_IMAGE
        command: ["sh"]
        args: ["start-beat.sh"]
        resources:
          cpu: 0.5
          memory: 1Gi
        env:
$(backend_env)
    scale:
      minReplicas: 1
      maxReplicas: 1
EOF
} > $WORK/fddt-yaml/beat.yaml
az containerapp create -g $RG -n fddt-beat --yaml $WORK/fddt-yaml/beat.yaml
```

`start-beat.sh` runs Celery beat **and** a one-slot worker for `housekeeping_queue` in the same process. That worker runs the jobs beat schedules: the per-minute `queue_metrics` / `queue_alert` log lines, the stuck-document recovery every 5 minutes, and the nightly (02:00 UTC) usage reconciliation. **Two beats would run every job twice. Never scale it above 1.**

✅ **Verify (all four):** `for a in fddt-worker-extraction fddt-worker-vision fddt-worker-forensics fddt-beat; do az containerapp revision list -g $RG -n $a --query "[0].properties.runningState" -o tsv; done` → `Running` × 4. In the logs, each worker prints `Connected to rediss://...` and `celery@<queue>@... ready.`:

```
az containerapp logs show -g $RG -n fddt-worker-forensics --tail 50
```

---

## Step 16 — Web front end ( `fddt-web` )

📍 **Who:** you · **Type in:** Cloud Shell, then your **browser** · **Check in the portal** ( `fddt-web` ):

- **Overview › Application Url**: click it. The app's sign-in page must open (on the `*.azurecontainerapps.io` address, until step 18).
- **Networking › Ingress**: *Accepting traffic from anywhere*, target port 80.
- **Application › Containers › Environment variables**: `API_UPSTREAM` and the two body limits.

## 16.1 Create

```
cat > $WORK/fddt-yaml/web.yaml <<EOF
location: $LOC
identity:
  type: UserAssigned
  userAssignedIdentities:
    $UAMI_ID: {}
properties:
  environmentId: $ENV_ID
  workloadProfileName: Consumption
  configuration:
    activeRevisionsMode: Single
    ingress:
      external: true
      targetPort: 80
      transport: auto
      allowInsecure: false
    registries:
      - server: $ACR_SERVER
        identity: $UAMI_ID
  template:
    containers:
      - name: web
        image: $WEB_IMAGE
        resources:
          cpu: 0.25
          memory: 0.5Gi
        env:
          - name: API_UPSTREAM
            value: "http://fddt-api"
          - name: UPLOAD_BODY_LIMIT
            value: "11m"
          - name: BULK_UPLOAD_BODY_LIMIT
            value: "301m"
        probes:
          - type: Liveness
            httpGet: { path: /, port: 80 }
            periodSeconds: 15
          - type: Readiness
            httpGet: { path: /, port: 80 }
            periodSeconds: 10
    scale:
      minReplicas: $WEB_MIN
      maxReplicas: $WEB_MAX
      rules:
        - name: http-concurrency
          http:
            metadata:
              concurrentRequests: "100"
EOF
az containerapp create -g $RG -n fddt-web --yaml $WORK/fddt-yaml/web.yaml
export WEB_FQDN=$(az containerapp show -g $RG -n fddt-web --query properties.configuration.ingress.fqdn -o
tsv)
echo "https://$WEB_FQDN"
```

### 16.2 What nginx does ( `frontend/nginx.conf.template` )

| Setting | Value | Notes |
|---|---|---|
| `API_UPSTREAM` | `http://fddt-api` | nginx strips `/api` and forwards to the API; `Host` = `fddt-api` (Container Apps routes by host) |
| `UPLOAD_BODY_LIMIT` | `11m` | Single uploads. Must be ≥ the **largest per-company file limit** + ~1 MB multipart overhead |
| `BULK_UPLOAD_BODY_LIMIT` | `301m` | Bulk zip uploads (streamed, unbuffered). Must be ≥ the largest per-company zip limit |
| Proxy timeouts | 300 s | The Container Apps ingress itself caps a request at **240 s**. A 300 MB zip therefore needs ≥ ~10 Mbit/s of client upload bandwidth |
| `.mjs` MIME type | `application/javascript` | Needed for the pdf.js worker |

**When a platform admin raises a company's limits** (e.g. 20 MB / 500 MB), raise these too: `az containerapp update -g $RG -n fddt-web --set-env-vars UPLOAD_BODY_LIMIT=21m BULK_UPLOAD_BODY_LIMIT=501m`.

### 16.3 Verify the proxy path

```
curl -s https://$WEB_FQDN/api/health          # {"status":"ok","environment":"production"}
curl -s -o /dev/null -w "%{http_code}\n" https://$WEB_FQDN/        # 200
```

A `502` / `504` here means nginx cannot reach the API. Check `allowInsecure: true` on `fddt-api` and test from inside: `az containerapp exec -g $RG -n fddt-web --command "wget -qO- http://fddt-api/health"`.

### 16.4 Optional — restrict the UI to office IPs

```
for cidr in $OFFICE_CIDRS; do
  az containerapp ingress access-restriction set -g $RG -n fddt-web \
    --rule-name "office-${cidr//[\/.]/-}" --ip-address $cidr --action Allow
done
```

(Once any *Allow* rule exists, every other source is denied.) Blob SAS URLs stay reachable from anywhere, but they expire quickly.

---

## Step 17 — Blob CORS for the app origin

> 📍 **Who:** you · **Type in:** Cloud Shell · **Check in the portal: fddtprodfiles › Settings › Resource sharing (CORS) → Blob service** tab: one row with your origins and `GET, HEAD, OPTIONS`.

The PDF viewer fetches files from Blob Storage directly, so the storage account must allow the app's origin(s).

```
az storage cors clear --services b --connection-string "$SA_CONN"
az storage cors add --services b --methods GET HEAD OPTIONS \
  --origins "https://$APP_DOMAIN" "https://$WEB_FQDN" \
  --allowed-headers '*' --exposed-headers '*' --max-age 3600 --connection-string "$SA_CONN"
```

Add the Front Door endpoint hostname here too if you use step 19. **Re-run this step whenever an app hostname changes.**

✅ **Verify:** `az storage cors list --services b --connection-string "$SA_CONN" -o table`.

---

## Step 18 — Custom domain and TLS

> 📍 **Who: you** run the commands (Cloud Shell). The **client's DNS administrator** creates the two DNS records, wherever their domain is managed (Azure DNS, GoDaddy, Cloudflare, ...). Send them the table in 18.1. **Check:** `nslookup <APP_DOMAIN>` in Cloud Shell, or https://dnschecker.org, shows the CNAME. Then in the portal: **fddt-web › Settings › Custom domains**: your domain with *Binding type* **SNI SSL** / *Certificate* **Managed**, status **Secured**.

### 18.1 DNS records to create (in the client's DNS zone)

```
export VERIFY_ID=$(az containerapp show -g $RG -n fddt-web --query properties.customDomainVerificationId -o
tsv)
echo "CNAME  ${APP_DOMAIN}            ->  $WEB_FQDN"
echo "TXT    asuid.${APP_DOMAIN}      ->  $VERIFY_ID"
```

| Type | Name | Value | TTL |
|---|---|---|---|
| CNAME | `app` (the host part of `$APP_DOMAIN`) | `$WEB_FQDN` | 3600 |
| TXT | `asuid.app` | `$VERIFY_ID` | 3600 |
| *(apex domain only, instead of CNAME)* A | `@` | `$ENV_IP` | 3600 |

### 18.2 Bind the hostname with a free managed certificate

After the records resolve ( `nslookup $APP_DOMAIN` ):

```
az containerapp hostname add  -g $RG -n fddt-web --hostname $APP_DOMAIN
az containerapp hostname bind -g $RG -n fddt-web --hostname $APP_DOMAIN \
  --environment $ENV --validation-method CNAME          # use HTTP for an apex A record
```

The certificate is issued and renewed automatically (it can take up to ~20 minutes the first time).

✅ **Verify:** `curl -sI https://$APP_DOMAIN | head -1` → `HTTP/2 200`, and the browser shows a valid certificate.

---

## Step 19 — Optional: Front Door + WAF

> 📍 **Who:** you (commands); the client's DNS admin (records) · **Type in:** Cloud Shell, then the portal for the domain and WAF parts · **Check in the portal:** `fddtprod-afd` **› Settings › Front Door manager** (endpoint, route `default`, origin group `web`), **› Settings › Domains** (*Validation state* **Approved**, certificate *Deployed*), **› Settings › Security policies** (the WAF policy attached).

Optional, and not needed for sign-in protection: the app itself throttles sign-ins (5 failures lock an email address for 15 minutes; 20 attempts per IP per minute). Use Front Door only if you want an edge WAF, or global edge caching of the static files (~$35/month + usage). The Front Door **Standard** tier supports WAF *custom rules* (including rate limits). The managed OWASP rule sets and bot protection need **Premium**.

```
az afd profile create -g $RG --profile-name $PREFIX-afd --sku Standard_AzureFrontDoor \
  --origin-response-timeout-seconds 240
az afd endpoint create -g $RG --profile-name $PREFIX-afd --endpoint-name $PREFIX-web --enabled-state
Enabled
az afd origin-group create -g $RG --profile-name $PREFIX-afd --origin-group-name web \
  --probe-request-type HEAD --probe-protocol Https --probe-interval-in-seconds 60 --probe-path / \
  --sample-size 4 --successful-samples-required 3 --additional-latency-in-milliseconds 50
az afd origin create -g $RG --profile-name $PREFIX-afd --origin-group-name web --origin-name aca \
  --host-name $WEB_FQDN --origin-host-header $WEB_FQDN --https-port 443 --http-port 80 \
  --priority 1 --weight 1000 --enabled-state Enabled
az afd route create -g $RG --profile-name $PREFIX-afd --endpoint-name $PREFIX-web --route-name default \
  --origin-group web --supported-protocols Http Https --https-redirect Enabled \
  --forwarding-protocol HttpsOnly --link-to-default-domain Enabled --patterns-to-match "/*"
```

Then:

1. **Custom domain on Front Door** (instead of step 18): portal → Front Door → *Domains* → *Add* → `$APP_DOMAIN`, *AFD managed certificate*. Create the `_dnsauth.app` TXT record it shows, then a CNAME `app` → the endpoint hostname (`<endpoint>.z01.azurefd.net`). Associate the domain with route `default`.
2. **WAF policy** (portal → *Front Door* → *Security policies* → *Add*): mode **Prevention**, custom rule *Rate limit*: `RequestUri` contains `/api/auth/login`, threshold **20 requests per 1 minute per client IP**, action **Block**.
3. **Lock the origin to Front Door:** change `nsg-aca` rules 100/110 to source service tag `AzureFrontDoor.Backend`:

   ```
   az network nsg rule update -g $RG --nsg-name nsg-aca -n allow-https-in --source-address-prefixes
   AzureFrontDoor.Backend
   az network nsg rule update -g $RG --nsg-name nsg-aca -n allow-http-in  --source-address-prefixes
   AzureFrontDoor.Backend
   ```

4. Add the Front Door hostnames to the Blob CORS rule (step 17).
5. Front Door **does not cache** `/api/*`: caching is off on the route by default. Leave it off.

---

## Step 20 — Monitoring and alerts

> 📍 **Who:** you · **Type in:** Cloud Shell, then the portal for the extra alerts · **Check in the portal:** search **Monitor** → **Alerts › Alert rules**: filter by resource group `rg-fddtprod` → 7 log alerts + 3 PostgreSQL alerts. **Monitor › Alerts › Action groups** → `fddtprod-oncall` → **Test action group** sends a test email. To add a portal-only alert: open the resource → **Monitoring › Alerts** → **+ Create › Alert rule** → pick the *Signal name* from the table below → threshold → *Actions*: select `fddtprod-oncall`.

```
az monitor action-group create -g $RG -n $PREFIX-oncall --short-name fddtops \
  --action email oncall $ALERT_EMAIL
export AG_ID=$(az monitor action-group show -g $RG -n $PREFIX-oncall --query id -o tsv)

logalert() {   # $1 name  $2 KQL  $3 threshold(count >)  $4 window  $5 severity
  az monitor scheduled-query create -g $RG -n "$1" --scopes $LAW_RES_ID \
    --condition "count 'Q' > $3" --condition-query Q="$2" \
    --evaluation-frequency 5m --window-size $4 --severity $5 --action-groups $AG_ID -o none && echo "alert
$1"
}
logalert fddt-queue-backlog   "ContainerAppConsoleLogs_CL | where Log_s has 'queue_alert'" 0 5m 2
logalert fddt-worker-errors   "ContainerAppConsoleLogs_CL | where ContainerAppName_s startswith 'fddt-
worker' | where Log_s has_any ('ERROR','Traceback')" 5 15m 2
logalert fddt-db-permission   "ContainerAppConsoleLogs_CL | where Log_s has 'permission denied'" 0 15m 1
logalert fddt-azure-throttled "ContainerAppConsoleLogs_CL | where Log_s has '429'" 10 15m 3
logalert fddt-stuck-documents "ContainerAppConsoleLogs_CL | where Log_s has 'stuck_documents_requeued'" 0
15m 3
logalert fddt-login-lockouts  "ContainerAppConsoleLogs_CL | where Log_s has 'login_lockout'" 10 15m 2
logalert fddt-api-errors      "ContainerAppConsoleLogs_CL | where ContainerAppName_s == 'fddt-api' | where
Log_s has_any ('ERROR','Traceback')" 5 15m 2

export PG_ID=$(az postgres flexible-server show -g $RG -n $PG --query id -o tsv)
az monitor metrics alert create -g $RG -n fddt-pg-cpu     --scopes $PG_ID --condition "avg cpu_percent >
80"     --window-size 15m --evaluation-frequency 5m --action $AG_ID
az monitor metrics alert create -g $RG -n fddt-pg-storage --scopes $PG_ID --condition "avg storage_percent
> 80" --window-size 30m --evaluation-frequency 15m --action $AG_ID
az monitor metrics alert create -g $RG -n fddt-pg-memory  --scopes $PG_ID --condition "avg memory_percent >
90"  --window-size 15m --evaluation-frequency 5m --action $AG_ID
```

Add in the portal (metric names differ between API versions):

| Resource | Alert | Threshold |
|---|---|---|
| PostgreSQL (M/L) | PgBouncer *client connections waiting* | > 0 for 5 min |
| Managed Redis | *Used memory percentage* / *Server load* | > 70 % / > 80 % |
| `fddt-api`, `fddt-web` | *Requests* filtered to status 5xx | > 20 in 5 min |
| Each container app | *Restart count* | > 3 in 15 min |
| Document Intelligence / OpenAI | *Client errors* (429) | sustained |

**Dashboards:** the app writes one `queue_metrics {json}` line per queue per minute. Chart it in Log Analytics:

```
ContainerAppConsoleLogs_CL
| where Log_s has "queue_metrics "
| extend m = parse_json(extract("queue_metrics (.*)", 1, Log_s))
| project TimeGenerated, queue = tostring(m.queue), waiting = toint(m.waiting),
          running = toint(m.running), oldest = todouble(m.oldest_waiting_seconds), p95 =
todouble(m.p95_wait_seconds)
| render timechart
```

The live view is also in the app: **Platform › Processing queues**.

✅ **Verify:** `az monitor scheduled-query list -g $RG -o table` lists 7 alerts; the action group's test notification (portal → *Test action group*) reaches the on-call mailbox.

---

## Step 21 — Admin access to the private database and Redis

> 📍 **Who:** you, only when you need database access (not needed for go-live) · **Type in:** Cloud Shell (create and connect) · **Check in the portal: fddtprod-bastion › Overview** (*Succeeded*) and **fddtprod-jump › Overview** (*Running*). Stop the VM when done: **fddtprod-jump › Overview › Stop**.

PostgreSQL and Redis have **no public endpoint**. For `psql` / `redis-cli` (incident work, issuer-registry bulk loads, PgBouncer stats), use a small jump VM reached only through Azure Bastion. Delete or stop the jump VM when not in use; Bastion Standard costs about $210/month, so delete it too if access is rare.

```
az network vnet subnet create -g $RG --vnet-name $VNET -n AzureBastionSubnet --address-prefixes
$SNET_BASTION
az network public-ip create -g $RG -n $PREFIX-bastion-ip --sku Standard
az network bastion create -g $RG -n $PREFIX-bastion --vnet-name $VNET --public-ip-address $PREFIX-bastion-
ip \
  --sku Standard --enable-tunneling true
az vm create -g $RG -n $PREFIX-jump --image Ubuntu2404 --size Standard_B1s \
  --vnet-name $VNET --subnet snet-jump --public-ip-address "" --nsg "" \
  --admin-username azureops --generate-ssh-keys

# Connect from Cloud Shell (the key was created in ~/.ssh by --generate-ssh-keys above)
az network bastion ssh -g $RG -n $PREFIX-bastion --auth-type ssh-key --username azureops \
  --ssh-key ~/.ssh/id_rsa --target-resource-id $(az vm show -g $RG -n $PREFIX-jump --query id -o tsv)
# On the VM:
sudo apt-get update && sudo apt-get install -y postgresql-client redis-tools
psql "host=<PG_HOST> port=5432 dbname=fddt user=fddtowner sslmode=require"
redis-cli -h <REDIS_HOST> -p 10000 --tls -a '<REDIS_KEY>' LLEN forensics_queue
```

**When you are done, remove Bastion** (it costs ~$0.29/hour while it exists, ~$210/month if forgotten) and stop the VM; recreate both next time with the commands above:

```
az network bastion delete -g $RG -n $PREFIX-bastion
az network public-ip delete -g $RG -n $PREFIX-bastion-ip
az vm deallocate -g $RG -n $PREFIX-jump
```

---

## Step 22 — Smoke test and first-day setup

> 📍 **Who:** you, with the client's key user for items 2–3 and 9 · **Where:** a **new browser tab** on the client's screen at `https://<APP_DOMAIN>`; Cloud Shell only for the last block · **Also open: fddt-worker-forensics › Monitoring › Log stream**, to watch the document being processed.

1. Open `https://<APP_DOMAIN>` and sign in as the `SEED_ADMIN_EMAIL` with the password printed in step 11. **Change the password** right away: **Change password** in the top navigation bar.
2. **Platform › Companies** → create the client company. It receives the template risk rules and 30/60 thresholds, and its **issuer registry is empty**. Set its upload limits; if they exceed 10 MB / 300 MB, raise the nginx limits (step 16.2).
3. **Platform › Users** → create the company's users (submitter, Reviewer L1, Reviewer L2).
4. **Platform › Processing queues** → extraction and vision show **≥ 1 consumer**; waiting = 0. Forensics may show 0 consumers while idle (scale to zero); after the upload in item 5 it shows one.
5. As a company user, create a case and upload one sample PDF from `sample-documents/`. Confirm:
   - the document goes `pending → processing → completed` and gets a risk tier within a few minutes;

- **the PDF renders in the case viewer** (if it is blank, Blob CORS is wrong: step 17);
- the checks panel shows results from all queues (extraction, visual review, forensics).

6. Upload an **image** and a **password-protected PDF**. Each must be refused with its specific message.
7. Upload a small **bulk zip**. It must split into cases.
8. As a reviewer, **generate a report** and open it.
9. As Reviewer L2, load the client's **real issuer registry**. Until then almost every document flags `issuer.not_in_registry`.
10. **Sign-in throttling:** in a private browser window, sign in with a wrong password 6 times for a test user. The 6th answer must be *Too many sign-in attempts. Try again in 15 minutes.* (It clears by itself after 15 minutes, or at once after a successful sign-in from another session.)
11. Check the logs: no `ERROR` / `Traceback` / `429` from the Azure services in the last hour (the Log Analytics queries from step 20).

Then **remove the seed step** from the migration job, so later releases do not reset the admin password:

```
export MIGRATE_CMD="alembic upgrade head && alembic current"
sed -i 's|args: \[".*"\]|args: ["alembic upgrade head \&\& alembic current"]|' $WORK/fddt-yaml/migrate.yaml
az containerapp job update -g $RG -n fddt-migrate --yaml $WORK/fddt-yaml/migrate.yaml
```

---

## Step 23 — Post-go-live hardening

> 📍 **Who:** you, plus the client admin for Defender (it is enabled per subscription/storage account) · **Type in:** Cloud Shell, or the portal paths in the table.

| Item | How | Priority |
|---|---|---|
| Lock the AI services to our egress IP | `az cognitiveservices account network-rule add -g $RG -n $DI --ip-address $EGRESS_IP`, then `az resource update --ids $(az cognitiveservices account show -g $RG -n $DI --query id -o tsv) --set properties.networkAcls.defaultAction=Deny` (repeat both for `$AOAI`). Only with the NAT Gateway from step 4 | High |
| Defender for Storage malware scanning on upload | Portal → Storage account → *Microsoft Defender for Cloud* → enable *On-upload malware scanning* (billed per GB scanned) | High (the app has no malware scan) |
| Key Vault diagnostic logs → Log Analytics | Portal → Key Vault → *Diagnostic settings* | Medium |
| PostgreSQL reserved capacity (1 year ≈ −40 % of compute) | Portal → Reservations, after 1–2 months on a stable size | Medium (cost) |
| Document Intelligence commitment tier | When volume is steady (cost levers) | Medium (cost) |

| Item | How | Priority |
|---|---|---|
| Check the real bill and right-size | Cost Management, filter tag `app=fddt`; compare with the estimate; shrink apps whose CPU/memory stay low (each app › **Monitoring** › **Metrics**) | Medium (cost), after month 1 |
| Resource lock on the RG | `az lock create -g $RG -n no-delete --lock-type CanNotDelete` | High |

**Known application gaps to accept or schedule** (from azure-migration-notes.md): no forgot-password flow (users change their own password; admins reset others); JWT in `localStorage`; no automatic retry of a failed Azure call (documents whose queued tasks were lost are re-queued, but a check lost after extraction finished is not); keys and connection strings, not managed identity, for Storage / Document Intelligence / OpenAI; no page-count limit on uploads.

---

# Scaling playbook

## What limits throughput

| Stage | Bound by | Scales by |
|---|---|---|
| Upload / UI | API replicas (HTTP autoscale), nginx | Automatic: `API_MIN…API_MAX`, `WEB_MIN…WEB_MAX` |
| OCR (`extraction_queue`) | Document Intelligence cap **10 calls/s** (Azure S0 allows 15) | Rarely the bottleneck. Raise the cap only after an Azure TPS increase |
| AI review (`vision_queue`, plus the classification call in extraction) | **The gpt-4.1-mini deployment's TPM** (= the subscription's free capacity, step 10). The app reserves ~24k tokens per 1-page and ~32k per 2-page document | Only more TPM helps; replicas alone do nothing. We don't request quota, so this is the ceiling. Plan with it |
| Forensics (`forensics_queue`) | CPU | Automatic: KEDA on backlog, `FOR_MIN…FOR_MAX` (0 = scale to zero), 1 process per vCPU |
| Database | vCores / connections | Scale the SKU; PgBouncer already multiplexes connections |
| Redis | Memory / ops (small: queues and counters only) | B0 → B3 |

**Platform-wide document throughput ≈ `AZURE_OPENAI_MAX_TOKENS_PER_MINUTE` ÷ 32,000 per minute** for 2-page documents:

| Deployment TPM (free capacity) | App cap (80 %) | ≈ 2-page docs / minute | ≈ docs / day (24 h) |
|---|---|---|---|
| 100k (tier S target) | 80k | 2.5 | 3,600 |
| 150k (tier M target) | 120k | 3.75 | 5,400 |
| 500k (tier L target) | 400k | 12.5 | 18,000 |
| *yours:* `$AOAI_TPM` | `$AOAI_TPM_CAP` | `$AOAI_TPM_CAP ÷ 32,000` | × 1,440 |

**Fairness under load is automatic:** each task's priority is computed from its company's outstanding work (`FAIR_SHARE_BUCKET_SIZE` = 5), so a company that submits one document is served before the tail of another company's 1,000-document batch. Nothing to configure per company.

## Thread sizing for the Azure queues

Threads needed in flight ≈ (calls per minute ÷ 60) × average call latency (s). Calls per minute ≈ `TPM cap ÷ ~4,000` (average reservation per vision call).

| Tier | Vision calls/min | In-flight at ~8 s | Configured (replicas × `VISION_WORKER_CONCURRENCY`) |
|---|---|---|---|
| S | 20 | ~3 | 1 × 4 |
| M | 30 | ~4 | 1 × 4 (raise to 6 if the limiter shows spare tokens) |
| L | 100 | ~13 | 4 × 4 |

A thread waiting on the limiter holds its slot, so do **not** oversize: far more threads than the caps can feed only add waiting. Extraction: `EXTRACTION_WORKER_CONCURRENCY` 4 per replica saturates the 10 calls/s cap at typical latency.

## Symptom → action

| Symptom (where you see it) | Likely cause | Action |
|---|---|---|
| `queue_alert` on **vision** / **extraction**; limiter "throttled" counters high (Platform › Processing queues) | The deployment's TPM is the ceiling | Expected at peaks: documents queue and finish later. If capacity has been freed in the subscription since step 10 (another deployment removed or lowered), raise this deployment's capacity and the caps (below). Otherwise this is the platform's limit |
| `queue_alert` on **forensics**; forensics replicas at `FOR_MAX` | CPU | Raise `--max-replicas` and/or `--cpu 4 --memory 8Gi` |
| Forensics OOM-killed (restart count) | Very large/many-page PDFs | Keep ~2 GiB per vCPU: `--cpu 2 --memory 4Gi`, or set `FORENSICS_WORKER_CONCURRENCY=1` with more replicas |
| `429` in logs | Caps above the real quota (quota lowered, or different RPM/TPM) | Re-read `rateLimits` (step 10), set caps to 80 % |
| API p95 slow, CPU high | Too few API replicas | Raise `API_MAX`; check DB CPU |
| PgBouncer *client connections waiting* > 0 | Pool too small | Raise `pgbouncer.default_pool_size` (e.g. 80), confirm `max_connections` headroom |
| PostgreSQL CPU > 80 % sustained | DB compute | Scale the SKU up (see below); check the slow-query log |
| Bulk upload fails with **413** | nginx body limit below the company's limit | Raise `BULK_UPLOAD_BODY_LIMIT` / `UPLOAD_BODY_LIMIT` (step 16.2) |
| Bulk upload fails after ~4 minutes | Ingress 240 s request limit | Client needs more upload bandwidth, or split the zip |

## Scaling commands

```
# Container apps (no downtime: a new revision rolls in)
az containerapp update -g $RG -n fddt-worker-forensics --min-replicas 2 --max-replicas 8 --cpu 2 --memory
4Gi
az containerapp update -g $RG -n fddt-worker-vision    --min-replicas 2 --max-replicas 2
az containerapp update -g $RG -n fddt-api              --min-replicas 3 --max-replicas 6

# Azure OpenAI: only if capacity was freed in the subscription (check it as in section 0.3). Re-running
# the create with a bigger --sku-capacity updates the deployment; then set the caps on EVERY backend app
# (the limiter is global; all apps must agree). Example for 300k TPM:
az cognitiveservices account deployment create -g $RG -n $AOAI --deployment-name $AOAI_DEPLOYMENT \
  --model-name gpt-4.1-mini --model-version 2025-04-14 --model-format OpenAI \
  --sku-name GlobalStandard --sku-capacity 300
for a in fddt-api fddt-worker-extraction fddt-worker-vision fddt-worker-forensics fddt-beat; do
  az containerapp update -g $RG -n $a --set-env-vars AZURE_OPENAI_MAX_REQUESTS_PER_MINUTE=240
AZURE_OPENAI_MAX_TOKENS_PER_MINUTE=240000
done
az containerapp job update -g $RG -n fddt-migrate --set-env-vars AZURE_OPENAI_MAX_REQUESTS_PER_MINUTE=240
AZURE_OPENAI_MAX_TOKENS_PER_MINUTE=240000
# (Also update AOAI_RPM_CAP / AOAI_TPM_CAP in your session so regenerated YAML matches.)

# PostgreSQL compute (HA: failover-based, ~1–2 min interruption; schedule off-hours)
az postgres flexible-server update -g $RG -n $PG --sku-name Standard_D4ds_v5 --tier GeneralPurpose
# PostgreSQL storage grows automatically (auto-grow on); it can never shrink.

# Redis
az redisenterprise update -g $RG -n $REDIS --sku Balanced_B3
```

**Tier S → M** requires moving PostgreSQL from Burstable to General Purpose (the SKU update above with `--tier GeneralPurpose`), optionally HA (`az postgres flexible-server update --high-availability ZoneRedundant`), and PgBouncer (step 7), then switching every backend app from `DB_POOL_SIZE` / `DB_MAX_OVERFLOW` to `DATABASE_POOLER_HOST` / `PORT`. Regenerate the YAML with `TIER=M` and run `az containerapp update --yaml` per app.

## Things that must NOT be scaled

- `fddt-beat` **stays at exactly 1** replica (it also runs the housekeeping jobs).
- **Do not autoscale extraction/vision on queue length.** They are quota-bound. Keep one replica each (not zero), so OCR of a new document starts at once.
- Do not raise `AZURE_OPENAI_MAX_*` above 80 % of the real quota.
- Every backend app must carry the **same** `AZURE_OPENAI_MAX_*` and `AZURE_DOCUMENT_INTELLIGENCE_MAX_CALLS_PER_SECOND` values.

## Known scaling limits in the code (for capacity planning)

- **Duplicate detection** compares each new page hash with every stored page hash of the company (linear scan). Fine for hundreds of thousands of pages per company; needs an index before millions.
- **Report generation is synchronous** in the API request (bounded by the 240 s ingress limit).
- The **classification call runs inside the extraction task**: when the OpenAI cap is saturated, OCR of new documents also waits.

---

# Release, rollback and staging

## Release (every deployment after the first)

> 📍 **Who:** you, in the client's Cloud Shell session (push the release to GitHub `main` first) · **Type in:** Cloud Shell · **Check in the portal:** each app's **Application › Revisions and replicas**: the new revision is *Running / Healthy* and receives 100 % of traffic.

```
cd ~/FDDT && git checkout main && git pull        # asks for the GitHub token again
source ~/fddt-deploy/reload.sh
export VERSION=$(git rev-parse --short HEAD)
az acr build -r $ACR -t fddt-backend:$VERSION ./backend
az acr build -r $ACR -t fddt-web:$VERSION ./frontend
NEW_BACKEND=$ACR_SERVER/fddt-backend:$VERSION

# 1. Migrate first (migrations are additive; old workers keep working during the roll)
az containerapp job update -g $RG -n fddt-migrate --image $NEW_BACKEND
az containerapp job start  -g $RG -n fddt-migrate
az containerapp job execution list -g $RG -n fddt-migrate --query "[0].properties.status" -o tsv   # wait:
Succeeded

# 2. API, then workers + beat, then web
az containerapp update -g $RG -n fddt-api --image $NEW_BACKEND
for a in fddt-worker-extraction fddt-worker-vision fddt-worker-forensics fddt-beat; do
  az containerapp update -g $RG -n $a --image $NEW_BACKEND
done
az containerapp update -g $RG -n fddt-web --image $ACR_SERVER/fddt-web:$VERSION

# 3. Smoke: /api/health, one upload, Platform › Processing queues
curl -s https://$APP_DOMAIN/api/health
```

Before step 1, note the time: it is your **point-in-time-restore target** if the migration goes wrong.

## Rollback

- **App only** (no migration in the release): `az containerapp update -g $RG -n <app> --image <previous tag>` for each app. Previous tags stay in ACR.
- **With a migration:** Alembic migrations are forward-only in practice. Roll forward with a fix, or restore the database with PITR to the noted time ( `az postgres flexible-server restore -g $RG -n $PG-restore --source-server $PG --restore-time <UTC ISO time>` ), repoint `owner-db-url` / `DATABASE_POOLER_HOST` to the restored server, and redeploy the previous images.

## Staging

Run the same runbook with `PREFIX=fddtstg` , `TIER=S` , its own resource group, its own hostname ( `staging.<client-domain>` ) and a **different VNet range** (e.g. `10.21.0.0/16` ). Deploy every release to staging first. It costs about the same as tier S.

## CI/CD (when added)

Suggested pipeline: `oxlint` + `tsc -b` (frontend) → `pytest` incl. the Postgres RLS suite → `az acr build` both images → staging release (as above) → manual approval → production release. Authenticate the pipeline with a federated (OIDC) service principal holding *Contributor* on the RG and *AcrPush* on the registry.

---

## Backups, DR and routine operations

| Data | Protection | Restore |
|---|---|---|
| PostgreSQL | Automatic backups, **35-day PITR**; zone-redundant HA (M/L) fails over automatically | PITR to a new server, repoint the URL/pooler host |
| Blob Storage | ZRS + soft delete (14 d) + versioning | Undelete / restore a previous version |
| Redis | Only queued tasks, fair-share and sign-in counters, limiter windows | If lost: the stuck-document job queues affected documents again within ~15 minutes (checks lost after a document's extraction finished are not re-run; re-upload such a document if a check is missing) |
| Secrets | Key Vault soft delete + purge protection | Recover the deleted secret |
| App config | `~/fddt-deploy/fddt-yaml/*.yaml` + image tags in ACR | Re-apply with `az containerapp update --yaml` |

**Targets (tier M):** RPO ≈ 5 minutes, RTO ≈ 1 hour (restore + repoint), excluding in-flight pipeline tasks.

**Routine**

| Task | When | How |
|---|---|---|
| Rotate Storage / DI / OpenAI keys | Every 90 days | Switch the apps to key2 in Key Vault → restart the apps ( `az containerapp revision restart` ) → regenerate key1 |
| Rotate `fddt_app` / `fddt_platform` passwords | Every 180 days | `ALTER ROLE fddt_app PASSWORD '…'` as owner (jump VM) → update the KV secret → restart the backend apps |
| Rotate `JWT_SECRET_KEY` | On compromise only | Signs everyone out |
| Re-calibrate token reservations | After a model, prompt or render change | `python -m scripts.calibrate_llm_tokens` (a few cents) |
| Review failed documents | Daily | Platform audit log: `document_processing_failed` |
| Review costs | Monthly | Cost Management, tag `app=fddt` |
| Restore drill | Quarterly | PITR to a temporary server and query it |

Key Vault references are re-read when a revision starts, so **restart the apps after changing a secret**: `az containerapp revision restart -g $RG -n <app> --revision $(az containerapp show -g $RG -n <app> --query properties.latestRevisionName -o tsv)` .

## Troubleshooting

| Symptom | Check | Fix |
|---|---|---|
| Revision fails: *unable to fetch secret* | Identity has *Key Vault Secrets User*? Secret name matches? | Wait for role propagation; fix the name; `az containerapp update --yaml` |

| Symptom | Check | Fix |
|---|---|---|
| Revision fails: *image pull unauthorized* | Identity has *AcrPull*? `registries.identity` set? | Assign the role, re-apply |
| Worker log: `A rediss:// URL must have parameter ssl_cert_reqs` | `redis-url` secret | Add `?ssl_cert_reqs=required` (step 11), restart |
| Worker log: `CROSSSLOT` / `MOVED` errors | Redis clustering policy | The cache must be **Non-clustered**. Recreate it (the policy cannot be changed) |
| Worker log: `Name or service not known` for Redis or PG | Private DNS | Zone linked to the VNet? `privatelink.redis.azure.net` zone group on the endpoint? |
| `password authentication failed for user "fddt_app"` | The role was created with a different password | Migrations set it on create only: `ALTER ROLE fddt_app PASSWORD '<kv value>'` as owner |
| Documents stay `pending` | Platform › Processing queues: consumers online? | Start/repair the workers; re-upload documents uploaded while Redis was down |
| Documents `failed` at extraction | Worker log | Wrong DI/OpenAI endpoint or key; deployment name must contain `gpt-4.1` |
| PDF viewer blank, browser console shows CORS error | Blob CORS | Step 17 with the exact origin (scheme + host) |
| PDF viewer blank for some users only | Their proxy blocks `*.blob.core.windows.net` | Client IT must allow it (flow 2) |
| `502` on `/api/*` | `fddt-api` healthy? `allowInsecure: true` ? | `az containerapp exec -n fddt-web --command "wget -qO- http://fddt-api/health"` |
| `413 Request Entity Too Large` | nginx body limits | Step 16.2 |
| Many `429` s | Caps vs. quota | Step 10: caps = 80 % of `rateLimits` |
| Jobs run twice (nightly reconciliation twice) | `fddt-beat` replicas | Must be exactly 1 |
| No `queue_metrics` lines, stuck documents never re-queued | `fddt-beat` log stream shows `housekeeping@… ready.` ? | It must run `start-beat.sh` (beat **and** the housekeeping worker), not `celery beat` alone |
| Forensics checks never start | `fddt-worker-forensics` has `minReplicas: 0` but no Redis scale rules, or the rules can't reach Redis | Add the rules from 15.2 (secret `redis-key` , TLS on); or set `minReplicas: 1` |
| A user gets *Too many sign-in attempts* | 5 failed sign-ins for that address in 15 minutes (or 20 attempts from one IP in a minute) | Wait for the time shown, or sign in successfully once it has passed. Repeated lockouts across many addresses: see the `fddt-login-lockouts` alert |
| `document_requeued` rows in the audit log / `stuck_documents_requeued` alert | Redis lost queued tasks (restart, maintenance without HA, outage) | Nothing to do: the documents were queued again. After 3 attempts a document is marked failed with an explanation: re-upload it |
| Database `permission denied` in logs | A migration added a table without a grant | Re-run the migration job; report as a bug |

## Go-live checklist

**Infrastructure**

- [ ] Tier chosen; gpt-4.1-mini **Global Standard** deployment created with the subscription's free capacity (no quota request); caps = 80 % of its real RPM/TPM on **every** backend app; expected throughput told to the client.
- [ ] Document Intelligence on pricing tier **S0** (not F0).
- [ ] PostgreSQL 16, private access only, `sslmode=require`, 35-day backups; M/L: zone-redundant HA + PgBouncer on 6432.
- [ ] Managed Redis **Non-clustered**, private endpoint, `rediss://…:10000/0?ssl_cert_reqs=required` for both Celery URLs.
- [ ] HA decision for PostgreSQL and Redis recorded with the client (tier M launches without HA).
- [ ] Storage: container private, anonymous access off, soft delete + versioning, CORS = app origin(s) only.
- [ ] Only `fddt-web` has external ingress; `fddt-api` internal; workers/beat no ingress.
- [ ] `fddt-beat` min = max = **1**, running `start-beat.sh` (its log shows `housekeeping@… ready.`).
- [ ] `fddt-worker-forensics` scales from 0 and back (upload a document after an idle period).
- [ ] Log Analytics daily cap set (1 GB); Bastion deleted after use; NAT Gateway only if needed.
- [ ] NSGs on `snet-aca`, `snet-pg`, `snet-pe` as in 2.4.
- [ ] Custom domain bound, managed certificate valid; HTTP redirects to HTTPS.
- [ ] Alerts created and the action group test reached on-call.
- [ ] Resource lock on the resource group.

**Application**

- [ ] Migration job Succeeded at head `f3b7d1e8a4c5` (or later); seed step **removed** from the job.
- [ ] All secrets random and only in Key Vault; seeded admin password changed after first login.
- [ ] Smoke test (step 22) passed: upload → risk tier, PDF renders, invalid uploads refused, bulk zip, report.
- [ ] Client company created, users created, **real issuer registry loaded**, risk weights reviewed with the client.
- [ ] nginx body limits ≥ the largest per-company upload limits.
- [ ] A browser walkthrough of every role (submitter, Reviewer L1, Reviewer L2, platform admin).

**Governance**

- [ ] Client's written acceptance of Global Azure OpenAI processing (or a regional deployment chosen).
- [ ] Known gaps (step 23) accepted or scheduled; sign-in throttling tested (step 22, item 10).
- [ ] Handover sheet completed and stored with the operations team.

---

## Handover sheet

Fill in and store with the operations team (no secrets here: those stay in Key Vault).

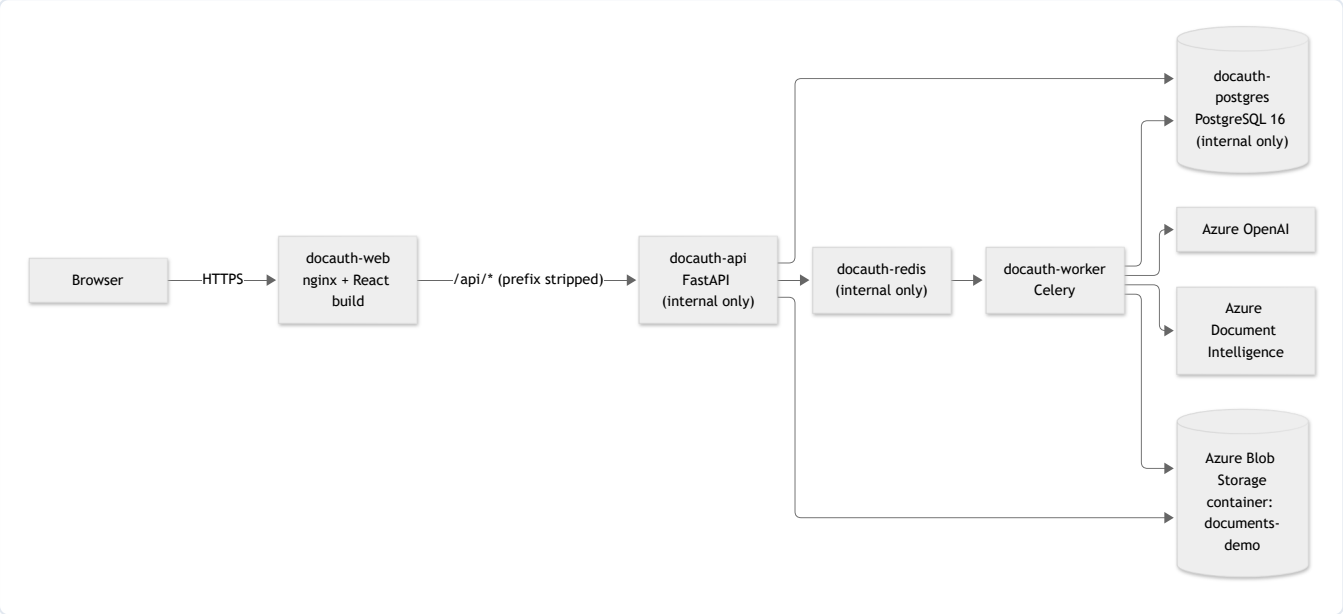
| Item | Value |
|---|---|
| Subscription / resource group | |
| Region / tier | |
| App URL | `https://` |
| Container Apps default FQDN ( `fddt-web` ) | |

| Item | Value |
|---|---|
| **Inbound public IP** (environment static IP) | |
| **Outbound public IP** (NAT Gateway, only if `USE_NAT=yes`) | |
| Front Door endpoint (if used) | |
| VNet / subnets | `10.20.0.0/16` — aca `/23`, pg `/24`, pe `/24`, bastion `/26`, jump `/28` |
| PostgreSQL FQDN / ports | `<pg>.postgres.database.azure.com` — 5432 direct, 6432 PgBouncer |
| PostgreSQL private IP | |
| Redis host / port / private IP | `<redis>.<region>.redis.azure.net` — 10000 TLS |
| Storage account / container | — `documents` |
| Azure OpenAI endpoint / deployment (gpt-4.1-mini, Global Standard) / capacity (RPM, TPM) / app caps | |
| Document Intelligence endpoint / pricing tier | — S0 |
| Key Vault | |
| Container Registry / current image tag | |
| Log Analytics workspace / action group | |
| Alembic head at go-live | |
| First platform admin (email only) | |
| DNS records created (CNAME / TXT) and DNS owner | |
| On-call contact | |

# Azure demo deployment

A small, low-cost Azure deployment used to demonstrate the platform. It is **not** the production design (see production-deployment.md for that). It predates multi-tenancy and the three processing queues — redeploying it with the current images needs the changes listed under *Updating the demo to the current build* below.

## Topology




Only `docauth-web` has a public address. The API, Redis and PostgreSQL are reachable only from inside the Container Apps environment, so the database is never exposed to the internet. The browser talks to a single origin, so no CORS configuration is needed.

## Resources

| Resource | Purpose |
|---|---|
| Azure Container Apps: `docauth-web`, `docauth-api`, `docauth-worker`, `docauth-redis`, `docauth-postgres` | The five runtime components (1 replica each) |
| Azure Container Registry | Holds the backend and frontend images |
| Azure Blob Storage container `documents-demo` | Original documents, signature crops and report PDFs (kept separate from development data) |
| Azure Document Intelligence, Azure OpenAI | Existing resources, used as in development |

## Images

Both images are built from the repository:

| Image | Source | Notes |
|---|---|---|
| Backend | `backend/Dockerfile` | One image for both the API (`start-api.sh`) and the worker (`start-worker.sh`) |

| Image | Source | Notes |
|---|---|---|
| Frontend | `frontend/Dockerfile`, `frontend/nginx.conf.template` | Static build served by nginx; `API_UPSTREAM` selects the backend address |

`start-api.sh` applies the Alembic migrations, seeds the administrator account, then starts the API, so a new environment bootstraps itself.

## Configuration

All settings are environment variables (see environment-variables.md). Secrets (database URL, JWT secret, storage connection string, Azure keys, seeded administrator password) are stored as Container Apps secrets and referenced by the apps; none are in the images or the repository. The seeded administrator email and password come from `SEED_ADMIN_EMAIL` / `SEED_ADMIN_PASSWORD`.

## Operating notes

- **PDF viewer requirements.** The browser loads the pdf.js worker from the web app and fetches each PDF directly from Blob Storage. This needs (1) the web server to serve `.mjs` files as `application/javascript` (configured in `frontend/nginx.conf.template`) and (2) a Blob Storage CORS rule allowing `GET`, `HEAD` and `OPTIONS` from the web app's address. Add the rule again if the web app's address changes.
- The Celery worker ran the default prefork pool with two processes (Linux) when the demo was set up, before the queue split (see *Updating the demo to the current build*).
- Redis and PostgreSQL run as containers on the container's temporary disk (no persistent volume). If either restarts, in-flight jobs (Redis) or all case data (PostgreSQL) are lost. Restart the API app to re-apply the migrations and re-seed the administrator, then re-upload the sample documents.
- This deployment is intended for demonstrations. For production use production-deployment.md (managed PostgreSQL and Redis with private networking, persistent storage, Key Vault, monitoring).

## Updating the demo to the current build

- **Workers:** the pipeline now uses three queues. A single `docauth-worker` must consume all of them: `celery -A app.tasks.celery_app worker -Q extraction_queue,vision_queue,forensics_queue,celery --pool=threads --concurrency=8` (or run `start-worker.sh extraction|vision|forensics` as three apps). `start-worker.sh` with no argument serves **only** `forensics_queue`, so documents would never be OCR'd.
- **Beat:** add one `start-beat.sh` app for the nightly usage reconciliation and queue metrics (optional for a demo).
- **Database roles:** the migrations now create `fddt_app` / `fddt_platform`; set `DATABASE_APP_PASSWORD` / `DATABASE_PLATFORM_PASSWORD` as secrets on every backend app. The Postgres container's `docauth` user is a superuser, which is fine for a demo.
- **Seeded account:** `admin@example.com` is now a **platform admin** — create a company and its users under *Platform* before submitting cases.
- **Uploads:** PDF only, per-company limits (10 MB / 300 MB zip by default); the nginx template caps request bodies at `UPLOAD_BODY_LIMIT` (11m) and bulk zips at `BULK_UPLOAD_BODY_LIMIT` (301m), both set by env vars.

## Tear-down

Delete the five `docauth-*` container apps, the container registry and the `documents-demo` container. The demo creates no other resources.